

Python E-Course

Module 1 — Digital Image Fundamentals

Detailed Notes

Module Overview

Level: Foundations

Duration: ~1 week (6-8 hours)

Prerequisites: Working Python install (see [setup-guide.md](#))

What You'll Learn

- Explain what a digital image actually is (grids of numbers).
- Read pixel coordinates and intensities correctly.
- Distinguish resolution from file size from color depth.
- Treat images as NumPy arrays — shape, dtype, channels.
- Navigate the (row, col) vs (x, y) coordinate confusion.

Lessons

#	Lesson	Duration
1	What is a Digital Image?	25 min
2	Pixels and Coordinates	30 min
3	Resolution and File Size	25 min
4	Color Depth and Bit-Depth	30 min
5	Image as a NumPy Array	35 min
6	Coordinate Systems & Conventions	25 min

Assessment

- Exercises — 18 (3 per lesson)
- Quiz — 10 MCQ
- Assignment — Image Info Inspector
- Project — Image Info Inspector CLI

What's Next

Module 2: Image I/O & Display — load, save, and view images correctly.

Lesson 1: What is a Digital Image?

Module: 1 — Digital Image Fundamentals

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Describe a digital image as a 2D or 3D grid of intensity values.
- Distinguish grayscale from color images at the data level.
- Read a tiny matrix as an image and predict what it looks like.

Prerequisites

- Working python + numpy + cv2.

1. Why This Matters

Every image processing operation is just arithmetic on a grid of numbers. Once you can stop thinking of an image as a photograph and start thinking of it as `np.ndarray`, everything from blurring to edge detection becomes "do this math at every pixel".

2. Concept

2.1 Grayscale = 2D grid

A grayscale image is a matrix of intensity values. In an 8-bit image, 0 = black, 255 = white, anything in between is gray.

A 5x5 image of a bright spot in the middle:

```
10  10  10  10  10
10  50  80  50  10
10  80 200  80  10
10  50  80  50  10
10  10  10  10  10
```

Eye this matrix — you'd see a dim background with a brighter spot in the centre.

2.2 Color = 3D grid (height × width × channels)

A color image stacks three grayscale grids — one per color channel. Standard order in OpenCV: B, G, R (blue, green, red).

A 3x3 pure-blue image (BGR):

Channel 0 (Blue):	Channel 1 (Green):	Channel 2 (Red):
255 255 255	0 0 0	0 0 0
255 255 255	0 0 0	0 0 0
255 255 255	0 0 0	0 0 0

So the array shape is (3, 3, 3) — height 3, width 3, 3 channels.

2.3 Continuous vs Sampled

A real-world scene is continuous; a camera samples it at a finite number of points (one per pixel). The number of samples = resolution. The precision of each sample = bit depth. We cover both in lessons 3 and 4.

2.4 Origin: top-left

By convention in computing, the origin (row 0, column 0) is the top-left corner. Row index grows downward; column index grows rightward.

	col 0	col 1	col 2
row 0	origin	→	→
row 1	↓		
row 2	↓		

(Math textbooks often put origin at bottom-left; image libraries don't.)

3. Code Examples

Example 1: Build a tiny image from scratch

```
import numpy as np
import matplotlib.pyplot as plt

img = np.array([
    [ 10,  10,  10,  10,  10],
    [ 10,  50,  80,  50,  10],
    [ 10,  80, 200,  80,  10],
    [ 10,  50,  80,  50,  10],
    [ 10,  10,  10,  10,  10],
], dtype=np.uint8)

print("shape:", img.shape, "dtype:", img.dtype)
plt.imshow(img, cmap="gray", vmin=0, vmax=255)
plt.title("hand-crafted 5x5")
plt.axis("off")
plt.show()
```

Expected Output (visual): A 5×5 grayscale image, almost-black around the edges, with a brighter pixel in the centre (intensity 200, almost white) and a halo of mid-gray values around it. Console prints shape: (5, 5) dtype: uint8.

Example 2: Load a real image and inspect it

```
from skimage.data import coins
import numpy as np

img = coins()
print("shape:", img.shape)          # (303, 384)
print("dtype:", img.dtype)          # uint8
```

```
print("min, max:", img.min(), img.max())
print("mean:", img.mean())
```

Expected Output (console):

```
shape: (303, 384)
dtype: uint8
min, max: 1 252
mean: 96.7...
```

What it means: A grayscale image, 303 rows × 384 cols, with darkest pixel at intensity 1 and brightest at 252. The "average" pixel is around 97 — moderately dim, consistent with the coins-on-dark-background look.

Example 3: Build a tiny color image

```
import numpy as np
import matplotlib.pyplot as plt

img = np.zeros((4, 4, 3), dtype=np.uint8) # BGR in OpenCV order
img[0, :] = (255, 0, 0) # top row -> blue
img[1, :] = (0, 255, 0) # green
img[2, :] = (0, 0, 255) # red
img[3, :] = (255, 255, 255) # white

# Display in RGB order for matplotlib
plt.imshow(img[..., ::-1]) # reverse last axis: BGR -> RGB
plt.axis("off")
plt.show()
```

Expected Output (visual): A 4×4 image: row 0 solid blue, row 1 solid green, row 2 solid red, row 3 white. (Without [..., ::-1] matplotlib would show the colours swapped because it expects RGB and we built BGR.)

4. Parameter Sensitivity

Variable	Lower value effect	Higher value effect
Pixel intensity (uint8)	Darker (toward black)	Brighter (toward white)
Image shape	Fewer total samples	More samples → finer detail
Number of channels	1 (grayscale only)	3 (color), 4 (with alpha)

5. Common Mistakes

Mistake	What Happens	Fix
Thinking pixels are tiny squares	They're samples, not squares	Treat each value as a measurement at a point
Confusing the origin	Drawing flipped vertically	Top-left is (0, 0); y grows down
(R, G, B) assumption	Colors look wrong in OpenCV	OpenCV is BGR — see Module 2

6. Practice Tasks

- By hand: Without running code, predict what a 3×3 matrix `[[255,0,0],[0,255,0],[0,0,255]]` looks like.
- Build: Make a 10×10 NumPy array that's black on the left half and white on the right half. Display it.
- Inspect: Load `skimage.data.camera()` and print its shape, dtype, min, max, and mean.

7. Key Takeaways

- A grayscale image is a 2D array; a color image is 3D (H, W, 3).
- 0 = black, 255 = white in uint8.
- Origin is top-left; rows go down, columns go right.
- OpenCV color order is BGR.

8. What's Next

Pixels and coordinates — how to address any one pixel and how to read its value safely.

Lesson 2: Pixels and Coordinates

Module: 1 — Digital Image Fundamentals

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Access any pixel in a grayscale or color image.
- Read the value at a pixel and modify it.
- Use slicing to grab a rectangular region.
- Tell [row, col] and (x, y) apart and convert between them.

Prerequisites

- Lesson 1: What is a Digital Image?

1. Why This Matters

Almost every image-processing bug at this level is a pixel-coordinate mistake. Get the indexing rules right once and they're settled forever. Get them wrong and you'll fight your own code on every project.

2. Concept

2.1 NumPy indexing: [row, col]

For a grayscale image:

```
img[r, c]
```

- r = row (0 at top)
- c = col (0 at left)

For a color image:

```
img[r, c]          # whole pixel: array of 3 values (B, G, R)
img[r, c, 0]       # blue channel at that pixel
img[r, c, 2]       # red channel
```

2.2 Modifying pixels

Assignment works the same way:

```
img[100, 200] = 255          # grayscale: set to white
img[100, 200] = (0, 0, 255)  # color BGR: set to red
img[100, 200, 2] = 0        # zero out only the red channel
```

2.3 Slicing — a rectangular region (ROI)

```
roi = img[y1:y2, x1:x2]
```

This returns rows y1 through y2-1, cols x1 through x2-1. Stop is exclusive, exactly like Python slicing on lists.

```
img[50:150, 80:240]    # 100 rows tall, 160 cols wide
```

2.4 Slicing is a VIEW, not a copy

This bites everyone once:

```
roi = img[50:150, 80:240]
roi[:] = 0    # also zeros img[50:150, 80:240]
```

To get an independent region:

```
roi = img[50:150, 80:240].copy()
```

2.5 The (x, y) convention (OpenCV drawing functions)

OpenCV's drawing functions (cv2.circle, cv2.rectangle, cv2.line, cv2.putText) expect coordinates as (x, y) tuples — x first, opposite of NumPy.

```
cv2.circle(img, (200, 100), radius=5, color=(0, 255, 0), thickness=-1)
# Draws a green dot at column 200, row 100.
# In NumPy terms: that's img[100, 200].
```

This isn't OpenCV being weird — it's the math-textbook convention of (x, y). Just keep it straight:

- Indexing/slicing arrays → [row, col] = [y, x]
- OpenCV drawing functions → (x, y)

2.6 Bounds checking

Out-of-bounds in NumPy raises IndexError. In OpenCV drawing functions, out-of-bounds coordinates silently clip — drawing simply happens outside the visible canvas. Useful to know when debugging "why is nothing showing up".

3. Code Examples

Example 1: Read and write pixels in a real image

```
import cv2
import numpy as np
from skimage.data import camera

img = camera().copy()    # grayscale (512, 512) uint8

print("pixel at (50, 100):", img[50, 100])    # row 50, col 100

# Set a 20x20 patch in the centre to white
center_y, center_x = img.shape[0] // 2, img.shape[1] // 2
img[center_y-10:center_y+10, center_x-10:center_x+10] = 255

print("after: that patch's mean is", img[center_y-10:center_y+10, center_x-10:center_x+10].mean())
```


Expected Output (console):

```
pixel at (50, 100): 200
after: that patch's mean is 255.0
```

Visual: Cameraman image with a small white square painted at the centre.

Example 2: NumPy vs OpenCV — same point, two notations

```
import cv2
import numpy as np

img = np.zeros((200, 300, 3), dtype=np.uint8)

# Same point, two ways:
#   NumPy: img[100, 200]          -> row 100, col 200
#   OpenCV drawing: (200, 100)   -> x=200, y=100

img[100, 200] = (255, 255, 255)          # one white pixel via NumPy
cv2.circle(img, (200, 100), 5, (0, 0, 255), -1) # red circle via OpenCV

# (200, 100) in OpenCV is the SAME location as [100, 200] in NumPy.
```

Expected Output (visual): A black 200×300 image with one tiny white pixel and a red 5-pixel-radius circle, both centred at the exact same spot.

Example 3: ROI view vs copy

```
import numpy as np
img = np.arange(20).reshape(4, 5).astype(np.uint8)
print("original:\n", img)

view = img[1:3, 1:4]
view[:] = 99
print("\nafter assigning to the VIEW:\n", img)
```

Expected Output:

```
original:
[[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]
 [15 16 17 18 19]]

after assigning to the VIEW:
[[ 0  1  2  3  4]
 [ 5 99 99 99  9]
 [10 99 99 99 14]
 [15 16 17 18 19]]
```

Lesson: The view modified the original. If you needed independence, you'd have called `.copy()`.

Example 4: Drawing a bounding box

```
import cv2
import numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR).copy()

# Draw a green rectangle around an approximate face region:
top_left = (180, 60) # (x, y)
bottom_right = (340, 220) # (x, y)
cv2.rectangle(img, top_left, bottom_right, (0, 255, 0), 3)

cv2.imwrite("astronaut_box.png", img)
```

Expected Output: Saves astronaut_box.png — the astronaut image with a 3-pixel-thick green rectangle drawn around the face area (top-left at column 180, row 60).

4. Parameter Sensitivity

Parameter	Effect of small value	Effect of large value
Slice height (y2-y1)	Thin horizontal strip	Tall block
Slice width (x2-x1)	Narrow vertical strip	Wide block
Channel index (0/1/2)	One channel isolated	Blue/Green/Red specifically (BGR order)

5. Common Mistakes

Mistake	What Happens	Fix
img[x, y] instead of img[y, x]	Wrong pixel	Y first in NumPy
cv2.circle(img, (y, x), ...)	Circle in the wrong place	OpenCV drawing wants (x, y)
Forgot .copy()	Modifying ROI modifies original	Use .copy() if you need independence
Slicing past bounds	Empty array silently	Print img.shape first, clip indices

6. Practice Tasks

- Load skimage.data.coins(). Print the pixel value at row 100, col 100.
- Set a 50×50 black square in the top-left of the same image. Display it.
- Draw a red bounding box (BGR, so red is (0,0,255)) around the top half of astronaut(). Save it.

7. Key Takeaways

- NumPy: img[row, col] = img[y, x] — Y first.
- OpenCV drawing: (x, y) tuples — X first.
- Slicing returns a view; .copy() for an independent array.
- Out-of-bounds: NumPy raises, OpenCV drawing silently clips.

8. What's Next

Resolution and file size — how the grid dimensions translate into bytes, megapixels, and quality.

Lesson 3: Resolution and File Size

Module: 1 — Digital Image Fundamentals

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Distinguish image resolution (pixels) from physical resolution (DPI).
- Predict an image's raw memory footprint from its shape and dtype.
- Distinguish raw size from file size (compression).
- Recognise when down-sampling is the right move.

Prerequisites

- Lesson 2: Pixels and Coordinates

1. Why This Matters

A 24-megapixel photo can be 5 MB on disk and 70 MB in RAM. Knowing why prevents out-of-memory crashes, slow pipelines, and decisions like "should I resize this before processing?" — which you'll be doing constantly.

2. Concept

2.1 Resolution = pixel count

For an image of shape (H, W):

- Total pixels = $H \times W$
- Megapixels (MP) = $H \times W / 1,000,000$

A 1920×1080 image: 2.07 MP. A 4000×3000 phone photo: 12 MP.

2.2 DPI is something else

DPI (dots per inch) describes how big each pixel is when printed or displayed at a particular physical size. It does not change the pixel data, only the print size:

```
6000 × 4000 image at 300 DPI -> printed at 20" × 13.3"
6000 × 4000 image at 600 DPI -> printed at 10" × 6.7"
```

Same pixel data — just a metadata hint. For processing, ignore DPI.

2.3 Raw memory footprint

For a NumPy image:

```
bytes = H × W × channels × (bits-per-channel / 8)
```

Examples (using uint8 = 1 byte per channel):

Image	Shape	Bytes	Display
HD grayscale	(1080, 1920)	2.07 MB	uint8
HD color	(1080, 1920, 3)	6.22 MB	uint8
12 MP phone color	(3000, 4000, 3)	36 MB	uint8
Same as float32	(3000, 4000, 3)	144 MB	float32
4K color	(2160, 3840, 3)	24.9 MB	uint8

The float32 row matters: many library functions return floats. Converting back to uint8 saves 4× memory.

2.4 File size = raw × (1 / compression ratio)

PNG and JPEG don't store raw arrays — they compress.

- PNG is lossless. Compression ratio depends on the image (uniform regions compress well; noisy images don't). Typical: 2-5× smaller than raw.
- JPEG is lossy. Compression ratio is dramatic — 10-30×. Trades quality for size via a quality setting (1-100).
- BMP is essentially raw + a tiny header.
- TIFF can be either.

So a 36 MB raw 12 MP photo might be a 4 MB JPEG, a 12 MB PNG, or a 36 MB BMP.

2.5 When to downscale

Working at full resolution is wasteful when:

- You're previewing or prototyping.
- The detail you care about (faces, license plates) is much bigger than 1 pixel.
- You're going to apply expensive operations (DFT, NLM denoising).

Rule of thumb: many image processing tasks work fine at the smallest resolution that still preserves the features you care about — often a tenth of the original.

```
small = cv2.resize(big, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)
# halves both dimensions, 4x less data
```

3. Code Examples

Example 1: Measure footprint

```
import numpy as np
from skimage.data import astronaut

img = astronaut()
print("shape:", img.shape)           # (512, 512, 3)
print("dtype:", img.dtype)           # uint8
print("nbytes:", img.nbytes)         # 786432 bytes
print("MB:", img.nbytes / (1024 * 1024)) # 0.75 MB
```

```
as_float = img.astype(np.float32)
print("float32 MB:", as_float.nbytes / (1024 * 1024))
```

Expected Output (console):

```
shape: (512, 512, 3)
dtype: uint8
nbytes: 786432
MB: 0.75
float32 MB: 3.0
```

What it means: $512 \times 512 \times 3 = 786,432$ pixels-of-bytes for uint8; quadruples to 3 MB as float32.

Always cast back to uint8 before saving.

Example 2: Quality vs file size for JPEG

```
import cv2
import os
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
for q in [10, 50, 90, 100]:
    fname = f"coffee_q{q}.jpg"
    cv2.imwrite(fname, img, [cv2.IMWRITE_JPEG_QUALITY, q])
    print(f"quality {q:3d} -> {os.path.getsize(fname) / 1024:6.1f} KB")
```

Expected Output (console, sizes will vary slightly):

```
quality 10 -> 8.4 KB
quality 50 -> 22.9 KB
quality 90 -> 60.7 KB
quality 100 -> 178.3 KB
```

Visual: quality 10 looks blocky and washed-out; quality 50 is acceptable for thumbnails; quality 90 is visually indistinguishable from the original at normal viewing distances.

Example 3: Compare PNG vs JPEG vs BMP

```
import cv2
import os
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
for ext in ["bmp", "png", "jpg"]:
    fname = f"coffee.{ext}"
    cv2.imwrite(fname, img)
    print(f"{ext}: {os.path.getsize(fname) / 1024:7.1f} KB")
```

Expected Output (sizes approximate):

```
bmp: 703.2 KB
png: 354.6 KB
jpg: 61.4 KB
```

Example 4: Downscale for previews

```
import cv2
from skimage.data import astronaut

big = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
small = cv2.resize(big, None, fx=0.25, fy=0.25, interpolation=cv2.INTER_AREA)

print("big: ", big.shape, big.nbytes // 1024, "KB")
print("small:", small.shape, small.nbytes // 1024, "KB")
```

Expected Output:

```
big: (512, 512, 3) 768 KB
small: (128, 128, 3) 48 KB
```

Visual: thumbnail-sized 128×128 version — recognisable as the astronaut portrait, fine detail (hair strands, helmet text) lost.

4. Parameter Sensitivity

Parameter	Lower	Higher
JPEG quality	smaller file, blocky artefacts	bigger file, no visible loss
Resolution (H, W)	smaller memory, less detail	bigger memory, finer detail
Bit depth	less precision, more banding	more precision, more memory
dtype	uint8 (1 byte)	float32 (4 bytes), float64 (8 bytes)

5. Common Mistakes

Mistake	What Happens	Fix
Loading 100 MP image into a 4 GB process	MemoryError	Downscale on load: cv2.imread(...); cv2.resize(...)
Saving a float image to PNG	Garbled colors or implicit cast	Convert to uint8 first with .clip(0,255).astype(np.uint8)
Treating JPEG-compressed = lossless	Compression artefacts accumulate	Use PNG for intermediate steps in a pipeline

6. Practice Tasks

- Compute the raw size in MB for a 6000×4000 RGB uint8 image.
- Save `skimage.data.coffee()` as JPEG at quality 10, 50, 95. Compare file sizes and inspect quality.
- Halve the dimensions of `skimage.data.astronaut()` using `cv2.resize` with `INTER_AREA`. Print before/after nbytes.

7. Key Takeaways

- Resolution = pixel count; DPI = print size hint.
- Raw bytes = $H \times W \times \text{channels} \times \text{bytes-per-channel}$.

- File size depends on format (PNG $\approx$ 2-5 $\times$, JPEG $\approx$ 10-30 $\times$ smaller than raw).
- Down-sample when the detail you care about doesn't need full resolution.

8. What's Next

Color depth — how many bits per channel and why it matters for medical and scientific imagery.

Lesson 4: Color Depth and Bit-Depth

Module: 1 — Digital Image Fundamentals

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Define bit-depth and link it to NumPy dtypes.
- Predict the range of values for 8-bit, 16-bit, and float images.
- Choose the right dtype for medical, satellite, or HDR images.
- Convert between dtypes without losing data or banding.

Prerequisites

- Lesson 3: Resolution and File Size

1. Why This Matters

A standard photo is 8 bits per channel — 256 levels. Most CT scans are 12 bits (4096 levels). Satellite imagery often 16 bits. If you load a 16-bit image and treat it as 8-bit, you'll silently throw away 8 bits of data and wonder why your contrast looks awful.

2. Concept

2.1 Bit-depth = bits per channel per pixel

Bit-depth	Range	NumPy dtype
1	0 or 1	bool
8	0–255	uint8
16	0–65535	uint16
32 (float)	typically 0.0–1.0	float32
64 (float)	typically 0.0–1.0	float64

A color image has bit-depth per channel times number of channels: a 24-bit color image = 8 bits × 3 channels.

2.2 256 levels is fine — until it isn't

For everyday photos, 256 gray levels per channel is far more than the eye can distinguish in any one region. Where 256 fails:

- Medical imaging — subtle tissue density differences span thousands of levels.
- Scientific imaging — telescope frames span huge dynamic ranges.
- HDR photography — captures and merges multiple exposures.
- Post-processing — heavy contrast adjustments on 8-bit reveal banding (visible stripes where smooth gradients should be).

2.3 Banding — what 8-bit can't do

If you stretch a low-contrast 8-bit image (say, intensities 100–110 → 0–255), you only have 11 source levels to spread over 256 target levels. The result is striped, not smooth. With a 16-bit source, you'd have 2,816 levels to work with — silky.

2.4 Choosing a dtype

Use	Dtype
Web/phone photos	uint8
Intermediate computation	float32 (then back to uint8)
Medical / scientific	uint16
HDR / merged exposures	float32
Masks (yes/no)	bool or uint8 (0/255)

2.5 Converting between dtypes safely

uint8 → float32 (normalize to 0–1):

```
img_f = img.astype(np.float32) / 255.0
```

float32 (0–1) → uint8 (0–255):

```
img_u = (img_f * 255).clip(0, 255).astype(np.uint8)
```

uint16 → uint8 (lose precision intentionally):

```
img_u = (img_16.astype(np.float32) / 256).astype(np.uint8)
# or use cv2.normalize
img_u = cv2.normalize(img_16, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
```

uint16 → uint8 (linear stretch — preserves contrast):

```
img_u = cv2.normalize(img_16, None, 0, 255, cv2.NORM_MINMAX, dtype=cv2.CV_8U)
```

3. Code Examples

Example 1: Demonstrate banding from over-stretching

```
import numpy as np
import matplotlib.pyplot as plt

# A near-uniform gray ramp with only 16 distinct levels
ramp = np.tile(np.linspace(100, 110, 16).astype(np.uint8), (100, 16))
print("source range:", ramp.min(), ramp.max(), " unique:", len(np.unique(ramp)))

# Stretch 100..110 -> 0..255
stretched = ((ramp - 100) * (255 / 10)).clip(0, 255).astype(np.uint8)

plt.figure(figsize=(8, 3))
plt.subplot(121); plt.imshow(ramp, cmap="gray", vmin=0, vmax=255); plt.title("low contrast"); plt.axis("off")
plt.subplot(122); plt.imshow(stretched, cmap="gray", vmin=0, vmax=255);
```

```
plt.title("stretched (banded)"); plt.axis("off")
plt.show()
```

Expected Output: Two grayscale strips side-by-side. The left looks almost uniformly dark gray. The right looks like 16 distinct vertical bands of gray — proof that 8-bit doesn't have enough precision to support aggressive contrast operations.

Example 2: Visit the unique levels

```
import numpy as np
from skimage.data import coins

img = coins()
print("dtype:", img.dtype)
print("min,max:", img.min(), img.max())
print("unique levels used:", len(np.unique(img)), "out of 256 possible")
```

Expected Output:

```
dtype: uint8
min,max: 1 252
unique levels used: 240 out of 256 possible
```

Example 3: uint8 ↔ float round-trip

```
import numpy as np

img8 = np.array([[0, 127, 255]], dtype=np.uint8)
imgF = img8.astype(np.float32) / 255.0      # 0.0, 0.498, 1.0
img8_back = (imgF * 255).clip(0, 255).astype(np.uint8)

print(img8)
print(imgF)
print(img8_back)
```

Expected Output:

```
[[ 0 127 255]]
[[0.         0.49803922 1.         ]]
[[ 0 127 255]]
```

Example 4: Stretching a 16-bit image down to 8-bit

```
import cv2
import numpy as np

# Synthesize a 16-bit gradient
img16 = np.tile(np.linspace(0, 65535, 512, dtype=np.uint16), (256, 1))
print("uint16 range:", img16.min(), img16.max())

# Naive cast (loses contrast if range doesn't span 0..65535)
naive = (img16 / 256).astype(np.uint8)
print("naive uint8 range:", naive.min(), naive.max())

# Normalize: linearly stretch min..max to 0..255
```

```
stretched = cv2.normalize(img16, None, 0, 255, cv2.NORM_MINMAX, dtype=cv2.CV_8U)
print("stretched uint8 range:", stretched.min(), stretched.max())
```

Expected Output:

```
uint16 range: 0 65535
naive uint8 range: 0 255
stretched uint8 range: 0 255
```

For this synthetic image both give 0–255 because the source already spans full range. For a real low-contrast 16-bit image, `cv2.normalize` would give much better dynamic range than naive division.

4. Parameter Sensitivity

Bit-depth	Memory cost	Quality after heavy edits	Best use
1 (bool)	tiny	n/a	masks
8 (uint8)	1×	banding under heavy stretches	display / web photos
16 (uint16)	2×	smooth under heavy stretches	medical / satellite / scientific
32 (float32)	4×	excellent precision	computation, HDR

5. Common Mistakes

Mistake	What Happens	Fix
Saving float (0..1) as PNG without scaling	All pixels look black (clipped to 0)	Multiply by 255, clip, cast to uint8
Treating uint16 as uint8	Lose 8 bits of precision	Use <code>cv2.normalize</code> or proper scaling
Forgetting to <code>.clip(0, 255)</code> after float math	Wrap-around / overflow on cast	Always clip before <code>.astype(np.uint8)</code>
Long pipelines without float intermediates	Banding from repeated uint8 quantization	Work in float32 internally, cast at the end

6. Practice Tasks

- Create a uint8 image full of value 200. Add 100 to it. Predict and verify the output.
- Convert that image to float32, divide by 255, add 0.4, then convert back to uint8 with clipping. Compare to task 1.
- Print the dtype, min, max, and nbytes of `skimage.data.moon()` (a 16-bit image after the conversion below): `from skimage import img_as_uint; m = img_as_uint(skimage.data.moon())`.

7. Key Takeaways

- Bit-depth = bits per channel per pixel.
- uint8 (256 levels) is enough for display but not for heavy editing.
- uint16 (65,536 levels) for medical / scientific data.

- Float32 for intermediate computation; cast back to uint8 to save.
- Always clip then cast when going from float to uint8.

8. What's Next

Treating an image as a NumPy array — the fluent ops you'll use every day.

Lesson 5: Image as a NumPy Array

Module: 1 — Digital Image Fundamentals

Duration: 35 minutes

Difficulty: Beginner

Learning Objectives

- Apply NumPy operations directly to images.
- Use broadcasting for per-pixel and per-channel ops.
- Apply boolean masks to images.
- Build a small image from scratch with `np.zeros`/`np.full`/`np.tile`/`np.arange`.

Prerequisites

- Lesson 4: Color Depth and Bit-Depth
- Basic NumPy familiarity. Refer to NumPy-for-Images cheatsheet as needed.

1. Why This Matters

NumPy IS the lingua franca of image processing in Python. OpenCV functions take and return NumPy arrays. scikit-image is built on NumPy. matplotlib displays NumPy arrays. Knowing the dozen NumPy patterns below replaces hundreds of lines of for-loops over pixels.

2. Concept

2.1 Vectorized arithmetic

Operate on the whole image at once:

```
img + 50          # brighten every pixel by 50 (watch overflow!)
img * 1.2         # scale; returns float
255 - img        # photographic negative
```

Always remember: + 50 on uint8 wraps around ($200 + 100 = 44$). Use `cv2.add` for saturating arithmetic, or cast to float first.

2.2 Broadcasting

Combine arrays of different shapes if axes line up:

```
img.shape          # (H, W, 3)
scale = np.array([1.0, 0.5, 0.5])  # (3,) - tints toward blue (BGR)
out = (img.astype(np.float32) * scale).clip(0, 255).astype(np.uint8)
```

Broadcasting matched the (3,) along the channel axis — equivalent to multiplying each channel separately, but in one line.

2.3 Boolean masks

```
mask = img > 128          # bool array shape (H, W) or (H, W, 3)
img[mask] = 255           # set bright pixels to max
print(mask.sum())         # how many pixels qualified
```

For per-pixel masks on color images, often build the mask from one channel:

```
gray = img.mean(axis=2)
bright = gray > 200       # (H, W) bool
```

2.4 Reductions (collapsing axes)

```
img.mean()                # one scalar: overall brightness
img.mean(axis=(0, 1))     # per-channel mean: shape (3,)
img.mean(axis=2)          # collapse channels -> grayscale shape (H, W)
img.sum(axis=0)           # column-wise sum: shape (W,)
```

Useful for histograms, projections, and feature stats.

2.5 Building images from scratch

```
black = np.zeros((100, 100, 3), dtype=np.uint8)
white = np.full((100, 100), 255, dtype=np.uint8)
gray_50 = np.full((100, 100), 128, dtype=np.uint8)

# horizontal gradient 0..255
gradient_h = np.tile(np.arange(256, dtype=np.uint8), (100, 1)) # shape (100, 256)
# vertical gradient
gradient_v = np.tile(np.arange(256, dtype=np.uint8)[: , None], (1, 100))

# random noise
noise = (np.random.rand(100, 100) * 255).astype(np.uint8)
```

2.6 Concatenation

```
np.hstack([a, b])         # side-by-side (heights must match)
np.vstack([a, b])         # stacked (widths must match)
```

Very handy for "before / after" displays.

3. Code Examples

Example 1: Photographic negative

```
import cv2
import numpy as np
from skimage.data import camera

img = camera()
neg = 255 - img

vis = np.hstack([img, neg])
cv2.imwrite("camera_vs_negative.png", vis)
print("saved:", vis.shape, vis.dtype)
```

Expected Output (console): saved: (512, 1024) uint8

Visual: the cameraman image on the left, his photographic negative on the right (bright sky becomes dark, dark coat becomes light).

Example 2: Brighten safely (no overflow)

```
import cv2
import numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)

# Wrong: wraps around
wrong = img + 80          # uint8 overflow!

# Right: saturating add
right = cv2.add(img, np.full_like(img, 80))

print("wrong sample pixels (B,G,R):", wrong[0, 0])
print("right sample pixels (B,G,R):", right[0, 0])
```

Expected Output (console, exact values depend on the pixel):

```
wrong sample pixels (B,G,R): [22 33 12]    <- looks darker than original due to
wrap
right sample pixels (B,G,R): [255 255 92]   <- saturates at 255
```

Example 3: Boolean mask — keep only bright pixels

```
import numpy as np
from skimage.data import coins
import matplotlib.pyplot as plt

img = coins()
bright = img > 150

out = np.zeros_like(img)
out[bright] = img[bright]

print("bright pixels:", bright.sum(), "out of", img.size)
plt.figure(figsize=(8, 4))
plt.subplot(121); plt.imshow(img, cmap="gray"); plt.axis("off");
plt.title("original")
plt.subplot(122); plt.imshow(out, cmap="gray"); plt.axis("off"); plt.title("bright
only")
plt.show()
```

Expected Output: Two panels — original coin image, and a version where only pixels brighter than 150 are kept (the coins themselves; background is black).

Example 4: Pure-NumPy grayscale conversion

```
import numpy as np
from skimage.data import astronaut

img = astronaut()          # (512, 512, 3) RGB
```

```
# Weighted gray: 0.299 R + 0.587 G + 0.114 B (ITU-R BT.601)
weights = np.array([0.299, 0.587, 0.114], dtype=np.float32)
gray = (img.astype(np.float32) * weights).sum(axis=2)
gray = gray.clip(0, 255).astype(np.uint8)

print(img.shape, "->", gray.shape, gray.dtype)
```

Expected Output:

```
(512, 512, 3) -> (512, 512) uint8
```

Visual: monochrome version of the astronaut portrait. (cv2 does this with `cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)` in one line — but it's good to see what's actually happening inside.)

Example 5: Build a checkerboard from scratch

```
import numpy as np
import matplotlib.pyplot as plt

H, W, square = 200, 200, 25
rows = (np.arange(H) // square) % 2
cols = (np.arange(W) // square) % 2
board = ((rows[:, None] ^ cols[None, :]) * 255).astype(np.uint8)

print(board.shape, board.dtype, "unique vals:", np.unique(board))
plt.imshow(board, cmap="gray"); plt.axis("off"); plt.show()
```

Expected Output:

```
(200, 200) uint8 unique vals: [ 0 255]
```

Visual: an 8×8 checkerboard, 25-pixel squares of pure black and pure white.

4. Parameter Sensitivity

Op	Small	Large
<code>img + N</code>	small brightness boost	wraps around (uint8) — switch to <code>cv2.add</code>
<code>img * f</code>	dims toward black	brightens but produces float >255
Mask threshold	many pixels selected	few pixels selected

5. Common Mistakes

Mistake	What Happens	Fix
<code>img + 100</code> on uint8	overflow / wrap	<code>cv2.add</code> or cast to float, clip, cast back
for r in range(H): for c in range(W): ...	painfully slow	use vectorized ops or broadcasting
Forgetting axis= on reductions	scalar when you wanted an array	specify the axis: axis=2 collapses channels
Wrong broadcast shape	ValueError	print shapes and align — (3,)

		works against (H, W, 3)
--	--	-------------------------

6. Practice Tasks

- Invert (negative of) `skimage.data.astronaut()` and display.
- Convert that astronaut to grayscale using only NumPy (the weighted-sum trick).
- Build a 256×256 horizontal gradient using `np.tile` and `np.arange`.

7. Key Takeaways

- Operate on whole images, not pixel-by-pixel.
- Broadcasting lets you apply per-channel or per-row ops without loops.
- Boolean masks select pixels by condition.
- Build images from scratch with `zeros/full/tile/arange/random`.
- Stack with `hstack/vstack` for side-by-side displays.

8. What's Next

The last lesson — pinning down coordinate conventions across libraries so you stop swapping x and y.

Lesson 6: Coordinate Systems & Conventions

Module: 1 — Digital Image Fundamentals

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Translate fluently between NumPy [row, col] and Cartesian (x, y).
- Use OpenCV functions that expect (x, y) tuples without confusion.
- Identify which library uses BGR vs RGB.
- Know skimage's (row, col) everywhere convention.

Prerequisites

- All previous Module-1 lessons.

1. Why This Matters

This one lesson exists because the single most common image-processing bug is swapping x and y or BGR and RGB. Once these are settled, you save weeks across your career.

2. Concept

2.1 The cheat sheet

Library / API	Pixel access	Point as argument	Color order
NumPy indexing	img[row, col]	n/a	depends on source
OpenCV drawing/geometry	n/a	(x, y) tuple	BGR
OpenCV cv2.warpAffine	n/a	(W, H) for dsize	BGR
OpenCV cv2.findContours	returns (x, y) arrays	—	—
scikit-image	img[row, col]	(row, col)	RGB
matplotlib imshow	—	uses array as-is	expects RGB
Pillow	img.getpixel((x, y))	(x, y)	RGB

2.2 Why this exists

- NumPy descends from math/scientific arrays — rows first, mirroring $A[i][j]$ in linear algebra.
- OpenCV descends from computer graphics — (x, y) matches the way you'd write coordinates in geometry.
- scikit-image chose to align with NumPy throughout — (row, col) everywhere, including arguments to its own functions.

There's no winning here. There's only knowing which library you're calling.

2.3 Visualising the difference

A 4x5 image:

NumPy view (`img[row, col]`):

	col 0	col 1	col 2	col 3	col 4
row 0	a	b	c	d	e
row 1	f	g	h	i	j
row 2	k	l	m	n	o
row 3	p	q	r	s	t

To address 'm' (row 2, col 2):

NumPy: `img[2, 2]`

OpenCV point: `(2, 2)` -> x=2, y=2 -> same location, same numbers (by luck of equal values)

To address 'i' (row 1, col 3):

NumPy: `img[1, 3]`

OpenCV point: `(3, 1)` -> x=3, y=1 -> swap!

When the row and column happen to be equal, both notations give the same tuple. That makes it harder to notice when you've swapped them. Don't rely on test cases where `row == col`.

2.4 Common conversion mistakes

OpenCV docs say: `cv2.line(img, pt1, pt2, color, thickness)`

pt1 and pt2 are (x, y).

`h, w = img.shape[:2]`

Wrong (swapped):

`cv2.line(img, (h, w), (0, 0), (0, 255, 0), 2)`

Right:

`cv2.line(img, (w, h), (0, 0), (0, 255, 0), 2)` # bottom-right to top-left

2.5 Width / height in function arguments

`cv2.resize(img, dsize)` — `dsize` is (width, height), not (height, width).

Resize to 200 wide, 100 tall:

`resized = cv2.resize(img, (200, 100))` # (W, H)

But the resulting array's shape is:

`resized.shape` # (100, 200, ...) -> (H, W, ...)

2.6 BGR vs RGB cheat sheet

Source	Order
<code>cv2.imread</code>	BGR
<code>cv2.imwrite (input)</code>	BGR
<code>cv2.cvtColor(img, cv2.COLOR_BGR2RGB)</code>	swap to RGB
<code>skimage.io.imread</code>	RGB
<code>skimage.data.*</code>	RGB

PIL.Image.open(...).convert("RGB")	RGB
matplotlib.pyplot.imshow	expects RGB

So the typical "load with OpenCV, display with matplotlib" recipe is:

```
img_bgr = cv2.imread("x.jpg")
plt.imshow(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB))
```

Skip the conversion and your photo will look weirdly blue-shifted (red and blue channels swapped).

3. Code Examples

Example 1: Same point, three notations

```
import cv2
import numpy as np

img = np.zeros((200, 300, 3), dtype=np.uint8) # H=200, W=300

# Target: row 75, col 150 (i.e. y=75, x=150)
img[75, 150] = (255, 255, 255) # NumPy
cv2.circle(img, (150, 75), 6, (0, 255, 0), 1) # OpenCV (x, y)
cv2.putText(img, "*", (150, 75), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255))

cv2.imwrite("same_point.png", img)
print("img shape (H,W,C):", img.shape)
```

Expected Output (console): img shape (H,W,C): (200, 300, 3)

Visual: at column 150, row 75 there's a single white pixel, a thin green circle around it, and a red asterisk nearby — all marking the same location, addressed three different ways.

Example 2: BGR vs RGB display

```
import cv2
import matplotlib.pyplot as plt
from skimage.data import astronaut

# skimage returns RGB
img_rgb = astronaut()

# Convert to BGR (what cv2.imread would give):
img_bgr = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2BGR)

# Display both with matplotlib (which expects RGB):
plt.figure(figsize=(8, 4))
plt.subplot(121); plt.imshow(img_rgb); plt.title("RGB (correct)"); plt.axis("off")
plt.subplot(122); plt.imshow(img_bgr); plt.title("BGR shown as RGB (wrong)");
plt.axis("off")
plt.show()
```

Expected Output (visual): Two panels. Left: the astronaut portrait in natural colors. Right: the same astronaut with the orange / red space-suit elements showing up as blue / cyan — because matplotlib interpreted the B channel as R.

Example 3: cv2.resize argument order

```
import cv2
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
print("original:", img.shape)          # (512, 512, 3) -> (H, W, C)

# Resize to width=300, height=100
small = cv2.resize(img, (300, 100))    # dsize is (W, H)
print("resized: ", small.shape)        # (100, 300, 3) -> (H, W, C)
```

Expected Output:

```
original: (512, 512, 3)
resized:  (100, 300, 3)
```

Example 4: scikit-image is row-col throughout

```
from skimage.draw import circle_perimeter
from skimage.data import camera
import numpy as np

img = camera().copy()
# skimage draws use (row, col)
rr, cc = circle_perimeter(r=100, c=200, radius=30)
img[rr, cc] = 255

print("shape:", img.shape, "modified pixels:", len(rr))
```

Expected Output: shape: (512, 512) modified pixels: ~189 and a white circle of radius 30 centred at row 100, col 200 of the cameraman image. Note that arguments are r= (row) and c= (col) — same convention as NumPy indexing.

4. Parameter Sensitivity

(This lesson is about conventions, not parameters — no sensitivity table.)

5. Common Mistakes

Mistake	What Happens	Fix
cv2.circle(img, (y, x), ...)	Circle in wrong place	OpenCV is (x, y)
cv2.resize(img, (H, W))	Image gets stretched	cv2.resize(img, (W, H))
Skipping cv2.cvtColor(img, COLOR_BGR2RGB) before plt.imshow	Reds look blue	Always convert before displaying with matplotlib
Mixing skimage and cv2 without converting	Channels swap silently	Pick one stack; convert at boundaries
Treating returned	Plot ends up rotated	They're (x, y); use pt[1] for

cv2.findContours points as (row, col)		row, pt[0] for col when indexing
---------------------------------------	--	----------------------------------

6. Practice Tasks

- Load astronaut() with skimage (RGB). Convert to BGR and save with cv2.imwrite. Reload with cv2.imread. Display correctly in matplotlib.
- Draw a cv2.rectangle from (50, 30) to (150, 100). Predict what the rectangle's corners look like in [row, col] terms.
- Resize an image to width=400, height=200 with cv2.resize. Print the resulting shape.

7. Key Takeaways

- NumPy / skimage: (row, col).
- OpenCV drawing & geometry: (x, y).
- cv2.resize(..., (W, H)) — argument order is opposite of .shape.
- OpenCV is BGR; matplotlib/skimage are RGB.
- Convert at library boundaries; don't mix conventions in one expression.

8. What's Next

End of Module 1. Module 2: Image I/O & Display — actually load and save images correctly across formats.

Module 1 — Exercises

18 exercises (3 per lesson).

1.1 What is a Digital Image?

- (E) Hand-decode: Without code, sketch what the 4×4 array `[[0,0,0,0],[0,255,255,0],[0,255,255,0],[0,0,0,0]]` looks like.
- (M) Stats: Load `skimage.data.camera()`. Print shape, dtype, min, max, mean, and total pixel count.
- (H) Build a color image: Make a (50, 50, 3) uint8 BGR image whose top half is red and bottom half is blue. Save as PNG.

1.2 Pixels and Coordinates

- (E) Set `img[50, 100]` of `skimage.data.coins()` to 255 and print it before and after.
- (M) Draw a green rectangle with `cv2.rectangle` from (50, 30) to (150, 100) on a 200×200 black image. Save it.
- (H) Demonstrate ROI-is-a-view: build a 4×5 array, slice the middle 2×3 region, modify the slice, print the original. Then redo with `.copy()` and compare.

1.3 Resolution and File Size

- (E) Compute raw uint8 bytes for shapes (1024,768), (1920,1080,3), (4000,3000,3). Print in MB.
- (M) Save `skimage.data.coffee()` as JPEG at qualities 10/50/95. Compare file sizes.
- (H) Halve dimensions of `astronaut()` with `cv2.resize(..., INTER_AREA)`. Compare nbytes before/after.

1.4 Color Depth

- (E) Predict what `np.array([200], dtype=np.uint8) + np.array([100], dtype=np.uint8)` returns. Verify.
- (M) Convert `coins()` to float32 in [0, 1], multiply by 1.3, clip, convert back to uint8. Compare to original.
- (H) Build a uint16 gradient `np.linspace(0, 65535, 256)` tiled into a 100×256 image. Convert to uint8 two ways: naive (`/256`) and `cv2.normalize`. Compare visually.

1.5 Image as NumPy Array

- (E) Build a 256×256 horizontal gradient using `np.tile + np.arange`. Save as PNG.
- (M) Convert `astronaut()` to grayscale using only NumPy (weighted-sum, no `cv2.cvtColor`).
- (H) Build a 200×200 checkerboard with 25×25 squares. No `cv2`. Save as PNG.

1.6 Coordinate Systems

- (E) Predict where `cv2.circle(img, (40, 70), 5, (0,255,0), -1)` draws — row and col, in NumPy terms.
- (M) Resize `astronaut()` to width=200, height=50. Print resulting shape.

- (H) Load `chelsea()` (skimage RGB), save as PNG with `cv2.imwrite`, reload with `cv2.imread`, display correctly with matplotlib using `cvtColor`.

Module 1 — Quiz

10 MCQs covering all 6 lessons.

Q1

What's the shape of a 1920×1080 BGR color image as a NumPy array? A. (1920, 1080, 3) B. (1080, 1920, 3) C. (1080, 1920) D. (3, 1080, 1920)

Answer: B — shape is (H, W, C) and H is the first axis. (L1, L5)

Q2

How do you access the pixel at row 50, col 200 of a NumPy image? A. `img[200, 50]` B. `img[50, 200]` C. `img[(50, 200)]` D. Both B and C

Answer: D — both work; B is clearer. (L2)

Q3

What argument order does `cv2.circle(img, center, radius, ...)` expect for center? A. (row, col) B. (x, y) C. (col, row) D. (y, x)

Answer: B (= C). (L2, L6)

Q4

What does `np.array([200], dtype=np.uint8) + np.array([100], dtype=np.uint8)` return? A. [300] B. [44] C. [255] D. error

Answer: B — uint8 wraps around. Use `cv2.add` for saturation. (L4, L5)

Q5

A 12 MP color image as uint8 occupies roughly how much RAM as a raw NumPy array? A. 4 MB B. 12 MB C. 36 MB D. 144 MB

Answer: C — $4000 \times 3000 \times 3 = 36,000,000$ bytes $\approx$ 36 MB. (L3)

Q6

Which of these is lossless? A. JPEG B. PNG C. WebP (default) D. JPEG XR

Answer: B (L3)

Q7

You're displaying a `cv2.imread`-loaded image with `plt.imshow`. Colors look wrong. The fix is: A. Save and reload B. `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` before `imshow` C. Normalize D. Use `cmap='gray'`

Answer: B — OpenCV is BGR; matplotlib expects RGB. (L6)

Q8

Slicing `roi = img[10:50, 20:80]` creates: A. A copy of the region B. A view into the original C. A list of pixels D. A bool mask

Answer: B — modifying `roi` modifies `img`. Use `.copy()` for independence. (L2)

Q9

Which dtype gives 65,536 levels per channel? A. `uint8` B. `uint16` C. `float32` D. `int8`

Answer: B (L4)

Q10

`cv2.resize(img, dsize)` — what does `dsize` mean? A. (height, width) B. (width, height) C. (channels, width) D. (rows, cols)

Answer: B — opposite of `.shape` order. (L6)

Module 1 — Assignment: Image Info Inspector

Estimated time: 1.5 hours

Combines concepts from: Module 1 (all lessons)

Brief

Write a program that takes an image path and prints a complete profile: dimensions, dtype, channel count, memory footprint, raw pixel statistics per channel, and then displays each channel separately.

Requirements

The program must:

- Accept a file path (CLI arg or hard-coded).
- Load the image with `cv2.imread` (and check for `None`).
- Print:
 - File path
 - File size on disk (bytes / KB)
 - Image shape (H, W) or (H, W, 3)
 - dtype
 - Megapixels (computed from $H \times W$)
 - Raw NumPy footprint in MB
 - For grayscale: min, max, mean, std
 - For color (BGR): per-channel min, max, mean, std — labelled B, G, R
- Display each channel separately as a grayscale image (matplotlib or save to disk):
 - 3 subplots labelled "Blue", "Green", "Red" for color images
 - Plus the original image (converted to RGB) on a 4th subplot for reference
- Output a clean text report and a single PNG visualisation (report.png).

Constraints

- Use Module 1 concepts only.
- No loops over pixels — use vectorised NumPy.
- Use matplotlib to save / display.
- BGR conventions must be respected in labels.

Expected Output (sample)

```
== Image Info Report ==
Path       : datasets/starter/astronaut.png
File size  : 410.5 KB
Shape      : (512, 512, 3)
dtype      : uint8
Megapixels : 0.26 MP
Raw memory : 0.75 MB
```

Per-channel statistics (BGR):

```
B min= 0 max=255 mean= 73.4 std= 60.1
G min= 0 max=255 mean= 78.2 std= 53.7
R min= 0 max=255 mean=125.4 std= 71.0
```

Saved report visualisation -> report.png

report.png is a 2x2 grid: original (top-left), blue / green / red channel as grayscale (top-right, bottom-left, bottom-right).

Hints

- File size: `os.path.getsize(path)` or `Path(path).stat().st_size`.
- Per-channel stats: `img[..., c].mean()`, `img[..., c].std()`, etc.
- Display order matters: `matplotlib.imshow(channel, cmap='gray', vmin=0, vmax=255)` for fair comparison.

Grading Rubric

Criterion	Weight
Loads + None check	15%
Correct stats + units	30%
Visualisation correct	25%
BGR labels correct	15%
Style + naming	15%

Submission

Save as `assignment_module1.py`.

Module 1 — Mini Project: Image Info Inspector CLI

Overview

A polished command-line tool that wraps the assignment into something you'd actually use as a developer. Takes one or more images, prints a report, and optionally exports a JSON summary.

Concepts Used

- File I/O with `cv2.imread` (M1 L5)
- NumPy stats (M1 L5)
- File-size & dtype reasoning (M1 L3, L4)
- Coordinate conventions (M1 L6)

Features

- `imginfo PATH [PATH ...]` — print a one-block report for each image.
- `--json FILE` — append every report as a JSON object.
- `--quiet` — only print one line per image (path, shape, size).
- `--per-channel` (default for color images) — show channel stats.

Starter Code

`imginfo.py`:

```
import argparse
import json
import os
import sys
from pathlib import Path

import cv2
import numpy as np

def inspect(path: str) -> dict:
    if not os.path.exists(path):
        raise FileNotFoundError(path)

    img = cv2.imread(path, cv2.IMREAD_UNCHANGED)
    if img is None:
        raise ValueError(f"OpenCV couldn't decode: {path}")

    file_bytes = os.path.getsize(path)
    h = img.shape[0]
    w = img.shape[1]
    channels = 1 if img.ndim == 2 else img.shape[2]
    nbytes = img.nbytes
    info = {
        "path": str(Path(path).resolve()),
        "file_bytes": file_bytes,
        "shape": list(img.shape),
```

```

        "dtype": str(img.dtype),
        "channels": channels,
        "megapixels": round(h * w / 1_000_000, 3),
        "raw_mb": round(nbytes / (1024 * 1024), 3),
    }

    if channels == 1:
        info["stats"] = {
            "min": int(img.min()),
            "max": int(img.max()),
            "mean": round(float(img.mean()), 2),
            "std": round(float(img.std()), 2),
        }
    else:
        labels = ["B", "G", "R", "A"][:channels]    # OpenCV order
        per_channel = {}
        for i, label in enumerate(labels):
            ch = img[..., i]
            per_channel[label] = {
                "min": int(ch.min()),
                "max": int(ch.max()),
                "mean": round(float(ch.mean()), 2),
                "std": round(float(ch.std()), 2),
            }
        info["stats"] = per_channel

    return info


def fmt_report(info: dict) -> str:
    lines = [
        "== Image Info Report ==",
        f"Path          : {info['path']}",
        f"File size      : {info['file_bytes'] / 1024:.1f} KB",
        f"Shape          : {tuple(info['shape'])}",
        f"dtype          : {info['dtype']}",
        f"Channels        : {info['channels']}",
        f"Megapixels     : {info['megapixels']} MP",
        f"Raw memory      : {info['raw_mb']} MB",
    ]
    stats = info["stats"]
    if "mean" in stats:
        s = stats
        lines.append(f"Stats          : min={s['min']:3d} max={s['max']:3d} "
                    f"mean={s['mean']:6.2f} std={s['std']:6.2f}")
    else:
        lines.append("Per-channel statistics:")
        for label, s in stats.items():
            lines.append(
                f" {label} min={s['min']:3d} max={s['max']:3d} "
                f"mean={s['mean']:6.2f} std={s['std']:6.2f}"
            )
    return "\n".join(lines)

```

```

def main(argv=None):
    p = argparse.ArgumentParser(prog="imginfo", description="Inspect image files.")
    p.add_argument("paths", nargs="+", help="image file paths")
    p.add_argument("--json", metavar="FILE", help="append all results as JSON")
    p.add_argument("--quiet", action="store_true", help="one line per file")
    args = p.parse_args(argv)

    all_info = []
    for path in args.paths:
        try:
            info = inspect(path)
        except (FileNotFoundError, ValueError) as e:
            print(f"[error] {path}: {e}", file=sys.stderr)
            continue
        all_info.append(info)
        if args.quiet:
            print(f"{path:40s} {tuple(info['shape'])} {info['file_bytes']/1024:.1f} KB")
        else:
            print(fmt_report(info))
            print()

    if args.json:
        with open(args.json, "w", encoding="utf-8") as f:
            json.dump(all_info, f, indent=2)
        print(f"saved -> {args.json}")

if __name__ == "__main__":
    main()

```

Expected Interaction

```

$ python imginfo.py datasets/starter/astronaut.png
== Image Info Report ==
Path       : D:\Image Processing e-course\datasets\starter\astronaut.png
File size  : 410.5 KB
Shape      : (512, 512, 3)
dtype      : uint8
Channels   : 3
Megapixels : 0.262 MP
Raw memory : 0.75 MB
Per-channel statistics:
  B min= 0 max=255 mean= 73.40 std= 60.10
  G min= 0 max=255 mean= 78.20 std= 53.70
  R min= 0 max=255 mean=125.40 std= 71.00

$ python imginfo.py datasets/starter/*.png --quiet --json all.json
datasets/starter/astronaut.png      (512, 512, 3) 410.5 KB
datasets/starter/brick.png          (512, 512)   223.4 KB
datasets/starter/camera.png         (512, 512)   258.9 KB

```

```
... etc
saved -> all.json
```

Extension Challenges

- Add an EXIF reader (use Pillow's ExifTags) for JPEGs.
- Add a --save-channels DIR flag that writes B/G/R as separate grayscale PNGs.
- Add a histogram summary (with text bars showing distribution per channel).

Grading Rubric

Criterion	Weight
All commands work	30%
Correct stats	25%
JSON output valid	15%
Friendly error msgs	15%
Style + functions	15%

Python E-Course

Module 2 — Image I/O & Display

Detailed Notes

Module Overview

Level: Foundations

Duration: ~1 week

Prerequisites: Module 1

What You'll Learn

- Read images with `cv2.imread` (and handle `None` returns).
- Write images with `cv2.imwrite` and tune format-specific parameters.
- Display images correctly with OpenCV's HighGUI vs `matplotlib`.
- Pick the right format (JPEG vs PNG vs BMP vs TIFF) for the job.
- Work with Pillow (PIL) and interop with NumPy / OpenCV.

Lessons

#	Lesson	Duration
1	Reading Images	30 min
2	Writing Images	25 min
3	Displaying with OpenCV	30 min
4	Displaying with Matplotlib	25 min
5	Image Formats Deep Dive	30 min
6	Working with Pillow	25 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 3: Pixel Manipulation

Lesson 1: Reading Images

Module: 2 — Image I/O & Display

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Use `cv2.imread` and handle `None` returns safely.
- Choose the right `IMREAD_*` flag for grayscale, color, alpha, or 16-bit images.
- Read images from filesystem paths and from byte buffers.

Prerequisites

- Module 1

1. Why This Matters

Half of beginner image-processing bugs trace back to `cv2.imread` returning `None` silently (typo in path, unsupported format) or to picking the wrong flag (you wanted alpha, got 3 channels). Five minutes here saves hours later.

2. Concept

2.1 Signature

```
img = cv2.imread(filename, flags=cv2.IMREAD_COLOR)
```

Returns a NumPy array on success, `None` on failure (bad path, unreadable format). It does NOT raise.

2.2 The flags

Flag	Behaviour	Output shape
<code>cv2.IMREAD_COLOR</code> (default)	Always 3-channel BGR; drops alpha	(H, W, 3) uint8
<code>cv2.IMREAD_GRAYSCALE</code>	Convert to grayscale on load	(H, W) uint8
<code>cv2.IMREAD_UNCHANGED</code>	As-is (preserves alpha, 16-bit)	varies
<code>cv2.IMREAD_ANYDEPTH</code>	Keep bit depth if >8	(H, W) uint8 or uint16
<code>cv2.IMREAD_REduced_COLOR_2</code>	Color, decoded at half resolution	smaller
<code>cv2.IMREAD_REduced_GRAYSCALE_4</code>	Grayscale at quarter resolution	smaller

When in doubt: `IMREAD_UNCHANGED` preserves what's in the file (alpha, 16-bit depth).

2.3 Always check for None

```
img = cv2.imread("photo.jpg")
if img is None:
```

```
        raise FileNotFoundError("Couldn't read 'photo.jpg' (bad path or unsupported
format)")
```

Skip this and the next line will crash with a much less helpful `AttributeError: 'NoneType' object has no attribute 'shape'`.

2.4 What imread decodes

OpenCV ships with codecs for JPEG, PNG, BMP, TIFF, WebP, PPM/PGM, and a few more. It does not read GIFs (use Pillow), proprietary RAW formats (use rawpy), or HEIC/HEIF on most platforms (use Pillow + plugin).

2.5 Reading from bytes (e.g. HTTP response)

```
import numpy as np
import cv2

with open("photo.jpg", "rb") as f:
    raw = f.read()
arr = np.frombuffer(raw, dtype=np.uint8)
img = cv2.imdecode(arr, cv2.IMREAD_COLOR)
```

Useful when you fetched the image from requests or a database BLOB.

2.6 Unicode paths on Windows

`cv2.imread` doesn't support non-ASCII paths on Windows. Workaround:

```
import numpy as np
import cv2
img = cv2.imdecode(np.fromfile("привет.jpg", dtype=np.uint8), cv2.IMREAD_COLOR)
```

`np.fromfile` handles the Unicode filename; `cv2.imdecode` does the decoding.

3. Code Examples

Example 1: Basic load with None check

```
import cv2

path = "datasets/starter/astronaut.png"
img = cv2.imread(path)
if img is None:
    raise FileNotFoundError(path)

print("shape:", img.shape, "dtype:", img.dtype)
```

Expected Output:

```
shape: (512, 512, 3) dtype: uint8
```

Example 2: Compare flags

```
import cv2

path = "datasets/starter/coffee.png"    # color png
```

```

for flag, name in [
    (cv2.IMREAD_COLOR, "COLOR"),
    (cv2.IMREAD_GRAYSCALE, "GRAYSCALE"),
    (cv2.IMREAD_UNCHANGED, "UNCHANGED"),
]:
    img = cv2.imread(path, flag)
    print(f"{name:<10} shape={img.shape} dtype={img.dtype}")

```

Expected Output (shapes for a 3-channel png):

```

COLOR      shape=(400, 600, 3) dtype=uint8
GRAYSCALE  shape=(400, 600)   dtype=uint8
UNCHANGED  shape=(400, 600, 3) dtype=uint8

```

For a PNG with alpha, IMREAD_COLOR would still give (H, W, 3) (alpha discarded), while IMREAD_UNCHANGED gives (H, W, 4).

Example 3: Decode from bytes

```

import cv2
import numpy as np

with open("datasets/starter/coins.png", "rb") as f:
    raw = f.read()

arr = np.frombuffer(raw, dtype=np.uint8)
img = cv2.imdecode(arr, cv2.IMREAD_GRAYSCALE)
print("shape:", img.shape, "dtype:", img.dtype)

```

Expected Output:

```

shape: (303, 384) dtype: uint8

```

Example 4: Robust loader function

```

import cv2
import numpy as np
from pathlib import Path

def load(path, mode="color"):
    flag = {"color": cv2.IMREAD_COLOR,
            "gray": cv2.IMREAD_GRAYSCALE,
            "asis": cv2.IMREAD_UNCHANGED}[mode]
    # Unicode-safe on Windows
    img = cv2.imdecode(np.fromfile(str(path), dtype=np.uint8), flag)
    if img is None:
        raise FileNotFoundError(f"can't load: {path}")
    return img

print(load("datasets/starter/camera.png", "gray").shape)

```

Expected Output: (512, 512)

4. Parameter Sensitivity

Flag	When to use
IMREAD_COLOR	Standard photos, you don't care about alpha
IMREAD_GRAYSCALE	Edge detection, OCR, simple thresholding
IMREAD_UNCHANGED	Need alpha, or processing 16-bit medical / satellite data
IMREAD_ANYDEPTH	16-bit grayscale specifically

5. Common Mistakes

Mistake	What Happens	Fix
No if img is None check	Crash on img.shape with confusing error	Always check
Path typo	Silent None return	None check + raise
Reading PNG-with-alpha as COLOR	Alpha silently dropped	Use IMREAD_UNCHANGED
Reading 16-bit as default	Quantized to uint8	Use IMREAD_UNCHANGED
Non-ASCII Windows path	None return	cv2.imdecode + np.fromfile

6. Practice Tasks

- Write a `safe_imread(path)` that raises `FileNotFoundError` if OpenCV returns `None`.
- Read `astronaut.png` three ways (color, grayscale, unchanged). Print each shape and dtype.
- Load an image from raw bytes by first reading the file with `open(..., 'rb')`, then `cv2.imdecode`.

7. Key Takeaways

- `cv2.imread` returns `None` silently on failure — always check.
- Pick the flag deliberately: `COLOR`, `GRAYSCALE`, or `UNCHANGED`.
- `cv2.imdecode(np.fromfile(...))` handles Unicode paths and byte buffers.

8. What's Next

Writing images — with format-specific quality/compression settings.

Lesson 2: Writing Images

Module: 2 — Image I/O & Display

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Use `cv2.imwrite` and read its return value.
- Tune JPEG quality and PNG compression.
- Encode an image to a byte buffer with `cv2.imencode`.

Prerequisites

- Module 2 Lesson 1

1. Why This Matters

You'll save images constantly — intermediate stages, debug outputs, final results. Wrong format = blocky JPEGs of pristine masks, or 10× larger files than needed.

2. Concept

2.1 Signature

```
ok = cv2.imwrite(filename, img, params=None)
```

Returns True / False. Format is inferred from the file extension (.jpg, .png, .bmp, .tif, .webp, ...). Image must be BGR or grayscale uint8 unless you use a format that supports more.

2.2 Format-specific params

Passed as a flat list [FLAG_ID, value, FLAG_ID, value, ...]:

Format	Param flag	Range	Effect
JPEG	<code>cv2.IMWRITE_JPEG_QUALITY</code>	0–100	Higher = better quality, bigger file
PNG	<code>cv2.IMWRITE_PNG_COMPRESSION</code>	0–9	Higher = smaller file, slower write
WebP	<code>cv2.IMWRITE_WEBP_QUALITY</code>	0–100	Same as JPEG
TIFF	<code>cv2.IMWRITE_TIFF_COMPRESSION</code>	enum	LZW, none, etc.
PNG	<code>cv2.IMWRITE_PNG_STRATEGY</code>	enum	Compression strategy

```
cv2.imwrite("photo.jpg", img, [cv2.IMWRITE_JPEG_QUALITY, 90])  
cv2.imwrite("photo.png", img, [cv2.IMWRITE_PNG_COMPRESSION, 9])
```

2.3 Common pitfalls

- Floating-point images: convert to uint8 first (clip + `astype`).
- 16-bit images: PNG and TIFF support 16-bit; JPEG doesn't (it'll truncate).
- Always check the return value if writing to a network share or USB.

2.4 Encode to bytes

For uploading or storing in a database:

```
ok, buf = cv2.imencode(".jpg", img, [cv2.IMWRITE_JPEG_QUALITY, 85])
if ok:
    data = buf.tobytes()
```

buf is a NumPy array of uint8; .tobytes() gives raw bytes you can shove anywhere.

2.5 Atomic writes

Half-written files happen if the process dies mid-save. Write to a temp file then rename:

```
from pathlib import Path
import os
tmp = Path("out.jpg.tmp")
cv2.imwrite(str(tmp), img)
os.replace(tmp, "out.jpg")    # atomic on POSIX & Windows
```

3. Code Examples

Example 1: Save with different qualities

```
import cv2
import os
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)

for q in [10, 50, 90]:
    fname = f"coffee_q{q}.jpg"
    ok = cv2.imwrite(fname, img, [cv2.IMWRITE_JPEG_QUALITY, q])
    print(f"q={q:2d}  ok={ok}  size={os.path.getsize(fname)/1024:6.1f} KB")
```

Expected Output:

```
q=10  ok=True  size=   8.4 KB
q=50  ok=True  size=  22.9 KB
q=90  ok=True  size=  60.7 KB
```

Visual: q=10 has visible blocky / blotchy artefacts; q=90 looks identical to original at normal viewing distance.

Example 2: PNG compression levels

```
import cv2
import os
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)

for c in [0, 3, 6, 9]:
    fname = f"astro_c{c}.png"
    cv2.imwrite(fname, img, [cv2.IMWRITE_PNG_COMPRESSION, c])
    print(f"compression={c}  size={os.path.getsize(fname)/1024:7.1f} KB")
```


Expected Output (sizes roughly):

```
compression=0  size= 790.1 KB
compression=3  size= 470.2 KB
compression=6  size= 410.8 KB
compression=9  size= 392.3 KB
```

Example 3: Float image → save correctly

```
import cv2
import numpy as np

# A float image in [0, 1]
img_f = np.linspace(0, 1, 256, dtype=np.float32)
img_f = np.tile(img_f, (100, 1))  # (100, 256)

# Wrong: save as-is – PNG interprets float weirdly or fails
cv2.imwrite("wrong.png", img_f)
print("wrong size:", __import__("os").path.getsize("wrong.png"))

# Right: scale and cast
img_u = (img_f * 255).clip(0, 255).astype(np.uint8)
cv2.imwrite("right.png", img_u)
print("right size:", __import__("os").path.getsize("right.png"))
```

Expected Output: "wrong.png" file will exist but typically all-black or all-white when reloaded.
"right.png" is a proper horizontal gradient.

Example 4: Encode to bytes (e.g., for upload)

```
import cv2
from skimage.data import chelsea

img = cv2.cvtColor(chelsea(), cv2.COLOR_RGB2BGR)
ok, buf = cv2.imencode(".jpg", img, [cv2.IMWRITE_JPEG_QUALITY, 80])
print("ok:", ok, " bytes:", len(buf.tobytes()))

# Round-trip:
import numpy as np
back = cv2.imdecode(np.frombuffer(buf.tobytes(), np.uint8), cv2.IMREAD_COLOR)
print("decoded shape:", back.shape)
```

Expected Output:

```
ok: True  bytes: 40132
decoded shape: (300, 451, 3)
```

4. Parameter Sensitivity

Param	Low	High
JPEG quality (0..100)	Smaller file, blocky	Big file, near-lossless
PNG compression (0..9)	Big file, fastest	Small file, slow write

5. Common Mistakes

Mistake	What Happens	Fix
Saving float (0..1) to PNG	All black	Multiply by 255, clip, cast to uint8
Ignoring return value	Silent failures on bad paths	Check ok
JPEG for masks / line art	Blurry / blocky	Use PNG (lossless) for non-photographic content
Wrong extension	Format mismatch	Match extension to actual format

6. Practice Tasks

- Save coffee() at JPEG quality 5 and quality 95. Display both and compare.
- Save the same image as PNG at compression 0 and 9. Compare sizes.
- Encode an image to JPEG bytes with cv2.imencode. Round-trip via cv2.imdecode.

7. Key Takeaways

- cv2.imwrite returns True/False — check it.
- Format params via [FLAG_ID, value] pairs.
- Float → uint8 conversion before saving.
- cv2.imencode for buffers (network, DB).

8. What's Next

Displaying images with OpenCV's HighGUI (imshow / waitKey).

Lesson 3: Displaying with OpenCV

Module: 2 — Image I/O & Display

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Display an image with `cv2.imshow` and the required event loop.
- Use `cv2.waitKey` correctly.
- Handle window resizing and multiple windows.
- Recognize when HighGUI isn't available (headless, Jupyter).

Prerequisites

- Module 2 Lesson 2

1. Why This Matters

`cv2.imshow` is the fastest way to peek at an image during development — but it requires an event loop, doesn't work in Jupyter or headless servers, and silently fails in surprising ways. Knowing the rules saves frustration.

2. Concept

2.1 The minimal pattern

```
import cv2
img = cv2.imread("photo.jpg")
cv2.imshow("preview", img)
cv2.waitKey(0)                # pause until any key
cv2.destroyAllWindows()
```

Three rules:

- `imshow` creates / updates a named window. Same name = reused window.
- `waitKey(ms)` is required for the window to actually render. Without it, nothing shows.
- Always `destroyAllWindows()` to clean up.

2.2 `cv2.waitKey(ms)`

- 0 = wait forever until any key is pressed.
- positive int = wait that many milliseconds; returns -1 if no key was pressed.
- Returns the key code (an int) if a key was pressed.

```
key = cv2.waitKey(0) & 0xFF      # mask to get the actual key
if key == ord('q'):
    print("quit pressed")
```

The `& 0xFF` masks higher bits set by some keyboards / OSes.

2.3 Video / animation loop

```
cap = cv2.VideoCapture(0)
while True:
    ok, frame = cap.read()
    if not ok:
        break
    cv2.imshow("camera", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()
```

`waitKey(1)` waits ~1ms — long enough to update the window, short enough to keep the framerate high.

2.4 Multiple windows

```
cv2.imshow("original", img)
cv2.imshow("blurred", cv2.blur(img, (5, 5)))
cv2.waitKey(0)
cv2.destroyAllWindows()
```

2.5 Resizable windows

```
cv2.namedWindow("win", cv2.WINDOW_NORMAL) # allows resizing
cv2.resizeWindow("win", 800, 600)
cv2.imshow("win", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Default windows fit the image; `WINDOW_NORMAL` lets the user drag-resize.

2.6 When imshow doesn't work

- Headless server / WSL without X: HighGUI crashes or hangs. Use `matplotlib` (next lesson) or `opencv-python-headless`.
- Jupyter Notebook: `imshow` opens a window outside the notebook — usually fails. Use `plt.imshow` inside notebooks.
- macOS: Requires the cocoa backend; usually fine with `opencv-python` from pip.

3. Code Examples

Example 1: Show one image

```
import cv2
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
cv2.imshow("astronaut", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Expected Output (visual): A window titled "astronaut" pops up showing the portrait. Window closes when you press any key.

Example 2: Compare two side-by-side via stacking

```
import cv2
import numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
blur = cv2.blur(img, (15, 15))

combined = np.hstack([img, blur])
cv2.imshow("orig | blur", combined)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Expected Output: Single window showing original on the left, heavily blurred version on the right.

Example 3: Press-q-to-quit loop

```
import cv2
from skimage.data import camera

img = camera()
while True:
    cv2.imshow("press q", img)
    key = cv2.waitKey(50) & 0xFF
    if key == ord('q'):
        break
cv2.destroyAllWindows()
print("done")
```

Expected Output: Window stays open; closes only when you press 'q'.

Example 4: Resizable window for big images

```
import cv2
img = cv2.imread("datasets/starter/astronaut.png")

cv2.namedWindow("astronaut", cv2.WINDOW_NORMAL)
cv2.resizeWindow("astronaut", 1000, 1000)
cv2.imshow("astronaut", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Expected Output: A 1000×1000 window — the image is upscaled to fit; you can drag-resize the window further.

4. Parameter Sensitivity

waitKey(ms)	Use case
0	One-shot preview
1	Live video loop
30+	Slow-motion playback
Skipped	Window never renders — common bug

5. Common Mistakes

Mistake	What Happens	Fix
No waitKey	Window appears for an instant, then disappears	Always call waitKey(0) for stills
cv2.imshow in Jupyter	Window can't display, kernel hangs	Use plt.imshow (next lesson)
cv2.imshow on headless server	Crashes or freezes	Use opencv-python-headless + matplotlib / save to file
Comparing cv2.waitKey() directly to ord('q')	Unicode codes can have extra bits	Use & 0xFF mask

6. Practice Tasks

- Show coffee() with cv2.imshow. Wait for 'q' before closing.
- Build a 2-up side-by-side display with np.hstack and show with cv2.imshow.
- Use cv2.namedWindow(..., WINDOW_NORMAL) to make a resizable window.

7. Key Takeaways

- imshow + waitKey + destroyAllWindows is the trio.
- waitKey(1) for video loops; waitKey(0) for stills.
- cv2.imshow doesn't work in notebooks / headless — use matplotlib then.
- WINDOW_NORMAL for resizable windows.

8. What's Next

Displaying with matplotlib — the workhorse for notebooks and headless setups.

Lesson 4: Displaying with Matplotlib

Module: 2 — Image I/O & Display

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Display images correctly with `plt.imshow`.
- Convert BGR→RGB before display.
- Use appropriate colormap and value range for grayscale.
- Build multi-panel comparisons with subplots.

Prerequisites

- Module 2 Lesson 3

1. Why This Matters

In Jupyter notebooks and on headless servers, matplotlib is the display tool. It expects RGB and either float [0,1] or uint8 [0,255] — mix them wrong and you'll see garbage.

2. Concept

2.1 The basics

```
import matplotlib.pyplot as plt
plt.imshow(img)
plt.axis("off")
plt.show()
```

2.2 BGR → RGB

If `img` came from `cv2.imread`, it's BGR. Convert before display:

```
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
```

Without this, reds appear blue and vice versa.

2.3 Grayscale

For (H, W) arrays, matplotlib applies the default viridis colormap (greenish-purple). For traditional grayscale appearance:

```
plt.imshow(gray, cmap="gray", vmin=0, vmax=255)
```

`vmin/vmax` lock the display range so two images can be compared fairly.

2.4 dtype rules

Input dtype	Range matplotlib expects
uint8	0..255
float32/64	0.0..1.0

int	clipped to 0..1 or 0..255 — confusing
-----	---------------------------------------

If you have a float image not in [0, 1], divide first.

2.5 Multi-panel comparison

```
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
axes[0].imshow(orig); axes[0].set_title("original"); axes[0].axis("off")
axes[1].imshow(blur); axes[1].set_title("blur"); axes[1].axis("off")
axes[2].imshow(edges, cmap="gray"); axes[2].set_title("edges"); axes[2].axis("off")
plt.tight_layout()
plt.show()
```

2.6 Saving the figure

```
plt.savefig("report.png", dpi=150, bbox_inches="tight")
```

For the underlying image itself, prefer `cv2.imwrite` — `plt.savefig` saves the whole figure (axes, titles).

3. Code Examples

Example 1: BGR vs RGB demo

```
import cv2
import matplotlib.pyplot as plt
from skimage.data import astronaut

img_rgb = astronaut()
img_bgr = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2BGR)

fig, ax = plt.subplots(1, 2, figsize=(8, 4))
ax[0].imshow(img_bgr); ax[0].set_title("BGR shown as RGB (WRONG)");
ax[0].axis("off")
ax[1].imshow(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB))
ax[1].set_title("correctly converted"); ax[1].axis("off")
plt.show()
```

Expected Output (visual): Left panel has reds/oranges showing as blue/teal (the astronaut helmet looks wrong). Right panel has natural colors.

Example 2: Grayscale display with cmap

```
import matplotlib.pyplot as plt
from skimage.data import coins

img = coins()
fig, ax = plt.subplots(1, 2, figsize=(8, 4))
ax[0].imshow(img); ax[0].set_title("default cmap (viridis)"); ax[0].axis("off")
ax[1].imshow(img, cmap="gray", vmin=0, vmax=255); ax[1].set_title("gray cmap");
ax[1].axis("off")
plt.show()
```

Expected Output: Left shows the coin image tinted greenish-purple (viridis default). Right shows it in natural grayscale.

Example 3: Multi-panel grid

```
import cv2
import matplotlib.pyplot as plt
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(img, (15, 15), 0)

fig, ax = plt.subplots(1, 3, figsize=(12, 4))
ax[0].imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)); ax[0].set_title("original")
ax[1].imshow(gray, cmap="gray"); ax[1].set_title("grayscale")
ax[2].imshow(cv2.cvtColor(blur, cv2.COLOR_BGR2RGB)); ax[2].set_title("blurred")
for a in ax: a.axis("off")
plt.tight_layout(); plt.show()
```

Expected Output (visual): Three-panel figure — color original, gray version, blurred version.

Example 4: Reusable helper

```
import cv2
import matplotlib.pyplot as plt

def show(img, title="", cmap=None):
    if img.ndim == 3:
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        plt.imshow(img)
    else:
        plt.imshow(img, cmap=cmap or "gray", vmin=0, vmax=255)
    plt.title(title)
    plt.axis("off")
    plt.show()

img = cv2.imread("datasets/starter/coins.png", cv2.IMREAD_GRAYSCALE)
show(img, "coins")
```

4. Parameter Sensitivity

Param	Effect
cmap="gray"	Traditional grayscale (vs default viridis)
vmin/vmax	Lock display range; needed for fair comparisons
figsize=(w, h)	Panel size in inches at default DPI
dpi=N (in savefig)	Output resolution

5. Common Mistakes

Mistake	What Happens	Fix
Skipping BGR→RGB	Reds look blue	cv2.cvtColor(img, COLOR_BGR2RGB)
Default cmap on grayscale	Greenish tint	cmap="gray"
Float image not in [0,1]	All white / all clipped	Divide by max or use uint8
Two panels with auto-scaled	Brightness looks different	Set vmin=0, vmax=255 on

vmin/vmax	between panels	both
-----------	----------------	------

6. Practice Tasks

- Display `coins()` two ways: with default `cmap` and with `cmap="gray"`, `vmin=0`, `vmax=255`. Compare.
- Load `astronaut.png` via `cv2.imread`, display with `matplotlib` without BGR conversion. Note how it looks wrong.
- Build a 2×2 subplot showing: original color, grayscale, blurred, and edges.

7. Key Takeaways

- `plt.imshow(...)` for in-notebook / headless display.
- BGR → RGB conversion before display.
- `cmap="gray"` + `vmin=0`, `vmax=255` for grayscale.
- Multi-panel comparisons via `plt.subplots`.

8. What's Next

Image format deep dive — when to use which container.

Lesson 5: Image Formats Deep Dive

Module: 2 — Image I/O & Display

Duration: 30 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Pick between JPEG, PNG, BMP, TIFF, WebP for the task.
- Recognize compression artefacts (blocky JPEGs).
- Understand which formats support alpha and 16-bit.

Prerequisites

- Module 2 Lesson 4

1. Why This Matters

JPEG-ing a binary mask is a great way to ruin it. Saving HDR data as a JPEG loses 8 bits. Knowing the format trade-offs saves rework and disk space.

2. Concept

2.1 Comparison table

Format	Compression	Lossy?	Alpha	Bit-depth	Best for
JPEG	DCT + quant	Yes	No	8	Photos
PNG	DEFLATE	No	Yes	8 or 16	Masks, line art, anything with sharp edges
BMP	None	No	Yes (rare)	8 / 24	Quick uncompressed dumps
TIFF	Many (LZW, ZIP, none)	Both	Yes	8 / 16 / 32	Scientific / publishing
WebP	Wavelet	Both (mode)	Yes	8	Modern web (smaller than JPEG/PNG)
GIF	LZW	Lossless*	No	8 (palette)	Animation; not for photos
HEIC/HEIF	HEVC	Yes	Yes	8/10	Apple devices; needs plugin

* GIF is technically lossless but limited to a 256-color palette — looks lossy on photos.

2.2 JPEG artefacts

JPEG splits the image into 8×8 blocks and quantises high-frequency content. At low quality, this causes:

- Blocking — visible 8×8 squares
- Ringing — wavy halos around sharp edges
- Color bleed — chroma subsampling

JPEG is terrible for screenshots with text and for binary masks — both rely on sharp edges.

2.3 PNG strengths

- Lossless — bit-exact round-trip
- Alpha channel
- 16-bit grayscale and 48-bit RGB
- Good compression on uniform regions (UI screenshots, masks)
- Bad on photos (still bigger than JPEG)

2.4 BMP

Almost no compression. Good for "quick dump that another tool will read instantly" or when you need raw pixel data with a header. Large file sizes.

2.5 TIFF

The Swiss Army knife. Optional compression, multiple pages, alpha, high bit-depth. Widely supported in scientific tools. OpenCV supports basic TIFF; for full features (multi-page, geo-TIFF), use specialized libraries.

2.6 WebP

Google's successor to JPEG+PNG. Both lossy and lossless modes. About 25-35% smaller than equivalent-quality JPEG. Now supported on all major browsers.

2.7 Rules of thumb

- Photo destined for web: JPEG q=80-90 or WebP.
- Mask / binary image: PNG.
- Intermediate pipeline output: PNG (lossless, no quality loss between stages).
- Scientific 16-bit data: PNG-16 or TIFF.
- Final deliverable for print: TIFF.

3. Code Examples

Example 1: Same image, three formats

```
import cv2, os
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
for ext in ["jpg", "png", "bmp"]:
    p = f"coffee.{ext}"
    cv2.imwrite(p, img)
    print(f"{ext:4s}: {os.path.getsize(p)/1024:7.1f} KB")
```

Expected Output:

```
jpg :    61.4 KB
png :   354.6 KB
bmp :   703.2 KB
```

Example 2: Demonstrate JPEG artefacts on a mask

```
import cv2, numpy as np

mask = np.zeros((200, 400), dtype=np.uint8)
cv2.putText(mask, "HELLO", (50, 130), cv2.FONT_HERSHEY_SIMPLEX, 3, 255, 8)

cv2.imwrite("mask.png", mask)
cv2.imwrite("mask.jpg", mask, [cv2.IMWRITE_JPEG_QUALITY, 30])

png_back = cv2.imread("mask.png", cv2.IMREAD_GRAYSCALE)
jpg_back = cv2.imread("mask.jpg", cv2.IMREAD_GRAYSCALE)

print("png unique values:", np.unique(png_back))    # [0, 255]
print("jpg unique values:", np.unique(jpg_back))    # many values - JPEG smeared
the edges
```

Expected Output:

```
png unique values: [0 255]
jpg unique values: [ 0  1  2 ... 253 254 255]
```

A binary mask saved as PNG round-trips exactly. As JPEG it becomes a smeary 0..255 gradient — useless for masking.

Example 3: PNG with alpha

```
import cv2, numpy as np

# Build RGBA image: red square with semi-transparent fade
img = np.zeros((200, 200, 4), dtype=np.uint8)
img[:, :, 2] = 255          # red channel (BGR order: index 2 = red)
img[:, :, 3] = np.tile(np.linspace(0, 255, 200, dtype=np.uint8), (200, 1))

cv2.imwrite("red_fade.png", img)

back = cv2.imread("red_fade.png", cv2.IMREAD_UNCHANGED)
print("loaded shape:", back.shape, "channels:", back.shape[2])
print("alpha min/max:", back[..., 3].min(), back[..., 3].max())
```

Expected Output:

```
loaded shape: (200, 200, 4) channels: 4
alpha min/max: 0 255
```

Example 4: 16-bit PNG round-trip

```
import cv2, numpy as np

img16 = (np.linspace(0, 65535, 256, dtype=np.uint16)
         .reshape(1, -1)
         .repeat(100, axis=0))
```

```
print("save dtype:", img16.dtype, "range:", img16.min(), img16.max())

cv2.imwrite("ramp16.png", img16)
back = cv2.imread("ramp16.png", cv2.IMREAD_UNCHANGED)
print("read dtype:", back.dtype, "range:", back.min(), back.max())
```

Expected Output:

```
save dtype: uint16 range: 0 65535
read dtype: uint16 range: 0 65535
```

4. Parameter Sensitivity

(Format is the parameter — see comparison table above.)

5. Common Mistakes

Mistake	What Happens	Fix
JPEG-ing a mask	Edges smear into gradients	Use PNG
Re-saving JPEG repeatedly	Quality degrades with each save	Store pipeline intermediates as PNG
Treating GIF as photo format	Banded color	Use JPEG / PNG / WebP
Saving 16-bit as JPEG	Truncated to 8-bit silently	Use PNG-16 or TIFF

6. Practice Tasks

- Save astronaut() as JPEG (q=85), PNG, BMP. Compare sizes.
- Save a binary mask as both JPEG and PNG. Reload and np.unique each.
- Save a 16-bit gradient as PNG and round-trip; verify range preserved.

7. Key Takeaways

- JPEG for photos; never for masks or screenshots.
- PNG is lossless and supports alpha + 16-bit.
- BMP for unconditional uncompressed dumps.
- TIFF for scientific / multi-page / high bit-depth.

8. What's Next

Pillow (PIL) — when and how to interop with the other major Python image library.

Lesson 6: Working with Pillow

Module: 2 — Image I/O & Display

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Open, convert, and save images with PIL.Image.
- Interop between Pillow and NumPy / OpenCV.
- Read formats OpenCV doesn't (GIF, HEIC with plugin).
- Use Pillow's text rendering (better fonts than cv2.putText).

Prerequisites

- Module 2 Lesson 5

1. Why This Matters

OpenCV is fast but limited (no GIF, awkward text). Pillow is slower but supports more formats, has nicer text rendering, and dominates the Python web/imaging ecosystem. Knowing both lets you pick the right tool.

2. Concept

2.1 Open / save

```
from PIL import Image
img = Image.open("photo.jpg")
print(img.size, img.mode)          # (W, H), 'RGB' / 'L' / 'RGBA' / ...
img.save("out.png")
```

Pillow returns its own Image object — not a NumPy array.

2.2 Mode

Mode	Meaning
L	8-bit grayscale
RGB	8-bit color
RGBA	8-bit color + alpha
1	1-bit binary
I;16	16-bit grayscale
CMYK	Print
P	Palette / GIF

Convert with `img.convert("L")`, etc.

2.3 Pillow ↔ NumPy

```
import numpy as np
from PIL import Image
```

```
# PIL -> NumPy
arr = np.array(Image.open("photo.jpg"))    # RGB by default
print(arr.shape, arr.dtype)                # (H, W, 3) uint8

# NumPy -> PIL
img = Image.fromarray(arr)                  # RGB
img.save("out.png")
```

2.4 Pillow ↔ OpenCV

Pillow is RGB; OpenCV is BGR. Convert at the boundary:

```
# PIL -> OpenCV
import cv2, numpy as np
from PIL import Image
pil = Image.open("p.jpg").convert("RGB")
cv2_img = cv2.cvtColor(np.array(pil), cv2.COLOR_RGB2BGR)

# OpenCV -> PIL
cv2_img = cv2.imread("p.jpg")
pil = Image.fromarray(cv2.cvtColor(cv2_img, cv2.COLOR_BGR2RGB))
```

2.5 Text with Pillow (better fonts)

cv2.putText only does block Hershey fonts. Pillow uses TrueType:

```
from PIL import Image, ImageDraw, ImageFont

img = Image.new("RGB", (400, 200), (30, 30, 30))
draw = ImageDraw.Draw(img)
font = ImageFont.truetype("arial.ttf", 48)    # path varies by OS
draw.text((20, 60), "Hello, World!", font=font, fill=(255, 255, 255))
img.save("text.png")
```

2.6 Format coverage

Pillow reads:

- Everything OpenCV reads
- Plus GIF (including animated), ICO, PSD (limited), EPS, more legacy formats
- HEIC/HEIF if you install pillow-heif
- AVIF if you install pillow-avif-plugin

For animated GIF:

```
from PIL import Image
gif = Image.open("animated.gif")
for frame_index in range(gif.n_frames):
    gif.seek(frame_index)
    gif.copy().save(f"frame_{frame_index:03d}.png")
```


3. Code Examples

Example 1: PIL → NumPy round-trip

```
from PIL import Image
import numpy as np

pil = Image.open("datasets/starter/astronaut.png").convert("RGB")
arr = np.array(pil)
print("array:", arr.shape, arr.dtype)

# Modify (set top 20 rows to white)
arr[:20] = 255
pil2 = Image.fromarray(arr)
pil2.save("astro_top_white.png")
```

Expected Output (console):

```
array: (512, 512, 3) uint8
```

Visual: astronaut image with a white horizontal strip at the top.

Example 2: PIL → OpenCV pipeline

```
import cv2, numpy as np
from PIL import Image

pil = Image.open("datasets/starter/coffee.png").convert("RGB")
cv_img = cv2.cvtColor(np.array(pil), cv2.COLOR_RGB2BGR)

blurred = cv2.GaussianBlur(cv_img, (15, 15), 0)
cv2.imwrite("coffee_blur.png", blurred)
```

Expected Output: coffee_blur.png — heavily-blurred coffee photo.

Example 3: TrueType text overlay

```
from PIL import Image, ImageDraw, ImageFont

img = Image.open("datasets/starter/astronaut.png").convert("RGB")
draw = ImageDraw.Draw(img)
try:
    font = ImageFont.truetype("arial.ttf", 36)
except OSError:
    font = ImageFont.load_default()
draw.rectangle((10, 460, 250, 510), fill=(0, 0, 0, 128))
draw.text((20, 470), "ASTRONAUT", font=font, fill=(255, 255, 255))
img.save("astro_caption.png")
```

Expected Output: Astronaut image with a small dark rectangle and "ASTRONAUT" label in the bottom-left.

Example 4: Save animated GIF

```
from PIL import Image
```

```
frames = [Image.new("RGB", (100, 100), (i * 25, 0, 255 - i * 25))
           for i in range(10)]
frames[0].save("anim.gif", save_all=True, append_images=frames[1:],
              duration=100, loop=0)
```

Expected Output: A 100×100 looping GIF where the color shifts from blue to red over 10 frames.

4. Parameter Sensitivity

Choice	Effect
convert("L")	grayscale
convert("RGBA")	adds alpha
convert("1")	binary, often lossy
Pillow vs OpenCV for resize	Pillow LANCZOS is slightly higher quality

5. Common Mistakes

Mistake	What Happens	Fix
Forgetting BGR↔RGB at the boundary	Colors swap	Convert at the interface
Treating Pillow Image like a NumPy array	TypeError	Wrap with np.array(img) first
Pillow .size is (W, H)	Mixing with NumPy .shape (H, W)	Remember Pillow uses (W, H)
Trying to read HEIC without plugin	UnidentifiedImageError	pip install pillow-heif

6. Practice Tasks

- Open coffee.png with Pillow, convert to grayscale, save as PNG.
- Round-trip a NumPy array (e.g., random) through Image.fromarray and back to np.array. Confirm equality.
- Overlay TrueType text "DEMO" on astronaut.png using Pillow's ImageDraw.

7. Key Takeaways

- Pillow uses RGB (vs OpenCV's BGR) and (W, H) sizes.
- np.array(pil_img) and Image.fromarray(arr) round-trip cleanly.
- Pillow handles formats OpenCV doesn't (GIF, HEIC plugin, etc.).
- Pillow has better text rendering via ImageFont.truetype.

8. What's Next

End of Module 2. Next: Module 3 — Pixel Manipulation.

Module 2 — Exercises

18 exercises (3 per lesson).

2.1 Reading Images

- (E) Write a `safe_load(path)` that raises `FileNotFoundError` on `None`.
- (M) Read `coffee.png` 3 ways: `COLOR`, `GRAYSCALE`, `UNCHANGED`. Print each shape.
- (H) Decode an image from bytes — use `open('rb') + cv2.imdecode`.

2.2 Writing Images

- (E) Save `coffee.png` as JPEG at `q=10, 50, 95`. Compare file sizes.
- (M) Save the same image as PNG with compression 0 and 9. Compare sizes.
- (H) Encode an image to JPEG bytes with `cv2.imencode`. Round-trip via `cv2.imdecode`.

2.3 Display OpenCV

- (E) Show `coins.png` with `cv2.imshow`. Quit on `'q'`.
- (M) Side-by-side display: stack original and Gaussian-blurred with `np.hstack`.
- (H) Make a resizable window with `WINDOW_NORMAL` and size it to `1000×1000`.

2.4 Display Matplotlib

- (E) Display `coins.png` with `cmap="gray"`, `vmin=0`, `vmax=255`.
- (M) Load `astronaut.png` with `cv2.imread`; display with matplotlib with BGR conversion.
- (H) Build a `2×2` subplot grid: original, gray, blurred, edges.

2.5 Formats

- (E) Save `astronaut.png` as JPG, PNG, BMP. Compare sizes.
- (M) Save a binary mask as JPEG and PNG; reload and `np.unique` each.
- (H) Save a 16-bit gradient as PNG; reload and verify range.

2.6 Pillow

- (E) Open `coffee.png` with Pillow, convert to grayscale, save as PNG.
- (M) Round-trip a random NumPy array through Pillow.
- (H) Add TrueType caption text to `astronaut.png` using `ImageFont.truetype`.

Module 2 — Quiz

10 MCQs.

Q1

`cv2.imread("nope.jpg")` on a missing file: A. Raises `FileNotFoundError` B. Returns `None` C. Returns empty array D. Crashes

Answer: B (L1)

Q2

Which flag preserves alpha channel? A. `IMREAD_COLOR` B. `IMREAD_GRAYSCALE` C. `IMREAD_UNCHANGED` D. `IMREAD_ANYDEPTH`

Answer: C (L1)

Q3

`cv2.imwrite("x.jpg", img, [cv2.IMWRITE_JPEG_QUALITY, 90])` does: A. Saves at quality 90 B. Compresses 90% C. Resizes to 90% D. Errors

Answer: A (L2)

Q4

What's required for `cv2.imshow` to render? A. nothing B. `cv2.waitKey` C. `plt.show` D. `cv2.show`

Answer: B (L3)

Q5

For grayscale display in matplotlib, the right call is: A. `plt.imshow(g)` B. `plt.imshow(g, cmap="viridis")` C. `plt.imshow(g, cmap="gray", vmin=0, vmax=255)` D. `plt.gray(g)`

Answer: C (L4)

Q6

For a binary mask, the best format is: A. JPEG B. PNG C. GIF D. WebP-lossy

Answer: B (L5)

Q7

`PIL.Image.size` returns: A. (H, W) B. (W, H) C. (H, W, C) D. (W, H, C)

Answer: B (L6)

Q8

Going from Pillow (RGB) to OpenCV requires: A. nothing B. `cv2.cvtColor(..., COLOR_RGB2BGR)` C. transpose D. flip

Answer: B (L6)

Q9

cv2.waitKey(1) in a video loop: A. waits forever B. waits 1 sec C. waits 1 ms D. skips

Answer: C (L3)

Q10

Saving a float [0,1] image with cv2.imwrite without scaling produces: A. correct PNG B. all-black or all-white image C. error D. uint16 result

Answer: B (L2)

Module 2 — Assignment: Batch Image Format Converter

Estimated time: 1.5-2 hours

Combines: Module 2 + Module 1

Brief

Build a CLI tool that converts every image in a folder to a target format with specified quality / compression. Print a summary table.

Requirements

- `convert.py INPUT_DIR OUTPUT_DIR --format jpg|png|webp [--quality 1-100]`
- Recursively (or just top-level — your choice) finds image files.
- Reads each safely (check for `None`).
- Saves into `OUTPUT_DIR` with the same base name + new extension.
- Applies format-specific parameters:
- JPEG/WebP: `--quality` (default 90).
- PNG: `--compression` (default 6).
- Prints a summary:
- per-file: name, input size, output size, ratio
- total: number converted, total in/out size, average ratio

Constraints

- Use `cv2.imread / cv2.imwrite`.
- Use `argparse`.
- Skip non-image files quietly.
- Create `OUTPUT_DIR` if missing.

Expected Output (sample)

```
$ python convert.py datasets/starter/ out/ --format jpg --quality 80
```

```
astronaut.png      410.5 KB ->  68.2 KB  (16.6%)
coffee.png        487.1 KB ->  56.8 KB  (11.7%)
coins.png          58.0 KB ->  20.3 KB  (35.0%)
...
```

```
Converted 10 files
Total input : 2843.1 KB
Total output: 401.7 KB
Average ratio: 14.1%
```

Grading Rubric

Criterion	Weight
Reads + None check	15%
Correct param mapping	25%

Output dir handling	15%
Summary table	20%
Style + argparse	25%

Submission

Save as assignment_module2.py.

Module 2 — Mini Project: Image Gallery Viewer

Overview

A keyboard-driven gallery: load all images from a folder, show them one at a time, navigate with arrow keys. Shows off cv2.imshow event loops and the difference between OpenCV display and matplotlib.

Concepts Used

- cv2.imread (M2 L1)
- cv2.imshow + waitKey event loop (M2 L3)
- BGR / RGB handling (M2 L4)
- pathlib (M1 / general Python)

Features

- Reads all images in a given folder (jpg/png/bmp/webp).
- Displays one at a time with file info overlaid (cv2.putText).
- Keys:
- n / → → next image
- p / ← → previous image
- s → save the current image to ./gallery_out/
- q / Esc → quit

Starter Code

gallery.py:

```
import sys
import cv2
from pathlib import Path

EXTS = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}

def find_images(folder):
    return sorted(p for p in Path(folder).iterdir() if p.suffix.lower() in EXTS)

def overlay_info(img, path, idx, total):
    h, w = img.shape[:2]
    info = f"[{idx + 1}/{total}] {path.name} ({w}x{h})"
    out = img.copy()
    cv2.rectangle(out, (0, 0), (w, 30), (0, 0, 0), -1)
    cv2.putText(out, info, (10, 22), cv2.FONT_HERSHEY_SIMPLEX, 0.6,
                (255, 255, 255), 1, cv2.LINE_AA)
    return out

def main(folder):
    paths = find_images(folder)
    if not paths:
        print("no images found in", folder); return
```



```

Path("gallery_out").mkdir(exist_ok=True)

idx = 0
while True:
    img = cv2.imread(str(paths[idx]))
    if img is None:
        print("skip (unreadable):", paths[idx]); idx = (idx + 1) % len(paths);
    continue

    cv2.imshow("gallery", overlay_info(img, paths[idx], idx, len(paths)))
    key = cv2.waitKey(0) & 0xFF

    if key in (ord('q'), 27):                # q or Esc
        break
    elif key in (ord('n'), 83):              # n or right arrow (83 on many
systems)
        idx = (idx + 1) % len(paths)
    elif key in (ord('p'), 81):              # p or left arrow (81)
        idx = (idx - 1) % len(paths)
    elif key == ord('s'):
        out = Path("gallery_out") / paths[idx].name
        cv2.imwrite(str(out), img)
        print("saved ->", out)

cv2.destroyAllWindows()

if __name__ == "__main__":
    folder = sys.argv[1] if len(sys.argv) > 1 else "datasets/starter"
    main(folder)

```

Expected Interaction

```

$ python gallery.py datasets/starter
(window opens showing first image with overlay "[1/10] astronaut.png (512x512)")
press n -> [2/10] brick.png (512x512)
press s -> saved -> gallery_out/brick.png
press q -> window closes

```

Extension Challenges

- Add a thumbnail strip at the bottom of the window.
- Add r to rotate 90° clockwise; r again rotates more.
- Save thumbnails into gallery_out/thumbs/.

Grading Rubric

Criterion	Weight
Loads & cycles images	30%
Save key works	15%
Info overlay correct	15%
Handles unreadable files	15%
Style + structure	25%

Python E-Course

Module 3 — Pixel Manipulation

Detailed Notes

Module Overview

Level: Foundations

Duration: ~1 week

Prerequisites: Modules 1-2

What You'll Learn

- Read and write individual pixels with the right coordinate order.
- Draw shapes and text on images.
- Extract and modify ROIs (and copy when you must).
- Split and merge channels.
- Apply per-pixel arithmetic with saturation.
- Build masks with bitwise operators.

Lessons

#	Lesson	Duration
1	Accessing Pixels	25 min
2	Drawing Shapes & Text	30 min
3	ROI Extraction & Insertion	25 min
4	Channel Operations	25 min
5	Image Arithmetic	30 min
6	Bitwise Operations & Masking	35 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 4: Color Spaces

Lesson 1: Accessing Pixels

Module: 3 — Pixel Manipulation

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Read and write individual pixels via NumPy indexing.
- Compare `img.item` / `img.itemset` against direct indexing.
- Apply changes to whole rows / columns.

Prerequisites

- Module 1, Module 2

1. Why This Matters

Working pixel-by-pixel is rarely the fastest, but it's the clearest mental model. Once you understand how a single pixel is addressed and modified, every higher-level op (drawing, masking, blending) becomes "do this for many pixels at once."

2. Concept

2.1 Reading

```
img[y, x]          # grayscale: scalar
img[y, x]          # color (BGR): array of 3
img[y, x, 0]       # color: blue channel
```

2.2 Writing

```
img[y, x] = 255          # grayscale to white
img[y, x] = (0, 0, 255)  # color to red (BGR)
img[y, x, 0] = 0         # zero out blue at one pixel
```

2.3 Faster single-pixel access: `item` / `itemset`

```
img.item(y, x)          # ~3x faster than img[y, x] for scalar reads
img.itemset((y, x), 255) # faster write
```

You won't notice the difference for occasional pixels. For tight loops, prefer vectorising entirely (M1 L5).

2.4 Whole rows / columns

```
img[100, :] = 0          # zero out row 100
img[:, 50] = 0           # zero out column 50
img[:, :, 2] = 0         # zero red channel everywhere (BGR)
```

2.5 Per-pixel loops (don't, but know they work)

```
h, w = img.shape[:2]
for y in range(h):
```

```

for x in range(w):
    img[y, x] = 255 - img[y, x]    # negative

```

Slow. The vectorised version is one line: `img = 255 - img`. Save loops for cases where every pixel needs different logic — even then, often `np.where` is cleaner.

3. Code Examples

Example 1: Read & write one pixel

```

import cv2, numpy as np
from skimage.data import coins

img = coins().copy()
print("before:", img[100, 200])
img[100, 200] = 255
print("after :", img[100, 200])

```

Expected Output (console):

```

before: 137
after : 255

```

Example 2: Row / column tricks

```

import cv2, numpy as np
from skimage.data import camera

img = camera().copy()                # (512, 512) grayscale
img[:, 0:5] = 255                    # paint a thin white stripe on the left
img[0:5, :] = 0                      # black stripe on top
cv2.imwrite("camera_stripes.png", img)

```

Expected Output: Cameraman image with a 5-pixel white stripe along the left edge and a 5-pixel black stripe along the top.

Example 3: Whole channel

```

import cv2
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR).copy()
img[:, :, 2] = 0                      # zero red channel (BGR index 2)
cv2.imwrite("astro_no_red.png", img)

```

Expected Output: Astronaut with all red removed — image looks cyan-tinted because only blue + green channels remain.

4. Parameter Sensitivity

(No params here — see Module 1 Lesson 5 for whole-image NumPy ops.)

5. Common Mistakes

Mistake	What Happens	Fix
---------	--------------	-----

<code>img[x, y]</code>	Wrong pixel	NumPy is <code>img[y, x]</code>
Looping pixels for trivial ops	Slow	Vectorise: <code>img = 255 - img</code> , etc.
<code>img[y, x] = 256</code> on <code>uint8</code>	Wraps to 0 silently	Clip values to 0..255 first

6. Practice Tasks

- Set `img[50, 50]` to white on a grayscale copy of `coins()`. Verify.
- Zero out row 100 and column 100 on `camera()`. Save and inspect.
- Remove the green channel from `astronaut()`. Save and view.

7. Key Takeaways

- `img[y, x]` reads/writes one pixel. Y first.
- Whole rows/cols/channels via slicing.
- Loops are slow — vectorise when you can.

8. What's Next

Drawing shapes and text on images.

Lesson 2: Drawing Shapes & Text

Module: 3 — Pixel Manipulation

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Draw lines, rectangles, circles, polygons.
- Render text with `cv2.putText`.
- Use `thickness=-1` to fill shapes.
- Use anti-aliasing for smooth edges.

Prerequisites

- Module 3 Lesson 1

1. Why This Matters

You'll annotate detections (bounding boxes), display HUD overlays, build masks programmatically. OpenCV's drawing functions are fast and ubiquitous — and they all use the (x, y) convention.

2. Concept

2.1 The shared signature

Almost every OpenCV draw call follows:

```
cv2.shape(img, geometry, color, thickness=1, lineType=cv2.LINE_8)
```

- `img` is modified in place. Pass a copy if you want to keep the original.
- `color` is (B, G, R) for color images, or a scalar for grayscale.
- `thickness=-1` (or `cv2.FILLED`) fills the shape.
- `lineType=cv2.LINE_AA` enables anti-aliasing.

2.2 Lines

```
cv2.line(img, (x1, y1), (x2, y2), (0, 255, 0), thickness=2)
```

2.3 Rectangles

```
cv2.rectangle(img, (x1, y1), (x2, y2), (0, 0, 255), 2)
cv2.rectangle(img, (x1, y1), (x2, y2), (0, 0, 255), -1) # filled
```

2.4 Circles

```
cv2.circle(img, (cx, cy), radius=20, color=(255, 0, 0), thickness=2)
```

2.5 Polygons

```
import numpy as np
pts = np.array([[100, 50], [200, 100], [180, 200], [80, 180]], dtype=np.int32)
```

```
cv2.polylines(img, [pts], isClosed=True, color=(0, 255, 0), thickness=2)
cv2.fillPoly(img, [pts], color=(0, 255, 0))
```

pts must be an array of shape (N, 2) int32 wrapped in a list.

2.6 Text

```
cv2.putText(img, "Hello", (x, y), cv2.FONT_HERSHEY_SIMPLEX,
            fontScale=1.0, color=(255, 255, 255),
            thickness=2, lineType=cv2.LINE_AA)
```

(x, y) is the bottom-left corner of the text baseline — counter-intuitive at first. For nicer fonts, use Pillow (M2 L6).

2.7 Anti-aliasing

lineType=cv2.LINE_AA smooths edges. Slower; use for final output, not video frames.

3. Code Examples

Example 1: Annotate an image with a bounding box and label

```
import cv2
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR).copy()

top_left, bottom_right = (180, 60), (340, 220)
cv2.rectangle(img, top_left, bottom_right, (0, 255, 0), 3)
cv2.putText(img, "face", (top_left[0], top_left[1] - 10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2, cv2.LINE_AA)

cv2.imwrite("astro_annotated.png", img)
```

Expected Output: Astronaut image with a 3-pixel green box around the face region and a "face" label above the top-left corner. Anti-aliased clean edges.

Example 2: Polygon + fill

```
import cv2, numpy as np

canvas = np.zeros((300, 400, 3), dtype=np.uint8)
pts = np.array([[100, 50], [300, 100], [350, 200], [200, 250], [80, 200]],
               dtype=np.int32)
cv2.fillPoly(canvas, [pts], (200, 100, 50))          # filled
cv2.polylines(canvas, [pts], True, (0, 255, 255), 2)  # yellow outline
cv2.imwrite("poly.png", canvas)
```

Expected Output: A pentagon shape filled with brownish-orange, with a bright yellow outline.

Example 3: Multi-shape composition

```
import cv2, numpy as np

canvas = np.full((300, 400, 3), 30, dtype=np.uint8) # dark gray background
cv2.line(canvas, (10, 10), (390, 10), (255, 255, 255), 1)
```



```

cv2.circle(canvas, (200, 150), 60, (0, 255, 0), -1)
cv2.rectangle(canvas, (60, 80), (160, 220), (0, 0, 255), 2)
cv2.putText(canvas, "shapes demo", (50, 280),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 1, cv2.LINE_AA)
cv2.imwrite("shapes.png", canvas)

```

Expected Output: Dark gray canvas with a thin white top border, a filled green circle in the middle, a red unfilled rectangle on the left, and white text "shapes demo" near the bottom.

4. Parameter Sensitivity

Parameter	Low	High
thickness	thin line	fat stroke; -1 = filled
fontScale	small text	huge text
lineType	LINE_8 (fast, jaggy)	LINE_AA (smooth, slower)

5. Common Mistakes

Mistake	What Happens	Fix
Color tuple as (R, G, B)	Colors swapped	Use BGR: (0, 0, 255) is red
Text disappears off-screen	(x, y) was baseline; text drawn upward off the image	Move y down (text baseline is bottom-left)
cv2.polylines(img, pts, ...) instead of [pts]	TypeError	Wrap points in a list
Drawing on original by accident	Original is modified	img.copy() first

6. Practice Tasks

- Draw a yellow circle of radius 50 at the centre of a 300×300 black canvas.
- Add a filled red rectangle around a hard-coded "face" region on astronaut().
- Render "DEMO" in 1.5× scale anti-aliased text at the bottom-right of any image.

7. Key Takeaways

- All drawing functions modify img in place; copy first if needed.
- Coordinates: (x, y) not [row, col].
- thickness=-1 fills.
- lineType=cv2.LINE_AA for smooth edges.

8. What's Next

ROIs — slice patches in and out of images.

Lesson 3: ROI Extraction & Insertion

Module: 3 — Pixel Manipulation

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Extract a rectangular region (ROI) from an image.
- Paste a patch back into a region with matching shape.
- Know when slicing returns a view vs a copy.
- Use ROIs to focus expensive operations.

Prerequisites

- Module 3 Lesson 2

1. Why This Matters

Most useful operations apply to a region, not the whole image — process only the face, blur only the background, replace one logo with another. ROIs make these one-line.

2. Concept

2.1 Extract

```
roi = img[y1:y2, x1:x2]
```

`y1:y2` selects rows, `x1:x2` selects cols. Both ends behave like Python slicing (start inclusive, stop exclusive).

2.2 Insert

```
img[y1:y2, x1:x2] = patch
```

`patch` must have the same shape as the slice. If shapes differ, you get a `ValueError`.

2.3 View vs copy

`roi = img[y1:y2, x1:x2]` is a view — modifying it modifies `img`. Use `.copy()` if you want an independent copy.

```
roi = img[y1:y2, x1:x2].copy()
```

2.4 ROIs across color

For color: `img[y1:y2, x1:x2]` includes all channels. To touch just one channel inside the ROI:

```
img[y1:y2, x1:x2, 0] = 0    # zero blue inside the ROI
```

2.5 Compute then paste

```
roi = img[y1:y2, x1:x2]
roi_blurred = cv2.GaussianBlur(roi, (15, 15), 0)
img[y1:y2, x1:x2] = roi_blurred
```

Blurs only the region. Faster than blurring the full image.

2.6 ROI bounds clipping

OpenCV doesn't crash if you slice past the edge — NumPy gives you the in-bounds part:

```
img[-50:, -50:] # bottom-right 50x50 (whatever exists)
```

But if you compute coordinates from elsewhere (a detection), clip them yourself:

```
y1 = max(0, y1); y2 = min(h, y2)
x1 = max(0, x1); x2 = min(w, x2)
```

3. Code Examples

Example 1: Copy a region to another location

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR).copy()

# Take the face area and paste a duplicate in the top-left corner
face = img[60:220, 180:340].copy() # (160, 160, 3)
img[10:170, 10:170] = face # paste in top-left
cv2.imwrite("astro_dup_face.png", img)
print("face shape:", face.shape)
```

Expected Output: Astronaut image with a 160×160 copy of the face floating in the top-left corner. Console prints face shape: (160, 160, 3).

Example 2: Blur only a region

```
import cv2
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR).copy()
y1, y2, x1, x2 = 60, 220, 180, 340

roi = img[y1:y2, x1:x2]
img[y1:y2, x1:x2] = cv2.GaussianBlur(roi, (35, 35), 0)
cv2.imwrite("astro_face_blurred.png", img)
```

Expected Output: Astronaut with the face region heavily blurred (privacy / censor effect), rest of the image sharp.

Example 3: View vs copy demo

```
import numpy as np
img = np.arange(25).reshape(5, 5).astype(np.uint8)
```

```

view = img[1:4, 1:4]
view[:] = 99
print("after view modify:\n", img)

img2 = np.arange(25).reshape(5, 5).astype(np.uint8)
roi = img2[1:4, 1:4].copy()
roi[:] = 0
print("\nafter copy modify (original unchanged):\n", img2)

```

Expected Output:

```

after view modify:
[[ 0  1  2  3  4]
 [ 5 99 99 99  9]
 [10 99 99 99 14]
 [15 99 99 99 19]
 [20 21 22 23 24]]

after copy modify (original unchanged):
[[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]
 [15 16 17 18 19]
 [20 21 22 23 24]]

```

4. Parameter Sensitivity

Slice size	Effect
Tiny (e.g. 5×5)	Local patch ops; cheap
Whole image	No real "ROI" — same as full-image op
Out-of-bounds	Numpy silently clips; manual checks recommended for computed coords

5. Common Mistakes

Mistake	What Happens	Fix
Forgetting .copy()	Modifying ROI also modifies original	Add .copy()
Shape mismatch when pasting	ValueError: could not broadcast ...	Ensure patch shape matches slice
Slicing negative ranges by accident	Empty array	Print roi.shape to confirm
Hardcoded coords past edge	Empty slice instead of error	Clip with max/min

6. Practice Tasks

- Crop a 100×100 ROI from `coffee()` and save it as `coffee_crop.png`.
- Blur only the bottom half of `astronaut()` and save.
- Demonstrate the view-vs-copy difference in a 4×4 array.

7. Key Takeaways

- `ROI = img[y1:y2, x1:x2]`.
- Pasting requires shape match.
- Slices are views — copy if independent.
- Compute-then-paste limits expensive ops to a region.

8. What's Next

Splitting and merging color channels.

Lesson 4: Channel Operations

Module: 3 — Pixel Manipulation

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Split a color image into B, G, R channels.
- Merge channels back into a color image.
- Display single channels as grayscale.
- Swap, zero, or scale individual channels.

Prerequisites

- Module 3 Lesson 3

1. Why This Matters

Many image processing techniques work on one channel at a time — e.g. detect skin in the red channel, find shadows in blue. Splitting and recombining is also the building block for converting between color spaces (Module 4).

2. Concept

2.1 Split

```
b, g, r = cv2.split(img)      # 3 separate (H, W) uint8 arrays
```

Equivalent to:

```
b, g, r = img[..., 0], img[..., 1], img[..., 2]
```

The NumPy form is faster and doesn't allocate intermediate buffers.

2.2 Merge

```
img = cv2.merge([b, g, r])    # back to (H, W, 3)
# or
img = np.stack([b, g, r], axis=2)
```

2.3 Display one channel

A single channel is a 2D grayscale image. Display with `cmap="gray"`:

```
plt.imshow(b, cmap="gray", vmin=0, vmax=255)
```

2.4 Channel swap

```
swapped = cv2.merge([r, g, b])    # swap blue and red
# or
swapped = img[..., ::-1]          # reverse last axis (BGR -> RGB)
```

2.5 Zero a channel

```
no_blue = img.copy()
no_blue[:, :, 0] = 0
```

2.6 Single-channel false-color tint

```
red_only = np.zeros_like(img)
red_only[:, :, 2] = img[:, :, 2]      # keep only the red channel content
```

3. Code Examples

Example 1: Split + display side by side

```
import cv2, numpy as np
import matplotlib.pyplot as plt
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
b, g, r = cv2.split(img)

fig, ax = plt.subplots(1, 4, figsize=(14, 4))
ax[0].imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)); ax[0].set_title("color")
ax[1].imshow(b, cmap="gray"); ax[1].set_title("Blue")
ax[2].imshow(g, cmap="gray"); ax[2].set_title("Green")
ax[3].imshow(r, cmap="gray"); ax[3].set_title("Red")
for a in ax: a.axis("off")
plt.tight_layout(); plt.show()
```

Expected Output: Four panels. Original color astronaut, then three grayscale views — the blue channel emphasises sky / cool tones; the red channel emphasises skin / helmet; the green is mostly mid-tones.

Example 2: Merge back (round-trip)

```
import cv2, numpy as np
from skimage.data import chelsea

img = cv2.cvtColor(chelsea(), cv2.COLOR_RGB2BGR)
b, g, r = cv2.split(img)
back = cv2.merge([b, g, r])
print("equal?", np.array_equal(img, back))
```

Expected Output: equal? True

Example 3: Swap red and blue (intentionally)

```
import cv2
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
swapped = img[..., ::-1].copy()      # reverse last axis
cv2.imwrite("coffee_swapped.png", swapped)
```

Expected Output: Coffee image with channels reversed — red coffee colors now appear blue, browns become teals.

Example 4: Keep only the red channel as a tinted output

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
red_only = np.zeros_like(img)
red_only[:, :, 2] = img[:, :, 2]      # keep red channel
cv2.imwrite("astro_red_only.png", red_only)
```

Expected Output: Astronaut image with only red intensities visible — looks like a red-toned monochrome of the original.

4. Parameter Sensitivity

(No params beyond channel index.)

5. Common Mistakes

Mistake	What Happens	Fix
b, g, r = img (without cv2.split)	Unpacks rows, not channels	Use cv2.split or img[..., i]
Treating channel as RGB on a BGR image	Wrong channel labels	Remember BGR order in OpenCV
Displaying single channel with default cmap	Greenish viridis	cmap="gray"
cv2.merge([r, g, b]) thinking it's RGB output	Still BGR shape, just wrong color	Be explicit about color order

6. Practice Tasks

- Split coffee(), save each channel as a separate grayscale PNG.
- Build a "red-only" version of astronaut() by zeroing green and blue.
- Confirm cv2.merge(cv2.split(img)) is bit-identical to img.

7. Key Takeaways

- cv2.split and cv2.merge reorder data; the cheap NumPy way is img[..., i] and np.stack.
- Each channel is a (H, W) grayscale image.
- Channel zeroing and swapping are one-liners.

8. What's Next

Image arithmetic — adding, subtracting, blending images.

Lesson 5: Image Arithmetic

Module: 3 — Pixel Manipulation

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Add, subtract, and scale images safely (saturation vs wrap).
- Use `cv2.addWeighted` for blending with α / β .
- Build a brightness / contrast effect.

Prerequisites

- Module 3 Lesson 4

1. Why This Matters

Image arithmetic underlies blending, alpha compositing, brightness/contrast, and difference detection. The dtype trap (uint8 wrap-around) bites everyone once — let's get past that.

2. Concept

2.1 NumPy + wraps; cv2.add saturates

```
import numpy as np, cv2
a = np.array([[200]], dtype=np.uint8)
b = np.array([[100]], dtype=np.uint8)

print(a + b)          # [[44]] -> wraps (300 mod 256)
print(cv2.add(a, b))  # [[255]] -> saturates
```

For visual operations on uint8, use `cv2.add` / `cv2.subtract` / `cv2.multiply`.

2.2 Subtract

```
diff = cv2.subtract(a, b)    # 0 if b > a, never negative
```

For signed differences (useful when finding changes), cast to int16 first:

```
diff = a.astype(np.int16) - b.astype(np.int16)
```

2.3 Weighted blend — cv2.addWeighted

```
out = cv2.addWeighted(img1, alpha, img2, beta, gamma)
# out = clip(img1 * alpha + img2 * beta + gamma)
```

For a 50/50 blend:

```
cv2.addWeighted(img1, 0.5, img2, 0.5, 0)
```

Both images must have the same shape and dtype.

2.4 Brightness & contrast

```
# brightness: add a constant
out = cv2.add(img, np.full_like(img, 30))

# contrast: scale around midpoint
out = cv2.convertScaleAbs(img, alpha=1.3, beta=0) # alpha = contrast, beta =
brightness
```

`cv2.convertScaleAbs(img, alpha, beta)` = `clip(alpha * img + beta)` and casts back to `uint8`.

2.5 Difference image (change detection)

```
abs_diff = cv2.absdiff(img1, img2)
```

`absdiff(a, b)` = $|a - b|$, saturated. Useful for motion / change detection.

3. Code Examples

Example 1: Wrap vs saturate

```
import cv2, numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
wrong = img + 80 # wraps
right = cv2.add(img, np.full_like(img, 80)) # saturates

print("wrong sample px:", wrong[0, 0])
print("right sample px:", right[0, 0])
```

Expected Output (exact pixel values vary):

```
wrong sample px: [22 33 12] <- darker than original due to wrap-around
right sample px: [255 255 92] <- saturated correctly
```

Example 2: Blend two images

```
import cv2
from skimage.data import astronaut, coffee

a = cv2.resize(cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR), (400, 400))
b = cv2.resize(cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR), (400, 400))

blend = cv2.addWeighted(a, 0.6, b, 0.4, 0)
cv2.imwrite("blend.png", blend)
```

Expected Output: A 400×400 ghostly overlay of astronaut + coffee, with astronaut more dominant (alpha 0.6 vs 0.4).

Example 3: Brightness & contrast

```
import cv2
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
```

```

bright = cv2.convertScaleAbs(img, alpha=1.0, beta=40)    # brightness +40
contrast = cv2.convertScaleAbs(img, alpha=1.5, beta=0)   # contrast 1.5x
both = cv2.convertScaleAbs(img, alpha=1.5, beta=-50)    # both

cv2.imwrite("bright.png", bright)
cv2.imwrite("contrast.png", contrast)
cv2.imwrite("both.png", both)

```

Expected Output: Three saved variants: brighter / more contrast / both. both.png looks punchy with deep shadows.

Example 4: Difference image

```

import cv2
from skimage.data import coffee

img1 = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
img2 = cv2.GaussianBlur(img1, (15, 15), 0)
diff = cv2.absdiff(img1, img2)
print("diff stats: min", diff.min(), "max", diff.max(), "mean", diff.mean())
cv2.imwrite("diff.png", diff)

```

Expected Output: diff stats: min 0 max ~85 mean ~5. The saved diff image is mostly black with brighter speckles where the original had edges (since edges are exactly what blurring loses).

4. Parameter Sensitivity

Param	Low	High
beta (brightness)	dim	bright; clipped to 255
alpha (contrast)	flat / dim	punchy, possibly clipped
alpha blend	second image dominates	first image dominates

5. Common Mistakes

Mistake	What Happens	Fix
a + b on uint8	wrap-around	cv2.add(a, b)
Negative subtract result on uint8	clipped at 0	cast to int16 if you need signed
Different shapes in addWeighted	error	resize / crop first
convertScaleAbs thinking it doesn't clip	it does clip	inspect range with .min()/ .max()

6. Practice Tasks

- Brighten coffee() by 60 using cv2.add. Save.
- Blend astronaut() and coffee() (resize to same shape) at 70/30. Save.
- Compute absdiff between an image and its 15×15 Gaussian-blurred version. Inspect the mean.

7. Key Takeaways

- Use `cv2.add` / `cv2.subtract` for saturating uint8 arithmetic.
- `cv2.addWeighted(a,  $\alpha$ , b,  $\beta$ ,  $\gamma$ )` for blends.
- `cv2.convertScaleAbs(img, alpha, beta)` for brightness/contrast.
- `cv2.absdiff` for change detection.

8. What's Next

Bitwise ops and masking — the workhorse for combining shapes and selections.

Lesson 6: Bitwise Operations & Masking

Module: 3 — Pixel Manipulation

Duration: 35 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Use `cv2.bitwise_and` / `or` / `xor` / `not` for masking.
- Build a mask from a shape or threshold.
- Combine masks; apply a mask to an image.
- Build a basic green-screen effect.

Prerequisites

- Module 3 Lesson 5

1. Why This Matters

A mask is a single-channel binary image (0 or 255) marking "where". Combine masks with images to isolate, replace, or composite regions. This is foundational for segmentation, background removal, and stamp / sticker effects.

2. Concept

2.1 The four bitwise functions

Function	Effect (per pixel)
<code>cv2.bitwise_and(a, b)</code>	bit-AND
<code>cv2.bitwise_or(a, b)</code>	bit-OR
<code>cv2.bitwise_xor(a, b)</code>	bit-XOR
<code>cv2.bitwise_not(a)</code>	invert

For uint8 images, $255 \text{ AND } 255 = 255$, $255 \text{ AND } 0 = 0$, so a mask of 0/255 cleanly selects.

2.2 The masking pattern

```
masked = cv2.bitwise_and(img, img, mask=mask)
```

- mask is a single-channel uint8, same H×W as img.
- Where `mask == 0`, output is 0.
- Where `mask > 0`, output is img.

Equivalent NumPy: `out = np.where(mask[..., None] > 0, img, 0)`.

2.3 Building masks

```
# from a threshold
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, mask = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY)

# from a shape
```

```
mask = np.zeros(img.shape[:2], dtype=np.uint8)
cv2.circle(mask, (200, 200), 80, 255, -1)
```

2.4 Inverting a mask

```
inv = cv2.bitwise_not(mask)
```

2.5 Composite: foreground + background

```
# fg masked by mask, bg masked by inverted mask, then add
fg = cv2.bitwise_and(foreground, foreground, mask=mask)
bg = cv2.bitwise_and(background, background, mask=cv2.bitwise_not(mask))
out = cv2.add(fg, bg)
```

2.6 Color masks (rare but useful)

You can apply bitwise ops between two color images — but for "select region" tasks, build a single-channel mask and use mask= instead.

3. Code Examples

Example 1: Apply a circular mask

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
h, w = img.shape[:2]
mask = np.zeros((h, w), dtype=np.uint8)
cv2.circle(mask, (w // 2, h // 2), min(h, w) // 3, 255, -1)

masked = cv2.bitwise_and(img, img, mask=mask)
cv2.imwrite("astro_circle.png", masked)
```

Expected Output: Astronaut image, but only a circular region in the centre is visible; the rest is solid black.

Example 2: Invert a mask

```
import cv2, numpy as np

mask = np.zeros((200, 200), dtype=np.uint8)
cv2.rectangle(mask, (50, 50), (150, 150), 255, -1)

inv = cv2.bitwise_not(mask)
cv2.imwrite("mask.png", mask)
cv2.imwrite("mask_inv.png", inv)
```

Expected Output: mask.png is a black square with a white inner rectangle; mask_inv.png is the opposite — white square with a black inner rectangle.

Example 3: Composite — green screen replacement

```
import cv2, numpy as np

# Build a "green screen" subject: a synthetic image with green background
subject = np.full((300, 400, 3), (0, 255, 0), dtype=np.uint8) # all green
```

```

cv2.circle(subject, (200, 150), 80, (50, 80, 200), -1)           # tan "face"

# Background: any image of the right size
bg = cv2.resize(cv2.cvtColor(__import__("skimage.data",
                                     fromlist=["coffee"]).coffee(),
                                     cv2.COLOR_RGB2BGR), (400, 300))

# Build a mask of "not green" pixels (the subject's foreground)
hsv = cv2.cvtColor(subject, cv2.COLOR_BGR2HSV)
mask_green = cv2.inRange(hsv, (40, 100, 100), (80, 255, 255))  # green range in
HSV
mask_fg = cv2.bitwise_not(mask_green)

fg = cv2.bitwise_and(subject, subject, mask=mask_fg)
bg_only = cv2.bitwise_and(bg, bg, mask=mask_green)
composite = cv2.add(fg, bg_only)

cv2.imwrite("composite.png", composite)

```

Expected Output: The synthetic tan circle floating on the coffee photo (the green background was replaced with the coffee image).

Example 4: Bitwise AND of two shape masks

```

import cv2, numpy as np

a = np.zeros((200, 200), dtype=np.uint8); cv2.circle(a, (80, 100), 70, 255, -1)
b = np.zeros((200, 200), dtype=np.uint8); cv2.circle(b, (120, 100), 70, 255, -1)

both = cv2.bitwise_and(a, b)
either = cv2.bitwise_or(a, b)
xor = cv2.bitwise_xor(a, b)

for name, m in [("a", a), ("b", b), ("and", both), ("or", either), ("xor", xor)]:
    cv2.imwrite(f"{name}.png", m)

```

Expected Output: Five files showing the two overlapping circles and their AND (just the lens of overlap), OR (the full vesica shape), XOR (only the non-overlapping crescents).

4. Parameter Sensitivity

Mask source	Effect
Threshold	binary based on brightness
Shape (cv2.circle etc.)	precise geometric region
HSV color range	object segmentation by color
Edge-based / contour	object outlines (M9-M12)

5. Common Mistakes

Mistake	What Happens	Fix
Mask not single-channel	Operations behave per-channel weirdly	Use (H, W) uint8 mask
Mask non-binary (gray)	Partial selection; soft edges	Use 0 / 255 for hard mask, or

values)	(sometimes desirable)	alpha blending for soft
Mask wrong shape	Crash or misaligned	<code>mask.shape[:2] == img.shape[:2]</code>
Treating <code>cv2.bitwise_and(img, mask)</code> as the masking pattern	Wrong — channels misalign	Use <code>cv2.bitwise_and(img, img, mask=mask)</code>

6. Practice Tasks

- Build a square mask at the centre of `coffee()`; keep only inside the square.
- Build a circular mask and a rectangular mask, AND them, apply to `astronaut()`.
- Implement a tiny green-screen: synthesise a green-background subject, swap the green out for any other image.

7. Key Takeaways

- Mask = (H, W) uint8, 0/255.
- `cv2.bitwise_and(img, img, mask=mask)` is the masking pattern.
- Build masks from thresholds, shapes, or HSV ranges.
- Composite by `cv2.add(fg, bg)` after masking both with mask + inverse.

8. What's Next

End of Module 3. Next: Module 4 — Color Spaces — HSV, LAB, and the color-picking trick for object detection.

Module 3 — Exercises

18 exercises (3 per lesson).

3.1 Accessing Pixels

- (E) Set `coins()[100,200]` to 0 and print before/after.
- (M) Zero out the entire 50th row and 50th column of `camera()`. Save.
- (H) Remove the blue channel from `astronaut()` (BGR index 0).

3.2 Drawing

- (E) Draw a green 5-pixel circle at the centre of a 200×200 black canvas.
- (M) Annotate `astronaut()` with a yellow bounding box and label "face".
- (H) Draw a filled pentagon on a 400×400 canvas using `cv2.fillPoly`.

3.3 ROI

- (E) Crop a 100×100 patch from the centre of `coffee()`. Save it.
- (M) Blur only the bottom half of `astronaut()`. Save.
- (H) Demonstrate view vs copy on a small 5×5 array.

3.4 Channels

- (E) Split `coffee()` into B, G, R; save each as grayscale.
- (M) Confirm `cv2.merge(cv2.split(img))` equals `img`.
- (H) Build a red-only tinted version of `astronaut()`.

3.5 Arithmetic

- (E) Brighten `coffee()` by 60 using `cv2.add`. Save.
- (M) Blend resized `astronaut()` and `coffee()` at 70/30 with `addWeighted`.
- (H) Compute `absdiff` between original `chelsea()` and its 21×21 `GaussianBlur`.

3.6 Bitwise / Masking

- (E) Apply a centered circular mask to `coffee()`.
- (M) Build a mask = circle OR rectangle, apply to `astronaut()`.
- (H) Implement a tiny green-screen: synthesise a green-background subject, swap for `coffee()` as the new background.

Module 3 — Quiz

10 MCQs.

Q1

`img[100, 200]` accesses which pixel? A. row 200, col 100 B. row 100, col 200 C. $x=100, y=200$ D. depends on shape

Answer: B (L1)

Q2

`cv2.rectangle(img, (50, 30), (150, 100), ...)` corners are: A. $(x1,y1)=(50,30), (x2,y2)=(150,100)$ B. $(row,col)=(50,30)$ and $(150,100)$ C. ambiguous D. $(y1,x1)=(50,30)$

Answer: A — OpenCV uses (x, y) . (L2)

Q3

A 100×100 roi = `img[50:150, 80:180]`. Modifying roi modifies: A. nothing B. `img` C. depends D. only if `.copy()`

Answer: B — slice is a view. (L3)

Q4

`cv2.split(img)` for a BGR image returns: A. (R, G, B) B. (B, G, R) C. (gray, alpha) D. (R, G, B, A)

Answer: B (L4)

Q5

`np.array([200], dtype=np.uint8) + np.array([100], dtype=np.uint8)` gives: A. [300] B. [44] C. [255] D. error

Answer: B — wraps. Use `cv2.add` for saturation. (L5)

Q6

`cv2.add` on `uint8`: A. saturates at 255 B. wraps C. errors D. casts to int

Answer: A (L5)

Q7

`cv2.addWeighted(a, 0.5, b, 0.5, 0)` produces: A. equal blend B. only a C. only b D. difference

Answer: A (L5)

Q8

`cv2.absdiff` is used for: A. addition B. absolute difference of two images C. blurring D. masking

Answer: B (L5)

Q9

The masking pattern is: A. `cv2.bitwise_and(img, mask)` B. `cv2.bitwise_and(img, img, mask=mask)`
C. `img * mask` D. `img & mask`

Answer: B (L6)

Q10

For a mask, `cv2.bitwise_not(mask)` does: A. flips horizontally B. inverts the mask C. rotates 90°
D. blurs

Answer: B (L6)

Module 3 — Assignment: Image Watermarking Tool

Estimated time: 2 hours

Combines: Module 3 + Modules 1-2

Brief

Build a watermarking CLI that overlays a text and/or a logo image (with transparency) onto a target image, then saves the result.

Requirements

- `watermark.py INPUT --output OUT [--text "WATERMARK"] [--logo path/to/logo.png] [--opacity 0.5] [--position bottom-right|top-left|center]`
- Text overlay:
- Use `cv2.putText` (or Pillow for nicer fonts).
- Configurable position keyword.
- Add a subtle dark rectangle behind the text for readability.
- Logo overlay:
- Logo should be a PNG with alpha; respect its alpha channel.
- Resize logo so its larger dimension is 1/6 of the target image.
- Blend with `--opacity`.
- Save the result to `--output`.

Constraints

- Module 3 concepts (ROI, arithmetic, bitwise masking).
- Wrap operations in named functions.
- Use `argparse`.

Expected Behaviour (sample)

```
$ python watermark.py astro.png --output out.png --text "(c) Maya" --opacity 0.6 --  
position bottom-right  
saved out.png
```

The output should have "(c) Maya" inside a subtle dark rectangle at the bottom-right, at 60% opacity.

If `--logo` is provided, the logo PNG should appear in the same corner with its alpha channel respected.

Grading Rubric

Criterion	Weight
Text watermark	25%
Logo overlay (alpha)	25%
Opacity control	15%
Position handling	15%

Style + argparse	20%
------------------	-----

Submission

Save as assignment_module3.py.

Module 3 — Mini Project: Green Screen / Background Replacement

Overview

Replace a green background with any other image. Uses HSV masking + bitwise compositing — the classic chroma-key recipe.

Concepts Used

- HSV preview (M4 will go deeper)
- `cv2.inRange` for masking
- `cv2.bitwise_and` / `bitwise_or` (M3 L6)
- ROI sizing (M3 L3)

Features

- `greenscreen.py SUBJECT BACKGROUND [--output OUT]`
- Detect green pixels in HSV, build a mask.
- Replace those pixels with corresponding pixels from `BACKGROUND` (resized to match).
- Save the composite.

Starter Code

`greenscreen.py`:

```
import argparse
import cv2
import numpy as np

def load(path):
    img = cv2.imread(path)
    if img is None:
        raise FileNotFoundError(path)
    return img

def composite(subject, bg, low=(35, 70, 70), high=(85, 255, 255)):
    bg_resized = cv2.resize(bg, (subject.shape[1], subject.shape[0]))
    hsv = cv2.cvtColor(subject, cv2.COLOR_BGR2HSV)
    green_mask = cv2.inRange(hsv, low, high)
    # green_mask: 255 where green (background), 0 where subject
    fg_mask = cv2.bitwise_not(green_mask)
    fg = cv2.bitwise_and(subject, subject, mask=fg_mask)
    bg_only = cv2.bitwise_and(bg_resized, bg_resized, mask=green_mask)
    return cv2.add(fg, bg_only)

def main():
    p = argparse.ArgumentParser()
    p.add_argument("subject", help="image with green background")
    p.add_argument("background", help="replacement background image")
    p.add_argument("--output", default="composite.png")
    args = p.parse_args()
```

```

subject = load(args.subject)
bg = load(args.background)
out = composite(subject, bg)
cv2.imwrite(args.output, out)
print(f"saved -> {args.output}")

if __name__ == "__main__":
    main()

```

Test Image

If you don't have a green-screen photo, synthesize one:

```

import cv2, numpy as np
img = np.full((400, 600, 3), (0, 255, 0), dtype=np.uint8) # all green
cv2.circle(img, (300, 200), 100, (40, 100, 200), -1)      # tan face
cv2.imwrite("test_subject.png", img)

```

Use datasets/starter/coffee.png as the background.

Expected Output

```

$ python greenscreen.py test_subject.png datasets/starter/coffee.png
saved -> composite.png

```

composite.png shows the tan circle floating on the coffee image — green has been swapped out.

Extension Challenges

- Add --hsv-low and --hsv-high flags so you can chroma-key any color (e.g., blue screen).
- Erode the mask before applying (M10 preview) to remove edge bleed.
- Soft-edge the mask with Gaussian blur on the mask, then use alpha blend instead of hard bitwise.

Grading Rubric

Criterion	Weight
Basic composite works	35%
Background resized OK	15%
Reasonable HSV range	20%
CLI + style	15%
Documentation / README	15%

Python E-Course

Module 4 — Color Spaces

Detailed Notes

Module Overview

Level: Color & Geometry

Duration: ~1 week

Prerequisites: Modules 1-3

What You'll Learn

- Convert between RGB/BGR, HSV, LAB, YCrCb.
- Read HSV histograms and pick color ranges for object detection.
- Choose the right color space for the task (segmentation, similarity, printing).
- Avoid the OpenCV HSV-range trap (H is 0–179, not 0–360).

Lessons

#	Lesson	Duration
1	Why Color Spaces Matter	25 min
2	RGB and BGR	20 min
3	HSV — the Object-Detection Workhorse	35 min
4	LAB and YCrCb	25 min
5	Converting with cv2.cvtColor	20 min
6	Color Picking for Object Detection	30 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 5: Geometric Transformations

Lesson 1: Why Color Spaces Matter

Module: 4 — Color Spaces

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Explain the limitations of RGB for perceptual tasks.
- Name the three main alternative color spaces and what each is good for.
- Predict which space to choose for: object detection by color, perceptual similarity, broadcast video.

Prerequisites

- Module 3

1. Why This Matters

RGB stores how much red, green, and blue light a pixel emits — great for displays, bad for "find every red apple". For most perceptual tasks (color matching, segmentation, lighting-invariant detection), an alternative color space makes the problem 10× easier.

2. Concept

2.1 RGB's weakness: lighting changes everything

A red apple in shade is (80, 30, 30). In sun it's (240, 90, 90). Both are red, but RGB values differ by 3×. Threshold-based "find red" in RGB has to deal with this constantly.

2.2 HSV separates color from lightness

HSV = Hue, Saturation, Value.

- H — what color is it? (0 = red, 60 = yellow, 120 = green, 240 = blue)
- S — how saturated / pure is it? (0 = gray, 255 = vivid)
- V — how bright is it? (0 = black, 255 = max)

Shade and sun change V (and a bit of S), but H stays the same. Filter on H → reliable color detection.

2.3 LAB is perceptually uniform

LAB: Lightness + a (green↔red) + b (blue↔yellow). Two LAB colors with the same Euclidean distance look about equally different to humans — RGB doesn't have this property. Use LAB when comparing colors meaningfully.

2.4 YCrCb separates luma from chroma

Y = brightness; Cr, Cb = color differences. This is how broadcast video (JPEG, MPEG) is stored — you can compress chroma more than luma without humans noticing. Useful for skin-tone detection too.

2.5 Cheat sheet

Goal	Use
Display	RGB / BGR
Color-based segmentation	HSV
Perceptual color distance	LAB
Broadcast / skin detection	YCrCb
Print	CMYK (not built into OpenCV)

3. Code Examples

Example 1: Same apple under two lights — RGB vs HSV

```
import numpy as np, cv2

# Synthesize: a 50x50 patch in two "lighting" conditions
shade = np.full((50, 50, 3), (30, 30, 80), dtype=np.uint8)    # BGR: dark red
sun    = np.full((50, 50, 3), (90, 90, 240), dtype=np.uint8)  # BGR: bright red

for name, img in [("shade", shade), ("sun", sun)]:
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    print(f"{name:5s}  BGR={tuple(img[0,0])}  HSV={tuple(hsv[0,0])}")
```

Expected Output:

```
shade  BGR=(30, 30, 80)  HSV=(0, 159, 80)
sun    BGR=(90, 90, 240) HSV=(0, 159, 240)
```

Notice: H is the same (0 = red) in both. Only V (brightness) differs. That's why HSV is the choice for object detection by color.

Example 2: Show all three spaces side by side

```
import cv2, matplotlib.pyplot as plt
from skimage.data import coffee

bgr = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)
lab = cv2.cvtColor(bgr, cv2.COLOR_BGR2LAB)

fig, ax = plt.subplots(1, 3, figsize=(12, 4))
ax[0].imshow(cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)); ax[0].set_title("RGB")
ax[1].imshow(hsv); ax[1].set_title("HSV (raw channels as RGB)")
ax[2].imshow(lab); ax[2].set_title("LAB (raw channels as RGB)")
for a in ax: a.axis("off")
plt.tight_layout(); plt.show()
```

Expected Output: Three panels. The first is the normal coffee image. The other two are nonsensical pseudo-colored versions because we're displaying HSV/LAB values as if they were RGB — that's the point: these spaces aren't meant for direct display.

4. Parameter Sensitivity

(No params here — each conversion is deterministic.)

5. Common Mistakes

Mistake	What Happens	Fix
Filtering by RGB for object color	Fails under lighting changes	Convert to HSV first
Displaying HSV image with imshow	Looks psychedelic	Convert back to BGR/RGB for display
Assuming H is 0..360	OpenCV uses 0..179 (next lesson)	Halve textbook H values

6. Practice Tasks

- Predict HSV of pure red (0, 0, 255) BGR. Verify with `cv2.cvtColor`.
- Predict HSV of pure green (0, 255, 0) BGR. Verify.
- Convert `astronaut()` to LAB and print the L channel's mean.

7. Key Takeaways

- RGB is for display.
- HSV separates hue from brightness — best for color detection.
- LAB is perceptually uniform — best for color similarity.
- YCrCb separates luma from chroma — used in video / skin detection.

8. What's Next

The boring necessary one — RGB and BGR (and why every library disagrees).

Lesson 2: RGB and BGR

Module: 4 — Color Spaces

Duration: 20 minutes

Difficulty: Beginner

Learning Objectives

- Tell RGB from BGR in code.
- Convert between them with `cv2.cvtColor` or slicing.
- Recognise visual symptoms of using the wrong order.

Prerequisites

- Module 4 Lesson 1

1. Why This Matters

This is the single most common bug in beginner OpenCV code: load with `cv2.imread` (BGR), pass directly to `matplotlib` (expects RGB), and wonder why reds look blue. Settling this is a 20-minute lesson that saves a career of confusion.

2. Concept

2.1 Three channels, two orders

- RGB: index 0 = Red, 1 = Green, 2 = Blue. Standard everywhere (web, scikit-image, matplotlib, Pillow).
- BGR: index 0 = Blue, 1 = Green, 2 = Red. OpenCV's historical choice.

The image data is the same; the interpretation of channel order differs.

2.2 Library cheat sheet

Library	Order
OpenCV (<code>cv2.imread</code> , <code>cv2.imwrite</code>)	BGR
skimage (<code>skimage.io.imread</code> , <code>skimage.data.*</code>)	RGB
Pillow	RGB
matplotlib <code>imshow</code>	RGB
HTML / CSS	RGB

2.3 Conversions

```
rgb = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)
bgr = cv2.cvtColor(rgb, cv2.COLOR_RGB2BGR)
```

Both are equivalent to swapping the last axis:

```
rgb = bgr[..., ::-1]    # reverses last axis
```

The slicing form is a view, the `cvtColor` form is a copy. For long-lived results prefer `cvtColor`.

2.4 Visual symptom

When you display BGR data as RGB:

- Reds appear blue
- Blues appear red
- Greens unchanged

Skin tones look ghastly; skies look orange.

3. Code Examples

Example 1: Same image, three orderings

```
import cv2, matplotlib.pyplot as plt
from skimage.data import astronaut

img_rgb = astronaut() # RGB
img_bgr = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2BGR)

fig, ax = plt.subplots(1, 3, figsize=(12, 4))
ax[0].imshow(img_rgb);
ax[0].set_title("RGB (correct)")
ax[1].imshow(img_bgr);
ax[1].set_title("BGR shown as RGB (WRONG)")
ax[2].imshow(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB));
ax[2].set_title("BGR → RGB (correct)")
for a in ax: a.axis("off")
plt.tight_layout(); plt.show()
```

Expected Output: Three panels. Panel 1 and 3 look identical (correct astronaut). Panel 2 has all the reds shifted to cyan / blue.

Example 2: Slicing vs cvtColor

```
import cv2, numpy as np
from skimage.data import coffee

bgr = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
rgb_a = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)
rgb_b = bgr[..., ::-1] # view

print("equal:", np.array_equal(rgb_a, rgb_b)) # True
print("same memory:", rgb_a.base is None,
      "vs slice base shared?", rgb_b.base is bgr)
```

Expected Output:

```
equal: True
same memory: True vs slice base shared? True
```

Example 3: Define a tiny show helper

```
import cv2, matplotlib.pyplot as plt
```

```
def show_bgr(img, title=""):
    plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    plt.title(title); plt.axis("off"); plt.show()

# always safe – never display BGR directly
```

4. Parameter Sensitivity

(No params — choice is RGB vs BGR.)

5. Common Mistakes

Mistake	What Happens	Fix
<code>plt.imshow(cv2.imread(...))</code>	Reds → blues	Wrap with <code>cv2.cvtColor(..., COLOR_BGR2RGB)</code>
<code>cv2.imwrite("x.jpg", rgb_image)</code> from skimage	Reds and blues swapped in saved file	Convert RGB → BGR before saving
Mixing skimage and cv2 without converting	Channels swap silently	Convert at every boundary

6. Practice Tasks

- Load `astronaut.png` with `cv2.imread` and display with matplotlib without converting. Note how it looks.
- Convert and display correctly. Confirm helmet looks orange / red, not blue.
- Build a `show(img)` helper that auto-converts BGR before display.

7. Key Takeaways

- OpenCV = BGR. Everything else = RGB.
- Convert at library boundaries.
- `cv2.cvtColor` (copy) or `[..., ::-1]` (view).

8. What's Next

HSV — the color space you'll use most for object detection.

Lesson 3: HSV — the Object-Detection Workhorse

Module: 4 — Color Spaces

Duration: 35 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Convert BGR → HSV and back.
- Understand OpenCV's 0–179 H range (the trap).
- Build a binary mask with `cv2.inRange`.
- Pick HSV ranges for common colors.

Prerequisites

- Module 4 Lesson 2

1. Why This Matters

HSV is the color space for color-based object detection. Splitting hue from brightness gives you robustness to lighting changes that's impossible in RGB. The catch — OpenCV's H range is unusual.

2. Concept

2.1 The OpenCV ranges

Channel	Range	Meaning
H	0–179	Hue (red, yellow, green, ...)
S	0–255	Saturation
V	0–255	Value (brightness)

The trap: Most references say H goes 0–360. OpenCV halves it to fit in uint8. So:

- Textbook H = 30 (orange) → OpenCV H = 15
- Textbook H = 120 (green) → OpenCV H = 60
- Textbook H = 240 (blue) → OpenCV H = 120

Halve any reference value before using it in `cv2.inRange`.

2.2 Color → H mapping

Color	OpenCV H
Red	0 or 179 (wraps around)
Orange	~15
Yellow	~30
Green	~60
Cyan	~90
Blue	~120

Magenta	~150
Red (other end)	179

Red is special — it spans both ends. You usually need two masks for red and OR them.

2.3 cv2.inRange for masking

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, (low_h, low_s, low_v), (high_h, high_s, high_v))
```

Returns a uint8 mask: 255 where in range, 0 otherwise.

2.4 Saturation and Value cutoffs

Almost always include lower bounds on S and V to exclude grays and shadows. A typical color range:

```
# Yellow
low = ( 20, 100, 100)
high = ( 40, 255, 255)
```

The 100, 100 lower bounds say "must be reasonably saturated and bright" — excludes washed-out grays.

2.5 The red wraparound

```
mask1 = cv2.inRange(hsv, (0, 100, 100), (10, 255, 255))
mask2 = cv2.inRange(hsv, (170, 100, 100), (179, 255, 255))
mask_red = cv2.bitwise_or(mask1, mask2)
```

3. Code Examples

Example 1: Pure colors → HSV values

```
import cv2, numpy as np

colors_bgr = {
    "Red": ( 0, 0, 255),
    "Green": ( 0, 255, 0),
    "Blue": (255, 0, 0),
    "Yellow": ( 0, 255, 255),
    "Cyan": (255, 255, 0),
}

for name, bgr in colors_bgr.items():
    px = np.full((1, 1, 3), bgr, dtype=np.uint8)
    h, s, v = cv2.cvtColor(px, cv2.COLOR_BGR2HSV)[0, 0]
    print(f"{name:7s} BGR={bgr} -> H={h:3d} S={s:3d} V={v:3d}")
```

Expected Output:

```
Red      BGR=(0, 0, 255)   -> H=  0 S=255 V=255
Green    BGR=(0, 255, 0)  -> H= 60 S=255 V=255
Blue     BGR=(255, 0, 0)  -> H=120 S=255 V=255
Yellow   BGR=(0, 255, 255) -> H= 30 S=255 V=255
Cyan     BGR=(255, 255, 0) -> H= 90 S=255 V=255
```

Example 2: Mask green pixels with inRange

```
import cv2, numpy as np

# build a synthetic image: half green, half blue
img = np.zeros((200, 400, 3), dtype=np.uint8)
img[:, :200] = (0, 255, 0)
img[:, 200:] = (255, 0, 0)

hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, (40, 100, 100), (80, 255, 255)) # green range

print("green pixels:", (mask == 255).sum(), "out of", mask.size)
cv2.imwrite("mask_green.png", mask)
```

Expected Output: green pixels: 40000 out of 80000 and a mask_green.png that's white on the left half, black on the right.

Example 3: Detect red — with wraparound

```
import cv2, numpy as np

img = cv2.cvtColor(np.tile(np.array(
    [[(0, 0, 255), (0, 0, 200), (0, 0, 150)]], dtype=np.uint8), (50, 1, 1)),
    cv2.COLOR_RGB2BGR)
# (just a synthetic red stripe)

hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
m1 = cv2.inRange(hsv, (0, 100, 50), (10, 255, 255))
m2 = cv2.inRange(hsv, (170, 100, 50), (179, 255, 255))
mask = cv2.bitwise_or(m1, m2)
print("red pixels detected:", (mask == 255).sum())
```

Expected Output: A non-zero count showing that the red region was captured by combining the two ranges.

Example 4: Real image — find red objects in a photo

```
import cv2
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

m1 = cv2.inRange(hsv, (0, 90, 50), (10, 255, 255))
m2 = cv2.inRange(hsv, (170, 90, 50), (179, 255, 255))
mask = cv2.bitwise_or(m1, m2)

result = cv2.bitwise_and(img, img, mask=mask)
cv2.imwrite("coffee_red_only.png", result)
print("red pixel fraction:", (mask > 0).mean())
```

Expected Output: coffee_red_only.png shows only the red parts of the coffee image (cherry / berry tones in the bean shadows). The rest is black. Console prints something like red pixel fraction: 0.08.

4. Parameter Sensitivity

Param	Lower	Higher
H low/high	tighter range (specific color)	broader range (more shades)
S low	includes pale / washed colors (more noise)	only vivid colors
V low	includes shadows	only bright objects

5. Common Mistakes

Mistake	What Happens	Fix
Using textbook H (0–360) directly	Wrong color matched	Halve to OpenCV 0–179
Red without wraparound	Misses half the reds	Two masks + OR
Too-low S/V bounds	Captures gray pixels	Set S, V $\geq$ 70–100
Forgetting to convert BGR→HSV	Garbage matches	cv2.cvtColor(img, cv2.COLOR_BGR2HSV) first

6. Practice Tasks

- Compute HSV for pure orange BGR (0, 128, 255). What H?
- Build a mask for "blue" pixels in astronaut() (the suit). Tune ranges visually.
- Build a red mask with wraparound; apply to coffee().

7. Key Takeaways

- HSV separates color (H) from brightness (V).
- OpenCV H is 0–179. Halve textbook values.
- cv2.inRange + tight S/V bounds = the classic color filter.
- Red wraps; use two ranges + OR.

8. What's Next

LAB and YCrCb — when perceptual distance or chroma separation matters.

Lesson 4: LAB and YCrCb

Module: 4 — Color Spaces

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Convert to/from LAB and YCrCb.
- Use LAB for perceptual color distance and lightness manipulation.
- Use YCrCb for skin detection and chroma-separated processing.
- Know which channel of each space carries which information.

Prerequisites

- Module 4 Lesson 3

1. Why This Matters

HSV is good but not perceptually uniform. LAB is, which makes it the right space for "are these two colors similar to a human?". YCrCb separates brightness from chroma — handy when you want to manipulate one without the other (the basis for JPEG compression and many skin-detection pipelines).

2. Concept

2.1 LAB

Channel	Range (OpenCV uint8)	Meaning
L	0–255	Lightness (0 = black, 255 = white)
a	0–255	green ↔ red (128 = neutral)
b	0–255	blue ↔ yellow (128 = neutral)

(In the academic LAB definition, L is 0–100 and a/b are signed. OpenCV scales them to fit uint8.)

Two LAB colors that differ by the same Euclidean distance look about equally different to the human eye. Use this for color similarity / nearest-color matching.

2.2 Common LAB recipes

CLAHE on L channel only (improves contrast without color shift — covered fully in Module 6):

```
lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
L, a, b = cv2.split(lab)
L_eq = cv2.createCLAHE(clipLimit=2.0).apply(L)
lab_eq = cv2.merge([L_eq, a, b])
out = cv2.cvtColor(lab_eq, cv2.COLOR_LAB2BGR)
```

2.3 YCrCb

Channel	Meaning
Y	Luma (grayscale-like)
Cr	Red-difference chroma
Cb	Blue-difference chroma

YCrCb separates brightness from color. Used in JPEG/MPEG (which can compress chroma harder than luma) and in skin detection (skin tends to fall in a tight Cr/Cb range across many ethnicities).

Skin range (rough): $Y \in [0, 255]$, $Cr \in [133, 173]$, $Cb \in [77, 127]$.

2.4 Pick the space for the task

Goal	Space
Make image more contrasty without shifting hue	LAB (CLAHE on L)
Find nearest color from a palette	LAB (Euclidean distance)
Detect skin in arbitrary lighting	YCrCb
Detect a specific color object	HSV
Convert for display	RGB / BGR

3. Code Examples

Example 1: Convert and inspect ranges

```
import cv2, numpy as np
from skimage.data import astronaut

bgr = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
lab = cv2.cvtColor(bgr, cv2.COLOR_BGR2LAB)
ycc = cv2.cvtColor(bgr, cv2.COLOR_BGR2YCrCb)

for name, img in [("LAB", lab), ("YCrCb", ycc)]:
    for i, ch in enumerate(["c1", "c2", "c3"]):
        v = img[..., i]
        print(f"{name} {ch}: min={v.min()} max={v.max()} mean={v.mean():.1f}")
    print()
```

Expected Output (rough values, depending on the image):

```
LAB c1: min=2 max=255 mean=124.2
LAB c2: min=88 max=183 mean=132.1
LAB c3: min=86 max=192 mean=148.7

YCrCb c1: min=14 max=251 mean=105.4
YCrCb c2: min=104 max=181 mean=137.1
YCrCb c3: min=90 max=151 mean=123.5
```

Example 2: Boost contrast via LAB (no hue shift)

```
import cv2
from skimage.data import coffee
```

```

bgr = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
lab = cv2.cvtColor(bgr, cv2.COLOR_BGR2LAB)
L, a, b = cv2.split(lab)
clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8))
L_eq = clahe.apply(L)
lab_eq = cv2.merge([L_eq, a, b])
out = cv2.cvtColor(lab_eq, cv2.COLOR_LAB2BGR)
cv2.imwrite("coffee_clahe_lab.png", out)

```

Expected Output: Coffee photo with sharper contrast in highlights / shadows, but colors are not over-saturated (unlike doing CLAHE per RGB channel, which can produce odd color shifts).

Example 3: Skin detection in YCrCb

```

import cv2, numpy as np
from skimage.data import astronaut

bgr = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
ycc = cv2.cvtColor(bgr, cv2.COLOR_BGR2YCrCb)

# Loose skin range
skin = cv2.inRange(ycc, (0, 133, 77), (255, 173, 127))
out = cv2.bitwise_and(bgr, bgr, mask=skin)
cv2.imwrite("astro_skin.png", out)
print("skin pixel fraction:", (skin > 0).mean())

```

Expected Output: astro_skin.png shows mostly the astronaut's face (and any other skin-tone regions) — the rest is black. Console prints something like skin pixel fraction: 0.04.

Example 4: Color-similarity in LAB

```

import cv2, numpy as np

# Palette and target color (BGR)
palette_bgr = [(0,0,255), (255,0,0), (0,255,0), (200,200,200)]
target_bgr = (40, 30, 220) # a slightly off-red

palette_lab = [cv2.cvtColor(np.array([[c]], dtype=np.uint8), cv2.COLOR_BGR2LAB)[0,0]
for c in palette_bgr]
target_lab = cv2.cvtColor(np.array([[target_bgr]], dtype=np.uint8),
cv2.COLOR_BGR2LAB)[0,0].astype(np.int32)

distances = [np.linalg.norm(target_lab - p.astype(np.int32)) for p in palette_lab]
best = int(np.argmin(distances))
print("nearest in palette:", palette_bgr[best], " distance:", distances[best])

```

Expected Output: Tells you the off-red target is closest to the pure-red (0,0,255) palette entry — and the distance number quantifies how close.

4. Parameter Sensitivity

(Conversions are fixed; no params beyond which channel you operate on.)

5. Common Mistakes

Mistake	What Happens	Fix
Forgetting LAB's a/b are centred at 128	Sign confusion	Subtract 128 if you need signed values
Doing CLAHE on RGB channels independently	Color shift	Do CLAHE on L of LAB, leave a/b alone
Using LAB for arbitrary "color detection"	Less convenient than HSV	HSV for detection; LAB for similarity

6. Practice Tasks

- Convert `coffee()` to LAB; print mean of L channel.
- Build a YCrCb skin mask on `astronaut()`. Inspect coverage.
- Find the nearest of [red, green, blue] to an off-orange BGR color, using LAB distance.

7. Key Takeaways

- LAB: perceptually uniform; use for color similarity and contrast manipulation.
- YCrCb: luma + chroma; useful for skin detection and video.
- Different tools for different goals — pick based on the question you're answering.

8. What's Next

The conversion API itself — `cv2.cvtColor` and the constants you'll meet.

Lesson 5: Converting with cv2.cvtColor

Module: 4 — Color Spaces

Duration: 20 minutes

Difficulty: Beginner

Learning Objectives

- Use cv2.cvtColor for any conversion you need.
- Remember the most-used conversion flags.
- Convert grayscale $\leftrightarrow$ color, do alpha extraction.

Prerequisites

- Module 4 Lesson 4

1. Why This Matters

cv2.cvtColor is the single function for almost every color-space change. Memorising 6 conversion flags covers 95% of real-world work.

2. Concept

2.1 Signature

```
out = cv2.cvtColor(src, code)
```

Where code is a constant like cv2.COLOR_BGR2GRAY. The result has whatever shape the destination space requires (e.g. (H, W) for grayscale, (H, W, 3) for color).

2.2 The flags you'll actually use

Conversion	Flag
BGR $\rightarrow$ RGB	COLOR_BGR2RGB
RGB $\rightarrow$ BGR	COLOR_RGB2BGR
BGR $\rightarrow$ grayscale	COLOR_BGR2GRAY
RGB $\rightarrow$ grayscale	COLOR_RGB2GRAY
gray $\rightarrow$ BGR (replicate)	COLOR_GRAY2BGR
BGR $\rightarrow$ HSV	COLOR_BGR2HSV
HSV $\rightarrow$ BGR	COLOR_HSV2BGR
BGR $\rightarrow$ LAB	COLOR_BGR2LAB
LAB $\rightarrow$ BGR	COLOR_LAB2BGR
BGR $\rightarrow$ YCrCb	COLOR_BGR2YCrCb
YCrCb $\rightarrow$ BGR	COLOR_YCrCb2BGR
BGRA $\rightarrow$ BGR	COLOR_BGRA2BGR (drop alpha)
BGRA $\rightarrow$ alpha only	(slice: bgra[..., 3])

2.3 Grayscale conversion math

COLOR_BGR2GRAY uses the ITU-R BT.601 weights:


```
gray = 0.299 R + 0.587 G + 0.114 B
```

The exact pure-NumPy form (covered in M1 L5):

```
gray = (img.astype(np.float32) * (0.114, 0.587, 0.299)).sum(axis=2).clip(0, 255).astype(np.uint8)
```

(Weights in BGR order to match cv2.cvtColor's input.)

2.4 Gray → BGR

```
bgr = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)
```

This just replicates the gray value into 3 channels — useful when you need to draw a colored annotation on a grayscale background.

2.5 Multi-step pipelines

```
img = cv2.imread("in.jpg") # BGR
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, lo, hi)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray_masked = cv2.bitwise_and(gray, gray, mask=mask)
```

3. Code Examples

Example 1: BGR ↔ grayscale round-trip

```
import cv2, numpy as np
from skimage.data import astronaut

bgr = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
gray = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY)
bgr2 = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)

print("bgr:", bgr.shape, "gray:", gray.shape, "back:", bgr2.shape)
# Channels of bgr2 are replicated, not the original — round-trip is lossy
print("identical?", np.array_equal(bgr, bgr2))
```

Expected Output:

```
bgr: (512, 512, 3) gray: (512, 512) back: (512, 512, 3)
identical? False
```

Example 2: Quick HSV pipeline

```
import cv2, numpy as np
from skimage.data import coffee

bgr = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, (0, 0, 0), (179, 255, 100)) # all dark pixels
print("dark pixel fraction:", (mask > 0).mean())
cv2.imwrite("coffee_dark_mask.png", mask)
```

Expected Output: dark pixel fraction: ~0.20 (depending on the image) and a mask highlighting the darkest regions.

Example 3: Alpha extraction

```
import cv2, numpy as np

# Build BGRA image
bgra = np.zeros((100, 100, 4), dtype=np.uint8)
bgra[:, :, 2] = 255          # red
bgra[:, :, 3] = np.tile(np.linspace(0, 255, 100, dtype=np.uint8), (100, 1))

bgr = cv2.cvtColor(bgra, cv2.COLOR_BGRA2BGR)
alpha = bgra[..., 3]
print("bgr shape:", bgr.shape, "alpha shape:", alpha.shape)
```

Expected Output:

```
bgr shape: (100, 100, 3) alpha shape: (100, 100)
```

4. Parameter Sensitivity

(Each code is a fixed conversion — no params.)

5. Common Mistakes

Mistake	What Happens	Fix
Wrong code direction	Wrong color or shape	Read direction in the flag name: BGR2GRAY not GRAY2BGR
Converting an already-gray image to gray	OpenCV may error or no-op	Check img.ndim first
Forgetting the result shape changes	Crash on later code expecting (H, W, 3)	Branch on shape after conversion

6. Practice Tasks

- Convert coffee() to grayscale and back to BGR; compare round-trip.
- Build a quick HSV → mask → result pipeline in 3 lines.
- Extract alpha from a PNG-with-alpha and save it as a grayscale image.

7. Key Takeaways

- cv2.cvtColor(src, code) covers nearly all color conversions.
- Remember code direction (SRC2DST).
- BGR2GRAY is the standard grayscale conversion.
- GRAY2BGR just replicates — round-trip is lossy.

8. What's Next

Color picking — turning an HSV understanding into actual object detection.

Lesson 6: Color Picking for Object Detection

Module: 4 — Color Spaces

Duration: 30 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Pick a target color from an image interactively.
- Choose HSV ranges that capture an object across lighting variation.
- Build an end-to-end detect-and-highlight pipeline.

Prerequisites

- Module 4 Lesson 5

1. Why This Matters

Hand-tuning HSV ranges is the most common debug step in color-based detection. Knowing a repeatable picker workflow saves hours.

2. Concept

2.1 The picker workflow

- Sample: take a small patch of pixels from a representative object.
- Convert to HSV.
- Measure: compute min/max/mean of each channel across the patch.
- Pad: widen the range a bit to handle variation.
- Apply: `cv2.inRange` over the whole image.
- Inspect: are too many or too few pixels selected? Iterate.

2.2 Sampling helper

```
def sample_hsv(img_bgr, x, y, half=5):
    patch = img_bgr[y - half:y + half, x - half:x + half]
    hsv = cv2.cvtColor(patch, cv2.COLOR_BGR2HSV).reshape(-1, 3)
    h, s, v = hsv[:, 0], hsv[:, 1], hsv[:, 2]
    return {"h": (int(h.min()), int(h.max())),
            "s": (int(s.min()), int(s.max())),
            "v": (int(v.min()), int(v.max())),
            "mean": (int(h.mean()), int(s.mean()), int(v.mean()))}
```

2.3 Range padding heuristic

For a sampled mean and spread:

```
H ± 10
S ∈ [max(50, S_min - 30), 255]
V ∈ [max(50, V_min - 30), 255]
```

These are starting values — tune per image.

2.4 Interactive picker with mouse callback

```
import cv2

img = cv2.imread("photo.jpg")
def on_mouse(event, x, y, flags, param):
    if event == cv2.EVENT_LBUTTONDOWN:
        sample = sample_hsv(img, x, y)
        print(sample)

cv2.namedWindow("pick")
cv2.setMouseCallback("pick", on_mouse)
while True:
    cv2.imshow("pick", img)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cv2.destroyAllWindows()
```

Click on the object to get its HSV ranges; ctrl-C to exit.

2.5 Mask post-processing

After `cv2.inRange`, masks often have small holes / specks. Clean with morphology (preview of Module 10):

```
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel) # remove specks
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel) # fill holes
```

2.6 Highlighting detection

Common patterns:

- `cv2.bitwise_and(img, img, mask=mask)` — show only the object.
- `cv2.bitwise_not(mask)` and apply to background — desaturate / grey-out everything else.
- Draw bounding box around the largest contour of the mask (Module 12).

3. Code Examples

Example 1: Detect red apples — full pipeline

```
import cv2, numpy as np
from skimage.data import coffee # stand-in: red bean tones

bgr = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)

# red ranges
m1 = cv2.inRange(hsv, (0, 80, 50), (10, 255, 255))
m2 = cv2.inRange(hsv, (170, 80, 50), (179, 255, 255))
mask = cv2.bitwise_or(m1, m2)

# clean
```

```

kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)

# highlight: keep red, darken the rest
dark_bg = (bgr * 0.3).astype(np.uint8)
fg = cv2.bitwise_and(bgr, bgr, mask=mask)
bg = cv2.bitwise_and(dark_bg, dark_bg, mask=cv2.bitwise_not(mask))
result = cv2.add(fg, bg)
cv2.imwrite("coffee_red_highlight.png", result)
print("red pixel fraction:", (mask > 0).mean())

```

Expected Output: coffee_red_highlight.png — the original image but with anything not-red darkened to 30% brightness; reds pop out.

Example 2: HSV sampler function in action

```

import cv2, numpy as np
from skimage.data import astronaut

bgr = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)

def sample_hsv(img_bgr, x, y, half=5):
    patch = img_bgr[y - half:y + half, x - half:x + half]
    hsv = cv2.cvtColor(patch, cv2.COLOR_BGR2HSV).reshape(-1, 3)
    h, s, v = hsv[:, 0], hsv[:, 1], hsv[:, 2]
    return dict(h_min=int(h.min()), h_max=int(h.max()),
                s_min=int(s.min()), s_max=int(s.max()),
                v_min=int(v.min()), v_max=int(v.max()))

# Sample a known patch (somewhere on the orange helmet area)
sample = sample_hsv(bgr, x=250, y=180)
print("sample:", sample)

```

Expected Output (something like):

```

sample: {'h_min': 5, 'h_max': 22, 's_min': 120, 's_max': 210, 'v_min': 100, 'v_max': 200}

```

Use these as the basis for a tuned cv2.inRange call.

Example 3: Mask cleanup demo

```

import cv2, numpy as np

# Noisy mask
np.random.seed(0)
mask = (np.random.rand(200, 200) < 0.2).astype(np.uint8) * 255
cv2.imwrite("mask_raw.png", mask)

k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
cleaned = cv2.morphologyEx(mask, cv2.MORPH_OPEN, k)
cv2.imwrite("mask_opened.png", cleaned)

print("raw white pixels: ", (mask > 0).sum())
print("opened white pixels: ", (cleaned > 0).sum())

```

Expected Output: Far fewer "white" pixels after opening — small specks removed.

4. Parameter Sensitivity

Adjustment	Effect
Widen H by ± 5	Catch more shades
Lower S min	Include washed colors (and grays!)
Lower V min	Include shadowed object parts (and darkness)
Bigger morph kernel	Remove more specks but eat into the object

5. Common Mistakes

Mistake	What Happens	Fix
Tuning H way too tight	Misses the object under different lighting	Sample multiple patches; pad
S, V min = 0	Lots of false positives	Set ≥ 50 –100
Skipping morphology	Mask is noisy, lots of specks	Open then close
Sampling a single pixel	Bad statistics	Sample a 10×10 patch min, mean, max

6. Practice Tasks

- Take any image with a clearly red object. Sample its center; print HSV stats.
- Tune an inRange to detect that object; show the masked-out highlight.
- Apply MORPH_OPEN then MORPH_CLOSE and compare the mask before/after.

7. Key Takeaways

- Sample a patch (not a pixel) to get range stats.
- Pad H by ± 10 , set S/V mins ≥ 50 –100.
- Always clean the mask with morphology.
- Iterate visually — color picking is an empirical loop.

8. What's Next

End of Module 4. Next: Module 5 — Geometric Transformations.

Module 4 — Exercises

18 exercises (3 per lesson).

4.1 Why

- (E) For pure red BGR (0,0,255), predict and verify HSV.
- (M) Predict that two patches of red (dark/bright) have the same H. Verify.
- (H) Identify which color space you'd use to: (a) detect a green ball, (b) compare two grays for similarity, (c) compress chroma. Justify each.

4.2 RGB/BGR

- (E) Load astronaut.png with cv2.imread; display with matplotlib without conversion.
- (M) Convert and display correctly.
- (H) Build a show(img) helper that auto-converts BGR before display.

4.3 HSV

- (E) Compute HSV for orange (0, 128, 255) BGR.
- (M) Build a blue mask on astronaut() (the suit).
- (H) Build a red mask with wraparound on coffee().

4.4 LAB / YCrCb

- (E) Convert coffee() to LAB; print mean of L channel.
- (M) Build a YCrCb skin mask on astronaut(); print coverage.
- (H) Find nearest of [red, green, blue] palette colors to off-orange via LAB distance.

4.5 cvtColor

- (E) Convert coffee() to gray and back to BGR. Are they identical? Why not.
- (M) Build a 3-step pipeline: BGR → HSV → inRange (dark) → save mask.
- (H) Extract the alpha channel from a PNG-with-alpha. Save as grayscale.

4.6 Color Picking

- (E) Sample a 10×10 HSV patch from any image. Print min/max/mean per channel.
- (M) Tune inRange to detect a red object in your own photo.
- (H) Apply MORPH_OPEN + MORPH_CLOSE to clean a noisy mask; compare counts.

Module 4 — Quiz

10 MCQs.

Q1

OpenCV's H channel range is: A. 0–360 B. 0–255 C. 0–179 D. 0–100

Answer: C (L3)

Q2

The S channel measures: A. brightness B. saturation C. hue D. shadow

Answer: B (L3)

Q3

For red detection in OpenCV HSV, you typically use: A. one range around H=0 B. one range around H=179 C. two ranges (around 0 and 179) OR'd D. H=120

Answer: C — red wraps. (L3)

Q4

LAB's L channel is: A. red↔green B. blue↔yellow C. lightness D. saturation

Answer: C (L4)

Q5

For perceptual color distance, use: A. RGB B. HSV C. LAB D. BGR

Answer: C (L4)

Q6

For skin detection, a good color space is: A. RGB B. YCrCb C. CMYK D. HSV palette only

Answer: B (L4)

Q7

cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) produces: A. (H, W, 3) B. (H, W) C. (H, W, 1) D. error

Answer: B (L5)

Q8

cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR): A. recovers original color B. replicates gray into 3 channels C. errors D. inverts

Answer: B (L5)

Q9

To detect a specific color robustly across lighting, the best space is: A. RGB B. HSV C. LAB D. YCrCb

Answer: B (L1, L3)

Q10

Why include lower S/V bounds in cv2.inRange? A. Speeds up B. Excludes grays / shadows C. Saves memory D. Required by API

Answer: B (L6)

Module 4 — Assignment: Color-Based Object Highlighter

Estimated time: 2 hours

Combines: Module 4 + Modules 2-3

Brief

CLI tool that takes an image and a target color name (red/green/blue/yellow/orange), detects all matching regions in HSV, and produces an output where the rest of the image is desaturated.

Requirements

- `highlight.py INPUT --color red|green|blue|yellow|orange [--output OUT] [--desaturate 0.3]`
- Hardcoded HSV ranges per color (your tuned values).
- Build the mask, clean with morphology open + close.
- Compose output:
- Where mask is 255: original pixels.
- Where mask is 0: $\text{original} \times \text{--desaturate}$ (default 0.3 = 30% brightness).
- Print: pixel coverage (% of image mask covers).

Constraints

- Module 4 + previous concepts only.
- Use named constants for color HSV ranges (dict).
- Handle the red wraparound.

Expected Behaviour (sample)

```
$ python highlight.py coffee.png --color red
saved -> coffee_highlight.png
red coverage: 8.2%
```

The output PNG shows the red regions of the coffee bean at full brightness; everything else dimmed.

Grading Rubric

Criterion	Weight
All 5 colors mapped	25%
Red wraparound handled	15%
Morphology cleanup	15%
Composite correct	25%
Style + argparse	20%

Submission

Save as `assignment_module4.py`.

Module 4 — Mini Project: HSV Color Picker (Trackbars)

Overview

Interactive cv2-based tool: load an image and adjust H/S/V min/max via trackbars. The mask and the masked-output update in real time. Once tuned, the tool prints the chosen range to stdout for re-use.

Concepts Used

- BGR → HSV (M4 L3)
- cv2.inRange masking (M3 L6 + M4 L3)
- cv2.imshow + trackbars (M2 L3)

Features

- 6 trackbars: H_low, H_high, S_low, S_high, V_low, V_high.
- Real-time mask preview.
- Real-time "masked image" preview.
- Press p → print current range to console.
- Press q → quit.

Starter Code

hsv_picker.py:

```
import sys
import cv2
import numpy as np

def nothing(_):
    pass

def main(path):
    img = cv2.imread(path)
    if img is None:
        raise FileNotFoundError(path)
    img = cv2.resize(img, None, fx=0.5, fy=0.5) if max(img.shape) > 800 else img
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

    cv2.namedWindow("controls")
    for name, init in [("H_low", 0), ("H_high", 179),
                      ("S_low", 0), ("S_high", 255),
                      ("V_low", 0), ("V_high", 255)]:
        cv2.createTrackbar(name, "controls",
                          init, 255 if "H" not in name else 179, nothing)

    while True:
        lo = (cv2.getTrackbarPos("H_low", "controls"),
              cv2.getTrackbarPos("S_low", "controls"),
              cv2.getTrackbarPos("V_low", "controls"))
        hi = (cv2.getTrackbarPos("H_high", "controls"),
```

```

cv2.getTrackbarPos("S_high", "controls"),
cv2.getTrackbarPos("V_high", "controls"))

mask = cv2.inRange(hsv, lo, hi)
masked = cv2.bitwise_and(img, img, mask=mask)
combo = np.hstack([img, cv2.cvtColor(mask, cv2.COLOR_GRAY2BGR), masked])
cv2.imshow("controls", combo)

key = cv2.waitKey(30) & 0xFF
if key == ord('q'):
    break
if key == ord('p'):
    print(f"low={lo} high={hi}")

cv2.destroyAllWindows()

if __name__ == "__main__":
    main(sys.argv[1] if len(sys.argv) > 1 else "datasets/starter/coffee.png")

```

Expected Interaction

```

$ python hsv_picker.py datasets/starter/coffee.png
(window opens; drag trackbars to find red regions; press p)
low=(0, 80, 50) high=(10, 255, 255)
press q to quit

```

Extension Challenges

- Add a second set of trackbars and OR the two masks (helps with the red wraparound).
- Save the current mask and result when s pressed.
- Auto-load multiple images and let user cycle with arrow keys.

Grading Rubric

Criterion	Weight
Trackbars work	25%
Real-time preview	25%
Mask + masked side-by-side	20%
Print key (p)	15%
Code clarity	15%

Python E-Course

Module 5 — Geometric Transformations

Detailed Notes

Module Overview

Level: Color & Geometry

Duration: ~1 week

Prerequisites: Modules 1-4

What You'll Learn

- Translate, rotate, and scale images correctly.
- Build and apply 2×3 affine transforms.
- Use perspective (4-point) transforms for document scanning.
- Compare interpolation methods (NEAREST, LINEAR, CUBIC, LANCZOS).
- Resize, crop, and flip.

Lessons

#	Lesson	Duration
1	Translation, Rotation, Scaling	30 min
2	The 2×3 Affine Matrix	30 min
3	Perspective Transformation	35 min
4	Resizing and Interpolation	30 min
5	Flip, Crop, Pad	20 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 6: Image Enhancement

Lesson 1: Translation, Rotation, Scaling

Module: 5 — Geometric Transformations

Duration: 30 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Build translation matrices.
- Rotate images with `cv2.getRotationMatrix2D` + `cv2.warpAffine`.
- Scale via `cv2.resize` or the affine matrix.
- Anticipate clipping when transforms push pixels off the canvas.

Prerequisites

- Module 1 coordinate systems

1. Why This Matters

Translating / rotating / scaling are the most common geometric ops — for data augmentation, alignment, and display. The math (a 2×3 matrix) is simple; the gotchas (canvas size, center of rotation) cause most bugs.

2. Concept

2.1 Translation = adding (tx, ty)

Matrix form:

$$M = \begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \end{bmatrix}$$

In code:

```
M = np.float32([[1, 0, tx], [0, 1, ty]])
out = cv2.warpAffine(img, M, (w, h))
```

(w, h) is the output size in pixels.

2.2 Rotation around a point

```
M = cv2.getRotationMatrix2D(center=(cx, cy), angle=30, scale=1.0)
out = cv2.warpAffine(img, M, (w, h))
```

- Angle in degrees, counter-clockwise.
- scale lets you zoom in/out simultaneously.

Default rotation around the centre:

```
h, w = img.shape[:2]
M = cv2.getRotationMatrix2D((w / 2, h / 2), 30, 1.0)
```

2.3 Canvas size after rotation

Rotated content often falls outside the original canvas. To compute the bounding box that fits the whole rotated image:

```
import numpy as np
def rotate_keep(img, angle):
    h, w = img.shape[:2]
    M = cv2.getRotationMatrix2D((w/2, h/2), angle, 1.0)
    cos, sin = abs(M[0, 0]), abs(M[0, 1])
    new_w = int(h * sin + w * cos)
    new_h = int(h * cos + w * sin)
    M[0, 2] += (new_w / 2) - w / 2      # shift to centre in new canvas
    M[1, 2] += (new_h / 2) - h / 2
    return cv2.warpAffine(img, M, (new_w, new_h))
```

2.4 Scaling via cv2.resize

```
out = cv2.resize(img, (new_w, new_h))
out = cv2.resize(img, None, fx=0.5, fy=0.5)
```

For pure scaling, cv2.resize is simpler and faster than warpAffine.

2.5 borderMode — what's outside the image

cv2.warpAffine(..., borderMode=cv2.BORDER_CONSTANT, borderValue=(0, 0, 0))

Default is black. Other useful options:

- BORDER_REPLICATE — copy the edge pixel outward
- BORDER_REFLECT — mirror
- BORDER_WRAP — tile

3. Code Examples

Example 1: Translate by (50, 30)

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
h, w = img.shape[:2]
M = np.float32([[1, 0, 50], [0, 1, 30]])
out = cv2.warpAffine(img, M, (w, h))
cv2.imwrite("translated.png", out)
```

Expected Output: Astronaut shifted 50 px right and 30 px down; the freshly-uncovered area on the top-left is black (border default).

Example 2: Rotate 45° around centre

```
import cv2
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
```



```

h, w = img.shape[:2]
M = cv2.getRotationMatrix2D((w / 2, h / 2), 45, 1.0)
out = cv2.warpAffine(img, M, (w, h))
cv2.imwrite("rotated45.png", out)

```

Expected Output: Astronaut rotated 45° counter-clockwise around the centre, with the four corners pushed off the canvas (black triangles at the corners).

Example 3: Rotate without clipping

```

import cv2, numpy as np
from skimage.data import astronaut

def rotate_keep(img, angle):
    h, w = img.shape[:2]
    M = cv2.getRotationMatrix2D((w/2, h/2), angle, 1.0)
    cos, sin = abs(M[0, 0]), abs(M[0, 1])
    new_w = int(h * sin + w * cos)
    new_h = int(h * cos + w * sin)
    M[0, 2] += (new_w / 2) - w / 2
    M[1, 2] += (new_h / 2) - h / 2
    return cv2.warpAffine(img, M, (new_w, new_h))

out = rotate_keep(cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR), 30)
print("output shape:", out.shape)
cv2.imwrite("rotated30_keep.png", out)

```

Expected Output: A larger canvas with the rotated astronaut fully visible inside. output shape: will be (700-ish, 700-ish, 3) instead of the original 512².

Example 4: Scale by 0.5

```

import cv2
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
small = cv2.resize(img, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)
print(img.shape, "->", small.shape)
cv2.imwrite("coffee_half.png", small)

```

Expected Output:

(400, 600, 3) -> (200, 300, 3)

4. Parameter Sensitivity

Param	Effect
Angle	how much rotation (CCW positive)
Scale (in getRotationMatrix2D)	uniform zoom
Center	pivot point
borderMode	what's outside the source image

5. Common Mistakes

Mistake	What Happens	Fix
Wrong dsize argument order	Stretches one axis	(W, H) not (H, W)
Rotating then losing corners	Image is clipped	Compute new canvas size
Angle in radians	Tiny / weird rotation	OpenCV uses degrees
Translating in wrong direction	Confusion	tx positive = right; ty positive = down

6. Practice Tasks

- Translate astronaut() by 100 px right, 0 down.
- Rotate coffee() 15° around its centre, keep original canvas size.
- Rotate coffee() 30° without clipping — write a rotate_keep helper.

7. Key Takeaways

- Translation matrix: `[[1,0,tx],[0,1,ty]]`.
- Rotation matrix: `cv2.getRotationMatrix2D(center, angle_deg, scale)`.
- `cv2.warpAffine` needs an output `dsize=(W, H)`.
- Rotating without resizing clips; build a new canvas if you need everything visible.

8. What's Next

The 2×3 affine matrix in depth — combining translate / rotate / shear / scale into one.

Lesson 2: The 2×3 Affine Matrix

Module: 5 — Geometric Transformations

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Read the meaning of every number in a 2×3 affine matrix.
- Build affine matrices from 3 source/destination point pairs.
- Combine multiple affine transforms.

Prerequisites

- Module 5 Lesson 1, reference/math-refresher.md §5

1. Why This Matters

An affine transform packs translate + rotate + scale + shear into one matrix. Knowing what each number does lets you debug "why is my warp diagonal", build custom transforms, and compose ops without applying them one at a time.

2. Concept

2.1 The matrix

$$M = \begin{bmatrix} a & b & tx \\ c & d & ty \end{bmatrix}$$

- a, d — scale on x and y respectively (1.0 = no scale).
- b, c — shear.
- tx, ty — translation in pixels.

Mathematically: $[x'; y'; 1] = M \cdot [x; y; 1]$ (with an implicit $[0 \ 0 \ 1]$ row).

2.2 Pure forms

Op	Matrix
Identity	$[[1,0,0],[0,1,0]]$
Translate (tx, ty)	$[[1,0,tx],[0,1,ty]]$
Scale (sx, sy)	$[[sx,0,0],[0,sy,0]]$
Rotate θ (CCW)	$[[\cos\theta,-\sin\theta,0],[\sin\theta,\cos\theta,0]]$
Shear x by k	$[[1,k,0],[0,1,0]]$
Shear y by k	$[[1,0,0],[k,1,0]]$
Reflect over y-axis (flip horizontal)	$[[-1,0,W],[0,1,0]]$

(Reflection requires translating after flipping to keep content visible.)

2.3 From 3 point pairs — cv2.getAffineTransform

If you know where 3 source points should land in the destination, OpenCV finds the unique affine matrix that does it:

```
src = np.float32([[0, 0], [w, 0], [0, h]])          # top-left, top-right, bottom-
left
dst = np.float32([[0, h*0.3], [w*0.8, 0], [w*0.2, h]])
M = cv2.getAffineTransform(src, dst)
out = cv2.warpAffine(img, M, (w, h))
```

Affine has 6 unknowns (the 6 numbers of M); 3 point pairs $\times$ 2 coords = 6 equations. So 3 pairs uniquely define an affine.

For 4+ pairs use cv2.estimateAffinePartial2D / cv2.estimateAffine2D (least-squares fit).

2.4 Composing transforms

Combine ops by multiplying matrices. OpenCV doesn't have a built-in multiply for the 2 $\times$ 3 form (it's a chopped 3 $\times$ 3); the cleanest way is to pad to 3 $\times$ 3, multiply with NumPy, and slice back:

```
def compose(*Ms):
    result = np.eye(3, dtype=np.float32)
    for M in Ms:
        M3 = np.vstack([M, [0, 0, 1]])
        result = M3 @ result
    return result[:2]
```

Then cv2.warpAffine(img, compose(M2, M1), dsize) applies M1 first then M2.

2.5 The order matters

Matrix multiplication is non-commutative. rotate(translate(img)) $\neq$ translate(rotate(img)).

3. Code Examples

Example 1: Custom affine via 3 points

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
h, w = img.shape[:2]

src = np.float32([[0, 0], [w, 0], [0, h]])
dst = np.float32([[40, 30], [w - 10, 60], [60, h - 20]])
M = cv2.getAffineTransform(src, dst)
print("M =\n", M)

out = cv2.warpAffine(img, M, (w, h))
cv2.imwrite("affine_custom.png", out)
```

Expected Output: Astronaut warped — top-left corner pulled inward, top-right pulled down, bottom-left pulled right. Console prints a 2 $\times$ 3 matrix with non-zero tx, ty.

Example 2: Manual shear

```
import cv2, numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
h, w = img.shape[:2]

k = 0.3
M = np.float32([[1, k, 0], [0, 1, 0]])
out = cv2.warpAffine(img, M, (w + int(h * k), h))
cv2.imwrite("sheared.png", out)
```

Expected Output: Image sheared horizontally — top stays put, bottom shifted right (or vice versa depending on k sign). Canvas widened so nothing is clipped.

Example 3: Compose translate + rotate

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
h, w = img.shape[:2]

# 1. translate so the "centre of interest" is at the origin
T1 = np.float32([[1, 0, -w/2], [0, 1, -h/2]])
# 2. rotate by angle around the new origin
ang = np.deg2rad(20)
R = np.float32([[np.cos(ang), -np.sin(ang), 0],
                [np.sin(ang), np.cos(ang), 0]])
# 3. translate back
T2 = np.float32([[1, 0, w/2], [0, 1, h/2]])

def compose(*Ms):
    result = np.eye(3, dtype=np.float32)
    for M in Ms:
        result = np.vstack([M, [0, 0, 1]]) @ result
    return result[:2]

M = compose(T1, R, T2)
out = cv2.warpAffine(img, M, (w, h))
cv2.imwrite("composed.png", out)
```

Expected Output: Same as `cv2.getRotationMatrix2D(center=(w/2, h/2), angle=20, scale=1)` would have done — demonstrates that compose gives you full control.

4. Parameter Sensitivity

Matrix element	Larger value	Smaller value
a (sx)	wider	narrower / squashed
d (sy)	taller	shorter
b (shear x)	more lean to the right	leans left if negative
c (shear y)	leans down to the right	inverse

tx, ty	shift further	less shift
--------	---------------	------------

5. Common Mistakes

Mistake	What Happens	Fix
Pass dst, src in wrong order to getAffineTransform	Inverse transform	Match the call signature
Forget to pad to 3×3 before matrix-multiplying	Wrong composition	Use a helper like compose
Apply rotation without recentring	Image swings off-canvas	Translate to origin, rotate, translate back

6. Practice Tasks

- Build a translate-only affine $[[1,0,40],[0,1,-20]]$. Apply it.
- Use `cv2.getAffineTransform` with three custom point pairs of your choice.
- Compose a 30° rotation around the centre using the compose helper.

7. Key Takeaways

- 2×3 affine: 6 numbers (scale x/y, shear x/y, translate x/y).
- `cv2.getAffineTransform(src, dst)` — solve from 3 point pairs.
- Compose by multiplying 3×3 forms.
- Order matters: translate-rotate ≠ rotate-translate.

8. What's Next

Perspective — when 4 points are needed (affine is too restrictive for tilt).

Lesson 3: Perspective Transformation

Module: 5 — Geometric Transformations

Duration: 35 minutes

Difficulty: Intermediate

Learning Objectives

- Distinguish affine from perspective.
- Build a perspective transform from 4 point pairs.
- Implement a document scanner (4-point straighten).

Prerequisites

- Module 5 Lesson 2

1. Why This Matters

Affine keeps parallel lines parallel. Perspective doesn't — which is what you need when fixing a phone photo of a tilted document: the page's parallel edges converge in the image, and you need to map them back to a rectangle.

2. Concept

2.1 The 3×3 perspective matrix

Where affine is 2×3, perspective is 3×3 with 8 free parameters (the 9th is normalised).

Mathematically:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Then $x_out = x'/w'$, $y_out = y'/w'$. The non-zero g, h are what give perspective its "tilt".

2.2 4 point pairs uniquely define it

```
src = np.float32([[x1,y1], [x2,y2], [x3,y3], [x4,y4]])
dst = np.float32([[X1,Y1], [X2,Y2], [X3,Y3], [X4,Y4]])
M = cv2.getPerspectiveTransform(src, dst)
out = cv2.warpPerspective(img, M, (W, H))
```

(W, H) is the output canvas — typically the size of the rectangle you're mapping to.

2.3 Document-scanner recipe

Goal: photo of a tilted document → flat rectangular scan.

- Detect the 4 corners of the document (manually or with edge detection, M9).
- Order them consistently: top-left, top-right, bottom-right, bottom-left.
- Decide output size — usually (max_width, max_height) from corner distances.

- cv2.getPerspectiveTransform + cv2.warpPerspective.

2.4 Corner ordering helper

A simple recipe: top-left has the smallest x+y, bottom-right the largest. Top-right has the smallest y-x, bottom-left the largest.

```
def order_corners(pts):
    s = pts.sum(axis=1)
    d = np.diff(pts, axis=1).ravel()
    return np.array([
        pts[np.argmin(s)],      # top-left
        pts[np.argmax(d)],      # top-right
        pts[np.argmax(s)],      # bottom-right
        pts[np.argmin(d)],      # bottom-left
    ], dtype=np.float32)
```

2.5 Output size from corners

```
(tl, tr, br, bl) = ordered
width  = int(max(np.linalg.norm(tr - tl), np.linalg.norm(br - bl)))
height = int(max(np.linalg.norm(bl - tl), np.linalg.norm(br - tr)))
```

This preserves the document's actual proportions.

3. Code Examples

Example 1: Bird's-eye warp of an arbitrary 4-point region

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
h, w = img.shape[:2]

# Pick 4 corners of a "trapezoid" in the image
src = np.float32([[100, 100], [400, 80], [450, 380], [50, 400]])
# Map to a flat 300x300 rectangle
dst = np.float32([[0, 0], [300, 0], [300, 300], [0, 300]])

M = cv2.getPerspectiveTransform(src, dst)
out = cv2.warpPerspective(img, M, (300, 300))
cv2.imwrite("warped.png", out)
```

Expected Output: A 300×300 image — the trapezoid region of the astronaut "straightened" into a square.

Example 2: Order corners helper

```
import numpy as np

def order_corners(pts):
    s = pts.sum(axis=1)
    d = np.diff(pts, axis=1).ravel()
    return np.array([
```



```

        pts[np.argmax(s)],
        pts[np.argmax(d)],
        pts[np.argmax(s)],
        pts[np.argmax(d)],
    ], dtype=np.float32)

raw = np.array([[400, 80], [50, 400], [100, 100], [450, 380]])
print(order_corners(raw))

```

Expected Output:

```

[[100. 100.]
 [400.  80.]
 [450. 380.]
 [ 50. 400.]]

```

Same four points, in TL, TR, BR, BL order.

Example 3: Synthetic "document" scanner

```

import cv2, numpy as np

# Build a fake skewed "document" on a background
canvas = np.full((500, 700, 3), 50, dtype=np.uint8)
src_pts = np.float32([[120, 80], [620, 120], [580, 460], [100, 420]])
doc_pts = np.float32([[0, 0], [400, 0], [400, 500], [0, 500]])

# Create a small "page" image to project into the canvas
page = np.full((500, 400, 3), 240, dtype=np.uint8)
cv2.putText(page, "DOCUMENT", (50, 250),
            cv2.FONT_HERSHEY_SIMPLEX, 1.5, (30, 30, 30), 4)

# Project page onto canvas
M_into = cv2.getPerspectiveTransform(doc_pts, src_pts)
warped_page = cv2.warpPerspective(page, M_into, (canvas.shape[1], canvas.shape[0]))
mask = (warped_page.sum(axis=2) > 0).astype(np.uint8) * 255
canvas[mask > 0] = warped_page[mask > 0]
cv2.imwrite("skewed_doc.png", canvas)

# Now "scan": reverse the transform from src_pts -> flat doc_pts
M_back = cv2.getPerspectiveTransform(src_pts, doc_pts)
scanned = cv2.warpPerspective(canvas, M_back, (400, 500))
cv2.imwrite("scanned.png", scanned)

```

Expected Output:

- skewed_doc.png: dark canvas with a skewed white page reading "DOCUMENT".
- scanned.png: 400×500 image of the page straightened out, "DOCUMENT" perfectly readable.

4. Parameter Sensitivity

Param	Effect
dst size (W, H)	Output resolution

corner ordering	Wrong order = mirror / flip
extreme tilt	Bad output — perspective loses precision when source is near-degenerate

5. Common Mistakes

Mistake	What Happens	Fix
Wrong corner order	Output is flipped or twisted	Use <code>order_corners</code>
dst size doesn't match aspect	Stretched output	Compute size from corner distances
Using 3 points (affine sig) for perspective	Wrong function	<code>cv2.getPerspectiveTransform</code> needs 4
Passing int arrays	"wrong type" error	Cast to <code>np.float32</code>

6. Practice Tasks

- Pick 4 points on any image, warp to a 200×200 square.
- Implement `order_corners`; verify on a shuffled 4-point set.
- Build the document-scanner round-trip: skew a page into a canvas, then un-skew.

7. Key Takeaways

- Perspective uses a 3×3 matrix; 4 point pairs define it.
- `cv2.getPerspectiveTransform(src, dst) + cv2.warpPerspective`.
- Document scanning = 4-corner perspective straighten.
- Order corners consistently for predictable output.

8. What's Next

Resizing & interpolation — when shrinking and enlarging, which interpolation should you use?

Lesson 4: Resizing and Interpolation

Module: 5 — Geometric Transformations

Duration: 30 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Use `cv2.resize` correctly (argument order matters).
- Choose the right interpolation: `NEAREST`, `LINEAR`, `CUBIC`, `LANCZOS4`, `AREA`.
- Pick `INTER_AREA` for downsizing, `INTER_CUBIC` / `LANCZOS` for upsizing.

Prerequisites

- Module 5 Lesson 3

1. Why This Matters

Resize is the most-called geometric op — to fit a model input, generate thumbnails, save memory. The default interpolation isn't always right; the wrong choice gives mushy or jaggy output.

2. Concept

2.1 Two signature forms

```
cv2.resize(img, dsize, interpolation=cv2.INTER_LINEAR)
cv2.resize(img, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)
```

`dsize` is (W, H) — opposite of `.shape`. `fx`, `fy` are multiplicative scale factors.

2.2 The 5 interpolation modes

Mode	Cost	Quality going UP	Quality going DOWN
<code>INTER_NEAREST</code>	cheapest	blocky	aliasing
<code>INTER_LINEAR</code> (default)	cheap	smooth, slightly blurry	decent
<code>INTER_CUBIC</code>	medium	sharper than linear	decent
<code>INTER_LANCZOS4</code>	expensive	sharpest	excellent but slow
<code>INTER_AREA</code>	medium	terrible (don't)	best for downsizing

2.3 The shortcut

- Going down (smaller) → `INTER_AREA`.
- Going up (larger) → `INTER_CUBIC` (good default) or `INTER_LANCZOS4` (slow but sharpest).
- Pixel art / masks where blocky is intentional → `INTER_NEAREST`.

2.4 What's actually happening

- `NEAREST` picks the closest source pixel.
- `LINEAR` averages 2×2 source neighbours.

- CUBIC fits a smoother polynomial over 4×4 neighbours.
- LANCZOS4 uses a sinc kernel over 8×8 neighbours.
- AREA averages all source pixels that fall into each destination pixel — proper for downscaling because it's an actual low-pass filter (no aliasing).

2.5 Anti-aliasing on downscale

If you use LINEAR or CUBIC to downscale by a lot, you get aliasing (moiré patterns on fine textures). AREA prevents this.

3. Code Examples

Example 1: Same downscale, different methods

```
import cv2
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
for name, mode in [("NEAREST", cv2.INTER_NEAREST),
                  ("LINEAR", cv2.INTER_LINEAR),
                  ("CUBIC", cv2.INTER_CUBIC),
                  ("AREA", cv2.INTER_AREA),
                  ("LANCZOS", cv2.INTER_LANCZOS4)]:
    small = cv2.resize(img, (64, 64), interpolation=mode)
    cv2.imwrite(f"down_{name}.png", small)
    print(name, small.shape)
```

Expected Output: Five 64×64 thumbnails. NEAREST is blocky and aliased; AREA looks smoothest and faithful; the others are similar to each other (LINEAR slightly softer than CUBIC / LANCZOS).

Example 2: Same upscale

```
import cv2
from skimage.data import coins

img = coins()
small = cv2.resize(img, (64, 64), interpolation=cv2.INTER_AREA)

for name, mode in [("NEAREST", cv2.INTER_NEAREST),
                  ("LINEAR", cv2.INTER_LINEAR),
                  ("CUBIC", cv2.INTER_CUBIC),
                  ("LANCZOS", cv2.INTER_LANCZOS4)]:
    big = cv2.resize(small, (256, 256), interpolation=mode)
    cv2.imwrite(f"up_{name}.png", big)
```

Expected Output: Four 256×256 upscales. NEAREST looks pixelated (each source pixel is a 4×4 block). LINEAR is smooth but soft. CUBIC sharper. LANCZOS sharpest but with the slowest run time.

Example 3: Don't use AREA for upscaling

```
import cv2
from skimage.data import camera
```

```

img = camera()
small = cv2.resize(img, (50, 50), interpolation=cv2.INTER_AREA)

big_area = cv2.resize(small, (400, 400), interpolation=cv2.INTER_AREA)
big_cubic = cv2.resize(small, (400, 400), interpolation=cv2.INTER_CUBIC)
cv2.imwrite("up_area.png", big_area)
cv2.imwrite("up_cubic.png", big_cubic)

```

Expected Output: AREA upscale looks blocky / averaged poorly; CUBIC looks smooth.

Example 4: Maintain aspect ratio

```

import cv2
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
h, w = img.shape[:2]
target_w = 400
ratio = target_w / w
out = cv2.resize(img, (target_w, int(h * ratio)), interpolation=cv2.INTER_AREA)
print(img.shape, "->", out.shape)

```

Expected Output: (400, 600, 3) -> (266, 400, 3) (aspect preserved).

4. Parameter Sensitivity

Mode	Best at	Worst at
NEAREST	Pixel art, mask scaling	Photo upscale
LINEAR	Generic default	Sharp results when upscaling
CUBIC	Photo upscale	Slow for video
LANCZOS4	Highest quality upscale	Slowest
AREA	Photo downscale	Any upscale

5. Common Mistakes

Mistake	What Happens	Fix
(H, W) in dsize	Stretched	(W, H)
LINEAR for huge downscale	Aliasing	AREA
AREA for upscale	Blocky	CUBIC / LANCZOS
Forgetting aspect ratio	Squashed image	Compute one axis from the other

6. Practice Tasks

- Resize astronaut() to 100×100 with each of the 5 modes; compare.
- Upscale that 100×100 back to 512×512 with NEAREST and LANCZOS; compare.
- Resize coffee() so the width is 300, preserving aspect ratio.

7. Key Takeaways

- cv2.resize(..., (W, H), interpolation=...).
- Downscale: INTER_AREA. Upscale: INTER_CUBIC or INTER_LANCZOS4.

- Pixel art / masks: INTER_NEAREST.
- Maintain aspect ratio manually.

8. What's Next

Flip, crop, and pad — the small but ever-present resize cousins.

Lesson 5: Flip, Crop, Pad

Module: 5 — Geometric Transformations

Duration: 20 minutes

Difficulty: Beginner

Learning Objectives

- Flip images horizontally / vertically / both with `cv2.flip`.
- Crop with NumPy slicing.
- Pad with `cv2.copyMakeBorder` and the various border modes.

Prerequisites

- Module 5 Lesson 4

1. Why This Matters

Three small ops you'll use constantly — for data augmentation, batch alignment, and resolving size mismatches before stacking / concatenating.

2. Concept

2.1 Flip

```
cv2.flip(img, 0)    # vertical flip (around x-axis)
cv2.flip(img, 1)    # horizontal flip (around y-axis)
cv2.flip(img, -1)   # both
```

Mnemonic: positive = horizontal mirror (most useful for augmentation).

2.2 Crop = slicing

```
crop = img[y1:y2, x1:x2]
```

Already covered (M3 L3). Worth repeating in the geometry context.

2.3 Pad — `cv2.copyMakeBorder`

```
out = cv2.copyMakeBorder(img, top, bottom, left, right,
                          borderType, value=0)
```

`borderType` options:

Type	Effect
<code>cv2.BORDER_CONSTANT</code>	Solid color (use <code>value=...</code>)
<code>cv2.BORDER_REPLICATE</code>	Repeat the edge pixel outward
<code>cv2.BORDER_REFLECT</code>	Mirror
<code>cv2.BORDER_REFLECT_101</code>	Mirror without repeating the edge (default for many filters)
<code>cv2.BORDER_WRAP</code>	Tile the image

2.4 Resize-with-padding (letterbox)

A common trick: fit an image into a target size by resizing the longest side and padding the rest:

```
def letterbox(img, target=(224, 224), color=(0, 0, 0)):
    th, tw = target
    h, w = img.shape[:2]
    s = min(tw / w, th / h)
    nw, nh = int(w * s), int(h * s)
    resized = cv2.resize(img, (nw, nh), interpolation=cv2.INTER_AREA)
    top = (th - nh) // 2
    bottom = th - nh - top
    left = (tw - nw) // 2
    right = tw - nw - left
    return cv2.copyMakeBorder(resized, top, bottom, left, right,
                              cv2.BORDER_CONSTANT, value=color)
```

This preserves aspect ratio — used a lot in ML pipelines that need a fixed input size.

3. Code Examples

Example 1: Horizontal flip (data augmentation)

```
import cv2
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
mirrored = cv2.flip(img, 1)
cv2.imwrite("astro_flip_h.png", mirrored)
```

Expected Output: Astronaut mirrored left-to-right.

Example 2: All four flips

```
import cv2, numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
out = np.hstack([img, cv2.flip(img, 1), cv2.flip(img, 0), cv2.flip(img, -1)])
cv2.imwrite("flips.png", out)
```

Expected Output: 4-up: original, horizontal mirror, vertical mirror, both axes.

Example 3: Pad with each border type

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
common = dict(top=30, bottom=30, left=30, right=30)
panels = [
    ("CONSTANT", cv2.copyMakeBorder(img, **common,
    borderType=cv2.BORDER_CONSTANT, value=128)),
    ("REPLICATE", cv2.copyMakeBorder(img, **common,
    borderType=cv2.BORDER_REPLICATE)),
    ("REFLECT", cv2.copyMakeBorder(img, **common,
```



```

borderType=cv2.BORDER_REFLECT)),
    ("REFLECT_101", cv2.copyMakeBorder(img, **common,
borderType=cv2.BORDER_REFLECT_101)),
    ("WRAP",          cv2.copyMakeBorder(img, **common, borderType=cv2.BORDER_WRAP)),
]
for name, p in panels:
    cv2.imwrite(f"pad_{name}.png", p)

```

Expected Output: Five files showing the different border behaviours — CONSTANT is gray, REPLICATE stretches the edge outward, REFLECT mirrors, WRAP tiles.

Example 4: Letterbox to square

```

import cv2
from skimage.data import coffee

def letterbox(img, target=(300, 300), color=(0, 0, 0)):
    th, tw = target
    h, w = img.shape[:2]
    s = min(tw / w, th / h)
    nw, nh = int(w * s), int(h * s)
    resized = cv2.resize(img, (nw, nh), interpolation=cv2.INTER_AREA)
    top     = (th - nh) // 2
    bottom  = th - nh - top
    left    = (tw - nw) // 2
    right   = tw - nw - left
    return cv2.copyMakeBorder(resized, top, bottom, left, right,
                              cv2.BORDER_CONSTANT, value=color)

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
out = letterbox(img, (300, 300))
print(out.shape)      # (300, 300, 3)
cv2.imwrite("letterbox.png", out)

```

Expected Output: A 300×300 image with the coffee photo centred and black bars filling the unused vertical space.

4. Parameter Sensitivity

(Each op is its own thing — no shared param table.)

5. Common Mistakes

Mistake	What Happens	Fix
cv2.flip(img, 0) thinking it's horizontal	It's vertical	Remember: flipCode == 1 for horizontal
Pad with negative border size	Crops instead	Use slicing for crop
Letterbox stretching	Aspect ratio lost	Pick s = min(...), not max
BORDER_CONSTANT without value	Black border	Pass value=(B,G,R) for color

6. Practice Tasks

- Flip astronaut() along all 3 axes (h, v, both) and save each.
- Pad coins() with 30 px black borders on all sides.
- Letterbox coffee() into a 256×256 square with white padding.

7. Key Takeaways

- cv2.flip(img, code): 0 vertical, 1 horizontal, -1 both.
- Crop = slicing.
- cv2.copyMakeBorder for padding; many border types.
- Letterbox preserves aspect ratio when fitting to a target size.

8. What's Next

End of Module 5. Next: Module 6 — Image Enhancement — histograms, equalisation, CLAHE.

Module 5 — Exercises

15 exercises (3 per lesson).

5.1 Translate/Rotate/Scale

- (E) Translate astronaut() by (100, 0).
- (M) Rotate coffee() 30° around its centre, keep original canvas.
- (H) Rotate coffee() 30° without clipping (rotate_keep).

5.2 Affine

- (E) Build a translate-only affine $[[1,0,40],[0,1,-20]]$. Apply.
- (M) Use cv2.getAffineTransform with 3 custom point pairs.
- (H) Compose a 30° rotation about the centre with the compose helper.

5.3 Perspective

- (E) Warp a 4-point region into a 200×200 square.
- (M) Implement order_corners; test on a shuffled list.
- (H) Build the document-scanner round-trip (skew then un-skew).

5.4 Resize

- (E) Resize astronaut() to 100×100 with each of NEAREST/LINEAR/CUBIC/AREA/LANCZOS4. Compare.
- (M) Upscale a small image to 512² with NEAREST vs LANCZOS. Compare.
- (H) Resize coffee() so width is 300 and aspect ratio is preserved.

5.5 Flip / Crop / Pad

- (E) Flip astronaut() along all 3 axes.
- (M) Pad coins() with 30 px black borders.
- (H) Letterbox coffee() to a 256×256 square with white padding.

Module 5 — Quiz

10 MCQs.

Q1

cv2.warpAffine expects dsize as: A. (H, W) B. (W, H) C. depends D. (W, H, C)

Answer: B (L1)

Q2

cv2.getRotationMatrix2D(center, angle, scale) angle is in: A. radians B. degrees C. percent D. cm

Answer: B (L1)

Q3

Affine has how many free parameters? A. 4 B. 6 C. 8 D. 9

Answer: B (L2)

Q4

Perspective has how many free parameters? A. 4 B. 6 C. 8 D. 12

Answer: C (L3)

Q5

For downscaling photos, the recommended interpolation is: A. NEAREST B. LINEAR C. AREA D. CUBIC

Answer: C (L4)

Q6

For sharp upscaling, you'd use: A. NEAREST B. LINEAR C. AREA D. LANCZOS4

Answer: D (L4)

Q7

cv2.flip(img, 1) does: A. vertical flip B. horizontal flip C. both axes D. no-op

Answer: B (L5)

Q8

For a document scanner (tilted photo → flat rectangle), use: A. rotation B. affine C. perspective D. resize

Answer: C (L3)

Q9

cv2.getAffineTransform needs how many point pairs? A. 2 B. 3 C. 4 D. 6

Answer: B (L2)

Q10

cv2.copyMakeBorder(..., BORDER_REFLECT) produces: A. solid color B. mirrored edge C. repeated edge pixel D. no border

Answer: B (L5)

Module 5 — Assignment: Image Aligner

Estimated time: 2 hours

Combines: Module 5 + Modules 1-4

Brief

Build a CLI that aligns an input image to a reference by user-supplied corresponding points.

Requirements

- align.py INPUT REFERENCE --points "x1,y1;x2,y2;x3,y3" (3 pairs in input AND 3 in reference, or 4 if --perspective).
- Use cv2.getAffineTransform (default) or cv2.getPerspectiveTransform (--perspective).
- Warp INPUT to align with REFERENCE.
- Save side-by-side comparison: original | aligned | reference.

Constraints

- Module 5 only.
- Use argparse to parse point strings.
- Validate 3 (affine) or 4 (perspective) point pairs.

Expected Behaviour

```
$ python align.py input.png reference.png \  
  --input-points "0,0;500,0;0,500" \  
  --ref-points   "20,40;510,10;30,495"
```

```
saved -> comparison.png  
warp type: affine
```

Grading Rubric

Criterion	Weight
Affine path works	25%
Perspective path works	25%
Point parsing + validation	20%
Side-by-side output	15%
Style + argparse	15%

Submission

Save as assignment_module5.py.

Module 5 — Mini Project: 4-Point Document Scanner

Overview

Build a tool that takes a photo of a tilted document and produces a flat, rectangular "scan". The 4 corners are supplied by the user (click in a window).

Concepts Used

- Perspective transform (M5 L3)
- Mouse callbacks (M2 L3)
- Corner ordering

Features

- scanner.py INPUT_IMAGE
- Opens the image; user clicks 4 corners.
- Auto-orders corners (TL, TR, BR, BL).
- Computes output size from the corner distances.
- Warps to flat rectangle.
- Saves scanned.png.

Starter Code

scanner.py:

```
import sys
import cv2
import numpy as np

def order_corners(pts):
    s = pts.sum(axis=1)
    d = np.diff(pts, axis=1).ravel()
    return np.array([
        pts[np.argmin(s)], pts[np.argmax(d)],
        pts[np.argmax(s)], pts[np.argmin(d)],
    ], dtype=np.float32)

def scan(img, pts):
    ordered = order_corners(np.array(pts))
    tl, tr, br, bl = ordered
    w = int(max(np.linalg.norm(tr - tl), np.linalg.norm(br - bl)))
    h = int(max(np.linalg.norm(bl - tl), np.linalg.norm(br - tr)))
    dst = np.float32([[0, 0], [w, 0], [w, h], [0, h]])
    M = cv2.getPerspectiveTransform(ordered, dst)
    return cv2.warpPerspective(img, M, (w, h))

def run(path):
    img = cv2.imread(path)
    if img is None:
        raise FileNotFoundError(path)
    if max(img.shape) > 1200:
```

```

img = cv2.resize(img, None, fx=0.5, fy=0.5)

pts = []
disp = img.copy()

def on_mouse(event, x, y, flags, param):
    nonlocal disp
    if event == cv2.EVENT_LBUTTONDOWN and len(pts) < 4:
        pts.append([x, y])
        cv2.circle(disp, (x, y), 5, (0, 255, 0), -1)
        if len(pts) > 1:
            cv2.line(disp, tuple(pts[-2]), tuple(pts[-1]), (0, 255, 0), 1)
        if len(pts) == 4:
            cv2.line(disp, tuple(pts[3]), tuple(pts[0]), (0, 255, 0), 1)

cv2.namedWindow("scan")
cv2.setMouseCallback("scan", on_mouse)
while True:
    cv2.imshow("scan", disp)
    if cv2.waitKey(20) & 0xFF == ord('q'):
        break
    if len(pts) == 4:
        cv2.waitKey(500)
        break
cv2.destroyAllWindows()

if len(pts) == 4:
    scanned = scan(img, pts)
    cv2.imwrite("scanned.png", scanned)
    print("saved -> scanned.png shape:", scanned.shape)
else:
    print("need 4 points; quit early")

if __name__ == "__main__":
    run(sys.argv[1] if len(sys.argv) > 1 else "datasets/starter/coffee.png")

```

Expected Interaction

```

$ python scanner.py phone_photo.jpg
(window opens; click 4 corners of document)
saved -> scanned.png shape: (800, 600, 3)

```

Extension Challenges

- Auto-detect 4 corners with edge detection + contour analysis (preview of M9-M12).
- Add adaptive thresholding to the result for "black ink on white paper" look (preview of M11).
- Support saving multi-page PDFs.

Grading Rubric

Criterion	Weight
4-point click works	25%
Correct warp	30%

Corner ordering	15%
Output size from corners	15%
Code clarity	15%

Python E-Course

Module 6 — Image Enhancement

Detailed Notes

Module Overview

Level: Enhancement

Duration: ~1 week

Prerequisites: Modules 1-5

What You'll Learn

- Read and interpret image histograms.
- Apply histogram equalization (global) and CLAHE (adaptive).
- Tune brightness and contrast cleanly.
- Apply gamma correction for perceptual fixes.
- Use LUTs (lookup tables) for arbitrary tone mapping.

Lessons

#	Lesson	Duration
1	Histograms — Reading and Plotting	30 min
2	Histogram Equalization	30 min
3	CLAHE (Adaptive Equalization)	30 min
4	Brightness & Contrast	25 min
5	Gamma Correction	25 min
6	LUTs — Lookup Tables	25 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 7: Smoothing & Filtering

Lesson 1: Histograms — Reading and Plotting

Module: 6 — Image Enhancement

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Compute an image histogram with `cv2.calcHist` or NumPy.
- Plot histograms (grayscale and per-channel color).
- Identify dark / bright / low-contrast / clipped images from their histograms.

Prerequisites

- Modules 1-5, matplotlib basics.

1. Why This Matters

The histogram of an image is its "fingerprint." A glance tells you whether it's too dark, too bright, has low contrast, or has clipped highlights. Every enhancement decision is informed by reading a histogram first.

2. Concept

2.1 What a histogram counts

For each possible intensity (0..255 for uint8), how many pixels have that value. A grayscale histogram is a length-256 array.

2.2 Compute with NumPy

```
hist, edges = np.histogram(img, bins=256, range=(0, 256))
# hist.shape = (256,)
```

2.3 Compute with OpenCV

```
hist = cv2.calcHist([img], channels=[0], mask=None, histSize=[256], ranges=[0, 256])
# hist.shape = (256, 1)
```

`cv2.calcHist` is faster on large images and supports multi-channel directly.

2.4 Plot it

```
import matplotlib.pyplot as plt
plt.plot(hist)
plt.xlim(0, 256); plt.xlabel("intensity"); plt.ylabel("count")
plt.show()
```

2.5 Per-channel for color

```
for i, color in enumerate(["b", "g", "r"]):
    h = cv2.calcHist([img], [i], None, [256], [0, 256])
    plt.plot(h, color=color)
```

2.6 What you're looking for

- Concentrated on the left → image is dark.
- Concentrated on the right → image is bright.
- Narrow range (e.g. 80-180) → low contrast.
- Spike at 0 or 255 → clipping (lost detail in shadows / highlights).
- Bimodal (two peaks) → likely foreground + background — good for thresholding.

2.7 Cumulative distribution function (CDF)

Sum of the histogram. Used by histogram equalization (Lesson 2). For a perfectly uniform distribution, the CDF is a straight diagonal line.

```
cdf = hist.cumsum()
cdf_normalized = cdf * float(hist.max()) / cdf.max()    # for overlaying on the
histogram plot
```

3. Code Examples

Example 1: Grayscale histogram

```
import cv2, numpy as np
import matplotlib.pyplot as plt
from skimage.data import coins

img = coins()
hist = cv2.calcHist([img], [0], None, [256], [0, 256])

plt.plot(hist); plt.xlim(0, 256)
plt.title("coins() histogram"); plt.xlabel("intensity"); plt.ylabel("count")
plt.show()
print("min", img.min(), "max", img.max(), "mean", img.mean())
```

Expected Output: A histogram peaked roughly in the 30-90 intensity range (dark background) with a secondary mode around 130-180 (the coins). Console prints range and mean confirming this.

Example 2: RGB / BGR per-channel histogram

```
import cv2, numpy as np
import matplotlib.pyplot as plt
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)

for i, color in enumerate(["b", "g", "r"]):
    h = cv2.calcHist([img], [i], None, [256], [0, 256])
    plt.plot(h, color=color, label=color)
plt.xlim(0, 256); plt.legend(); plt.title("coffee per-channel"); plt.show()
```

Expected Output: Three overlaid curves; the red channel is shifted toward higher intensities (lots of warm coffee tones), while blue is concentrated lower.

Example 3: Identify a dark image

```
import cv2, numpy as np

# Dim copy of a bright image
img = (cv2.cvtColor(__import__("skimage.data", fromlist=["camera"]).camera(),
                    cv2.COLOR_GRAY2BGR) * 0.3).astype(np.uint8)

print("mean:", img.mean())
# Most pixels in [0..80] -> heavy left skew, "dark image" diagnosis
```

Expected Output: mean: ~35 and a histogram bunched on the left — diagnostic for "needs brightening".

Example 4: CDF overlay

```
import cv2, numpy as np
import matplotlib.pyplot as plt
from skimage.data import coins

img = coins()
hist = cv2.calcHist([img], [0], None, [256], [0, 256]).ravel()
cdf = hist.cumsum()
cdf_n = cdf * hist.max() / cdf.max()

plt.plot(hist, color='gray'); plt.plot(cdf_n, color='red', label="CDF")
plt.xlim(0, 256); plt.legend(); plt.title("coins histogram + CDF"); plt.show()
```

Expected Output: The gray histogram with a red CDF rising from 0 to the histogram's peak; you can read the percentage of pixels below any intensity by looking at the red curve.

4. Parameter Sensitivity

Param	Effect
bins	More bins = finer resolution but spikier
range	Restrict to a sub-band (e.g. only mid-tones)

5. Common Mistakes

Mistake	What Happens	Fix
Wrap arg as img not [img] in cv2.calcHist	TypeError	Wrap in list
Computing histogram of color image as one channel	Wrong / misleading	Per-channel loop or convert to gray
Forgetting xlim(0, 256)	Plot has lots of empty space	Set the limit

6. Practice Tasks

- Plot the grayscale histogram of coins().
- Plot per-channel histograms for coffee().
- Make a copy of camera() darkened by $\times 0.3$. Plot the histogram. Diagnose "dark".

7. Key Takeaways

- Histogram = pixel-value distribution.

- Shape tells you about brightness, contrast, clipping, bimodality.
- `cv2.calcHist([img], [0], None, [256], [0, 256])` is the standard call.
- CDF is the sum — used by equalization.

8. What's Next

Histogram equalization — automatic global contrast boost driven by the CDF.

Lesson 2: Histogram Equalization

Module: 6 — Image Enhancement

Duration: 30 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Apply `cv2.equalizeHist` on grayscale.
- Understand the CDF-based mapping behind it.
- Recognise when global equalization fails (uneven lighting → CLAHE next).

Prerequisites

- Module 6 Lesson 1

1. Why This Matters

A one-line contrast booster. Often the cheapest way to make a dim image readable. But "global" — it can wash out bright regions while fixing dark ones.

2. Concept

2.1 The idea

Map intensities so the histogram becomes as uniform as possible. The mapping function = the normalised CDF.

If your image has 10% of pixels below intensity 50, equalization maps intensity 50 to intensity 25 (10% of 255 $\approx$ 25.5). After the map, the new histogram is roughly flat.

2.2 The math

$$\text{mapping}(v) = \text{round}((\text{CDF}(v) - \text{CDF}_{\min}) / (N - \text{CDF}_{\min}) \times 255)$$

Where N is the total pixel count. OpenCV does this for you.

2.3 The function

```
eq = cv2.equalizeHist(gray)
```

Input must be (H, W) `uint8`.

2.4 Color images — don't equalize per channel

`cv2.equalizeHist` on each B/G/R channel separately produces color shifts. Better:

```
ycc = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)
ycc[..., 0] = cv2.equalizeHist(ycc[..., 0])
out = cv2.cvtColor(ycc, cv2.COLOR_YCrCb2BGR)
```

Equalize only the Y (luma) channel — fixes contrast without color drift.

2.5 Pure-NumPy equivalent

```
def equalize(gray):
    hist, _ = np.histogram(gray, 256, (0, 256))
    cdf = hist.cumsum()
    cdf_m = np.ma.masked_equal(cdf, 0)
    cdf_m = (cdf_m - cdf_m.min()) * 255 / (cdf_m.max() - cdf_m.min())
    cdf_final = np.ma.filled(cdf_m, 0).astype('uint8')
    return cdf_final[gray]
```

2.6 Where it fails

If different regions of the image have very different lighting (bright window, dark interior), global equalization brightens shadows but blows out highlights. CLAHE (Lesson 3) fixes this.

3. Code Examples

Example 1: Basic equalize

```
import cv2, numpy as np
import matplotlib.pyplot as plt
from skimage.data import moon

img = moon()
eq = cv2.equalizeHist(img)

fig, ax = plt.subplots(2, 2, figsize=(10, 8))
ax[0,0].imshow(img, cmap='gray'); ax[0,0].set_title("original")
ax[0,1].imshow(eq, cmap='gray'); ax[0,1].set_title("equalized")
ax[1,0].hist(img.ravel(), 256, [0, 256]); ax[1,0].set_title("orig histogram")
ax[1,1].hist(eq.ravel(), 256, [0, 256]); ax[1,1].set_title("eq histogram")
for a in ax.ravel(): a.set_xlim(0, 256) if "hist" not in a.get_title() else None
plt.tight_layout(); plt.show()
```

Expected Output: Top-left: original moon (mid-gray, low contrast). Top-right: same shape with much more visible craters / shadows. Bottom: histograms — original is narrow / centred; equalized is spread across 0..255.

Example 2: Color equalize via YCrCb

```
import cv2
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
ycc = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)
ycc[..., 0] = cv2.equalizeHist(ycc[..., 0])
out = cv2.cvtColor(ycc, cv2.COLOR_YCrCb2BGR)
cv2.imwrite("coffee_eq.png", out)
```

Expected Output: Coffee image with boosted contrast — shadows darker, highlights brighter, colors approximately preserved (not as washed out as per-channel BGR equalization would be).

Example 3: Per-channel BGR is wrong

```
import cv2, numpy as np
from skimage.data import coffee
```

```
img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
wrong = cv2.merge([cv2.equalizeHist(c) for c in cv2.split(img)])
cv2.imwrite("coffee_eq_wrong.png", wrong)
```

Expected Output: Coffee image with oversaturated, slightly-off colors — proof that per-channel equalization shifts hue.

Example 4: Pure-NumPy version

```
import numpy as np
from skimage.data import moon

def equalize(gray):
    hist, _ = np.histogram(gray, 256, (0, 256))
    cdf = hist.cumsum()
    cdf_m = np.ma.masked_equal(cdf, 0)
    cdf_m = (cdf_m - cdf_m.min()) * 255 / (cdf_m.max() - cdf_m.min())
    return np.ma.filled(cdf_m, 0).astype('uint8')[gray]

eq = equalize(moon())
print(eq.min(), eq.max(), eq.mean())
```

Expected Output: 0 255 ~127 — eq spans full range, mean near the midpoint of the histogram.

4. Parameter Sensitivity

Equalization itself has no params — but YCrCb-based color equalization preserves color much better than naive per-channel.

5. Common Mistakes

Mistake	What Happens	Fix
Equalize each BGR channel	Color shift / oversaturation	Equalize Y of YCrCb, or L of LAB
Apply on already-uniform image	No visible change	Save the cycles
Use on uneven-lit images	Highlights blown out	Use CLAHE (next lesson)
Forget input must be uint8	TypeError	Cast first

6. Practice Tasks

- Equalize moon(); show before/after histograms.
- Equalize the Y channel of coffee() via YCrCb.
- Implement the pure-NumPy equivalent and confirm output matches cv2.equalizeHist.

7. Key Takeaways

- cv2.equalizeHist flattens grayscale histograms based on the CDF.
- Apply on luma (Y) for color images — never per BGR channel.
- Global → great for uniform images, bad for uneven lighting.

8. What's Next

CLAHE — adaptive equalization that handles uneven lighting.

Lesson 3: CLAHE (Adaptive Equalization)

Module: 6 — Image Enhancement

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Apply `cv2.createCLAHE` for adaptive contrast enhancement.
- Tune `clipLimit` and `tileGridSize`.
- Decide when CLAHE beats global `equalizeHist`.

Prerequisites

- Module 6 Lesson 2

1. Why This Matters

CLAHE = Contrast Limited Adaptive Histogram Equalization. It equalizes per-tile so different regions can have different contrast adjustments. Result: a window scene with a bright sky and a dark interior gets a fix that works for both.

2. Concept

2.1 How it works (intuition)

- Split image into small tiles (e.g. 8×8 grid).
- Equalize each tile's histogram independently.
- Bilinearly interpolate between neighbours to avoid visible tile boundaries.
- Clip the histogram peaks before equalization so noise doesn't get amplified.

2.2 OpenCV API

```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
out = clahe.apply(gray)
```

2.3 Params

Param	Effect
<code>clipLimit</code>	How aggressively to clip histogram peaks. Higher = more contrast, more noise. Typical 1.0–4.0.
<code>tileGridSize</code>	Tiles per axis. Smaller (e.g. 8×8) = more local. Larger (e.g. 16×16) = closer to global.

2.4 Color images — Y channel only

Same recipe as plain `equalize`:

```
ycc = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)
clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8))
```

```
ycc[..., 0] = clahe.apply(ycc[..., 0])
out = cv2.cvtColor(ycc, cv2.COLOR_YCrCb2BGR)
```

Or do CLAHE on L of LAB — same idea, slightly different perceptual flavour.

2.5 Why it beats global

Global equalize uses one mapping for the whole image. CLAHE uses a different (but smoothly interpolated) mapping per region — handles a bright window and a dark wall in the same image.

3. Code Examples

Example 1: Global vs CLAHE on uneven lighting

```
import cv2
import matplotlib.pyplot as plt
from skimage.data import moon

img = moon()
eq = cv2.equalizeHist(img)
clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8)).apply(img)

fig, ax = plt.subplots(1, 3, figsize=(12, 4))
ax[0].imshow(img, cmap='gray'); ax[0].set_title("original")
ax[1].imshow(eq, cmap='gray'); ax[1].set_title("equalizeHist")
ax[2].imshow(clahe, cmap='gray'); ax[2].set_title("CLAHE")
for a in ax: a.axis("off")
plt.tight_layout(); plt.show()
```

Expected Output: Three moons. Original is low contrast. equalizeHist over-amplifies the bright crater regions. CLAHE produces a balanced result with subtle detail across the surface.

Example 2: Color CLAHE

```
import cv2
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
L, a, b = cv2.split(lab)
clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8))
L_eq = clahe.apply(L)
out = cv2.cvtColor(cv2.merge([L_eq, a, b]), cv2.COLOR_LAB2BGR)
cv2.imwrite("coffee_clahe.png", out)
```

Expected Output: Coffee photo with localised contrast boost — shadows in the cup's interior become more visible without over-blowing the cup's rim highlights.

Example 3: Tune clipLimit

```
import cv2
from skimage.data import moon

img = moon()
for cl in [1.0, 2.0, 4.0, 8.0]:
```

```
clahe = cv2.createCLAHE(clipLimit=cl, tileGridSize=(8, 8))
cv2.imwrite(f"moon_clahe_cl{int(cl)}.png", clahe.apply(img))
```

Expected Output: Four versions. cl=1.0 is subtle; cl=8.0 is aggressive with visible noise in flat areas.

Example 4: Tune tile size

```
import cv2
from skimage.data import moon

img = moon()
for ts in [(4, 4), (8, 8), (16, 16)]:
    clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=ts)
    cv2.imwrite(f"moon_clahe_t{ts[0]}.png", clahe.apply(img))
```

Expected Output: 4×4 shows visible blocky transitions in flat areas; 16×16 is closer to global; 8×8 is the usual sweet spot.

4. Parameter Sensitivity

Param	Low	High
clipLimit	subtle, no noise	aggressive, can amplify noise
tileGridSize width	very local, blocky	smoother, closer to global

5. Common Mistakes

Mistake	What Happens	Fix
Apply CLAHE per BGR channel	Color shift	Use L of LAB or Y of YCrCb
clipLimit too high	Noise amplified	Try 2.0-3.0 first
tileGridSize too small	Visible tile artefacts	Stick with (8, 8) usually
Input is not uint8	Error	Convert first

6. Practice Tasks

- Apply CLAHE to moon(); compare with global equalizeHist.
- Try clipLimit ∈ {1, 3, 6}; pick which looks best.
- Apply CLAHE to the L channel of coffee() (LAB).

7. Key Takeaways

- cv2.createCLAHE(clipLimit, tileGridSize).apply(gray).
- Defaults: clipLimit 2–4, tile 8×8.
- For color: do CLAHE on Y (YCrCb) or L (LAB), not per BGR.
- Handles uneven lighting that global equalize can't.

8. What's Next

Brightness and contrast — the explicit knobs, often combined with CLAHE.

Lesson 4: Brightness & Contrast

Module: 6 — Image Enhancement

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Adjust brightness and contrast safely with `cv2.convertScaleAbs`.
- Build an interactive trackbar version.
- Understand why "scale and shift" is the simplest contrast model.

Prerequisites

- Module 6 Lesson 3

1. Why This Matters

Brightness/contrast knobs are sometimes all an image needs. The math is one line; the trick is understanding what you're adjusting and clipping correctly.

2. Concept

2.1 Linear adjustment

```
out = clip( $\alpha$  * img +  $\beta$ , 0, 255)
```

- α (alpha) = contrast scale. >1 = more contrast. <1 = less.
- β (beta) = brightness offset (added intensity).

2.2 OpenCV's helper

```
out = cv2.convertScaleAbs(img, alpha=1.5, beta=20)
```

It does the math, clips to 0–255, and casts to `uint8` in one call.

2.3 Why "scale and shift" is the contrast model

Scaling by α stretches the histogram around 0; shifting by β slides it. To centre contrast around mid-gray (128), use:

```
out = clip( $\alpha$  * (img - 128) + 128 +  $\beta$ , 0, 255)
```

(This is essentially what photo apps' "contrast" slider does.)

2.4 Interactive trackbars

```
cv2.createTrackbar("alpha x10", "win", 10, 30, lambda v: None)
cv2.createTrackbar("beta", "win", 0, 100, lambda v: None)
```

Then read with `cv2.getTrackbarPos`.

3. Code Examples

Example 1: Constant brightness boost

```
import cv2
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
out = cv2.convertScaleAbs(img, alpha=1.0, beta=40)
cv2.imwrite("brighter.png", out)
```

Expected Output: Coffee image, +40 on all pixels (saturated at 255 where needed) — looks brighter overall.

Example 2: Contrast scale

```
import cv2
from skimage.data import camera

img = cv2.cvtColor(camera(), cv2.COLOR_GRAY2BGR)
out = cv2.convertScaleAbs(img, alpha=1.6, beta=0)
cv2.imwrite("contrast.png", out)
```

Expected Output: Cameraman with darker shadows and brighter highlights — overall punchier.

Example 3: Contrast around mid-gray

```
import cv2, numpy as np
from skimage.data import camera

img = camera().astype(np.float32)
alpha, beta = 1.6, 0
out = alpha * (img - 128) + 128 + beta
out = np.clip(out, 0, 255).astype(np.uint8)
cv2.imwrite("contrast_centered.png", out)
```

Expected Output: Similar to Example 2 but contrast is symmetric around 128 — useful when you want to preserve overall brightness.

Example 4: Trackbar tuner

```
import cv2
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
cv2.namedWindow("tune")
cv2.createTrackbar("alpha x10", "tune", 10, 30, lambda v: None)
cv2.createTrackbar("beta", "tune", 0, 100, lambda v: None)

while True:
    a = cv2.getTrackbarPos("alpha x10", "tune") / 10
    b = cv2.getTrackbarPos("beta", "tune")
    out = cv2.convertScaleAbs(img, alpha=a, beta=b)
    cv2.imshow("tune", out)
    if cv2.waitKey(30) & 0xFF == ord('q'):
```



```
        break
    cv2.destroyAllWindows()
```

Expected Interaction: A window with two sliders; the image updates live as you drag.

4. Parameter Sensitivity

Param	Lower	Higher
α	flatter / less contrast	punchier; clipping risk
β	darker	brighter; clipping risk

5. Common Mistakes

Mistake	What Happens	Fix
<code>img * 1.5</code> on uint8	float result; not displayable directly	Use <code>cv2.convertScaleAbs</code>
Not clipping after manual math	Bright pixels wrap to 0	Always <code>np.clip(0, 255)</code> before cast
Treating α as additive	Wrong knob	α multiplies, β adds

6. Practice Tasks

- Apply $\alpha=1.5$, $\beta=20$ to `coffee()`. Save.
- Apply $\alpha=0.7$, $\beta=-30$ (less contrast, dim). Save.
- Build the trackbar example and find a setting you like.

7. Key Takeaways

- `out =  $\alpha$  * img +  $\beta$` , then clip + cast.
- `cv2.convertScaleAbs(img, alpha, beta)` handles all of it.
- Centring contrast around 128 preserves average brightness.

8. What's Next

Gamma correction — non-linear brightness adjustment matching human vision.

Lesson 5: Gamma Correction

Module: 6 — Image Enhancement

Duration: 25 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Apply gamma correction.
- Choose $\gamma < 1$ (brighten shadows) vs $\gamma > 1$ (darken / boost highlights).
- Implement gamma using a precomputed LUT for speed.

Prerequisites

- Module 6 Lesson 4

1. Why This Matters

Linear brightness adjustment is rarely what your eye wants. Human vision is non-linear — we discriminate darker tones much better than lighter ones. Gamma correction matches this behaviour. It's also what your monitor does internally to compensate for sRGB encoding.

2. Concept

2.1 The formula

```
out = ((in / 255) **  $\gamma$ ) * 255
```

Per pixel. After clipping and casting to uint8.

- $\gamma = 1.0 \rightarrow$ identity (no change)
- $\gamma < 1.0 \rightarrow$ brighten dark areas (shadows lifted)
- $\gamma > 1.0 \rightarrow$ darken dark areas (more contrast in highlights)

2.2 Visual intuition

- A dark image (mean ~ 50)? Use $\gamma \approx 0.4\text{--}0.6$.
- A washed-out image (mean ~ 190)? Use $\gamma \approx 1.5\text{--}2.2$.

2.3 Implement with a LUT (fast)

For uint8 there are only 256 possible input intensities — compute their mapped outputs once, then index:

```
import numpy as np, cv2

def gamma_correct(img, gamma):
    lut = np.array([(i / 255.0) ** gamma) * 255 for i in range(256)],
                  dtype=np.uint8)
    return cv2.LUT(img, lut)
```

cv2.LUT applies the lookup table to every pixel — much faster than $(img / 255) ** \gamma$ for large images.

2.4 Why sRGB matters

Most images on disk are stored with an implicit gamma (~ 2.2). When you do arithmetic in pixel space (blending, averaging), you're working with the encoded values — not perceived linear light. For perceptually-correct blending, convert to linear (divide by 255, raise to 2.2), do math, convert back. For most apps, the encoded space is fine.

3. Code Examples

Example 1: Brighten shadows

```
import cv2, numpy as np
from skimage.data import camera

def gamma_correct(img, g):
    lut = np.array([(i / 255.0) ** g) * 255 for i in range(256)], dtype=np.uint8)
    return cv2.LUT(img, lut)

img = camera()
for g in [0.4, 0.7, 1.0, 1.5, 2.2]:
    out = gamma_correct(img, g)
    cv2.imwrite(f"camera_g{g}.png", out)
```

Expected Output: Five copies. $g=0.4$ is very bright (shadows lifted heavily). $g=1.0$ is unchanged. $g=2.2$ is very dark.

Example 2: Color gamma

```
import cv2, numpy as np
from skimage.data import coffee

def gamma_correct(img, g):
    lut = np.array([(i / 255.0) ** g) * 255 for i in range(256)], dtype=np.uint8)
    return cv2.LUT(img, lut)

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
cv2.imwrite("coffee_g0_5.png", gamma_correct(img, 0.5))
cv2.imwrite("coffee_g2_0.png", gamma_correct(img, 2.0))
```

Expected Output: Two versions — $g=0.5$ looks washed-out/bright; $g=2.0$ looks dark and moody.

Example 3: Compare gamma vs linear brightness

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
linear = cv2.convertScaleAbs(img, alpha=1.0, beta=50)
gamma = np.array([(i/255.0)**0.5)*255 for i in range(256)], dtype=np.uint8)[img]

print("means: linear", linear.mean(), " gamma", gamma.mean())
```

Expected Output: Both brighten the image, but linear adds 50 everywhere (clipping highlights), while gamma lifts shadows more than mid-tones (no clipping at the top). Means differ slightly.

4. Parameter Sensitivity

γ	Effect
0.3-0.6	strong lift of dark regions
0.7-0.9	mild lift
1.0	no change
1.1-1.5	mild darken of dark regions
1.5-2.5	strong darken; punchier highlights

5. Common Mistakes

Mistake	What Happens	Fix
Forgetting to divide by 255 first	Massive overflow	The formula is $(i/255)^\gamma * 255$
Using gamma on a normalised float	Already correct space — double-correction	Pick one space and stick with it
Doing pixel-by-pixel python loop	Slow	Build a LUT once

6. Practice Tasks

- Apply $\gamma=0.5$ to camera(); save and inspect.
- Apply $\gamma=2.0$; save.
- Implement gamma_correct once, then time it on a 4000×3000 image — should be fast thanks to LUT.

7. Key Takeaways

- Gamma: $\text{out} = (\text{in}/255)^\gamma * 255$.
- $\gamma < 1$ brightens shadows; $\gamma > 1$ darkens / boosts highlights.
- Use a LUT for speed.

8. What's Next

LUTs in general — any tone mapping you can imagine.

Lesson 6: LUTs — Lookup Tables

Module: 6 — Image Enhancement

Duration: 25 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Build a lookup table for any per-pixel function.
- Apply LUTs with `cv2.LUT`.
- Implement common effects: negative, posterize, sepia, threshold-like maps.

Prerequisites

- Module 6 Lesson 5

1. Why This Matters

Any per-pixel function with `uint8` inputs can be precomputed into a 256-entry table. `cv2.LUT` then applies it across the whole image in microseconds, no matter how complex the function.

2. Concept

2.1 The LUT

```
lut = np.array([f(i) for i in range(256)], dtype=np.uint8)
out = cv2.LUT(img, lut)
```

- `f(i)` defines what each input intensity maps to.
- `lut.dtype` must match `img.dtype` (typically `uint8`).
- For color: pass a (256, 3) LUT for per-channel maps, or a single (256,) applied to each channel.

2.2 Useful presets

Negative:

```
lut = np.arange(255, -1, -1, dtype=np.uint8)
```

Posterize (4 levels):

```
n = 4
step = 256 // n
lut = np.array([min((i // step) * step, 255) for i in range(256)], dtype=np.uint8)
```

Sepia (per-channel LUTs for BGR):

```
def sepia(img):
    # Convert to grayscale and tint
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    # Tint: B 50, G 100, R 150 mapping (approximate sepia)
    tint = np.zeros_like(img)
```

```
tint[..., 0] = (gray * 0.40).astype(np.uint8)
tint[..., 1] = (gray * 0.65).astype(np.uint8)
tint[..., 2] = (gray * 0.85).astype(np.uint8)
return tint
```

(For sepia you usually combine LUT with a small linear transform — LUT alone can do it via per-channel LUTs.)

2.3 Why faster than direct math

cv2.LUT is $O(N)$ but with one cheap memory access per pixel. A Python loop over pixels is hundreds of times slower; even vectorised `f(img)` may compute repeated values for the same input intensity. LUT precomputes 256 outputs, then looks them up.

2.4 Per-channel LUTs (color)

```
b_lut = ... # (256,) uint8
g_lut = ...
r_lut = ...
img[..., 0] = cv2.LUT(img[..., 0], b_lut)
img[..., 1] = cv2.LUT(img[..., 1], g_lut)
img[..., 2] = cv2.LUT(img[..., 2], r_lut)
```

Or build a (256, 3) LUT and pass it as one.

3. Code Examples

Example 1: Negative

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
lut = np.arange(255, -1, -1, dtype=np.uint8)
neg = cv2.LUT(img, lut)
cv2.imwrite("astro_negative.png", neg)
```

Expected Output: Photographic negative of astronaut — bright becomes dark and vice versa.

Example 2: Posterize

```
import cv2, numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
n = 4
step = 256 // n
lut = np.array([min((i // step) * step, 255) for i in range(256)], dtype=np.uint8)
post = cv2.LUT(img, lut)
cv2.imwrite("coffee_poster.png", post)
print("unique values per channel after posterize:", len(np.unique(post[..., 0])))
```

Expected Output: Coffee image with banded "posterized" colour regions (only 4 levels per channel). Console prints 4 for unique values.

Example 3: Step-threshold via LUT

```
import cv2, numpy as np
from skimage.data import coins

img = coins()
lut = np.where(np.arange(256) > 128, 255, 0).astype(np.uint8)
binary = cv2.LUT(img, lut)
cv2.imwrite("coins_binary.png", binary)
```

Expected Output: Coins image converted to pure black/white (intensity above 128 → 255; below → 0). Note: same as `cv2.threshold(..., 128, 255, cv2.THRESH_BINARY)`.

Example 4: Sepia-ish tone via per-channel LUT

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
b_lut = np.clip(np.arange(256) * 0.40, 0, 255).astype(np.uint8)
g_lut = np.clip(np.arange(256) * 0.65, 0, 255).astype(np.uint8)
r_lut = np.clip(np.arange(256) * 0.85, 0, 255).astype(np.uint8)

out = img.copy()
out[..., 0] = cv2.LUT(out[..., 0], b_lut)
out[..., 1] = cv2.LUT(out[..., 1], g_lut)
out[..., 2] = cv2.LUT(out[..., 2], r_lut)
cv2.imwrite("astro_sepia.png", out)
```

Expected Output: Astronaut tinted with warm brown / sepia tones — like an old photograph.

4. Parameter Sensitivity

(Any LUT is a 256-entry array — sensitivity = the function you choose.)

5. Common Mistakes

Mistake	What Happens	Fix
LUT not uint8	Wrong output dtype	<code>dtype=np.uint8</code>
LUT not length 256	Error	Build exactly 256 entries
Applying to non-uint8 image	Behavior undefined	Convert input to uint8 first
Building LUT each frame in a loop	Slow	Build once, reuse

6. Practice Tasks

- Build a posterize LUT for 8 levels. Apply to `coffee()`.
- Build a "boost contrast" LUT: stretch range 50..200 to 0..255. Apply to `camera()`.
- Build the sepia 3-channel LUT effect.

7. Key Takeaways

- LUT = 256-entry table mapping input → output for each intensity.
- `cv2.LUT(img, lut)` is the fastest way to apply any per-pixel function.

- Use for negatives, posterize, thresholds, gamma, sepia, custom tone curves.

8. What's Next

End of Module 6. Next: Module 7 — Smoothing & Filtering — the heart of image processing.

Module 6 — Exercises

18 exercises.

6.1 Histograms

- (E) Plot grayscale histogram of `coins()`.
- (M) Plot per-channel histogram of `coffee()`.
- (H) Darken `camera()` $\times 0.3$ and inspect histogram — diagnose "dark".

6.2 Histogram Equalization

- (E) Equalize `moon()`; show before/after.
- (M) Equalize Y of YCrCb on `coffee()`.
- (H) Implement pure-NumPy equalize; compare to OpenCV.

6.3 CLAHE

- (E) Apply CLAHE to `moon()`. Compare with `equalizeHist`.
- (M) Try `clipLimit` $\in \{1, 3, 6\}$.
- (H) Apply CLAHE to L of LAB on `coffee()`.

6.4 Brightness/Contrast

- (E) `cv2.convertScaleAbs(coffee, alpha=1.5, beta=20)`.
- (M) Center contrast around 128 (manual NumPy).
- (H) Build the trackbar tuner.

6.5 Gamma

- (E) $\gamma=0.5$ on `camera()`.
- (M) $\gamma=2.0$ on `coffee()`.
- (H) Implement `gamma_correct` with LUT; measure timing on a big image.

6.6 LUTs

- (E) Photographic negative LUT on `astronaut()`.
- (M) 8-level posterize LUT on `coffee()`.
- (H) 3-channel sepia LUT.

Module 6 — Quiz

10 MCQs.

Q1

A histogram peaked near 0 indicates: A. bright image B. dark image C. high contrast D. low contrast

Answer: B (L1)

Q2

The CDF is: A. cumulative sum of histogram B. derivative C. log D. inverse

Answer: A (L1, L2)

Q3

cv2.equalizeHist works on: A. color images directly B. grayscale uint8 C. float D. binary

Answer: B (L2)

Q4

For color equalization, apply to: A. each BGR channel B. only B C. Y of YCrCb (or L of LAB) D. RGB after conversion

Answer: C (L2)

Q5

CLAHE is better than global equalize when: A. image is dark B. lighting is uneven C. image is small D. image is grayscale

Answer: B (L3)

Q6

clipLimit in CLAHE controls: A. tile size B. how aggressively to limit histogram peaks C. clipping range D. color space

Answer: B (L3)

Q7

cv2.convertScaleAbs(img, alpha=1.5, beta=20): A. $(1.5 \cdot \text{img} + 20)$ clipped to 0..255 as uint8 B. multiplies hue C. converts color space D. blurs

Answer: A (L4)

Q8

For gamma $\gamma < 1$, the result is: A. darker B. brighter (shadows lifted) C. inverted D. unchanged

Answer: B (L5)

Q9

LUTs are useful because: A. they save memory B. they're vectorised per-pixel mapping C. they handle floats D. they convert formats

Answer: B (L6)

Q10

A "negative" can be done with a LUT containing: A. `np.arange(256)` B. `np.arange(255, -1, -1)` C. `np.zeros(256)` D. `np.ones(256) * 128`

Answer: B (L6)

Module 6 — Assignment: Auto-Enhance Tool

Estimated time: 2 hours

Combines: Module 6 + Modules 4-5

Brief

Build an auto-enhance CLI that picks brightness / contrast / gamma adjustments based on the image's histogram statistics.

Requirements

- `enhance.py INPUT [--output OUT] [--mode auto|clahe|gamma]`
- auto mode:
 - Compute mean and std.
 - If $\text{mean} < 80 \rightarrow$ apply CLAHE on luma (YCrCb Y).
 - If $\text{mean} > 180 \rightarrow$ apply $\gamma = 1.4$.
 - If $80 \leq \text{mean} \leq 180$ and $\text{std} < 40 \rightarrow$ light contrast boost ($\alpha=1.3$, $\beta=0$).
 - Else $\rightarrow$ no-op.
- clahe mode: always apply CLAHE on Y (`clipLimit=3`, `tile=(8,8)`).
- gamma mode: apply γ based on auto rule.
- Print: chosen mode + reasoning + before/after mean/std.
- Save the result.

Constraints

- Module 6 + prior concepts only.
- Use functions named per-step.
- Argparse.

Expected Output

```
$ python enhance.py too_dark.jpg
mean before: 38.4   std before: 22.1
chosen mode: clahe (mean below 80)
mean after : 124.7  std after : 56.3
saved -> too_dark_enhanced.jpg
```

Grading Rubric

Criterion	Weight
Auto-mode logic	30%
CLAHE path	20%
Gamma path	20%
Reporting	15%
Style	15%

Submission

Save as `assignment_module6.py`.

Module 6 — Mini Project: Histogram-Driven Interactive Editor

Overview

A small cv2-window editor with trackbars for brightness, contrast, gamma, and CLAHE. Real-time preview alongside the histogram of the current output.

Concepts Used

- All of Module 6 (histograms, CLAHE, brightness/contrast, gamma).
- Trackbars (M2 L3).

Features

- 5 trackbars: brightness β ($-100..100$), contrast $\alpha \times 10$ ($5..30$), gamma $\times 10$ ($3..30$), CLAHE on/off (0/1), CLAHE clipLimit $\times 10$ ($5..80$).
- Window shows the result + an inline matplotlib-style histogram strip below.
- s saves; q quits.

Starter Code

hist_editor.py:

```
import sys, cv2, numpy as np

def histogram_strip(gray, width):
    hist = cv2.calcHist([gray], [0], None, [256], [0, 256]).ravel()
    hist = (hist / hist.max() * 80).astype(np.int32)
    strip = np.full((100, width, 3), 30, dtype=np.uint8)
    for x in range(256):
        col = int(x * width / 256)
        cv2.line(strip, (col, 100), (col, 100 - int(hist[x])), (200, 200, 200), 1)
    return strip

def apply_pipeline(img, beta, alpha, gamma, clahe_on, clahe_clip):
    out = cv2.convertScaleAbs(img, alpha=alpha, beta=beta)
    lut = np.array([((i/255)**gamma)*255 for i in range(256)], dtype=np.uint8)
    out = cv2.LUT(out, lut)
    if clahe_on:
        ycc = cv2.cvtColor(out, cv2.COLOR_BGR2YCrCb)
        clahe = cv2.createCLAHE(clipLimit=clahe_clip, tileGridSize=(8, 8))
        ycc[..., 0] = clahe.apply(ycc[..., 0])
        out = cv2.cvtColor(ycc, cv2.COLOR_YCrCb2BGR)
    return out

def run(path):
    img = cv2.imread(path)
    if img is None:
        raise FileNotFoundError(path)

    cv2.namedWindow("editor")
    cv2.createTrackbar("brightness+100", "editor", 100, 200, lambda v: None)
    cv2.createTrackbar("contrast x10", "editor", 10, 30, lambda v: None)
```

```

cv2.createTrackbar("gamma x10",      "editor", 10, 30, lambda v: None)
cv2.createTrackbar("clahe (0/1)",    "editor", 0, 1, lambda v: None)
cv2.createTrackbar("clip x10",       "editor", 30, 80, lambda v: None)

while True:
    b = cv2.getTrackbarPos("brightness+100", "editor") - 100
    a = cv2.getTrackbarPos("contrast x10",    "editor") / 10
    g = cv2.getTrackbarPos("gamma x10",      "editor") / 10
    c = cv2.getTrackbarPos("clahe (0/1)",    "editor")
    cl = cv2.getTrackbarPos("clip x10",      "editor") / 10
    out = apply_pipeline(img, b, a, g, c == 1, cl)
    gray = cv2.cvtColor(out, cv2.COLOR_BGR2GRAY)
    strip = histogram_strip(gray, out.shape[1])
    combo = np.vstack([out, strip])
    cv2.imshow("editor", combo)
    k = cv2.waitKey(30) & 0xFF
    if k == ord('q'):
        break
    if k == ord('s'):
        cv2.imwrite("edited.png", out); print("saved -> edited.png")
cv2.destroyAllWindows()

if __name__ == "__main__":
    run(sys.argv[1] if len(sys.argv) > 1 else "datasets/starter/coffee.png")

```

Expected Interaction

```

$ python hist_editor.py coffee.png
(window opens with the image on top and a histogram strip below)
move sliders; histogram updates in real time
press s -> saved -> edited.png
press q -> quit

```

Extension Challenges

- Add a "before/after" toggle (b).
- Add per-channel histograms in the strip.
- Add undo / redo.

Grading Rubric

Criterion	Weight
All trackbars work	25%
Pipeline correct	25%
Histogram strip works	20%
Save / quit	10%
Style	20%

Python E-Course

Module 7 — Smoothing & Filtering

Detailed Notes

Module Overview

Level: Filtering

Duration: ~1.5 weeks

Prerequisites: Modules 1-6

What You'll Learn

- Explain convolution with the sliding-window picture.
- Use box, Gaussian, median, and bilateral filters.
- Understand kernel size and sigma trade-offs.
- Write a pure-NumPy convolution.
- Pick the right filter for noise / smoothness / edge preservation.

Lessons

#	Lesson	Duration
1	Convolution & Correlation	35 min
2	Box / Mean Filter	25 min
3	Gaussian Filter	30 min
4	Median Filter	25 min
5	Bilateral Filter	30 min
6	Comparing Filters	25 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 8: Sharpening

Lesson 1: Convolution & Correlation

Module: 7 — Smoothing & Filtering

Duration: 35 minutes

Difficulty: Intermediate

Learning Objectives

- Visualise convolution as a sliding window of multiply-and-sum.
- Distinguish convolution from correlation.
- Use `cv2.filter2D` for an arbitrary kernel.
- Write a pure-NumPy convolution to demystify what libraries do.

Prerequisites

- Module 1, [reference/math-refresher.md §3](#)

1. Why This Matters

Every linear filter (blur, sharpen, edge) is a convolution with a different kernel. Once you can picture the sliding window, every filter in this module is the same operation with a different small grid of numbers.

2. Concept

2.1 The sliding window picture

For a 3×3 kernel and a small image patch:

Image patch:		Kernel:		Output pixel at the patch centre:
$\begin{bmatrix} p_0 & p_1 & p_2 \\ p_3 & p_4 & p_5 \\ p_6 & p_7 & p_8 \end{bmatrix}$	*	$\begin{bmatrix} k_0 & k_1 & k_2 \\ k_3 & k_4 & k_5 \\ k_6 & k_7 & k_8 \end{bmatrix}$	=	$p_0 \cdot k_0 + p_1 \cdot k_1 + p_2 \cdot k_2 +$ $p_3 \cdot k_3 + p_4 \cdot k_4 + p_5 \cdot k_5 +$ $p_6 \cdot k_6 + p_7 \cdot k_7 + p_8 \cdot k_8$

Slide the kernel one pixel at a time over the image. At each location, multiply element-wise and sum. The result is one output pixel.

2.2 ASCII walkthrough

For 1D simplicity:

```
input :   5 10 15 20 25 30 35 40
kernel:  [1 2 1]   (sum 4 → divide by 4 for average)

position 0 (centred on 10):
  5*1 + 10*2 + 15*1 = 40   /4 = 10

position 1 (centred on 15):
  10*1 + 15*2 + 20*1 = 60  /4 = 15
```

```
position 2 (centred on 20):  
15*1 + 20*2 + 25*1 = 80 /4 = 20
```

The kernel $[1, 2, 1] / 4$ is the simplest weighted blur — it picks up the centre value most strongly.

2.3 Convolution vs Correlation

	Operation	Kernel handling
Correlation	multiply-sum	as-is
Convolution	multiply-sum	flip both axes first

For symmetric kernels (most blurs), they're identical. For directional kernels (Sobel), they differ. OpenCV's `cv2.filter2D` is correlation by default — flip your kernel yourself if you want strict convolution.

2.4 Borders

What happens at the edge, where the kernel hangs off the image? Same `borderType` options as `cv2.copyMakeBorder`:

- `cv2.BORDER_REFLECT_101` (default for most filters) — mirror
- `BORDER_CONSTANT` — pad with 0
- `BORDER_REPLICATE` — repeat edge
- `BORDER_WRAP` — tile

2.5 cv2.filter2D

```
out = cv2.filter2D(img, ddepth=-1, kernel=K)
```

- `ddepth=-1` keeps the input's dtype. Use `cv2.CV_32F` if you need float output (for kernels with negative values).
- `kernel` is a NumPy array, typically float32, summing to 1.0 for true averaging.

2.6 Pure-NumPy convolution (educational)

```
import numpy as np  
  
def convolve2d(img, k):  
    kh, kw = k.shape  
    pad_h, pad_w = kh // 2, kw // 2  
    padded = np.pad(img, ((pad_h, pad_h), (pad_w, pad_w)), mode='reflect')  
    out = np.zeros_like(img, dtype=np.float32)  
    for y in range(img.shape[0]):  
        for x in range(img.shape[1]):  
            patch = padded[y:y+kh, x:x+kw]  
            out[y, x] = (patch * k).sum()  
    return np.clip(out, 0, 255).astype(np.uint8)
```

Slow but illustrative. OpenCV's C-level implementation is hundreds of times faster.

3. Code Examples

Example 1: filter2D with a custom kernel

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
# Simple 3x3 averaging kernel
k = np.ones((3, 3), np.float32) / 9
out = cv2.filter2D(img, -1, k)
cv2.imwrite("box3.png", out)
print("input range:", img.min(), img.max())
print("output range:", out.min(), out.max())
```

Expected Output: A slightly blurred cameraman. Console confirms output stays in 0-255.

Example 2: Pure-NumPy convolve, verify against OpenCV

```
import cv2, numpy as np
from skimage.data import camera

def convolve2d(img, k):
    kh, kw = k.shape
    pad_h, pad_w = kh // 2, kw // 2
    padded = np.pad(img, ((pad_h, pad_h), (pad_w, pad_w)), mode='reflect')
    out = np.zeros_like(img, dtype=np.float32)
    for y in range(img.shape[0]):
        for x in range(img.shape[1]):
            out[y, x] = (padded[y:y+kh, x:x+kw] * k).sum()
    return np.clip(out, 0, 255).astype(np.uint8)

img = camera()[:100, :100] # tiny patch (slow loop)
k = np.ones((3, 3), np.float32) / 9
mine = convolve2d(img, k)
cv2_out = cv2.filter2D(img, -1, k)

print("max abs diff:", np.abs(mine.astype(int) - cv2_out.astype(int)).max())
```

Expected Output: max abs diff: 0 or 1 — tiny rounding difference, otherwise identical.

Example 3: Directional kernel

```
import cv2, numpy as np
from skimage.data import coins

img = coins().astype(np.float32)
sobel_x = np.array([[ -1,  0,  1],
                    [ -2,  0,  2],
                    [ -1,  0,  1]], dtype=np.float32)
out = cv2.filter2D(img, cv2.CV_32F, sobel_x) # float output (has negatives)
out_vis = np.abs(out).clip(0, 255).astype(np.uint8)
cv2.imwrite("sobel_x.png", out_vis)
```

Expected Output: A grayscale image where vertical edges (the coin edges, mostly vertical) are bright. Horizontal edges are dim. (Fully covered in M9.)

Example 4: Border behaviour

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
k = np.ones((11, 11), np.float32) / 121

for border, name in [(cv2.BORDER_REFLECT_101, "reflect101"),
                    (cv2.BORDER_REPLICATE, "replicate"),
                    (cv2.BORDER_CONSTANT, "constant")]:
    out = cv2.filter2D(img, -1, k, borderType=border)
    cv2.imwrite(f"blur_{name}.png", out)
```

Expected Output: Three blurred versions. constant (zero-pad) darkens the edge of the result; replicate and reflect look natural.

4. Parameter Sensitivity

Param	Effect
Kernel sum	=1: brightness preserved. >1: brighter. <1: darker.
Kernel size	Bigger = wider influence per output pixel
ddepth=-1	Output dtype = input

5. Common Mistakes

Mistake	What Happens	Fix
Kernel doesn't sum to 1	Brightness shifts	Normalize: $k / k.sum()$
Use uint8 ddepth for negative kernel	Wraps to garbage	Use cv2.CV_32F, then np.abs if needed
Forget that filter2D is correlation	Wrong direction for Sobel	Flip kernel for strict convolution
Wrong border default	Edges look weird	Stick with BORDER_REFLECT_101

6. Practice Tasks

- Build a 5x5 averaging kernel and apply it to camera().
- Build a 3x3 sharpening kernel: $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$. Apply, save.
- Implement the pure-NumPy convolve2d and verify it matches OpenCV on a small patch.

7. Key Takeaways

- Convolution = sliding-window multiply-sum.
- Convolution vs correlation: flip the kernel (matters only for asymmetric kernels).
- `cv2.filter2D(img, -1, kernel)` is the workhorse.
- Negative kernel values → use CV_32F output then handle the sign.

8. What's Next

The simplest blur — box / mean filter.

Lesson 2: Box / Mean Filter

Module: 7 — Smoothing & Filtering

Duration: 25 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Apply `cv2.blur` and `cv2.boxFilter`.
- Predict the effect of kernel size.
- Recognize why box blur is rarely the best choice.

Prerequisites

- Module 7 Lesson 1

1. Why This Matters

The simplest blur — uniform average over a window. Cheap, easy to reason about, but visually it produces blocky "boxy" softening. It's a useful baseline before reaching for Gaussian.

2. Concept

2.1 The kernel

For a (k, k) box filter, every entry is $1/(k*k)$. A 3×3 box:

```
[ 1/9  1/9  1/9 ]  
[ 1/9  1/9  1/9 ]  
[ 1/9  1/9  1/9 ]
```

Each output pixel is the average of the $k*k$ pixels around it.

2.2 OpenCV functions

```
cv2.blur(img, (k, k))                # normalised box (most common)  
cv2.boxFilter(img, -1, (k, k), normalize=True)  # equivalent  
cv2.boxFilter(img, -1, (k, k), normalize=False) # SUM (not average)
```

`cv2.blur` is the same as `cv2.boxFilter` with `normalize=True`.

2.3 Kernel size

Bigger $k \rightarrow$ more blur, but also slower ($O(k^2)$ per pixel). For very large blurs use Gaussian (separable, faster) or repeat a small box blur.

2.4 Why "boxy"

The kernel has hard edges — pixels just inside the window contribute as much as pixels at the centre. Result: blurring is not smooth. Diagonal lines look stepped.

3. Code Examples

Example 1: Box blur, multiple sizes

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
for k in [3, 7, 15, 31]:
    out = cv2.blur(img, (k, k))
    cv2.imwrite(f"box_k{k}.png", out)
```

Expected Output: Four versions. $k=3$ is barely blurred. $k=31$ looks heavily smoothed; fine detail (hair, helmet text) is gone.

Example 2: blur vs boxFilter

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
a = cv2.blur(img, (5, 5))
b = cv2.boxFilter(img, -1, (5, 5), normalize=True)
print("equal:", np.array_equal(a, b))
```

Expected Output: equal: True

Example 3: Box vs Gaussian (preview of next lesson)

```
import cv2
from skimage.data import camera

img = camera()
box = cv2.blur(img, (15, 15))
gaus = cv2.GaussianBlur(img, (15, 15), 0)

cv2.imwrite("box_15.png", box)
cv2.imwrite("gaus_15.png", gaus)
```

Expected Output: Both blurred but Gaussian looks smoother. Box has a slightly "blocky" quality on diagonal edges.

4. Parameter Sensitivity

Kernel size	Effect
3×3	barely visible smoothing
7×7	clear softening of small detail
15×15	heavy blur; text becomes unreadable
31+	extreme — usually better off with Gaussian σ tuning

5. Common Mistakes

Mistake	What Happens	Fix
---------	--------------	-----

Even-sized kernel	OpenCV anchor offset; works but check	Prefer odd sizes
normalize=False accidentally	Output is SUM, overflows uint8	Use cv2.blur for averaging
Box for "natural" blur	Looks boxy	Use Gaussian

6. Practice Tasks

- Apply box blur sizes 3, 7, 15, 31 to camera(). Compare.
- Verify `cv2.blur == cv2.boxFilter(..., normalize=True)`.
- Time `cv2.blur((31,31))` vs `cv2.GaussianBlur((31,31))` on a 1MP image.

7. Key Takeaways

- Box filter = uniform average.
- `cv2.blur(img, (k, k))` is the standard call.
- Odd kernel sizes; bigger = more blur, slower.
- Less natural-looking than Gaussian — use Gaussian when in doubt.

8. What's Next

Gaussian blur — the smooth, natural-looking default.

Lesson 3: Gaussian Filter

Module: 7 — Smoothing & Filtering

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Apply `cv2.GaussianBlur`.
- Tune kernel size and sigma σ .
- Know that Gaussian is separable ($1D \times 1D$), which makes it fast.

Prerequisites

- Module 7 Lesson 2

1. Why This Matters

Gaussian blur is the default blur for almost everything: pre-smoothing for edge detection (M9), noise reduction (M15), preview thumbnails. It produces visually pleasing, isotropic softening with no boxy artefacts.

2. Concept

2.1 The Gaussian function

In 2D:

$$G(x, y) = (1 / (2\pi\sigma^2)) \cdot \exp(- (x^2 + y^2) / (2\sigma^2))$$

- σ (sigma) controls the width of the bell curve. Bigger σ = wider, smoother blur.
- The kernel is normalised so it sums to 1.

2.2 The kernel visualisation

A 5x5 Gaussian kernel with $\sigma = 1$ looks roughly:

```
[0.003 0.013 0.022 0.013 0.003]
[0.013 0.060 0.098 0.060 0.013]
[0.022 0.098 0.162 0.098 0.022]
[0.013 0.060 0.098 0.060 0.013]
[0.003 0.013 0.022 0.013 0.003]
```

Heavier weight at the centre, falling off smoothly to the edges — that's why it doesn't look boxy.

2.3 OpenCV API

```
out = cv2.GaussianBlur(img, (k, k), sigmaX)
```

- (k, k) kernel size — must be odd.
- `sigmaX = 0` lets OpenCV pick σ from the kernel size: $\sigma = 0.3 * ((k-1)/2 - 1) + 0.8$.
- You can also call `cv2.GaussianBlur(img, (0, 0), sigma)` to let σ drive size.

2.4 Separability — why it's fast

Gaussian is separable: a 2D Gaussian blur is mathematically equivalent to a 1D horizontal Gaussian blur followed by a 1D vertical Gaussian blur. That's $O(2k)$ per pixel instead of $O(k^2)$.

OpenCV does this automatically. To see it explicitly:

```
out = cv2.sepFilter2D(img, -1, kx, ky) # general separable filter
```

2.5 Picking σ

- Small (0.5–1.5) → mild blur, preserves edges decently.
- Medium (2–4) → noise reduction without losing structure.
- Large (5+) → strong smoothing, edges lost.

3. Code Examples

Example 1: Tune sigma

```
import cv2
from skimage.data import camera

img = camera()
for sigma in [0.5, 1.5, 3.0, 6.0]:
    out = cv2.GaussianBlur(img, (0, 0), sigma)
    cv2.imwrite(f"gauss_{sigma}.png", out)
```

Expected Output: Four versions — $\sigma=0.5$ nearly unchanged; $\sigma=6$ heavily smoothed (cameraman becomes blurry).

Example 2: Build the kernel explicitly

```
import cv2, numpy as np

k = 5
sigma = 1.0
kx = cv2.getGaussianKernel(k, sigma) # column (k, 1)
ky = cv2.getGaussianKernel(k, sigma)
g2d = kx @ ky.T # (k, k)
print("kernel sum:", g2d.sum()) # ~1.0
print(g2d.round(3))
```

Expected Output: A 5×5 matrix summing to ~1.0, with the peak at the centre.

Example 3: Verify separability

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
direct = cv2.GaussianBlur(img, (7, 7), 1.5)

k = cv2.getGaussianKernel(7, 1.5)
sep = cv2.sepFilter2D(img, -1, k, k)
```

```
print("max abs diff:", np.abs(direct.astype(int) - sep.astype(int)).max())
```

Expected Output: max abs diff: 0 (or ≤ 1 due to rounding). Same result, both paths.

Example 4: Speed comparison vs box on big kernel

```
import cv2, time
import numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
big = cv2.resize(img, (2048, 2048))

for name, fn in [("box", lambda: cv2.blur(big, (31, 31))),
                 ("gaussian", lambda: cv2.GaussianBlur(big, (31, 31), 0))]:
    t0 = time.perf_counter()
    fn()
    print(f"{name}: {(time.perf_counter() - t0) * 1000:.1f} ms")
```

Expected Output: Both are fast at this kernel size (a few ms each). For much larger kernels (101+) the separability advantage of Gaussian shows more clearly.

4. Parameter Sensitivity

σ	Effect
0.5	Subtle anti-aliasing
1–2	Good for noise reduction
3–4	Heavy smoothing — pre-blur for edge detection
6+	Extreme; almost destroys structure
Kernel size	Effect
Smaller than $\sim 6\sigma$	Truncates the Gaussian; less smooth
$\geq 6\sigma$	Captures essentially all of the Gaussian

Rule of thumb: $k \approx 6\sigma + 1$ (rounded to odd).

5. Common Mistakes

Mistake	What Happens	Fix
Even kernel size	Error	Use odd
sigma = 0 but tiny kernel	OpenCV picks tiny $\sigma \rightarrow$ almost no blur	Pass a sane σ explicitly
Huge kernel, σ too small	Wasted work; same as small kernel	Match $k \approx 6\sigma$
Use box where Gaussian would be cleaner	Boxy artefacts	Use Gaussian by default

6. Practice Tasks

- Apply $\sigma \in \{0.5, 1.5, 3, 6\}$ on camera(); visually compare.
- Build a 7×7 Gaussian kernel from cv2.getGaussianKernel.

- Verify separable form gives the same output as direct GaussianBlur.

7. Key Takeaways

- Gaussian = smooth bell-curve weights, σ controls width.
- `cv2.GaussianBlur(img, (k, k),  $\sigma$ )`.
- Separable: 2D is two 1D passes — fast.
- $k \approx 6\sigma + 1$ as a sizing rule.

8. What's Next

Median filter — non-linear, great for salt-and-pepper noise.

Lesson 4: Median Filter

Module: 7 — Smoothing & Filtering

Duration: 25 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Apply `cv2.medianBlur`.
- Understand why median preserves edges while removing salt-and-pepper.
- Pick kernel size for impulse-noise removal.

Prerequisites

- Module 7 Lesson 3

1. Why This Matters

Salt-and-pepper noise (random pure-white and pure-black pixels) destroys Gaussian / box blur output — the noise is averaged in. Median filter removes it outright while keeping edges sharp. It's the go-to for impulse noise.

2. Concept

2.1 The idea — non-linear

For each pixel, sort the window's pixel values and take the median. Outliers (the salt and pepper) sit at the extremes and never get picked.

```
window: [10, 12, 15, 255, 11, 13, 14, 9, 16]
sorted:  [9, 10, 11, 12, 13, 14, 15, 16, 255]
median: 13      <- the outlier (255) is ignored
```

2.2 OpenCV API

```
out = cv2.medianBlur(img, ksize)
```

`ksize` is the edge length of the square window. Must be odd. For `ksize > 5`, OpenCV uses an optimized histogram-based algorithm.

2.3 Edge preservation

Average-based filters (box, Gaussian) blur edges because they include both sides of an edge in the average. Median picks one side or the other — it doesn't smear.

2.4 Where it fails

- Heavy Gaussian noise (continuous, not impulse) — median doesn't help much.
- Very fine textures — median can wipe them out.
- Large kernel sizes (>9) — gets slow and can over-smooth.

2.5 Comparison context

Filter	Best at	Hurts
Box	Quick blur	Edges, looks boxy
Gaussian	Smooth blur, Gaussian noise	Edges (some)
Median	Salt-and-pepper / impulse noise	Fine textures
Bilateral	Smooth, edge-preserving denoising	Slow

3. Code Examples

Example 1: Add salt-and-pepper, then median

```
import cv2, numpy as np
from skimage.data import camera

img = camera().copy()
noisy = img.copy()
np.random.seed(0)
# 5% white spots, 5% black spots
mask = np.random.rand(*img.shape)
noisy[mask < 0.05] = 0
noisy[mask > 0.95] = 255

cv2.imwrite("cam_noisy.png", noisy)
for k in [3, 5, 7]:
    out = cv2.medianBlur(noisy, k)
    cv2.imwrite(f"cam_median_k{k}.png", out)
```

Expected Output: cam_noisy.png is dotted with pure-white and pure-black specks. cam_median_k3.png removes most specks while keeping the cameraman's outline sharp. k=5/7 removes all noise but slightly soften textures.

Example 2: Median vs Gaussian on noisy image

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
np.random.seed(0)
noisy = img.copy()
mask = np.random.rand(*img.shape)
noisy[mask < 0.05] = 0
noisy[mask > 0.95] = 255

med = cv2.medianBlur(noisy, 3)
gau = cv2.GaussianBlur(noisy, (3, 3), 0)

cv2.imwrite("compare_median.png", med)
cv2.imwrite("compare_gaussian.png", gau)
```

Expected Output: Median: noise gone, edges still sharp. Gaussian: noise smeared into blurry blobs — visibly worse.

Example 3: Larger kernel on heavier noise

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
np.random.seed(0)
noisy = img.copy()
mask = np.random.rand(*img.shape)
noisy[mask < 0.15] = 0
noisy[mask > 0.85] = 255

for k in [3, 5, 7, 9]:
    cv2.imwrite(f"heavy_k{k}.png", cv2.medianBlur(noisy, k))
```

Expected Output: k=3 leaves a fair amount of speckle (since 30% of pixels are noisy, sometimes the median window has more bad than good); k=5–7 cleans it; k=9 starts to soften texture.

4. Parameter Sensitivity

ksize	Effect
3	Removes light impulse noise; preserves edges well
5	Stronger; handles ~10% noise
7	Heavy; handles ~20% noise; some texture loss
9+	Slow; over-smooth; rarely needed

5. Common Mistakes

Mistake	What Happens	Fix
Even ksize	Error	Use odd
Median on color directly	Per-channel median; OK but slight color shift possible	OK in practice; convert to LAB if perfectionist
Median to fix Gaussian noise	Doesn't help much	Use Gaussian / NLM / bilateral (M15)
Massive ksize	Slow + texture loss	Try ksize 3 or 5 first

6. Practice Tasks

- Add 5% salt-and-pepper to camera(). Filter with median(3) and Gaussian(3,3). Compare.
- Try ksize 3, 5, 7 on 10% noise. Find the smallest that cleans it.
- Apply median to a color image (e.g. coffee()). Check for color shifts.

7. Key Takeaways

- Median = pick the middle value of the window.
- Best at salt-and-pepper / impulse noise.

- Preserves edges (unlike averaging filters).
- ksize odd; 3 or 5 is usually enough.

8. What's Next

Bilateral filter — the smart one that combines smoothing with edge preservation for any noise type.

Lesson 5: Bilateral Filter

Module: 7 — Smoothing & Filtering

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Apply `cv2.bilateralFilter`.
- Explain why bilateral preserves edges (the range kernel).
- Tune `d`, `sigmaColor`, `sigmaSpace`.

Prerequisites

- Module 7 Lesson 4

1. Why This Matters

Gaussian smooths but blurs edges. Median preserves edges but only handles impulse noise. Bilateral does both: smooth flat regions while keeping edges crisp. It's the magic filter for portrait skin-smoothing, denoising photos before processing, and cartoon-style effects.

2. Concept

2.1 Two kernels at once

A regular Gaussian weights pixels by spatial distance. The bilateral filter also weights by intensity distance:

$$\text{weight} = G_{\text{spatial}}(dx, dy) \cdot G_{\text{range}}(I_{\text{neighbour}} - I_{\text{centre}})$$

If a neighbour pixel is far in intensity (across an edge), its range weight is near zero — it doesn't influence the output. Result: edges are preserved.

2.2 OpenCV API

```
out = cv2.bilateralFilter(img, d, sigmaColor, sigmaSpace)
```

Param	Meaning
d	Diameter of the spatial neighbourhood. 0 or -1 = computed from sigmaSpace. Typical 5-9 for fast, 15+ for heavy.
sigmaColor	σ of the range Gaussian — how dissimilar (in intensity) a neighbour can be and still count. Bigger = more aggressive smoothing across edges.
sigmaSpace	σ of the spatial Gaussian — how wide the neighbourhood.

2.3 Pick params

- Light smoothing: d=5, sigmaColor=25, sigmaSpace=25
- Heavy "skin-smooth" effect: d=9, sigmaColor=75, sigmaSpace=75
- Even heavier / cartoon-like: d=15, sigmaColor=150, sigmaSpace=150

2.4 Cost

Bilateral is slow — non-separable, expensive math. For previews / 1-MP images it's fine; for video at 1080p you'll need cv2.adaptiveBilateralFilter (not available in all builds) or a downsample-then-bilateral-then-upsample trick.

2.5 Color images

Works directly on BGR — each channel filtered independently in OpenCV's implementation, but using a joint range kernel computed in some color metric. Result preserves color edges well.

3. Code Examples

Example 1: Bilateral vs Gaussian

```
import cv2
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
gaus = cv2.GaussianBlur(img, (9, 9), 5)
bilat = cv2.bilateralFilter(img, 9, 75, 75)
cv2.imwrite("astro_gauss.png", gaus)
cv2.imwrite("astro_bilat.png", bilat)
```

Expected Output: astro_gauss.png softens everything including edges (face features blurry). astro_bilat.png smooths smooth regions (cheeks, helmet plastic) but keeps eyes, mouth, helmet outline crisp.

Example 2: Cartoon-like effect (heavy bilateral + edges)

```
import cv2
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
b = cv2.bilateralFilter(img, 15, 150, 150)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_MEAN_C,
                             cv2.THRESH_BINARY, 9, 8)
edges_bgr = cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)
cartoon = cv2.bitwise_and(b, edges_bgr)
cv2.imwrite("cartoon.png", cartoon)
```

Expected Output: Astronaut as a cartoon — flat color regions (heavy bilateral smoothing) with strong black outlines (adaptive threshold edges).

Example 3: Tune sigmaColor

```
import cv2
from skimage.data import coffee
```

```
img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
for sc in [25, 75, 150]:
    out = cv2.bilateralFilter(img, 9, sc, 50)
    cv2.imwrite(f"coffee_sc{sc}.png", out)
```

Expected Output: Three versions. sc=25 barely smooths across small intensity differences (preserves nearly all detail). sc=150 smooths across larger gradients (looks more painterly).

Example 4: Speed timing

```
import cv2, time
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
img = cv2.resize(img, (1024, 1024))

t = time.perf_counter()
cv2.bilateralFilter(img, 9, 75, 75)
print(f"bilateral d=9 : {(time.perf_counter()-t)*1000:.1f} ms")

t = time.perf_counter()
cv2.bilateralFilter(img, 15, 150, 150)
print(f"bilateral d=15 : {(time.perf_counter()-t)*1000:.1f} ms")

t = time.perf_counter()
cv2.GaussianBlur(img, (15, 15), 0)
print(f"gaussian k=15 : {(time.perf_counter()-t)*1000:.1f} ms")
```

Expected Output (timings vary):

```
bilateral d=9 : 280.0 ms
bilateral d=15 : 720.0 ms
gaussian k=15 : 4.5 ms
```

4. Parameter Sensitivity

Param	Lower	Higher
d	faster, smaller neighbourhood	slower, broader smoothing
sigmaColor	less smoothing across intensity steps (more edges preserved)	smooths across bigger steps (less edge preservation)
sigmaSpace	small region	wider region

Rule of thumb: keep sigmaColor and sigmaSpace roughly equal.

5. Common Mistakes

Mistake	What Happens	Fix
Use on large images at full res in a loop	Painfully slow	Downsample first
sigmaColor too large	Acts like Gaussian (loses edge preservation)	Try 25-75

d very large with small sigmas	Wasted work	Match $d \approx 6 * \text{sigmaSpace}$
Use bilateral as the only denoiser	Misses impulse noise	Combine with median

6. Practice Tasks

- Apply bilateral and Gaussian with similar smoothing strength on astronaut(). Compare edge preservation.
- Build a cartoon effect by combining heavy bilateral + adaptive threshold edges.
- Time bilateral vs Gaussian on a 1MP image — note the order-of-magnitude difference.

7. Key Takeaways

- Bilateral = spatial Gaussian $\times$ range Gaussian.
- Preserves edges while smoothing flat regions.
- Slower than Gaussian by 50-100 $\times$.
- Sweet spot params: $d=9$, $\text{sigmaColor}=75$, $\text{sigmaSpace}=75$ for moderate skin-smooth.

8. What's Next

Compare all four filters side by side.

Lesson 6: Comparing Filters

Module: 7 — Smoothing & Filtering

Duration: 25 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Pick the right filter for the job.
- Build a side-by-side comparison rig.
- Use the comparison rig to debug noise problems.

Prerequisites

- All of Module 7 lessons 2-5.

1. Why This Matters

Knowing the four filters individually is one thing. Knowing which to reach for when you see a particular noise pattern is the practical skill. This lesson is a decision tree + a comparison harness you can keep.

2. Concept

2.1 Decision tree

Is the noise mostly random white/black specks (salt-and-pepper)?

YES -> medianBlur (k=3 or 5)

NO -> Is it grainy / Gaussian-style?

YES -> Need edges sharp?

YES -> bilateralFilter (or NLN, M15)

NO -> GaussianBlur (sigma ~ 1-2)

NO -> Just want a soft preview?

YES -> GaussianBlur or blur

2.2 Summary table

Filter	Cost	Removes	Preserves edges?	Good for
blur (box)	low	mild noise	no	quick preview
GaussianBlur	low	mild Gaussian noise	no	denoise / pre-edge
medianBlur	medium	salt-and-pepper	yes	impulse noise
bilateralFilter	high	heavy noise / smoothing	yes	portraits / cartoons

2.3 The comparison rig

A 5-panel display: original + 4 filters. Helps you eyeball which is best:

```
def compare(img, k=5, sigma=1.5, sigmaColor=75, sigmaSpace=75):  
    box = cv2.blur(img, (k, k))
```

```

    gaus = cv2.GaussianBlur(img, (0, 0), sigma)
    med = cv2.medianBlur(img, k if k % 2 else k + 1)
    bilt = cv2.bilateralFilter(img, 9, sigmaColor, sigmaSpace)
    return np.hstack([img, box, gaus, med, bilt])

```

2.4 Synthetic noise for testing

```

def add_gaussian_noise(img, sigma=20):
    n = np.random.normal(0, sigma, img.shape).astype(np.int16)
    return np.clip(img.astype(np.int16) + n, 0, 255).astype(np.uint8)

def add_salt_pepper(img, p=0.05):
    out = img.copy()
    m = np.random.rand(*img.shape[:2])
    out[m < p] = 0
    out[m > 1 - p] = 255
    return out

```

Use these to demonstrate which filter wins on which noise.

3. Code Examples

Example 1: Compare on Gaussian noise

```

import cv2, numpy as np
from skimage.data import camera

def add_gaussian_noise(img, sigma=20):
    n = np.random.normal(0, sigma, img.shape).astype(np.int16)
    return np.clip(img.astype(np.int16) + n, 0, 255).astype(np.uint8)

img = camera()
noisy = add_gaussian_noise(img, sigma=25)

box = cv2.blur(noisy, (5, 5))
gaus = cv2.GaussianBlur(noisy, (0, 0), 1.5)
med = cv2.medianBlur(noisy, 5)
bilt = cv2.bilateralFilter(noisy, 9, 75, 75)

combo = np.hstack([noisy, box, gaus, med, bilt])
cv2.imwrite("compare_gaussian_noise.png", combo)

```

Expected Output: 5-panel image: noisy, then 4 cleaned versions. Gaussian and bilateral handle Gaussian noise well (bilateral preserves edges better); median is mediocre on this kind of noise.

Example 2: Compare on salt-and-pepper

```

import cv2, numpy as np
from skimage.data import camera

img = camera()
np.random.seed(0)
m = np.random.rand(*img.shape)
sp = img.copy()
sp[m < 0.05] = 0; sp[m > 0.95] = 255

```

```

box = cv2.blur(sp, (5, 5))
gaus = cv2.GaussianBlur(sp, (0, 0), 1.5)
med = cv2.medianBlur(sp, 3)
bilt = cv2.bilateralFilter(sp, 9, 75, 75)

combo = np.hstack([sp, box, gaus, med, bilt])
cv2.imwrite("compare_salt_pepper.png", combo)

```

Expected Output: Same 5-panel. Median wins clearly — the others smear the noise into blurry blobs.

Example 3: Print which params for which task

```

recipes = {
    "preview blur": ("blur", {"ksize": (15, 15)}),
    "pre-edge smoothing": ("Gaussian", {"sigma": 1.4}),
    "salt-and-pepper noise": ("median", {"ksize": 3}),
    "portrait skin smoothing": ("bilateral", {"d": 9, "sigmaColor": 75, "sigmaSpace":
75}),
    "cartoon effect": ("bilateral", {"d": 15, "sigmaColor": 150,
"sigmaSpace": 150}),
}
for k, v in recipes.items():
    print(f"{k:30s} -> {v[0]:10s} {v[1]}")

```

Expected Output: A neat printed table of which filter and params to reach for in common situations.

4. Parameter Sensitivity

(Per-filter — see Lessons 2-5.)

5. Common Mistakes

Mistake	What Happens	Fix
Pick the most expensive filter by default	Slow	Start with Gaussian; escalate if needed
Use one filter for every noise	Sub-optimal	Match filter to noise type
Wrong ksize parity	Error or odd behaviour	Always odd

6. Practice Tasks

- Build a compare function and run on camera() + Gaussian noise.
- Build a compare function and run on camera() + salt-and-pepper.
- Add timing per filter; print them in the comparison.

7. Key Takeaways

- Decision tree: noise type → filter.
- Median for impulse; Gaussian for mild Gaussian; bilateral for heavy + edge-aware.
- Comparison rig is the fastest debugging tool when you're unsure.

8. What's Next

End of Module 7. Next: Module 8 — Sharpening — the opposite operation.

Module 7 — Exercises

18 exercises.

7.1 Convolution

- (E) Apply a 5×5 averaging kernel with `cv2.filter2D` to `camera()`.
- (M) Apply the 3×3 sharpen kernel `[[0,-1,0],[-1,5,-1],[0,-1,0]]`.
- (H) Implement pure-NumPy `convolve2d`; verify against OpenCV on a small patch.

7.2 Box

- (E) Apply `cv2.blur((3,3))` and `(15,15)` to `camera()`; compare.
- (M) Time `cv2.blur((31,31))` on a 2000×2000 image.
- (H) Compare `box(15,15)` vs `Gaussian(15,15)` visually on `astronaut()`.

7.3 Gaussian

- (E) Apply $\sigma \in \{0.5, 2, 5\}$ to `camera()`.
- (M) Build a 7×7 Gaussian kernel from `cv2.getGaussianKernel` and apply via `sepFilter2D`.
- (H) Verify separable form == direct `GaussianBlur`.

7.4 Median

- (E) Add 5% salt-pepper; filter with `median(3)`.
- (M) Compare `median(3)` vs `Gaussian(3,3)` on salt-pepper noise.
- (H) Find smallest `ksize` that cleans 15% noise.

7.5 Bilateral

- (E) Apply `bilateralFilter(d=9, sigmaColor=75, sigmaSpace=75)` to `astronaut()`.
- (M) Tune `sigmaColor` $\in \{25, 75, 150\}$.
- (H) Build a cartoon effect: heavy bilateral + adaptive threshold edges.

7.6 Compare

- (E) Add Gaussian noise; run 4-filter compare rig.
- (M) Add salt-pepper; do the same.
- (H) Time each filter; print "noise type recommendation" table.

Module 7 — Quiz

10 MCQs.

Q1

Convolution differs from correlation by: A. flipping the kernel B. summing C. dot product D. transpose

Answer: A (L1)

Q2

Gaussian blur is fast because it's: A. cached B. separable ($1D \times 1D$) C. in C++ D. uses GPU

Answer: B (L3)

Q3

For salt-and-pepper noise, the right filter is: A. box B. Gaussian C. median D. bilateral

Answer: C (L4)

Q4

For smooth edge-preserving denoising, use: A. box B. Gaussian C. median D. bilateral

Answer: D (L5)

Q5

`cv2.blur(img, (7,7))` is the same as: A. `cv2.GaussianBlur` with $\sigma=7$ B. `cv2.boxFilter(... normalize=True)` C. `cv2.medianBlur` D. `cv2.filter2D` with sharpening

Answer: B (L2)

Q6

The rule of thumb relating Gaussian kernel size to σ : A. $k = \sigma$ B. $k = 3\sigma$ C. $k \approx 6\sigma + 1$ (odd) D. $k = 100$

Answer: C (L3)

Q7

Bilateral params: roughly which orders of magnitude? A. $d=1$, $\sigma=1$ B. $d=9$, $\sigma=75$ C. $d=100$, $\sigma=1000$ D. $d=0$, $\sigma=0$

Answer: B (L5)

Q8

`cv2.medianBlur` ksize must be: A. even B. odd C. multiple of 4 D. ≥ 7

Answer: B (L4)

Q9

A 3×3 kernel of all $1/9$ produces: A. sharpening B. edge detection C. box blur D. negative

Answer: C (L2)

Q10

Box blur looks "boxy" because: A. the kernel has hard edges B. it's only 3×3 C. it's slow D. it doesn't normalise

Answer: A (L2)

Module 7 — Assignment: Adaptive Denoiser

Estimated time: 2 hours

Combines: Module 7 + Modules 2-6

Brief

Build a CLI that auto-picks the right denoising filter based on simple noise diagnostics.

Requirements

- `denoise.py INPUT [--output OUT]`
- Diagnose noise type:
- Count "extreme" pixels (value 0 or 255). If >0.5% of total → impulse noise.
- Estimate Gaussian noise σ by std of a small high-pass filtered region.
- Choose filter:
- Impulse-heavy → `cv2.medianBlur(img, 5)`.
- Mostly Gaussian → `cv2.bilateralFilter(img, 9, 75, 75)`.
- Low overall noise → `cv2.GaussianBlur(img, (3,3), 0)`.
- Print: diagnosis stats + chosen filter.
- Save result.

Constraints

- Modules 1-7 only.
- Use `argparse`.
- Diagnostic functions in their own namespaces.

Expected Output

```
$ python denoise.py noisy.png
extreme pixel %: 6.8
estimated gaussian sigma: 2.1
chosen filter: medianBlur(k=5) (heavy impulse noise)
saved -> noisy_denoised.png
```

Grading Rubric

Criterion	Weight
Diagnosis correct	30%
Right filter chosen	25%
Clear reporting	15%
Code organisation	15%
Argparse + style	15%

Submission

Save as `assignment_module7.py`.

Module 7 — Mini Project: Filter Playground (Trackbars)

Overview

An interactive playground with trackbars for all four smoothing filters: box, Gaussian, median, bilateral. Apply any of them in real time and compare against the original.

Concepts Used

- All Module 7 filters
- Trackbars (M2 L3)

Features

- One trackbar per param: filter type (0..3), kernel size, Gaussian σ , bilateral sigmaColor, bilateral sigmaSpace.
- Side-by-side display: original on the left, filtered on the right.
- s saves; q quits.

Starter Code

filter_playground.py:

```
import sys, cv2, numpy as np

FILTERS = ["box", "gaussian", "median", "bilateral"]

def apply_filter(img, name, k, sigma, sc, ss):
    k = max(1, k)
    if k % 2 == 0: k += 1
    if name == "box":
        return cv2.blur(img, (k, k))
    if name == "gaussian":
        return cv2.GaussianBlur(img, (k, k), sigma)
    if name == "median":
        return cv2.medianBlur(img, k)
    if name == "bilateral":
        return cv2.bilateralFilter(img, k, sc, ss)

def run(path):
    img = cv2.imread(path)
    if img is None: raise FileNotFoundError(path)
    if max(img.shape) > 700:
        img = cv2.resize(img, None, fx=0.5, fy=0.5)

    cv2.namedWindow("filters")
    cv2.createTrackbar("filter 0=box 1=gauss 2=med 3=bilat", "filters", 1, 3,
lambda v: None)
    cv2.createTrackbar("ksize", "filters", 5, 31, lambda v: None)
    cv2.createTrackbar("gauss sigma", "filters", 0, 10, lambda v: None)
    cv2.createTrackbar("bilat sColor", "filters", 75, 200, lambda v: None)
    cv2.createTrackbar("bilat sSpace", "filters", 75, 200, lambda v: None)
```

```

while True:
    f = FILTERS[cv2.getTrackbarPos("filter 0=box 1=gauss 2=med 3=bilat",
"filters")]
    k = cv2.getTrackbarPos("ksize", "filters") or 1
    sg = cv2.getTrackbarPos("gauss sigma", "filters") or 0
    sc = cv2.getTrackbarPos("bilat sColor", "filters")
    ss = cv2.getTrackbarPos("bilat sSpace", "filters")
    out = apply_filter(img, f, k, sg, sc, ss)
    combo = np.hstack([img, out])
    cv2.putText(combo, f"{f} k={k}", (10, 25),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
    cv2.imshow("filters", combo)
    kk = cv2.waitKey(30) & 0xFF
    if kk == ord('q'): break
    if kk == ord('s'): cv2.imwrite("filtered.png", out); print("saved
filtered.png")
    cv2.destroyAllWindows()

if __name__ == "__main__":
    run(sys.argv[1] if len(sys.argv) > 1 else "datasets/starter/coffee.png")

```

Expected Interaction

```

$ python filter_playground.py coffee.png
(window opens; original | filtered side-by-side)
toggle filter trackbar, change ksize, etc.
press s -> saved filtered.png
press q -> quit

```

Extension Challenges

- Add a "synthetic noise" toggle (Gaussian / salt-and-pepper) so you can stress-test filters.
- Add a PSNR / SSIM readout vs original.
- Add a "before / after" toggle key.

Grading Rubric

Criterion	Weight
All 4 filters work	35%
Trackbars responsive	20%
Side-by-side display	15%
Save / quit	10%
Style	20%

Python E-Course

Module 8 — Sharpening

Detailed Notes

Module Overview

Level: Filtering

Duration: ~1 week

Prerequisites: Modules 1-7

What You'll Learn

- Sharpen with the Laplacian and custom kernels.
- Apply unsharp masking (the photographer's trick).
- Understand sharpening = original + amount * (original - blurred).
- Use frequency-domain intuition for high-pass filters.

Lessons

#	Lesson	Duration
1	Why Sharpen?	20 min
2	Laplacian Kernel	30 min
3	Unsharp Masking	30 min
4	High-Pass Filtering	25 min
5	Custom Sharpening Kernels	25 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 9: Edge Detection

Lesson 1: Why Sharpen?

Module: 8 — Sharpening

Duration: 20 minutes

Difficulty: Beginner

Learning Objectives

- Explain what "sharpness" means at the pixel level.
- Identify situations where sharpening helps and where it hurts.
- Predict noise amplification from naive sharpening.

Prerequisites

- Module 7

1. Why This Matters

Sharpening is the photographer's everyday tool — slightly soft images become crisp; out-of-focus regions appear focused (within limits). But sharpening also amplifies noise, so understanding the trade-off matters.

2. Concept

2.1 What sharpness is

Sharpness = high local contrast at edges. A sharp image has steep transitions; a blurry one has gradual ramps. Mathematically, the derivative (or gradient) is large at sharp edges and small in flat regions.

2.2 The core idea

Almost every sharpening method boils down to:

$$\text{sharpened} = \text{original} + \text{amount} \times (\text{high_pass_of_original})$$

Where "high pass" emphasises the rapid-changing parts (edges) and suppresses the smooth parts. Add a scaled high-pass back to the original → edges become exaggerated.

Equivalently:

$$\text{sharpened} = \text{original} + \text{amount} \times (\text{original} - \text{blurred})$$

The blurred image has the high frequencies removed; subtracting recovers them.

2.3 When sharpening helps

- Slightly soft photos (camera focus close but not perfect).
- Down-scaled images (scaling tends to soften).
- Pre-processing before OCR or QR-code detection.

2.4 When sharpening hurts

- Already-sharp images → ringing / halos.
- Noisy images → noise becomes more visible.
- High-ISO photos → grain pops out.

2.5 Halos and ringing

Over-sharpening creates bright/dark halos around edges. This is the easiest way to spot too much sharpening.

3. Code Examples

Example 1: A "soft" image vs a "sharp" image

```
import cv2
from skimage.data import camera

img = camera()
blur = cv2.GaussianBlur(img, (0, 0), 2.0) # simulate softening

# Print pixel-derivative magnitude (a proxy for sharpness)
import numpy as np
def edge_energy(x):
    gx = np.gradient(x.astype(np.float32), axis=1)
    gy = np.gradient(x.astype(np.float32), axis=0)
    return float(np.sqrt(gx**2 + gy**2).mean())

print("sharp original :", edge_energy(img))
print("blurred copy   :", edge_energy(blur))
```

Expected Output:

```
sharp original : 13.4
blurred copy   : 5.8
```

The blurred copy has less than half the average gradient magnitude — quantitatively softer.

Example 2: A sharpening preview (covered in detail in next lessons)

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
blur = cv2.GaussianBlur(img, (0, 0), 1.5)
sharp = cv2.addWeighted(img, 1.5, blur, -0.5, 0)
cv2.imwrite("preview_sharp.png", sharp)
```

Expected Output: Cameraman image with slightly enhanced edges — buildings and figures look crisper.

Example 3: Demonstrate noise amplification

```
import cv2, numpy as np
from skimage.data import camera
```

```

img = camera()
np.random.seed(0)
noisy = np.clip(img + np.random.normal(0, 10, img.shape), 0, 255).astype(np.uint8)

blur = cv2.GaussianBlur(noisy, (0, 0), 1.5)
sharp = cv2.addWeighted(noisy, 1.5, blur, -0.5, 0)

cv2.imwrite("noise_sharp.png", sharp)
print("noisy std :", noisy.std())
print("sharp std :", sharp.std())

```

Expected Output: The "sharp" version has a noticeably higher std (more contrast everywhere — including noise). The image visually looks grainier than noisy.

4. Parameter Sensitivity

(Lessons 2-5 cover specific parameters per technique.)

5. Common Mistakes

Mistake	What Happens	Fix
Sharpening a noisy image first	Noise amplified	Denoise first (M7), then sharpen lightly
Sharpening more than needed	Halos / ringing	Use small amount values
Sharpening already-sharp images	Ugly artefacts	Don't sharpen for the sake of it

6. Practice Tasks

- Compute edge_energy for camera() and a 2σ -blurred copy. Note the difference.
- Apply a light sharpen with addWeighted(img, 1.3, blur, -0.3, 0).
- Add Gaussian noise ($\sigma=10$) before sharpening; observe the noise amplification.

7. Key Takeaways

- Sharpening = original + amount $\times$ (original – blurred).
- Edges have high gradient; sharpening boosts them.
- Noise has high gradient too — sharpening amplifies it.
- Always denoise first if the image is noisy.

8. What's Next

The Laplacian — the simplest second-derivative operator behind sharpening.

Lesson 2: Laplacian Kernel

Module: 8 — Sharpening

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Use `cv2.Laplacian` to compute the second derivative.
- Apply a Laplacian-based sharpening kernel directly.
- Recognize the sign of the Laplacian as the "edge" indicator.

Prerequisites

- Module 7 (convolution), Module 8 Lesson 1.

1. Why This Matters

The Laplacian is the canonical second-derivative operator in image processing. It responds to all edges, regardless of orientation. Many sharpening kernels are just "identity – Laplacian-ish" — knowing the Laplacian unlocks both.

2. Concept

2.1 The math

The Laplacian is the sum of second partial derivatives:

$$\nabla^2 I = \partial^2 I / \partial x^2 + \partial^2 I / \partial y^2$$

For a 3×3 discrete approximation:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad \text{OR (8-conn)} \quad \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Both kernels respond to rapid intensity changes. In flat regions the output is ~0; at edges it spikes positive or negative.

2.2 The OpenCV call

```
lap = cv2.Laplacian(img, cv2.CV_64F, ksize=3)
```

- `ddepth = CV_64F` or `CV_32F` is required — the Laplacian can be negative.
- `ksize=1` uses the 4-connected kernel; larger `ksize` uses Sobel-derived approximations.

2.3 Visualising the Laplacian

```
lap_vis = cv2.convertScaleAbs(lap) # |lap|, clipped to uint8
```

This collapses negative values to positive — good for visualisation, but loses the sign information you need for sharpening.

2.4 Sharpening via Laplacian

$\text{sharpened} = \text{original} - \text{amount} \times \text{Laplacian}$

Because the Laplacian is positive on the dark side of an edge and negative on the bright side, subtracting it pushes edges further apart — making the bright side brighter and the dark side darker.

A common one-shot kernel:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

That's identity (centre=1) – Laplacian (centre=-4) = centre 5, edges -1, scaled. Sums to 1 (preserves brightness).

2.5 Pre-blur to avoid noise

The Laplacian amplifies anything with a high second derivative — including noise. Standard practice: blur slightly first (Gaussian $\sigma \sim 1.0$).

2.6 LoG — Laplacian of Gaussian

Gaussian blur → Laplacian is one common edge / blob detector (preview of M9 / M13). Saves work because Gaussian blur is separable.

3. Code Examples

Example 1: Laplacian visualisation

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
lap = cv2.Laplacian(img, cv2.CV_64F, ksize=3)
lap_vis = cv2.convertScaleAbs(lap)
cv2.imwrite("laplacian.png", lap_vis)
print("lap range:", lap.min(), lap.max())
```

Expected Output: laplacian.png highlights edges — building outlines, the cameraman's silhouette — bright on black background. Console prints negative min, positive max (e.g. -120 to 90).

Example 2: Sharpening via single kernel

```
import cv2, numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
k = np.array([[ 0, -1,  0],
              [-1,  5, -1],
              [ 0, -1,  0]], dtype=np.float32)
```

```
sharp = cv2.filter2D(img, -1, k)
cv2.imwrite("coffee_lap_sharp.png", sharp)
```

Expected Output: Coffee image with crisper edges — bean outlines and cup rim look sharper.

Example 3: Pre-blur + Laplacian for cleaner sharpening

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
blur = cv2.GaussianBlur(img, (0, 0), 1.0)
lap = cv2.Laplacian(blur, cv2.CV_64F, ksize=3)

# sharpened = original - amount * lap
sharp = np.clip(img.astype(np.float32) - 0.7 * lap, 0, 255).astype(np.uint8)
cv2.imwrite("camera_lap_blur_sharp.png", sharp)
```

Expected Output: Cameraman with enhanced edges; less noisy than a no-blur Laplacian sharpen.

Example 4: Amount tuning

```
import cv2, numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR).astype(np.float32)
blur = cv2.GaussianBlur(img, (0, 0), 1.0)
lap = cv2.Laplacian(blur, cv2.CV_32F, ksize=3)

for amt in [0.3, 0.7, 1.5]:
    sharp = np.clip(img - amt * lap, 0, 255).astype(np.uint8)
    cv2.imwrite(f"coffee_amt_{amt}.png", sharp)
```

Expected Output: Three saved files. 0.3 is subtle. 1.5 over-sharpens with visible halos.

4. Parameter Sensitivity

Param	Effect
amount (subtracted)	larger = sharper, more haloes
Pre-blur σ	larger = cleaner sharpen, less detail captured
ksize in cv2.Laplacian	larger = smoother / more spread response

5. Common Mistakes

Mistake	What Happens	Fix
ddepth=-1 to Laplacian	Negative values wrap	Use CV_64F / CV_32F
Skip pre-blur	Noise amplified	Light Gaussian first
Visualise Laplacian directly	All-black image (signed values mostly small)	Use cv2.convertScaleAbs
Apply sharpen multiple times	Heavy halos	One pass with right amount

6. Practice Tasks

- Compute the Laplacian of camera(). Visualise with convertScaleAbs.

- Apply the 3×3 sharpening kernel $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$ to `coffee()`.
- Tune amount $\in \{0.3, 0.7, 1.5\}$ in the "original – amount × Laplacian" formulation.

7. Key Takeaways

- Laplacian = sum of second derivatives; responds to all edges.
- Use `CV_64F` to avoid negative-value wrap.
- original – amount × Laplacian = sharpening recipe.
- Pre-blur slightly to avoid noise amplification.

8. What's Next

Unsharp masking — the historical, gentler way to sharpen.

Lesson 3: Unsharp Masking

Module: 8 — Sharpening

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Implement unsharp masking by hand with `cv2.GaussianBlur` + `cv2.addWeighted`.
- Tune amount, radius, threshold like Photoshop's USM dialog.
- Understand the darkroom origin (and the confusing name).

Prerequisites

- Module 8 Lesson 2

1. Why This Matters

Unsharp Masking (USM) is the standard sharpening method in Photoshop, Lightroom, GIMP, RawTherapee — everywhere. It produces controllable, photographer-friendly results. The name is confusing: it does not mean "remove sharpness" — it means "sharpen using an UNSHARP (blurred) mask".

2. Concept

2.1 The darkroom origin

In film printing, photographers placed a slightly out-of-focus negative copy ("unsharp mask") on top of the original during exposure to subtract low-frequency content — leaving the high frequencies (edges) more visible in the print. Digital USM mimics this.

2.2 The formula

$$\text{sharpened} = \text{original} + \text{amount} \times (\text{original} - \text{blurred})$$

Where `blurred` = `GaussianBlur(original, radius)`.

The middle term (`original - blurred`) is a high-pass image — small in flat regions, large at edges. Multiply by amount, add back to the original → edges exaggerated.

2.3 In code (one line)

```
blur = cv2.GaussianBlur(img, (0, 0), radius)
sharp = cv2.addWeighted(img, 1 + amount, blur, -amount, 0)
```

Equivalent algebra:

$$\begin{aligned} \text{sharp} &= (1 + \text{amount}) \times \text{original} - \text{amount} \times \text{blurred} \\ &= \text{original} + \text{amount} \times (\text{original} - \text{blurred}) \end{aligned}$$

2.4 The three knobs (Photoshop USM)

Param	Effect
Amount	How strong (typical 0.5–2.0)
Radius (Gaussian σ)	Edge halo width: small = fine detail, large = big features
Threshold	Only sharpen where local contrast exceeds this — prevents amplifying flat-area noise

2.5 Threshold implementation

```
diff = img.astype(np.int16) - blur.astype(np.int16)
mask = np.abs(diff) > threshold
sharp = img.copy()
sharp[mask] = np.clip(img[mask] + amount * diff[mask], 0, 255).astype(np.uint8)
```

Below the threshold, pixels are left untouched — the smooth sky stays smooth.

2.6 Color images

Apply USM to the luminance channel only (Y of YCrCb or L of LAB) to avoid colour shifts:

```
ycc = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)
Y = ycc[..., 0]
blur_Y = cv2.GaussianBlur(Y, (0, 0), radius)
ycc[..., 0] = cv2.addWeighted(Y, 1 + amount, blur_Y, -amount, 0)
out = cv2.cvtColor(ycc, cv2.COLOR_YCrCb2BGR)
```

3. Code Examples

Example 1: Basic USM

```
import cv2
from skimage.data import camera

img = camera()
blur = cv2.GaussianBlur(img, (0, 0), sigmaX=1.5)
sharp = cv2.addWeighted(img, 1.5, blur, -0.5, 0)
cv2.imwrite("camera_usm.png", sharp)
```

Expected Output: Cameraman image with crisper edges (building outlines, figure silhouette pop more) compared to original.

Example 2: Tune the three knobs

```
import cv2
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
for radius in [0.8, 2.0, 5.0]:
    for amount in [0.5, 1.5]:
        blur = cv2.GaussianBlur(img, (0, 0), radius)
        out = cv2.addWeighted(img, 1 + amount, blur, -amount, 0)
        cv2.imwrite(f"coffee_r{radius}_a{amount}.png", out)
```

Expected Output: 6 files. Larger radius produces wider halos. Larger amount produces stronger contrast.

Example 3: With threshold

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
blur = cv2.GaussianBlur(img, (0, 0), 1.5)
amount, threshold = 1.0, 8

diff = img.astype(np.int16) - blur.astype(np.int16)
mask = np.abs(diff) > threshold
sharp = img.copy()
sharp[mask] = np.clip(img[mask].astype(np.int16) + (amount *
diff[mask]).astype(np.int16), 0, 255).astype(np.uint8)
cv2.imwrite("camera_usm_thresh.png", sharp)
print("sharpened pixels:", mask.sum(), "out of", mask.size)
```

Expected Output: Same sharpening on edges but the sky stays smooth (no grain amplification). Console prints something like sharpened pixels: 30000 out of 262144.

Example 4: Color-safe USM via Y channel

```
import cv2
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
ycc = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)
Y = ycc[..., 0]
blur_Y = cv2.GaussianBlur(Y, (0, 0), 1.5)
ycc[..., 0] = cv2.addWeighted(Y, 1.5, blur_Y, -0.5, 0)
out = cv2.cvtColor(ycc, cv2.COLOR_YCrCb2BGR)
cv2.imwrite("coffee_usm_y.png", out)
```

Expected Output: Coffee image with sharper texture; colors not shifted (compare to per-channel USM, which can saturate colors weirdly).

4. Parameter Sensitivity

Param	Lower	Higher
amount	barely visible	aggressive, halos
radius (σ)	sharpens fine detail	broad halos around big features
threshold	sharpens everywhere (noise too)	sharpens only strong edges

Typical presets:

- "Web photo sharpen": amount=0.5, radius=1, threshold=3
- "Print sharpen": amount=1.0-1.5, radius=1-2, threshold=0
- "Creative crunch": amount=2, radius=4

5. Common Mistakes

Mistake	What Happens	Fix
USM on noisy image	Noise amplified	Denoise (M7) first or use threshold
Per-channel BGR USM	Color shifts	Apply on Y of YCrCb
Skipping clip	Wraparound	cv2.addWeighted clips automatically
radius too large	Soft, halo'd result	Start with 1-2

6. Practice Tasks

- Apply USM with amount=1.5, radius=1.5 to camera(). Save.
- Compare radius=0.5 vs radius=5 — note the halo width.
- Implement threshold-based USM and apply with threshold=8 on coffee().

7. Key Takeaways

- $\text{USM} = \text{original} + \text{amount} \times (\text{original} - \text{blurred})$.
- Three knobs: amount, radius (σ), threshold.
- Use Y channel for color to avoid hue shifts.
- "Unsharp" refers to the mask, not the result.

8. What's Next

High-pass filtering — the frequency-domain view of what USM is doing.

Lesson 4: High-Pass Filtering

Module: 8 — Sharpening

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Construct a high-pass image as original – blurred.
- Visualise the high-pass content of an image.
- Apply a high-pass kernel directly with `cv2.filter2D`.

Prerequisites

- Module 8 Lesson 3

1. Why This Matters

Sharpening, edge detection, and many forensics tasks all start with isolating the high-frequency content. A low-pass filter (blur) keeps slow changes; subtract it from the original and you have only the fast changes.

2. Concept

2.1 Frequency picture

- Low frequencies = slowly varying intensity (smooth regions, gradients).
- High frequencies = rapidly varying intensity (edges, fine texture, noise).

A Gaussian blur is a low-pass filter — it keeps the lows, attenuates the highs.

2.2 Building a high-pass

```
high_pass = original - blurred
```

The output centres around 0 — flat regions are near-zero, edges are large positive or negative.

To visualise it as an image, add an offset (e.g. 128) so 0 → mid-gray:

```
hp_vis = ((img.astype(np.int16) - blur.astype(np.int16)) + 128).clip(0, 255).astype(np.uint8)
```

2.3 Direct high-pass kernel

A 3×3 high-pass that approximates "subtract average":

```
[ -1  -1  -1 ]
[ -1   8  -1 ]
[ -1  -1  -1 ]
```

Sums to 0 → output near zero in flat regions; nonzero at edges. Pure high-pass.

2.4 Sharpening via high-pass

```
sharpened = original + amount × high_pass
```

Identical to USM — just stating it in frequency-domain language.

2.5 Frequency-domain preview

In Module 14 (DFT) you'll do the same thing via the Fourier transform — explicitly multiplying frequency coefficients. The spatial-domain methods here are mathematically equivalent for symmetric kernels, faster, and easier to reason about for everyday work.

3. Code Examples

Example 1: Visualise high-pass content

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
blur = cv2.GaussianBlur(img, (0, 0), 2.0)
hp = (img.astype(np.int16) - blur.astype(np.int16))
hp_vis = (hp + 128).clip(0, 255).astype(np.uint8)
cv2.imwrite("camera_highpass.png", hp_vis)
print("hp range:", hp.min(), hp.max(), " mean ~ 0?", round(hp.mean(), 2))
```

Expected Output: camera_highpass.png is mid-gray everywhere with edges showing as bright/dark traces. Console prints range like -90 80 and a mean very close to 0.

Example 2: Direct high-pass kernel

```
import cv2, numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
k = np.array([[ -1, -1, -1],
               [ -1,  8, -1],
               [ -1, -1, -1]], dtype=np.float32)
hp = cv2.filter2D(img, cv2.CV_32F, k)
hp_vis = cv2.convertScaleAbs(hp)
cv2.imwrite("coffee_hp_kernel.png", hp_vis)
```

Expected Output: coffee_hp_kernel.png shows only the edges and fine texture — mostly black with bright outlines.

Example 3: Sharpen by adding high-pass back

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
blur = cv2.GaussianBlur(img, (0, 0), 1.5)
hp = img.astype(np.int16) - blur.astype(np.int16)
sharp = np.clip(img.astype(np.int16) + (1.0 * hp), 0, 255).astype(np.uint8)
cv2.imwrite("camera_hp_sharp.png", sharp)
```

Expected Output: Same kind of sharpening as USM — confirms the equivalence.

4. Parameter Sensitivity

Param	Effect
Blur radius σ	smaller σ = finer-detail high-pass; bigger σ = broader features in high-pass
Kernel "centre" weight	controls amplitude — 8 vs 4 etc.

5. Common Mistakes

Mistake	What Happens	Fix
Storing high-pass as uint8	Negative parts wrap	Use int16 / float and shift for display
Sum of kernel $\neq 0$	Constant offset in output	Make sure rows sum to 0 for pure high-pass
Multiplying high-pass too much	Halos	Use small amounts (0.5-1.5)

6. Practice Tasks

- Build a high-pass of camera() with $\sigma=2.0$; visualise with +128 offset.
- Apply the direct 3×3 high-pass kernel to coffee() (cv2.CV_32F output).
- Sharpen using original + high_pass; confirm identical to USM with amount=1.

7. Key Takeaways

- High-pass = original – blurred.
- Centres around 0 in flat regions; large at edges.
- Sharpening = add scaled high-pass back to original.
- Direct kernels summing to 0 work too.

8. What's Next

Custom kernels — designing your own sharpening kernels for special effects.

Lesson 5: Custom Sharpening Kernels

Module: 8 — Sharpening

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Design custom sharpening kernels.
- Use emboss-style and edge-enhance kernels.
- Tune kernel sum to control brightness preservation.

Prerequisites

- Module 8 Lesson 4

1. Why This Matters

The Photoshop "smart sharpen", many camera in-body sharpening profiles, and emboss-like artistic effects all come from custom kernels. Knowing the structure lets you read any kernel and predict what it does.

2. Concept

2.1 Kernel anatomy

A sharpening kernel always has:

- A large positive centre weight (boosts the current pixel).
- Negative weights around it (subtracts neighbours).
- Sum ≈ 1 (or 0, with a bias added later) — preserves brightness.

2.2 Classic sharpening (4-neighbour Laplacian + identity)

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Sums to 1. Standard "sharpen" filter.

2.3 Strong sharpening (8-neighbour)

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Sums to 1. More aggressive — affects diagonals too.

2.4 Soft sharpening (less aggressive)

$$\begin{bmatrix} 0 & -0.5 & 0 \\ -0.5 & 3 & -0.5 \\ 0 & -0.5 & 0 \end{bmatrix}$$

Sums to 1. Centre weight only 3 (vs 5) — subtler.

2.5 Emboss (artistic effect)

```
[ -2  -1   0 ]
[ -1   1   1 ]
[  0   1   2 ]
```

Sums to 1. Produces a "3D embossed" look — directional sharpening that makes the image look like raised metal. Often add +128 bias and clip to keep mid-gray neutral.

2.6 Brightness preservation

If kernel sum = 1, average brightness is preserved. If sum > 1 → image becomes brighter. If sum < 1 → image becomes darker. If sum = 0 (pure high-pass) → output centred on zero; add a 128 offset for display.

2.7 Designing your own

Start from a "stronger" template (centre 5 or 9), then adjust:

- Reduce centre + edge magnitudes proportionally for subtler effect.
- Skew weights asymmetrically for directional effects (like emboss).
- Always check the sum.

3. Code Examples

Example 1: Classic 5-kernel

```
import cv2, numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
k = np.array([[ 0, -1,  0],
              [-1,  5, -1],
              [ 0, -1,  0]], dtype=np.float32)
print("sum =", k.sum())
sharp = cv2.filter2D(img, -1, k)
cv2.imwrite("coffee_classic.png", sharp)
```

Expected Output: sum = 1.0 and coffee_classic.png is the coffee image with crisper edges.

Example 2: Strong 9-kernel

```
import cv2, numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
k = np.array([[-1, -1, -1],
              [-1,  9, -1],
              [-1, -1, -1]], dtype=np.float32)
sharp = cv2.filter2D(img, -1, k)
cv2.imwrite("coffee_strong.png", sharp)
```

Expected Output: Coffee with more aggressive sharpening — visible halos on the cup rim.

Example 3: Emboss

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
k = np.array([[ -2, -1, 0],
              [-1,  1, 1],
              [ 0,  1, 2]], dtype=np.float32)
emb = cv2.filter2D(img, cv2.CV_32F, k) + 128
emb_vis = np.clip(emb, 0, 255).astype(np.uint8)
cv2.imwrite("camera_emboss.png", emb_vis)
```

Expected Output: Cameraman appearing as a 3D-stamped relief on a mid-gray background — top-left edges dark, bottom-right edges bright.

Example 4: Custom soft sharpen

```
import cv2, numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
k = np.array([[ 0, -0.5,  0],
              [-0.5, 3, -0.5],
              [ 0, -0.5,  0]], dtype=np.float32)
print("sum =", k.sum())
out = cv2.filter2D(img, -1, k)
cv2.imwrite("coffee_soft.png", out)
```

Expected Output: sum = 1.0. coffee_soft.png is more subtly sharper than the classic kernel.

4. Parameter Sensitivity

Kernel "strength" (centre weight)	Effect
centre = 3	very subtle
centre = 5	classic / good default
centre = 9	strong — risk of halos
centre = 1 (emboss)	directional / artistic

5. Common Mistakes

Mistake	What Happens	Fix
Kernel sum > 1	image brightens overall	rescale or accept brightness shift
Forgetting CV_32F for negative-sum kernels	clipping at 0 wipes out shape	use float, add bias, clip at end
Designing kernels without checking sum	unexpected brightness/contrast shift	always print k.sum()

6. Practice Tasks

- Apply the classic, strong, and soft sharpen kernels to coffee(). Compare.
- Apply emboss kernel to camera(). Add +128 bias.

- Design a "vertical-only" sharpen kernel; explain when it might be useful.

7. Key Takeaways

- Sharpening kernels: positive centre, negative neighbours.
- Sum to 1 = preserves brightness.
- Stronger centre = more sharpening = more halo risk.
- Asymmetric kernels (emboss) give artistic 3D effects.

8. What's Next

End of Module 8. Next: Module 9 — Edge Detection — where the high-pass ideas you've built around get formalised.

Module 8 — Exercises

15 exercises.

8.1 Why

- (E) Compute `edge_energy` for `camera()` and $\sigma=2$ blurred copy.
- (M) Apply `USM(amount=1.3,  $\sigma=1.5$ )` to `camera()`.
- (H) Add Gaussian noise $\sigma=10$; sharpen; show noise amplification.

8.2 Laplacian

- (E) Compute `cv2.Laplacian(camera, CV_64F, ksize=3)`. Visualise.
- (M) Apply 3×3 sharpen kernel $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$ to `coffee()`.
- (H) Tune amount $\in \{0.3, 0.7, 1.5\}$ in "`img - amount  $\times$  Laplacian`".

8.3 USM

- (E) USM `amount=1.5, radius=1.5` on `camera()`.
- (M) Compare radius 0.5 vs 5 — note halo width.
- (H) Add threshold-based USM (only sharpen where $|\text{diff}| > 8$).

8.4 High-pass

- (E) Compute $\sigma=2$ high-pass of `camera()`; visualise with +128 offset.
- (M) Apply direct 3×3 high-pass kernel to `coffee()`.
- (H) Sharpen by adding scaled high-pass to original; verify same as `USM(amount=1)`.

8.5 Custom kernels

- (E) Apply classic, strong, soft kernels to `coffee()`; compare.
- (M) Apply emboss kernel to `camera()` (+128 bias).
- (H) Design a vertical-only sharpening kernel; describe its use.

Module 8 — Quiz

10 MCQs.

Q1

Sharpening = original + amount $\times$? A. blurred B. original C. (original – blurred) D. Laplacian²

Answer: C (L1)

Q2

The Laplacian is the: A. first derivative B. second derivative sum C. inverse D. mean

Answer: B (L2)

Q3

Why use CV_64F (not -1) for Laplacian output? A. higher precision B. Laplacian can be negative — uint8 wraps C. faster D. required by API

Answer: B (L2)

Q4

"Unsharp Mask" name comes from: A. removing sharpness B. using a blurred (unsharp) mask in the formula C. raw photo metadata D. RAW format

Answer: B (L3)

Q5

USM amount=1.0 with radius=1.5 produces: A. heavy halos B. subtle edge crispness C. blurry image D. negative image

Answer: B (L3)

Q6

For color USM, apply to: A. each BGR channel B. only B C. Y of YCrCb or L of LAB D. RGB after conversion

Answer: C (L3)

Q7

A 3 $\times$ 3 high-pass kernel sums to: A. 1 B. 0 C. 255 D. depends

Answer: B (L4)

Q8

A classic sharpen kernel has centre value: A. 1 B. 5 C. 0 D. 255

Answer: B (L5)

Q9

The strong 8-neighbour kernel $\begin{bmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{bmatrix}$ sums to: A. 9 B. 1 C. 0 D. -8

Answer: B (L5)

Q10

Sharpening a noisy image: A. removes noise B. amplifies noise C. inverts noise D. has no effect on noise

Answer: B (L1)

Module 8 — Assignment: Smart Sharpener

Estimated time: 1.5-2 hours

Combines: Module 8 + Modules 6-7

Brief

Build a CLI smart sharpener that:

- Optionally denoises first.
- Applies USM with a noise-aware threshold.
- Reports before/after edge-energy.

Requirements

- sharpen.py INPUT [--output OUT] [--amount 1.0] [--radius 1.5] [--threshold 5] [--denoise]
- If --denoise: pass through cv2.bilateralFilter(d=7, sigmaColor=50, sigmaSpace=50) first.
- USM on Y of YCrCb (color-safe).
- Threshold: only sharpen where local diff exceeds threshold.
- Print mean edge energy before/after.
- Save result.

Constraints

- Modules 1-8 only.
- USM applied via the standard addWeighted formula.

Expected Output

```
$ python sharpen.py portrait.jpg --amount 1.2 --radius 1.5 --threshold 6 --denoise
denoised:    edge energy 12.4 -> 9.1
sharpened:   edge energy 9.1 -> 15.8
saved -> portrait_sharp.jpg
```

Grading Rubric

Criterion	Weight
USM on Y channel	25%
Threshold logic	20%
Denoise pass	15%
Edge energy reporting	15%
Argparse + style	25%

Submission

Save as assignment_module8.py.

Module 8 — Mini Project: Photoshop-Style USM Tool

Overview

Replicate Photoshop's "Unsharp Mask" dialog with three trackbars: amount, radius, threshold. Real-time preview alongside the original.

Concepts Used

- USM (M8 L3)
- Threshold-only sharpen (M8 L3 advanced)
- Trackbars (M2 L3)

Features

- 3 trackbars: amount ($\times 10$), radius ($\times 10 = \sigma$), threshold.
- Side-by-side: original | sharpened.
- s saves; q quits.

Starter Code

usm_tool.py:

```
import sys, cv2, numpy as np

def usm_threshold(img, amount, radius, threshold):
    if img.ndim == 3:
        ycc = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)
        Y = ycc[..., 0]
        Y_sharp = usm_threshold(Y, amount, radius, threshold)
        ycc[..., 0] = Y_sharp
        return cv2.cvtColor(ycc, cv2.COLOR_YCrCb2BGR)
    blur = cv2.GaussianBlur(img, (0, 0), max(0.1, radius))
    diff = img.astype(np.int16) - blur.astype(np.int16)
    mask = np.abs(diff) > threshold
    out = img.copy().astype(np.int16)
    out[mask] = np.clip(out[mask] + (amount * diff[mask])).astype(np.int16), 0, 255)
    return out.astype(np.uint8)

def run(path):
    img = cv2.imread(path)
    if img is None: raise FileNotFoundError(path)
    if max(img.shape) > 700:
        img = cv2.resize(img, None, fx=0.5, fy=0.5)

    cv2.namedWindow("usm")
    cv2.createTrackbar("amount x10", "usm", 10, 30, lambda v: None)
    cv2.createTrackbar("radius x10", "usm", 15, 50, lambda v: None)
    cv2.createTrackbar("threshold", "usm", 0, 50, lambda v: None)

    while True:
        a = cv2.getTrackbarPos("amount x10", "usm") / 10
        r = cv2.getTrackbarPos("radius x10", "usm") / 10
```



```

th = cv2.getTrackbarPos("threshold", "usm")
out = usm_threshold(img, a, r, th)
combo = np.hstack([img, out])
cv2.putText(combo, f"a={a} r={r} th={th}", (10, 25),
              cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
cv2.imshow("usm", combo)
k = cv2.waitKey(30) & 0xFF
if k == ord('q'): break
if k == ord('s'): cv2.imwrite("usm_out.png", out); print("saved
usm_out.png")
cv2.destroyAllWindows()

if __name__ == "__main__":
    run(sys.argv[1] if len(sys.argv) > 1 else "datasets/starter/coffee.png")

```

Expected Interaction

Move sliders; the sharpened panel updates in real time. Halos appear when amount > 2.0 or radius > 3.

Extension Challenges

- Add a "luminance only" toggle (already implemented; expose as flag).
- Show the high-pass mask in a third panel.
- Add a "before/after" toggle key.

Grading Rubric

Criterion	Weight
Three trackbars work	25%
USM math correct	25%
Threshold path	20%
Side-by-side display	15%
Style	15%

Python E-Course

Module 9 — Edge Detection

Detailed Notes

Module Overview

Level: Detection

Duration: ~1 week

Prerequisites: Modules 1-8

What You'll Learn

- Compute gradients with Sobel and Scharr.
- Detect edges with Laplacian.
- Run the Canny pipeline end-to-end (Gaussian → gradient → NMS → hysteresis).
- Tune low / high thresholds.

Lessons

#	Lesson	Duration
1	What is an Edge?	25 min
2	Sobel and Scharr Gradients	30 min
3	Laplacian as Edge Detector	25 min
4	Canny — The Full Pipeline	35 min
5	Tuning Canny Thresholds	25 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 10: Morphological Operations

Lesson 1: What is an Edge?

Module: 9 — Edge Detection

Duration: 25 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Define an edge in terms of intensity gradients.
- Distinguish step edges, ramp edges, and ridge edges.
- Identify orientation and magnitude of a gradient.

Prerequisites

- Modules 1-8

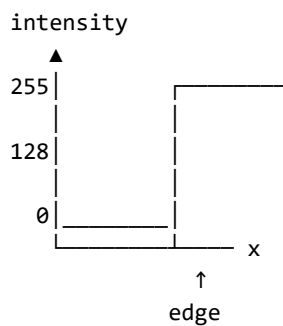
1. Why This Matters

Edges are the most informative pixels in an image — they outline objects. Almost every higher-level task (segmentation, contour detection, object recognition) starts from edges.

2. Concept

2.1 An edge in 1D

Plot intensity along a row:



The derivative (gradient) is large at the edge, small elsewhere.

2.2 Edge types

- Step: a sudden jump from one value to another (synthetic / extreme).
- Ramp: a gradual transition over several pixels (most real edges).
- Ridge: a thin bright line on a dark background — peak surrounded by drops.

2.3 In 2D — gradient = vector

The gradient has two components:

$$\nabla I = (\partial I / \partial x, \partial I / \partial y)$$

- Magnitude: $|\nabla I| = \sqrt{(\partial I / \partial x)^2 + (\partial I / \partial y)^2}$ — strength of the edge.
- Orientation: $\theta = \text{atan2}(\partial I / \partial y, \partial I / \partial x)$ — direction perpendicular to the edge.

2.4 The three classical detectors

Detector	What it measures
Sobel / Scharr	First derivative (gradient magnitude)
Laplacian	Second derivative (zero-crossings = edges)
Canny	Multi-step pipeline using Sobel + non-max suppression + hysteresis

2.5 Pre-blur

Derivatives amplify noise. Always apply a small Gaussian blur ($\sigma \approx 1$) before any edge detector. Canny does this internally.

3. Code Examples

Example 1: 1D gradient of a row

```
import numpy as np
import matplotlib.pyplot as plt

row = np.concatenate([np.full(20, 30), np.full(20, 200), np.full(20,
30)]).astype(np.float32)
grad = np.diff(row)

fig, ax = plt.subplots(2, 1, figsize=(8, 4))
ax[0].plot(row); ax[0].set_title("intensity"); ax[0].set_ylim(0, 255)
ax[1].plot(grad); ax[1].set_title("gradient"); ax[1].axhline(0, color="gray",
lw=0.5)
plt.tight_layout(); plt.show()
print("max |gradient|:", np.abs(grad).max())
```

Expected Output: Top plot: flat-low, jump up, flat-high, jump down, flat-low. Bottom plot: zero everywhere except a positive spike at the rising edge and a negative spike at the falling edge.

Console: max |gradient|: 170.

Example 2: Gradient magnitude with NumPy

```
import numpy as np, cv2
from skimage.data import camera

img = camera().astype(np.float32)
gx = np.gradient(img, axis=1)
gy = np.gradient(img, axis=0)
mag = np.sqrt(gx**2 + gy**2)
mag_vis = np.clip(mag, 0, 255).astype(np.uint8)
cv2.imwrite("cam_gradient_mag.png", mag_vis)
print("mag range:", mag.min(), mag.max())
```

Expected Output: cam_gradient_mag.png highlights edges — building outlines, the cameraman silhouette. Console shows max around 200-300.

Example 3: Orientation visualisation

```
import numpy as np, cv2
from skimage.data import camera

img = cv2.GaussianBlur(camera(), (0, 0), 1.0).astype(np.float32)
gx = cv2.Sobel(img, cv2.CV_32F, 1, 0, ksize=3)
gy = cv2.Sobel(img, cv2.CV_32F, 0, 1, ksize=3)
ang = np.arctan2(gy, gx) * 180 / np.pi    # degrees, [-180, 180]
print("orientation range:", ang.min(), ang.max())
```

Expected Output:

```
orientation range: -180.0 180.0
```

4. Parameter Sensitivity

(Per-detector — see Lessons 2-5.)

5. Common Mistakes

Mistake	What Happens	Fix
Skip pre-blur	Noisy edge map	Gaussian $\sigma=1$ before edge detection
Use uint8 output for gradients	Negative values wrap	Use CV_32F
Treat one gradient axis as the edge	Misses orthogonal edges	Compute magnitude from both axes

6. Practice Tasks

- Plot a 1D step-edge row and its `np.diff`. Identify the spike.
- Compute gradient magnitude of `camera()` via `np.gradient`. Visualise.
- Compute orientation map (degrees); print min/max.

7. Key Takeaways

- Edge = location of rapid intensity change.
- 2D gradient has magnitude (strength) and orientation (perpendicular to edge).
- Always blur slightly first to reduce noise.

8. What's Next

Sobel — the standard discrete first-derivative operator.

Lesson 2: Sobel and Scharr Gradients

Module: 9 — Edge Detection

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Apply cv2.Sobel and cv2.Scharr for gradients.
- Combine x and y gradients into a magnitude image.
- Know when Scharr beats Sobel.

Prerequisites

- Module 9 Lesson 1

1. Why This Matters

Sobel is the de facto discrete derivative operator. Almost every edge / corner / feature detector uses it under the hood (including Canny). Scharr is a small upgrade with better isotropy at the cost of speed.

2. Concept

2.1 The Sobel kernels (3×3)

Horizontal gradient ($\partial I / \partial x$ — detects vertical edges):

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Vertical gradient ($\partial I / \partial y$ — detects horizontal edges):

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

The middle row/col weighted by 2 emphasises the centre, which gives better noise rejection than a plain $[-1, 0, 1]$.

2.2 The Scharr kernels (3×3)

$\partial / \partial x:$	$\partial / \partial y:$
$\begin{bmatrix} -3 & 0 & 3 \\ -10 & 0 & 10 \\ -3 & 0 & 3 \end{bmatrix}$	$\begin{bmatrix} -3 & -10 & -3 \\ 0 & 0 & 0 \\ 3 & 10 & 3 \end{bmatrix}$

Bigger central weights — more rotationally symmetric responses. Slightly more accurate for small kernels.

2.3 OpenCV API

```
gx = cv2.Sobel(img, cv2.CV_32F, 1, 0, ksize=3)
gy = cv2.Sobel(img, cv2.CV_32F, 0, 1, ksize=3)
```

Or:

```
gx = cv2.Scharr(img, cv2.CV_32F, 1, 0)
gy = cv2.Scharr(img, cv2.CV_32F, 0, 1)
```

dx, dy = 1, 0 means "first derivative in x, zero in y".

2.4 Magnitude and direction

```
mag = cv2.magnitude(gx, gy)          # sqrt(gx2 + gy2)
ang = cv2.phase(gx, gy, angleInDegrees=True)
```

Or just NumPy:

```
mag = np.sqrt(gx**2 + gy**2)
ang = np.arctan2(gy, gx) * 180 / np.pi
```

2.5 Larger kernels

cv2.Sobel(..., ksize=5) or 7 smooths first (acts like Sobel applied to a blurred image). Use larger ksize when you want coarser edge maps.

2.6 Pre-blur recommendation

Even with Sobel's built-in smoothing, a 3×3 Gaussian $\sigma \approx 1$ before gradient gives noticeably cleaner edges on noisy images.

3. Code Examples

Example 1: Sobel X / Y

```
import cv2, numpy as np
from skimage.data import camera

img = cv2.GaussianBlur(camera(), (3, 3), 0)
gx = cv2.Sobel(img, cv2.CV_32F, 1, 0, ksize=3)
gy = cv2.Sobel(img, cv2.CV_32F, 0, 1, ksize=3)

cv2.imwrite("sobel_x.png", cv2.convertScaleAbs(gx))
cv2.imwrite("sobel_y.png", cv2.convertScaleAbs(gy))
print("gx range:", gx.min(), gx.max())
print("gy range:", gy.min(), gy.max())
```

Expected Output: sobel_x.png highlights vertical edges; sobel_y.png highlights horizontal edges. Console shows negative and positive ranges.

Example 2: Magnitude

```
import cv2, numpy as np
from skimage.data import coins

img = cv2.GaussianBlur(coins(), (3, 3), 0)
gx = cv2.Sobel(img, cv2.CV_32F, 1, 0, ksize=3)
```



```

gy = cv2.Sobel(img, cv2.CV_32F, 0, 1, ksize=3)
mag = cv2.magnitude(gx, gy)
mag_vis = np.clip(mag, 0, 255).astype(np.uint8)
cv2.imwrite("coins_mag.png", mag_vis)
print("max mag:", mag.max())

```

Expected Output: coins_mag.png shows clean circular outlines around each coin.

Example 3: Sobel vs Scharr

```

import cv2, numpy as np
from skimage.data import camera

img = cv2.GaussianBlur(camera(), (3, 3), 0)
sb_x = cv2.Sobel(img, cv2.CV_32F, 1, 0, ksize=3)
sb_y = cv2.Sobel(img, cv2.CV_32F, 0, 1, ksize=3)
sc_x = cv2.Scharr(img, cv2.CV_32F, 1, 0)
sc_y = cv2.Scharr(img, cv2.CV_32F, 0, 1)

sb_mag = np.clip(np.sqrt(sb_x**2 + sb_y**2), 0, 255).astype(np.uint8)
sc_mag = np.clip(np.sqrt(sc_x**2 + sc_y**2), 0, 255).astype(np.uint8)
cv2.imwrite("sobel_mag.png", sb_mag)
cv2.imwrite("scharr_mag.png", sc_mag)

```

Expected Output: Both show edge maps; Scharr's are slightly stronger and more uniform across orientations.

Example 4: ksize tuning

```

import cv2, numpy as np
from skimage.data import camera

img = cv2.GaussianBlur(camera(), (3, 3), 0)
for k in [3, 5, 7]:
    gx = cv2.Sobel(img, cv2.CV_32F, 1, 0, ksize=k)
    gy = cv2.Sobel(img, cv2.CV_32F, 0, 1, ksize=k)
    mag = np.clip(np.sqrt(gx**2 + gy**2), 0, 255).astype(np.uint8)
    cv2.imwrite(f"sobel_{k}.png", mag)

```

Expected Output: k=3 has thin precise edges; k=7 has thicker, smoother edges (more spatial integration).

4. Parameter Sensitivity

Param	Effect
ksize	larger = thicker / smoother edges
pre-blur σ	larger = fewer fine-detail responses
Sobel vs Scharr	Scharr more isotropic for ksize=3

5. Common Mistakes

Mistake	What Happens	Fix
Use uint8 ddepth	Negative gradients clipped to 0 → asymmetric response	Use CV_32F

Skip pre-blur	Noisy edge map	Light Gaussian first
Combine via `	gx	+

6. Practice Tasks

- Compute Sobel X, Y, magnitude for coins(). Save all three.
- Compare Sobel vs Scharr magnitudes on camera().
- Try ksize 3, 5, 7; compare edge thickness.

7. Key Takeaways

- Sobel = standard first-derivative operator; pair x + y.
- cv2.magnitude(gx, gy) for edge strength.
- Use CV_32F to keep negative values.
- Scharr is the higher-quality 3×3 alternative.

8. What's Next

Laplacian as a second-derivative edge detector.

Lesson 3: Laplacian as Edge Detector

Module: 9 — Edge Detection

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Use cv2.Laplacian as an isotropic edge detector.
- Find zero-crossings as edge locations.
- Use LoG (Laplacian of Gaussian) for blob and edge detection.

Prerequisites

- Module 9 Lesson 2, Module 8 Lesson 2.

1. Why This Matters

Laplacian gives one isotropic response (vs Sobel's separate x/y gradients). It's the basis for blob detection, scale-space (SIFT pyramid), and Marr-Hildreth edge detection. It's also more noise-sensitive than Sobel — pre-blur is essential.

2. Concept

2.1 Second derivative — zero-crossings = edges

Where the second derivative changes sign, the first derivative has an extremum — that's where the edge is. So edge detection via Laplacian = find zero-crossings of the Laplacian image.

2.2 LoG (Laplacian of Gaussian)

Apply a Gaussian blur first, then Laplacian. Or equivalently, apply the LoG kernel directly (math-equivalent, fewer steps).

Smoothing first matters because the Laplacian amplifies noise; the Gaussian σ controls the scale of edges you detect — small σ for fine edges, large σ for big features.

2.3 OpenCV API

```
blur = cv2.GaussianBlur(img, (0, 0), sigma=1.5)
lap = cv2.Laplacian(blur, cv2.CV_32F, ksize=3)
```

2.4 Visualisation choices

- cv2.convertScaleAbs(lap) — show |laplacian| (loses sign).
- Add bias 128, clip, cast — preserves sign for visualization.
- Threshold |lap| > T for a binary edge map.

2.5 Zero-crossings

```
sign = np.sign(lap)
zero_cross = (np.diff(sign, axis=0) != 0) | (np.diff(sign, axis=1) != 0)
```

A boolean image where sign-changes occur. Combined with $|\text{lap}| > T$ gives precise edge locations.

2.6 When to use

- Want isotropic, scale-aware edges → LoG.
- Want fast, classic edges → Canny (next lesson).
- Want gradient direction → Sobel.

3. Code Examples

Example 1: Basic Laplacian

```
import cv2, numpy as np
from skimage.data import camera

img = cv2.GaussianBlur(camera(), (0, 0), 1.5)
lap = cv2.Laplacian(img, cv2.CV_32F, ksize=3)
lap_vis = cv2.convertScaleAbs(lap)
cv2.imwrite("cam_lap.png", lap_vis)
print("lap range:", lap.min(), lap.max())
```

Expected Output: Edge map with bright outlines on a dark background. Range typically -150 to 150.

Example 2: Threshold |Laplacian|

```
import cv2, numpy as np
from skimage.data import coins

img = cv2.GaussianBlur(coins(), (0, 0), 1.5)
lap = cv2.Laplacian(img, cv2.CV_32F, ksize=3)
mask = (np.abs(lap) > 20).astype(np.uint8) * 255
cv2.imwrite("coins_lap_mask.png", mask)
print("edge pixels:", (mask > 0).sum())
```

Expected Output: Binary edge map of the coins (outlines of each coin).

Example 3: LoG with different scales

```
import cv2, numpy as np
from skimage.data import camera

for sigma in [0.5, 1.5, 4.0]:
    blur = cv2.GaussianBlur(camera(), (0, 0), sigma)
    lap = cv2.Laplacian(blur, cv2.CV_32F, ksize=3)
    cv2.imwrite(f"cam_log_s{sigma}.png", cv2.convertScaleAbs(lap))
```

Expected Output: $\sigma=0.5$ catches fine detail (texture, grain). $\sigma=4.0$ catches only coarse structures (building outlines, figure).

Example 4: Zero-crossings

```
import cv2, numpy as np
from skimage.data import coins
```

```

img = cv2.GaussianBlur(coins(), (0, 0), 1.5).astype(np.float32)
lap = cv2.Laplacian(img, cv2.CV_32F, ksize=3)

sign = np.sign(lap)
zc = np.zeros_like(lap, dtype=np.uint8)
zc[:-1] |= (sign[:-1] * sign[1:] < 0).astype(np.uint8)
zc[:, :-1] |= (sign[:, :-1] * sign[:, 1:] < 0).astype(np.uint8)
zc = (zc & (np.abs(lap) > 10).astype(np.uint8)) * 255
cv2.imwrite("coins_zerocross.png", zc)

```

Expected Output: Thin precise edges following the coin boundaries.

4. Parameter Sensitivity

Param	Effect
pre-blur σ	small = fine edges; large = blob-scale edges
ksize	small = local response; large = smoother
threshold	how strong an edge needs to be

5. Common Mistakes

Mistake	What Happens	Fix
No pre-blur	Lots of noise spikes	Always Gaussian first
Use uint8 ddepth	Negative parts lost	CV_32F
Use Laplacian for orientation	It's scalar; no direction	Use Sobel

6. Practice Tasks

- Apply LoG ($\sigma=1.5$) on camera(); visualise.
- Threshold $|\text{Laplacian}| > 20$ on coins(). Save mask.
- Compute zero-crossings; compare to threshold-only edges.

7. Key Takeaways

- Laplacian = second derivative, isotropic.
- Pre-blur with Gaussian — LoG is the standard pairing.
- Zero-crossings or $|\text{lap}| > T$ for binary edges.
- For most practical work, Canny (next lesson) is the default; LoG is a useful alternative for scale-space.

8. What's Next

Canny — the multi-step state-of-the-art edge detector.

Lesson 4: Canny — The Full Pipeline

Module: 9 — Edge Detection

Duration: 35 minutes

Difficulty: Intermediate

Learning Objectives

- Run `cv2.Canny` with sensible thresholds.
- Explain the 4 stages: Gaussian → gradient → non-max suppression → hysteresis.
- Visualise each stage independently.

Prerequisites

- Module 9 Lessons 1-3

1. Why This Matters

Canny is the most-used edge detector in the world. It produces thin, well-localised, connected edges with controllable strictness — the gold standard for general edge detection.

2. Concept

2.1 The four stages

- Smooth with Gaussian to reduce noise.
- Gradient via Sobel: magnitude + orientation.
- Non-maximum suppression (NMS): thin the gradient ridges to 1-pixel-wide edges by keeping only local maxima along the gradient direction.
- Hysteresis thresholding:
 - Pixels above high threshold → definite edges.
 - Pixels below low threshold → discarded.
 - Pixels between → kept only if connected (via 8-neighbours) to a definite edge.

The hysteresis step is what makes Canny robust: weak edges that are part of a continuous strong contour survive; isolated weak edges (noise) are dropped.

2.2 OpenCV API

```
edges = cv2.Canny(img, threshold1, threshold2, apertureSize=3, L2gradient=False)
```

- `threshold1, threshold2` = low, high (OpenCV accepts either order; conventionally `low < high`).
- `apertureSize` = Sobel kernel size (3, 5, or 7).
- `L2gradient=True` uses the true $\sqrt{g_x^2 + g_y^2}$ instead of the cheaper $|g_x| + |g_y|$. Slightly more accurate.

2.3 Output

Binary `uint8`: 0 (non-edge) or 255 (edge). Already thinned to one pixel wide.

2.4 Picking thresholds

A common heuristic:

```
v = np.median(img)
low = int(max(0, 0.66 * v))
high = int(min(255, 1.33 * v))
edges = cv2.Canny(img, low, high)
```

Sets the band around the image's median brightness. Works decently for many images; for tuned applications, expose them as parameters.

2.5 Implementation note

OpenCV already does Sobel + NMS + hysteresis. You don't need to chain manual stages — but visualising them helps understanding.

3. Code Examples

Example 1: Basic Canny

```
import cv2
from skimage.data import camera

img = camera()
edges = cv2.Canny(img, 100, 200)
cv2.imwrite("cam_canny.png", edges)
print("edge pixel %:", (edges > 0).mean() * 100)
```

Expected Output: cam_canny.png shows thin clean edges — the cameraman's silhouette, building outlines. Console prints a percentage usually 3-8%.

Example 2: Visualise the four stages manually

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
blur = cv2.GaussianBlur(img, (5, 5), 1.4)
gx = cv2.Sobel(blur, cv2.CV_32F, 1, 0, ksize=3)
gy = cv2.Sobel(blur, cv2.CV_32F, 0, 1, ksize=3)
mag = cv2.magnitude(gx, gy)
mag_vis = np.clip(mag, 0, 255).astype(np.uint8)
edges = cv2.Canny(img, 100, 200)

stages = np.hstack([img, blur, mag_vis, edges])
cv2.imwrite("canny_stages.png", stages)
```

Expected Output: 4-up image: original | blurred | gradient magnitude | final Canny edges.

Example 3: Auto-threshold via median

```
import cv2, numpy as np
from skimage.data import coins

img = coins()
```

```

v = np.median(img)
low, high = int(max(0, 0.66 * v)), int(min(255, 1.33 * v))
edges = cv2.Canny(img, low, high)
print(f"auto: low={low} high={high}, edge %={ (edges > 0).mean()*100:.2f}")
cv2.imwrite("coins_canny_auto.png", edges)

```

Expected Output: coins_canny_auto.png with clean coin outlines. Console prints chosen thresholds.

Example 4: L2 vs L1 gradient

```

import cv2
from skimage.data import camera

img = camera()
e1 = cv2.Canny(img, 100, 200, L2gradient=False)
e2 = cv2.Canny(img, 100, 200, L2gradient=True)
print("L1 edges:", (e1>0).sum(), " L2 edges:", (e2>0).sum())
cv2.imwrite("canny_L1.png", e1)
cv2.imwrite("canny_L2.png", e2)

```

Expected Output: Slightly different counts; L2 typically picks up a few extra subtle edges.

4. Parameter Sensitivity

Param	Lower	Higher
threshold1 (low)	more weak edges kept	fewer weak edges
threshold2 (high)	more edges accepted as definite	only strong edges accepted
apertureSize	fine edges	thicker / smoother edges
L2gradient	faster, less accurate	slower, more accurate

5. Common Mistakes

Mistake	What Happens	Fix
low > high	OpenCV swaps; works but unclear	Set low < high
Thresholds too low	Noisy "edge spaghetti"	Increase both
Thresholds too high	Missing edges	Decrease both
Apply Canny on color image	OpenCV converts to gray automatically	Be explicit: cv2.cvtColor(..., COLOR_BGR2GRAY) first

6. Practice Tasks

- Apply cv2.Canny(camera, 100, 200). Save.
- Build the auto-threshold helper using median.
- Visualise the 4 stages of Canny side-by-side.

7. Key Takeaways

- 4 stages: Gaussian → Sobel → NMS → hysteresis.
- cv2.Canny(img, low, high) runs all of them.

- Auto-thresholds from the image median is a good default.
- Output is binary (0/255), thinned.

8. What's Next

Tuning Canny thresholds in practice — what to do when defaults aren't enough.

Lesson 5: Tuning Canny Thresholds

Module: 9 — Edge Detection

Duration: 25 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Build a trackbar tuner for Canny.
- Use Otsu's method to pick thresholds automatically.
- Recognize the symptom-cause table for bad edge maps.

Prerequisites

- Module 9 Lesson 4

1. Why This Matters

Most real Canny work is tuning — image content, lighting, noise all change what's "right." A 20-line trackbar tool saves hours.

2. Concept

2.1 Symptoms and fixes

You see	Likely cause	Fix
Lots of fragmented noise edges	Thresholds too low	Raise both, especially high
Missing edges that should be there	Thresholds too high	Lower both, especially low
Thick / fuzzy edges	apertureSize too large	Use 3
Object outlines broken into segments	Image too noisy → NMS unstable	Pre-blur more (Gaussian $\sigma \sim 2$)
Texture interfering with real edges	Pre-blur too small	Increase pre-blur σ

2.2 Otsu-derived thresholds

Run Otsu's binary threshold on the smoothed image to get a "natural" gray-level split, then use that as high:

```
import cv2
blur = cv2.GaussianBlur(img, (5, 5), 0)
otsu, _ = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
high = otsu
low = 0.5 * high
edges = cv2.Canny(blur, low, high)
```

(Otsu is covered fully in M11.)

2.3 Tracker tuner pattern

```
cv2.createTrackbar("low", "tune", 50, 255, lambda v: None)
cv2.createTrackbar("high", "tune", 150, 255, lambda v: None)
while True:
    low = cv2.getTrackbarPos("low", "tune")
    high = cv2.getTrackbarPos("high", "tune")
    edges = cv2.Canny(img, low, high)
    cv2.imshow("tune", edges)
    if cv2.waitKey(30) & 0xFF == ord('q'):
        break
```

2.4 Multi-scale Canny

For mixed-scale content (small text + big shapes), run Canny at two pre-blur scales and OR the results:

```
e_fine = cv2.Canny(cv2.GaussianBlur(img, (3, 3), 1), low, high)
e_coarse = cv2.Canny(cv2.GaussianBlur(img, (7, 7), 3), low, high)
combined = cv2.bitwise_or(e_fine, e_coarse)
```

2.5 Recommended starting recipe

```
import cv2, numpy as np
def auto_canny(img, sigma=0.33):
    v = float(np.median(img))
    low = int(max(0, (1.0 - sigma) * v))
    high = int(min(255, (1.0 + sigma) * v))
    return cv2.Canny(img, low, high)
```

3. Code Examples

Example 1: Threshold tracker

```
import cv2, sys
from skimage.data import camera

img = camera()
cv2.namedWindow("tune")
cv2.createTrackbar("low", "tune", 50, 255, lambda v: None)
cv2.createTrackbar("high", "tune", 150, 255, lambda v: None)

while True:
    lo = cv2.getTrackbarPos("low", "tune")
    hi = cv2.getTrackbarPos("high", "tune")
    edges = cv2.Canny(img, lo, hi)
    cv2.imshow("tune", edges)
    if cv2.waitKey(30) & 0xFF == ord('q'):
        break
cv2.destroyAllWindows()
```

Expected Interaction: Drag thresholds in real time; observe how more / fewer edges appear.

Example 2: Otsu-derived

```
import cv2
from skimage.data import camera
```

```

img = camera()
blur = cv2.GaussianBlur(img, (5, 5), 0)
otsu, _ = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
edges = cv2.Canny(blur, otsu * 0.5, otsu)
cv2.imwrite("cam_canny_otsu.png", edges)
print("otsu threshold:", otsu)

```

Expected Output: cam_canny_otsu.png with reasonable edges. Otsu's threshold typically lands in the 90-130 range for typical images.

Example 3: auto_canny function

```

import cv2, numpy as np
from skimage.data import coffee

def auto_canny(img, sigma=0.33):
    v = float(np.median(img))
    low = int(max(0, (1.0 - sigma) * v))
    high = int(min(255, (1.0 + sigma) * v))
    return cv2.Canny(img, low, high)

gray = cv2.cvtColor(cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR), cv2.COLOR_BGR2GRAY)
cv2.imwrite("coffee_canny_auto.png", auto_canny(gray))

```

Expected Output: Coffee bean outlines on black.

Example 4: Multi-scale

```

import cv2
from skimage.data import camera

img = camera()
e_fine = cv2.Canny(cv2.GaussianBlur(img, (3, 3), 1), 60, 150)
e_coarse = cv2.Canny(cv2.GaussianBlur(img, (7, 7), 3), 60, 150)
combined = cv2.bitwise_or(e_fine, e_coarse)
cv2.imwrite("cam_canny_multi.png", combined)

```

Expected Output: Edge map that catches both small text and large object outlines.

4. Parameter Sensitivity

Symptom	Try
Too few edges	Halve low
Too many "spaghetti" edges	Raise both thresholds 50%
Broken thin lines	Reduce low only
Texture noise	Pre-blur with larger σ

5. Common Mistakes

Mistake	What Happens	Fix
Tuning on one image, deploying on others	Brittle	Use auto / Otsu thresholds
Combining Canny with very	Edge overlap mess	Use multi-scale OR carefully

different scales naively		
Setting low $\approx$ high	No hysteresis benefit	Keep low $\approx 0.5 \times$ high

6. Practice Tasks

- Build the trackbar tuner; find good thresholds for astronaut().
- Apply the Otsu-derived recipe on camera().
- Implement auto_canny; compare to manual.

7. Key Takeaways

- Tune via trackbar for one-off images.
- Use Otsu or median-based auto-thresholds in pipelines.
- Symptom $\rightarrow$ fix table guides what to twist.
- Multi-scale Canny for mixed-detail content.

8. What's Next

End of Module 9. Next: Module 10 — Morphological Operations.

Module 9 — Exercises

15 exercises.

9.1 What is an edge?

- (E) Plot 1D step-edge intensity and its `np.diff`.
- (M) Compute gradient magnitude of `camera()` with `np.gradient`.
- (H) Compute and visualise gradient orientation (degrees) for `coins()`.

9.2 Sobel

- (E) Sobel X / Y on `camera()`; save both.
- (M) Compute magnitude with `cv2.magnitude` on `coins()`.
- (H) Compare Sobel vs Scharr magnitudes visually.

9.3 Laplacian

- (E) LoG ($\sigma=1.5$) on `camera()`; save.
- (M) Threshold $|\text{Laplacian}| > 20$ on `coins()`.
- (H) Compute zero-crossings; compare to threshold edges.

9.4 Canny

- (E) `cv2.Canny(camera, 100, 200)`. Save.
- (M) Auto-threshold via median; apply on `coffee()`.
- (H) Visualise the 4 Canny stages side-by-side.

9.5 Tuning

- (E) Build the trackbar tuner for `astronaut()`.
- (M) Otsu-derived thresholds on `camera()`.
- (H) Multi-scale OR of two pre-blur Cannys.

Module 9 — Quiz

10 MCQs.

Q1

Edges are characterised by: A. low intensity B. high gradient magnitude C. uniform color D. random noise

Answer: B (L1)

Q2

Sobel kernel detects: A. brightness B. first derivative C. second derivative D. color

Answer: B (L2)

Q3

For an isotropic edge response (one operator, not two), use: A. Sobel B. Scharr C. Laplacian D. Prewitt

Answer: C (L3)

Q4

Why pre-blur before edge detection? A. reduces noise amplification B. compresses C. changes color D. shrinks image

Answer: A (L1, L3)

Q5

Canny stages, in order: A. NMS → Gaussian → gradient → hysteresis B. Gaussian → gradient → NMS → hysteresis C. Gradient → Gaussian → hysteresis → NMS D. Hysteresis → NMS → Gradient → Gaussian

Answer: B (L4)

Q6

Canny's hysteresis stage: A. removes weak edges entirely B. keeps weak edges only if connected to strong edges C. amplifies all edges D. blurs edges

Answer: B (L4)

Q7

For cv2.Canny, the typical low:high ratio is: A. 1:1 B. 1:2 to 1:3 C. 1:10 D. 10:1

Answer: B (L4, L5)

Q8

LoG combines: A. Sobel + Laplacian B. Gaussian + Laplacian C. Sobel + Gaussian D. Canny + Sobel

Answer: B (L3)

Q9

Output of cv2.Canny is: A. grayscale B. RGB C. binary uint8 (0/255), thinned D. float

Answer: C (L4)

Q10

Symptom: too many noisy "spaghetti" edges. Fix: A. lower thresholds B. raise both thresholds C. apply more sharpening D. remove blur

Answer: B (L5)

Module 9 — Assignment: Multi-Method Edge Comparator

Estimated time: 1.5 hours

Combines: Module 9 + Modules 6-7

Brief

Build a CLI that runs Sobel, Laplacian, and Canny on a given image and saves a side-by-side comparison.

Requirements

- `edges.py INPUT [--output OUT] [--canny-low 100] [--canny-high 200]`
- Pre-blur with Gaussian $\sigma=1$.
- Compute Sobel magnitude (visualise), Laplacian (`|abs|`), Canny.
- Concatenate horizontally: `original | sobel | laplacian | canny`.
- Save as a single image. Print edge-pixel percentages for Canny.

Constraints

- Modules 1-9 only.
- Use `argparse`.

Expected Output

```
$ python edges.py coins.png
canny edges: 4.8%
saved -> coins_edges.png
```

`coins_edges.png` is a horizontal strip of 4 panels.

Grading Rubric

Criterion	Weight
All 3 methods correct	35%
Visualisation correct	25%
Argparse + flags	15%
Stats reporting	10%
Style	15%

Submission

Save as `assignment_module9.py`.

Module 9 — Mini Project: Auto-Canny with Threshold Sliders

Overview

A Canny tuner with low/high sliders, plus a button-key for auto-threshold mode (median or Otsu).

Concepts Used

- Canny (M9 L4)
- Auto-threshold (M9 L5)
- Trackbars (M2 L3)

Features

- Sliders for low, high, pre-blur $\sigma \times 10$.
- a cycles between manual / median-auto / Otsu-auto modes.
- s saves; q quits.
- Window shows original | edges side-by-side, with mode label.

Starter Code

canny_tuner.py:

```
import sys, cv2, numpy as np

MODES = ["manual", "median-auto", "otsu-auto"]

def auto_thresholds(img, mode):
    if mode == "median-auto":
        v = float(np.median(img))
        return int(max(0, 0.66*v)), int(min(255, 1.33*v))
    if mode == "otsu-auto":
        otsu, _ = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
        return int(0.5*otsu), int(otsu)
    return None

def run(path):
    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if img is None: raise FileNotFoundError(path)
    if max(img.shape) > 700: img = cv2.resize(img, None, fx=0.5, fy=0.5)

    cv2.namedWindow("canny")
    cv2.createTrackbar("low", "canny", 50, 255, lambda v: None)
    cv2.createTrackbar("high", "canny", 150, 255, lambda v: None)
    cv2.createTrackbar("blur sigma x10", "canny", 10, 50, lambda v: None)
    mode_idx = 0

    while True:
        sig = max(0.1, cv2.getTrackbarPos("blur sigma x10", "canny") / 10)
        blurred = cv2.GaussianBlur(img, (0, 0), sig)
        mode = MODES[mode_idx]
```

```

if mode == "manual":
    lo = cv2.getTrackbarPos("low", "canny")
    hi = cv2.getTrackbarPos("high", "canny")
else:
    lo, hi = auto_thresholds(blurred, mode)
edges = cv2.Canny(blurred, lo, hi)
combo = np.hstack([img, edges])
cv2.putText(combo, f"[{mode}] low={lo} high={hi} sig={sig:.1f}",
            (10, 20), cv2.FONT_HERSHEY_SIMPLEX, 0.5, 200, 1)
cv2.imshow("canny", combo)
k = cv2.waitKey(30) & 0xFF
if k == ord('q'): break
if k == ord('a'): mode_idx = (mode_idx + 1) % len(MODES)
if k == ord('s'): cv2.imwrite("edges.png", edges); print("saved edges.png")
cv2.destroyAllWindows()

if __name__ == "__main__":
    run(sys.argv[1] if len(sys.argv) > 1 else "datasets/starter/camera.png")

```

Expected Interaction

Drag sliders, press a to cycle modes, s to save the current edges.

Extension Challenges

- Add a "histogram of edge angles" panel.
- Add multi-scale toggle (m).
- Overlay edges on the original in red for color visualisation.

Grading Rubric

Criterion	Weight
Manual mode works	25%
Auto modes work	25%
Sigma slider	15%
Mode label	10%
Style	25%

Python E-Course

Module 10 — Morphological Operations

Detailed Notes

Module Overview

Level: Detection

Duration: ~1 week

Prerequisites: Modules 1-9

What You'll Learn

- Apply erosion, dilation, opening, closing on binary and grayscale images.
- Use morphological gradient, top-hat, black-hat.
- Build structuring elements with `cv2.getStructuringElement`.
- Clean noisy masks reliably.

Lessons

#	Lesson	Duration
1	Binary Images & Structuring Elements	25 min
2	Erosion and Dilation	30 min
3	Opening and Closing	25 min
4	Gradient, Top-Hat, Black-Hat	25 min
5	Practical Mask Cleaning	25 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 11: Segmentation & Thresholding

Lesson 1: Binary Images & Structuring Elements

Module: 10 — Morphological Operations

Duration: 25 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Define a binary image and a structuring element (SE / kernel).
- Build SEs of different shapes with `cv2.getStructuringElement`.
- Understand how the SE shape determines a morphology op's effect.

Prerequisites

- Modules 1-9

1. Why This Matters

Morphology operates on binary masks (output of thresholding or color filtering) to clean them up — remove specks, fill holes, separate touching objects. The structuring element is the morphology equivalent of a convolution kernel — it determines what "a neighbourhood" means.

2. Concept

2.1 Binary image

A (H, W) uint8 image with only two values: 0 (off) and 255 (on). Output of `cv2.threshold` or `cv2.inRange`.

2.2 Structuring element (SE)

A small binary shape (3×3, 5×5, etc.) that defines the local neighbourhood examined at each pixel.

2.3 OpenCV API

```
se = cv2.getStructuringElement(shape, ksize, anchor=(-1,-1))
```

Shape constants:

Constant	Shape
<code>cv2.MORPH_RECT</code>	full rectangle of 1s
<code>cv2.MORPH_ELLIPSE</code>	round disc
<code>cv2.MORPH_CROSS</code>	+ shape

Example:

```
se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
print(se)
```

Output:

```
[[0 0 1 0 0]
 [1 1 1 1 1]
 [1 1 1 1 1]
 [1 1 1 1 1]
 [0 0 1 0 0]]
```

2.4 Shape choice — when it matters

Choice	Use
RECT (square)	Default; fast
ELLIPSE	Most natural for round / curved objects
CROSS	Preserves thin straight lines better

For most denoising work, ELLIPSE looks best. For text / line work, CROSS preserves more structure.

2.5 Size — kernel size matters more than shape

Bigger SE = bigger effect per op. A 9×9 erosion removes much more than a 3×3 erosion. Tune size first, shape second.

2.6 Custom shapes

You can also build SEs manually:

```
import numpy as np
se = np.array([[0, 0, 1, 0, 0],
               [0, 1, 1, 1, 0],
               [1, 1, 1, 1, 1],
               [0, 1, 1, 1, 0],
               [0, 0, 1, 0, 0]], dtype=np.uint8)
```

Useful for unusual shapes (diagonal lines, custom motifs).

3. Code Examples

Example 1: Build SEs

```
import cv2
for s, name in [(cv2.MORPH_RECT, "rect"),
                (cv2.MORPH_ELLIPSE, "ellipse"),
                (cv2.MORPH_CROSS, "cross")]:
    print(name)
    print(cv2.getStructuringElement(s, (5, 5)))
    print()
```

Expected Output: Three 5×5 matrices: solid 1s (rect), round disc (ellipse), + (cross).

Example 2: Visualise SE shapes at different sizes

```
import cv2, numpy as np
import matplotlib.pyplot as plt

shapes = [(cv2.MORPH_RECT, "rect"),
          (cv2.MORPH_ELLIPSE, "ellipse"),
          (cv2.MORPH_CROSS, "cross")]
```

```

sizes = [3, 5, 9, 15]

fig, axes = plt.subplots(len(shapes), len(sizes), figsize=(10, 6))
for i, (s, name) in enumerate(shapes):
    for j, k in enumerate(sizes):
        se = cv2.getStructuringElement(s, (k, k))
        axes[i, j].imshow(se, cmap='gray', vmin=0, vmax=1)
        axes[i, j].set_title(f"{name} {k}x{k}")
        axes[i, j].axis("off")
plt.tight_layout(); plt.show()

```

Expected Output: A 3×4 grid showing each shape at sizes 3, 5, 9, 15.

Example 3: Build a binary image to test on

```

import cv2, numpy as np
from skimage.data import coins

img = coins()
_, binary = cv2.threshold(img, 80, 255, cv2.THRESH_BINARY)
print("unique:", np.unique(binary))
cv2.imwrite("coins_binary.png", binary)

```

Expected Output: unique: [0 255] and a saved binary mask where the coins are white on black background.

4. Parameter Sensitivity

Param	Effect
ksize	bigger = stronger morphological effect
shape	small differences except for very thin / very round features

5. Common Mistakes

Mistake	What Happens	Fix
Apply morphology to non-binary image	Behaves like grayscale morphology — usually unintended	Threshold first
Forget odd kernel size	Anchor offset; weird shifts	Use odd ksize
Default MORPH_RECT for natural objects	Slightly boxy	Try MORPH_ELLIPSE

6. Practice Tasks

- Build a 7×7 ELLIPSE and print it.
- Build a 9×9 CROSS and print it.
- Threshold coins() at 80 to make a binary image. Save.

7. Key Takeaways

- Morphology = local neighbourhood operations on binary (or grayscale) images.
- Structuring element shape: RECT, ELLIPSE, CROSS.

- Size matters more than shape for most denoising.
- `cv2.getStructuringElement(shape, (k, k))` is the standard call.

8. What's Next

The two atomic operations: erosion and dilation.

Lesson 2: Erosion and Dilation

Module: 10 — Morphological Operations

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Apply `cv2.erode` and `cv2.dilate`.
- Predict the visual effect (objects shrink / grow).
- Use iterations to chain multiple passes.

Prerequisites

- Module 10 Lesson 1

1. Why This Matters

Erosion and dilation are the two atomic morphology operations. Every other morphology op (opening, closing, gradient, top-hat) is built from these two.

2. Concept

2.1 Erosion — shrink white regions

For each pixel:

- Output is 255 only if all pixels under the SE are 255 in the input.
- Otherwise, 0.

Effect: white regions shrink; thin white lines disappear; small white specks vanish.

2.2 Dilation — grow white regions

For each pixel:

- Output is 255 if any pixel under the SE is 255.
- Otherwise, 0.

Effect: white regions grow; thin gaps in white fill in; small black holes shrink.

2.3 OpenCV API

```
out = cv2.erode(img, kernel, iterations=1)
out = cv2.dilate(img, kernel, iterations=1)
```

- kernel is the SE.
- iterations chains the op N times. `erode(..., iterations=3) ≈ erode(...)` applied 3 times.

2.4 Duality

Erosion of white = dilation of black, and vice versa. If your foreground is black, just swap the operations.

2.5 Pure-NumPy versions (instructive)

```
def erode_np(bin_img, ksize=3):
    pad = ksize // 2
    padded = np.pad(bin_img, pad, mode='constant', constant_values=0)
    out = np.zeros_like(bin_img)
    for y in range(bin_img.shape[0]):
        for x in range(bin_img.shape[1]):
            if padded[y:y+ksize, x:x+ksize].min() == 255:
                out[y, x] = 255
    return out
```

Slow but shows the "all neighbours" rule clearly. OpenCV's version is hundreds of times faster.

2.6 Grayscale morphology

Erosion / dilation also work on grayscale: erosion = minimum over the SE, dilation = maximum. Useful for shadow / highlight extraction (see top-hat in L4).

3. Code Examples

Example 1: Erode and dilate a binary image

```
import cv2, numpy as np
from skimage.data import coins

img = coins()
_, binary = cv2.threshold(img, 80, 255, cv2.THRESH_BINARY)

se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
eroded = cv2.erode(binary, se)
dilated = cv2.dilate(binary, se)

combo = np.hstack([binary, eroded, dilated])
cv2.imwrite("erode_dilate.png", combo)
```

Expected Output: 3 panels — original mask | shrunken coins (some thin connections gone) | enlarged coins (touching ones merge).

Example 2: Iterations

```
import cv2
from skimage.data import coins

_, binary = cv2.threshold(coins(), 80, 255, cv2.THRESH_BINARY)
se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
for it in [1, 3, 5]:
    cv2.imwrite(f"erode_it{it}.png", cv2.erode(binary, se, iterations=it))
```

Expected Output: Iteration 1: barely shrunken. Iteration 5: coins much smaller; small ones gone.

Example 3: Pure-NumPy verify

```
import cv2, numpy as np

def erode_np(bin_img, ksize=3):
    pad = ksize // 2
    padded = np.pad(bin_img, pad, mode='constant', constant_values=0)
    out = np.zeros_like(bin_img)
    for y in range(bin_img.shape[0]):
        for x in range(bin_img.shape[1]):
            if padded[y:y+ksize, x:x+ksize].min() == 255:
                out[y, x] = 255
    return out

img = np.zeros((20, 20), dtype=np.uint8)
img[5:15, 5:15] = 255

mine = erode_np(img, 3)
cv = cv2.erode(img, cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3)))
print("equal?", np.array_equal(mine, cv))
```

Expected Output: equal? True

Example 4: Grayscale dilation = local max

```
import cv2
from skimage.data import camera

se = cv2.getStructuringElement(cv2.MORPH_RECT, (7, 7))
dilated = cv2.dilate(camera(), se)
cv2.imwrite("camera_dilate_gray.png", dilated)
```

Expected Output: Cameraman image becomes brighter, fine dark detail eroded; bright spots grow.

4. Parameter Sensitivity

Param	Effect
ksize	bigger = stronger shrink / grow per pass
iterations	linear stacking; iter=3 with k=3 $\approx$ k=7 once
SE shape	round / cross / rect — minor differences

5. Common Mistakes

Mistake	What Happens	Fix
Erode white when foreground is black	Wrong direction	Invert with cv2.bitwise_not first, or use dilate
Forgetting iterations exists	Need huge kernel for big effect	Use moderate kernel + iterations
Applying to grayscale by mistake	Min/max instead of binary	Threshold first if you wanted binary

6. Practice Tasks

- Threshold coins() and erode with 5×5 ellipse SE.
- Dilate the same with the same SE; compare.
- Try iterations=3; quantify how many coins remain.

7. Key Takeaways

- Erode: shrink whites; output = min over SE.
- Dilate: grow whites; output = max over SE.
- iterations stacks the effect.
- Together they form the foundation for all morphology.

8. What's Next

Opening and closing — the "denoising" combos.

Lesson 3: Opening and Closing

Module: 10 — Morphological Operations

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Apply MORPH_OPEN and MORPH_CLOSE via cv2.morphologyEx.
- Remember the "key in lock" mnemonic.
- Pick opening vs closing for a given mask defect.

Prerequisites

- Module 10 Lesson 2

1. Why This Matters

Erosion and dilation alone change object size. Opening and closing clean masks while preserving size — they're what you reach for to remove specks (opening) or fill small holes (closing).

2. Concept

2.1 Definitions

- Opening = erosion followed by dilation, same SE.
- Closing = dilation followed by erosion, same SE.

2.2 Visual effects

Operation	Removes	Preserves
Opening	small white specks; thin protrusions	overall object size
Closing	small black holes inside white; thin gaps	overall object size

2.3 The "key in lock" analogy

A structuring element is a "key". For opening: it must fit inside every preserved white region — specks too small for the key are dropped. For closing: it must fit inside every preserved black region — holes too small for the key are filled.

So pick SE size based on the smallest features you want to KEEP.

2.4 OpenCV API

```
cv2.morphologyEx(img, cv2.MORPH_OPEN, se)
cv2.morphologyEx(img, cv2.MORPH_CLOSE, se)
```

Equivalent manually:

```
opened = cv2.dilate(cv2.erode(img, se), se)
closed = cv2.erode (cv2.dilate(img, se), se)
```

2.5 Common pipeline

For a noisy mask:

```
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, small_se) # despeckle
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, small_se) # fill small holes
```

2.6 Confusion check

If you swap them and apply CLOSE first then OPEN, you might over-fill then re-create the same problem you opened to remove. Order matters.

3. Code Examples

Example 1: Despeckle with opening

```
import cv2, numpy as np

np.random.seed(0)
mask = (np.random.rand(200, 200) < 0.15).astype(np.uint8) * 255
cv2.imwrite("noisy_mask.png", mask)

se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
opened = cv2.morphologyEx(mask, cv2.MORPH_OPEN, se)
cv2.imwrite("opened.png", opened)
print("speckles before:", (mask > 0).sum(), "after:", (opened > 0).sum())
```

Expected Output: Far fewer white pixels after opening — small specks gone.

Example 2: Fill holes with closing

```
import cv2, numpy as np

# Build a square with little holes in it
mask = np.zeros((200, 200), dtype=np.uint8)
mask[50:150, 50:150] = 255
np.random.seed(0)
holes = np.random.rand(*mask.shape) < 0.05
mask[holes] = 0
cv2.imwrite("holed_mask.png", mask)

se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
closed = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, se)
cv2.imwrite("closed.png", closed)
```

Expected Output: The square has its small black holes filled in, while remaining a square (closing preserves overall size).

Example 3: OPEN-then-CLOSE pipeline

```
import cv2, numpy as np
from skimage.data import coins
```

```
_, mask = cv2.threshold(coins(), 80, 255, cv2.THRESH_BINARY)
se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, se)
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, se)
cv2.imwrite("coins_clean.png", mask)
```

Expected Output: Coin mask with edges smoothed and any small specks / holes cleaned up.

Example 4: SE size determines what survives

```
import cv2, numpy as np

mask = np.zeros((200, 200), dtype=np.uint8)
cv2.circle(mask, (50, 100), 10, 255, -1) # small disc
cv2.circle(mask, (150, 100), 30, 255, -1) # big disc

for k in [5, 15, 25]:
    se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (k, k))
    cv2.imwrite(f"open_{k}.png", cv2.morphologyEx(mask, cv2.MORPH_OPEN, se))
```

Expected Output: k=5 keeps both. k=15 drops the small disc. k=25 drops both. Demonstrates "key in lock" sizing.

4. Parameter Sensitivity

Param	Effect
SE size	larger = more aggressive removal / filling
SE shape	minor differences
iterations	stack the effect

5. Common Mistakes

Mistake	What Happens	Fix
Open then close when close then open was right	Different outcomes	Think about what you're removing vs filling
SE too small	No visible effect	Try a bigger SE
SE too large	Wipes out objects entirely	Match SE to smallest feature you want to keep

6. Practice Tasks

- Build a noisy 15%-speckle mask. Apply opening with 5×5 ellipse.
- Build a holed-square mask. Apply closing.
- Run open-then-close on the binarised coins() and compare to raw threshold.

7. Key Takeaways

- Opening = erode + dilate → removes specks.
- Closing = dilate + erode → fills small holes.
- SE size determines what's preserved.
- Standard mask cleanup: open then close.

8. What's Next

Gradient, top-hat, black-hat — the niche but valuable morphology ops.

Lesson 4: Gradient, Top-Hat, Black-Hat

Module: 10 — Morphological Operations

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Compute morphological gradient (boundary extraction).
- Use top-hat to find bright details on dark background.
- Use black-hat to find dark details on bright background.

Prerequisites

- Module 10 Lesson 3

1. Why This Matters

These three "compound" ops are the workhorse for boundary outlining (gradient) and removing uneven lighting (top-hat / black-hat). Top-hat in particular is used heavily in OCR pre-processing.

2. Concept

2.1 Morphological gradient

$\text{gradient} = \text{dilation} - \text{erosion}$

The thin outline of objects in a binary or grayscale image. Width $\approx$ SE size.

2.2 Top-hat (white top-hat)

$\text{top_hat} = \text{original} - \text{opening}$

Opening removes small bright features. Subtracting gives just those small bright features, on a near-black background. Great for finding bright spots, text on a dark background, or correcting uneven illumination.

2.3 Black-hat

$\text{black_hat} = \text{closing} - \text{original}$

Closing fills small dark features. Subtracting gives just those small dark features. Used for finding dark text on a bright background or dark dust spots.

2.4 OpenCV API

```
cv2.morphologyEx(img, cv2.MORPH_GRADIENT, se)
cv2.morphologyEx(img, cv2.MORPH_TOPHAT, se)
cv2.morphologyEx(img, cv2.MORPH_BLACKHAT, se)
```

2.5 Use case: text extraction

For text on uneven-lit paper:

```

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
se = cv2.getStructuringElement(cv2.MORPH_RECT, (25, 25))
# Dark text on bright paper → use black-hat
bh = cv2.morphologyEx(gray, cv2.MORPH_BLACKHAT, se)
_, text_mask = cv2.threshold(bh, 30, 255, cv2.THRESH_BINARY)

```

This isolates the text characters regardless of background brightness.

3. Code Examples

Example 1: Gradient — extract outlines

```

import cv2
from skimage.data import coins

_, binary = cv2.threshold(coins(), 80, 255, cv2.THRESH_BINARY)
se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
grad = cv2.morphologyEx(binary, cv2.MORPH_GRADIENT, se)
cv2.imwrite("coins_outlines.png", grad)

```

Expected Output: Thin outlines around each coin.

Example 2: Top-hat — bright details on dark

```

import cv2
from skimage.data import camera

img = camera()
se = cv2.getStructuringElement(cv2.MORPH_RECT, (15, 15))
tophat = cv2.morphologyEx(img, cv2.MORPH_TOPHAT, se)
cv2.imwrite("camera_tophat.png", tophat)

```

Expected Output: Image where bright small features (highlights in the sky, white shirt buttons) stand out against a near-black background.

Example 3: Black-hat — dark text extraction

```

import cv2
from skimage.data import text

img = text() # rough page-of-text image
se = cv2.getStructuringElement(cv2.MORPH_RECT, (25, 25))
bh = cv2.morphologyEx(img, cv2.MORPH_BLACKHAT, se)
_, mask = cv2.threshold(bh, 30, 255, cv2.THRESH_BINARY)
cv2.imwrite("text_mask.png", mask)

```

Expected Output: Just the text glyphs, isolated cleanly on a black background, regardless of paper brightness gradient.

Example 4: Illumination correction with top-hat

```

import cv2, numpy as np
from skimage.data import camera

# Simulate uneven lighting: multiply with a smooth ramp

```

```

img = camera().astype(np.float32)
h, w = img.shape
ramp = np.tile(np.linspace(0.5, 1.5, w), (h, 1))
uneven = np.clip(img * ramp, 0, 255).astype(np.uint8)
cv2.imwrite("uneven.png", uneven)

se = cv2.getStructuringElement(cv2.MORPH_RECT, (51, 51))
bg = cv2.morphologyEx(uneven, cv2.MORPH_OPEN, se) # background estimate
flat = cv2.subtract(uneven, bg) # remove background
cv2.imwrite("uneven_flat.png", flat)

```

Expected Output: uneven_flat.png has the brightness ramp removed — uniform background, foreground details preserved. This is the "rolling-ball" method, popular in microscopy.

4. Parameter Sensitivity

Param	Effect
SE size for gradient	thicker outlines
SE size for top-hat	larger = catches bigger features as "small"
SE size for black-hat	same logic for dark features

5. Common Mistakes

Mistake	What Happens	Fix
Wrong polarity (use top-hat when you needed black-hat)	Empty / inverted output	Decide: bright on dark or dark on bright
SE too small for top-hat	Output near zero	Use SE larger than the features you want to extract
Forgetting threshold after black-hat	Result is grayscale gradient, not binary	Threshold the output

6. Practice Tasks

- Compute morphological gradient of binarised coins().
- Apply top-hat (15×15) on camera().
- Use black-hat + threshold on text() to extract text.

7. Key Takeaways

- Gradient = dilation – erosion (boundaries).
- Top-hat = original – opening (bright small features).
- Black-hat = closing – original (dark small features).
- Top-hat / black-hat fix uneven lighting and isolate text / dust.

8. What's Next

Practical mask cleaning — putting the morphology toolbox to work in pipelines.

Lesson 5: Practical Mask Cleaning

Module: 10 — Morphological Operations

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Pick a mask-cleaning pipeline based on defects observed.
- Separate touching objects with erosion.
- Reconnect broken edges with closing.

Prerequisites

- Module 10 Lesson 4

1. Why This Matters

In real pipelines you almost never threshold once and use the result. Masks need cleanup. Knowing which morphology op to reach for in each situation saves trial-and-error.

2. Concept

2.1 Defect → fix cheat-sheet

You see	Fix
Salt-and-pepper noise in mask	MORPH_OPEN with small SE
Holes inside white regions	MORPH_CLOSE
Touching objects you want separated	extra cv2.erode iterations
Broken object boundaries	MORPH_CLOSE with larger SE
Need just object outlines	MORPH_GRADIENT
Outlines too thick after gradient	smaller SE

2.2 Standard cleanup pipeline

```
def clean(mask, k=5):  
    se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (k, k))  
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, se)  
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, se)  
    return mask
```

Adapt SE size to your image scale.

2.3 Separating touching objects

When two objects share a boundary, threshold treats them as one blob. Erode aggressively to "shrink" them into separate cores:

```
sure_fg = cv2.erode(mask, se, iterations=3)
```

The result is too small but each blob is now a distinct connected component. Use this with the distance transform / watershed (M11) for full separation.

2.4 Reconnecting broken edges

Pre-edge content with gaps can be sealed:

```
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE,
                          cv2.getStructuringElement(cv2.MORPH_RECT, (15, 1))) #
horizontal close
```

Asymmetric SE closes along one direction — great for sealing horizontal text lines.

2.5 Sanity check after cleaning

Count connected components before/after:

```
n, _ = cv2.connectedComponents(mask)
print("components:", n - 1) # subtract 1 for background
```

A successful clean usually reduces noise components dramatically.

3. Code Examples

Example 1: Full clean pipeline

```
import cv2, numpy as np
from skimage.data import coins

raw_mask = cv2.threshold(coins(), 80, 255, cv2.THRESH_BINARY)[1]
# Add some synthetic noise
np.random.seed(0)
sp = np.random.rand(*raw_mask.shape)
raw_mask[sp < 0.01] = 0
raw_mask[sp > 0.99] = 255

se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
opened = cv2.morphologyEx(raw_mask, cv2.MORPH_OPEN, se)
cleaned = cv2.morphologyEx(opened, cv2.MORPH_CLOSE, se)

combo = np.hstack([raw_mask, opened, cleaned])
cv2.imwrite("clean_pipeline.png", combo)
print("components before:", cv2.connectedComponents(raw_mask)[0] - 1)
print("components after :", cv2.connectedComponents(cleaned)[0] - 1)
```

Expected Output: 3 panels — noisy mask | speckles gone (open) | also holes filled (close).

Component count drops dramatically.

Example 2: Aggressive erosion to separate touching objects

```
import cv2, numpy as np

# Two touching circles
mask = np.zeros((200, 400), dtype=np.uint8)
cv2.circle(mask, (150, 100), 80, 255, -1)
```

```

cv2.circle(mask, (250, 100), 80, 255, -1)

se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
core = cv2.erode(mask, se, iterations=15)

n_raw, _ = cv2.connectedComponents(mask)
n_core, _ = cv2.connectedComponents(core)
print("raw components:", n_raw - 1, " core components:", n_core - 1)
cv2.imwrite("touching_raw.png", mask)
cv2.imwrite("touching_core.png", core)

```

Expected Output: raw=1 (one merged blob), core=2 (two separated). Eroded cores are smaller but distinct.

Example 3: Reconnect broken text lines

```

import cv2, numpy as np

# Synthesize broken horizontal lines
mask = np.zeros((100, 300), dtype=np.uint8)
mask[20:25, 10: 80] = 255
mask[20:25, 90:160] = 255      # gap at 80-90
mask[20:25, 170:240] = 255    # gap at 160-170

se = cv2.getStructuringElement(cv2.MORPH_RECT, (15, 1))
closed = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, se)
cv2.imwrite("text_lines_broken.png", mask)
cv2.imwrite("text_lines_closed.png", closed)

```

Expected Output: The broken horizontal segments become a single continuous line.

Example 4: Connected components for diagnosis

```

import cv2
from skimage.data import coins

_, mask = cv2.threshold(coins(), 80, 255, cv2.THRESH_BINARY)
n, _ = cv2.connectedComponents(mask)
print("blobs in raw mask:", n - 1)

se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
clean = cv2.morphologyEx(mask, cv2.MORPH_OPEN, se)
n2, _ = cv2.connectedComponents(clean)
print("blobs after open :", n2 - 1)

```

Expected Output: Component count typically drops after opening — noise specks removed.

4. Parameter Sensitivity

Param	Effect
SE size	bigger removes more / merges more
Iterations	linear stacking
SE asymmetry	direction-specific cleaning

5. Common Mistakes

Mistake	What Happens	Fix
Skip cleanup; pipe noisy mask to contour stage	Hundreds of bogus contours	Open + close first
Over-erode to separate objects	All objects vanish	Use minimum erosion + watershed (M11)
Wrong SE direction	Won't reconnect gaps	Use a row / column SE for directional ops

6. Practice Tasks

- Make a noisy binarised coins(); apply the standard open-close pipeline.
- Build two touching circles; erode them apart and count components.
- Build broken horizontal segments; reconnect with a (15, 1) closing.

7. Key Takeaways

- Match the op to the defect (cheat-sheet).
- Standard cleanup = OPEN then CLOSE.
- Aggressive erosion separates touching objects.
- Asymmetric SEs handle direction-specific defects.

8. What's Next

End of Module 10. Next: Module 11 — Segmentation & Thresholding.

Module 10 — Exercises

15 exercises.

10.1 Binary & SEs

- (E) Build a 7×7 ELLIPSE SE; print.
- (M) Build a 9×9 CROSS SE; print.
- (H) Threshold coins() at 80; save mask.

10.2 Erode / Dilate

- (E) Erode binarised coins with 5×5 ellipse.
- (M) Dilate the same; compare.
- (H) Verify pure-NumPy erode against cv2.erode on a small 20×20 patch.

10.3 Open / Close

- (E) Build 15% speckle mask; apply open(5).
- (M) Build holed square; apply close(5).
- (H) Open-then-close on binarised coins() and count components before/after.

10.4 Gradient / Top-hat / Black-hat

- (E) Gradient of binarised coins.
- (M) Top-hat (15×15) on camera().
- (H) Black-hat + threshold on text().

10.5 Practical

- (E) Standard clean pipeline on noisy coins() mask.
- (M) Erode 2 touching circles apart; count components.
- (H) Reconnect broken horizontal segments via (15,1) close.

Module 10 — Quiz

10 MCQs.

Q1

Erosion on white pixels: A. grows them B. shrinks them C. inverts them D. no change

Answer: B (L2)

Q2

Dilation on white pixels: A. grows them B. shrinks them C. inverts them D. blurs

Answer: A (L2)

Q3

Opening removes: A. small white specks B. holes C. edges D. everything

Answer: A (L3)

Q4

Closing fills: A. small white specks B. small black holes C. edges D. nothing

Answer: B (L3)

Q5

Opening = ? A. dilate then erode B. erode then dilate C. dilate twice D. erode twice

Answer: B (L3)

Q6

Morphological gradient = ? A. dilate – erode B. erode – dilate C. open – close D. close – open

Answer: A (L4)

Q7

Top-hat extracts: A. dark small features B. bright small features C. edges D. holes

Answer: B (L4)

Q8

For black text on bright paper, you'd use: A. top-hat B. black-hat C. gradient D. dilation

Answer: B (L4)

Q9

For separating two touching objects, the right first step is: A. dilation B. opening C. erosion (multiple iterations) D. closing

Answer: C (L5)

Q10

Standard noisy-mask cleanup is: A. open + close B. erode 10x C. blur D. invert

Answer: A (L5)

Module 10 — Assignment: Mask Cleaner

Estimated time: 1.5 hours

Combines: Module 10 + Modules 4, 6, 9

Brief

CLI tool that takes a binary mask (or makes one via thresholding), reports its noise stats, and cleans it.

Requirements

- `mask_clean.py INPUT [--threshold 128] [--ksize 5] [--output OUT]`
- If input is color, convert to gray and threshold.
- Compute: connected components before clean.
- Apply opening + closing with ellipse SE.
- Report components after.
- Save cleaned mask.

Constraints

- Modules 1-10 only.
- Use `cv2.connectedComponents`.
- Use `argparse`.

Expected Output

```
$ python mask_clean.py coins.png --threshold 80
components before: 73
components after : 18
saved -> coins_mask_clean.png
```

Grading Rubric

Criterion	Weight
Reads + thresholds	20%
Component count	20%
Open + close pipeline	30%
Reporting	15%
Style	15%

Submission

Save as `assignment_module10.py`.

Module 10 — Mini Project: Morphology Playground

Overview

Interactive morphology playground. Load any image, threshold it, then experiment with erosion / dilation / opening / closing / gradient / top-hat / black-hat live.

Concepts Used

- All of Module 10
- Trackbars (M2 L3), threshold (M11 preview)

Features

- Trackbars: threshold, ksize, op-type (0..6), iterations.
- Side-by-side: thresholded mask | morphology result.
- Save (s), quit (q).

Starter Code

morph_playground.py:

```
import sys, cv2, numpy as np

OPS = ["erode", "dilate", "open", "close", "gradient", "tophat", "blackhat"]

def apply_op(mask, op, k, it):
    se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (k, k))
    if op == "erode":    return cv2.erode(mask, se, iterations=it)
    if op == "dilate":   return cv2.dilate(mask, se, iterations=it)
    if op == "open":     return cv2.morphologyEx(mask, cv2.MORPH_OPEN, se,
iterations=it)
    if op == "close":    return cv2.morphologyEx(mask, cv2.MORPH_CLOSE, se,
iterations=it)
    if op == "gradient": return cv2.morphologyEx(mask, cv2.MORPH_GRADIENT, se)
    if op == "tophat":   return cv2.morphologyEx(mask, cv2.MORPH_TOPHAT, se)
    if op == "blackhat": return cv2.morphologyEx(mask, cv2.MORPH_BLACKHAT, se)

def run(path):
    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if img is None: raise FileNotFoundError(path)
    if max(img.shape) > 700: img = cv2.resize(img, None, fx=0.5, fy=0.5)

    cv2.namedWindow("morph")
    cv2.createTrackbar("threshold", "morph", 128, 255, lambda v: None)
    cv2.createTrackbar("ksize",      "morph",  5,  31, lambda v: None)
    cv2.createTrackbar("op",         "morph",  2,  6, lambda v: None)    # 0..6
    cv2.createTrackbar("iter",       "morph",  1, 10, lambda v: None)

    while True:
        th = cv2.getTrackbarPos("threshold", "morph")
        k  = max(1, cv2.getTrackbarPos("ksize", "morph"))
        if k % 2 == 0: k += 1
```

```

op_idx = cv2.getTrackbarPos("op", "morph")
it = max(1, cv2.getTrackbarPos("iter", "morph"))

_, mask = cv2.threshold(img, th, 255, cv2.THRESH_BINARY)
out = apply_op(mask, OPS[op_idx], k, it)
combo = np.hstack([mask, out])
cv2.putText(combo, f"{OPS[op_idx]} k={k} it={it} th={th}", (10, 20),
              cv2.FONT_HERSHEY_SIMPLEX, 0.5, 128, 1)
cv2.imshow("morph", combo)
kk = cv2.waitKey(30) & 0xFF
if kk == ord('q'): break
if kk == ord('s'): cv2.imwrite("morph_out.png", out); print("saved
morph_out.png")
cv2.destroyAllWindows()

if __name__ == "__main__":
    run(sys.argv[1] if len(sys.argv) > 1 else "datasets/starter/coins.png")

```

Expected Interaction

Slide threshold to set a mask, pick op, tune ksize / iterations live.

Extension Challenges

- Add a connected-component count badge on the result.
- Add "show original" toggle (o).
- Add asymmetric SE option ((15,1) etc.).

Grading Rubric

Criterion	Weight
All 7 ops work	35%
Threshold + ksize live	25%
Side-by-side display	15%
Save / quit	10%
Style	15%

Python E-Course

Module 11 — Segmentation & Thresholding

Detailed Notes

Module Overview

Level: Segmentation

Duration: ~1.5 weeks

Prerequisites: Modules 1-10

What You'll Learn

- Threshold images globally and adaptively.
- Use Otsu's method for automatic global thresholds.
- Run watershed for touching-object segmentation.
- Use GrabCut for interactive foreground extraction.

Lessons

#	Lesson	Duration
1	Binary Thresholding	25 min
2	Otsu's Method	25 min
3	Adaptive Thresholding	30 min
4	Watershed	35 min
5	GrabCut (Interactive Foreground)	30 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 12: Contours & Shape Analysis

Lesson 1: Binary Thresholding

Module: 11 — Segmentation & Thresholding

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Apply `cv2.threshold` with the five common type flags.
- Pick a sensible threshold from a histogram.
- Use binary masks as ROI selectors.

Prerequisites

- Modules 1-10

1. Why This Matters

Thresholding is the simplest segmentation method — turn a grayscale image into a binary "foreground vs background" mask. Despite its simplicity, it's surprisingly effective when the histogram is bimodal.

2. Concept

2.1 The function

```
ret, dst = cv2.threshold(src, thresh, maxval, type)
```

- `src` — grayscale uint8 image.
- `thresh` — the threshold value.
- `maxval` — the value used for pixels above the threshold (255 for binary).
- `type` — one of the flags below.

2.2 Threshold types

Type	Behaviour
<code>THRESH_BINARY</code>	<code>pixel > thresh → maxval</code> , else 0
<code>THRESH_BINARY_INV</code>	<code>pixel > thresh → 0</code> , else maxval
<code>THRESH_TRUNC</code>	<code>pixel > thresh → thresh</code> , else unchanged
<code>THRESH_TOZERO</code>	<code>pixel > thresh → unchanged</code> , else 0
<code>THRESH_TOZERO_INV</code>	<code>pixel > thresh → 0</code> , else unchanged

The first two are the "make a mask" forms; the others are clipping operations.

2.3 Reading a histogram for a threshold

Plot the histogram (M6 L1). If you see two clear peaks (bimodal), pick a value in the valley between them. If the peaks aren't clear, use Otsu (next lesson).

2.4 Apply on color requires a choice

`cv2.threshold` works on a single channel. For color images:

- Convert to grayscale, threshold.
- Or convert to HSV and threshold on a channel (e.g., V to find bright regions).
- Or use `cv2.inRange` for multi-channel thresholding (M4).

2.5 The simple recipe

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, mask = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY)
```

3. Code Examples

Example 1: Basic threshold

```
import cv2
from skimage.data import coins

img = coins()
_, mask = cv2.threshold(img, 80, 255, cv2.THRESH_BINARY)
cv2.imwrite("coins_thresh80.png", mask)
print("foreground %:", (mask > 0).mean() * 100)
```

Expected Output: `coins_thresh80.png` shows white coins on black background. foreground %: ~25 (varies).

Example 2: Try several thresholds

```
import cv2, numpy as np
from skimage.data import coins

img = coins()
for t in [60, 90, 120, 150]:
    _, mask = cv2.threshold(img, t, 255, cv2.THRESH_BINARY)
    cv2.imwrite(f"coins_t{t}.png", mask)
    print(f"t={t:3d} fg={ (mask > 0).mean()*100:5.1f}%")
```

Expected Output: Lower thresholds → more white (over-segmenting); higher thresholds → less white (loses coin tops).

Example 3: All five threshold types

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
types = [cv2.THRESH_BINARY, cv2.THRESH_BINARY_INV, cv2.THRESH_TRUNC,
         cv2.THRESH_TOZERO, cv2.THRESH_TOZERO_INV]
names = ["BINARY", "BINARY_INV", "TRUNC", "TOZERO", "TOZERO_INV"]
for t, n in zip(types, names):
    _, out = cv2.threshold(img, 127, 255, t)
    cv2.imwrite(f"types_{n}.png", out)
```

Expected Output: Five files showing the different behaviors — BINARY makes a hard mask, TRUNC caps bright values, TOZERO blackens dark values, etc.

Example 4: Use the mask as ROI selector

```
import cv2
from skimage.data import coins

img = coins()
_, mask = cv2.threshold(img, 80, 255, cv2.THRESH_BINARY)
# show only the coins, blacken the background
result = cv2.bitwise_and(img, img, mask=mask)
cv2.imwrite("coins_masked.png", result)
```

Expected Output: Coin image where the background is solid black; the coins themselves are preserved at original intensity.

4. Parameter Sensitivity

Param	Effect
thresh	low → more "foreground"; high → less
type	binary vs clipping behaviour
maxval	what to set pixels above thresh to (usually 255)

5. Common Mistakes

Mistake	What Happens	Fix
Passing a color image	Error or wrong result	Convert to gray first
Threshold on raw uneven lighting	Black blobs on one side, all-white on the other	Use adaptive (L3)
Forgetting ret is the threshold value	TypeError when unpacking	Always ret, mask = cv2.threshold(...)

6. Practice Tasks

- Threshold coins() at 60, 90, 120. Compare.
- Apply THRESH_TOZERO at 100 to camera().
- Use a threshold mask to keep only the coins, blacken background.

7. Key Takeaways

- cv2.threshold(src, thresh, maxval, type) returns (ret, dst).
- Five threshold types — binary forms make masks; others clip values.
- Picking thresh from a bimodal histogram works; otherwise use Otsu.
- Always convert color to gray first.

8. What's Next

Otsu's method — automatic threshold picking.

Lesson 2: Otsu's Method

Module: 11 — Segmentation & Thresholding

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Run Otsu thresholding with `cv2.THRESH_OTSU`.
- Explain the principle (maximise between-class variance).
- Recognise when Otsu fails (non-bimodal histograms).

Prerequisites

- Module 11 Lesson 1

1. Why This Matters

Otsu picks the optimal global threshold automatically — no human guessing. It's the right default when the histogram is bimodal (clear foreground / background separation).

2. Concept

2.1 The intuition

Imagine sliding a candidate threshold from 0 to 255. At each value, split the histogram into two groups: pixels below, pixels above. Compute the variance between the two groups (or equivalently, minimise within-group variance). The threshold that maximises between-class variance is Otsu's choice.

2.2 OpenCV API

```
otsu_value, mask = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

Pass 0 as the threshold — Otsu's algorithm picks it. The function also returns the chosen value in `otsu_value`.

You can combine with other flags:

```
cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
```

2.3 Pre-blur for stability

Like every threshold method, Otsu is noise-sensitive. A small Gaussian blur first stabilises the threshold:

```
blur = cv2.GaussianBlur(gray, (5, 5), 0)
otsu, mask = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

2.4 When Otsu fails

- Unimodal histogram (uniform image): no "valley" to pick → produces a meaningless mask.

- Uneven lighting: one global threshold can't fit; use adaptive (next lesson).
- More than 2 classes: Otsu only picks one boundary; use multi-level Otsu (not built into cv2; use `skimage.filters.threshold_multiotsu`).

2.5 Bimodality check

A quick test: compute histogram, find peaks, ratio of valley to peak. If the valley is at least 30% lower than both peaks, Otsu should work well.

3. Code Examples

Example 1: Basic Otsu

```
import cv2
from skimage.data import coins

img = coins()
otsu, mask = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
print("Otsu chose:", otsu)
cv2.imwrite("coins_otsu.png", mask)
```

Expected Output: Otsu chose: ~108 and coins_otsu.png showing clean coin separation from background.

Example 2: Bimodal vs unimodal histograms

```
import cv2, numpy as np
import matplotlib.pyplot as plt
from skimage.data import coins

bimodal = coins() # clear background + coins
unimodal = (np.random.rand(300, 400) * 255).astype(np.uint8)

for name, img in [("bimodal", bimodal), ("unimodal", unimodal)]:
    plt.figure()
    plt.hist(img.ravel(), 256, [0, 256])
    plt.title(name)
    plt.show()
    otsu, _ = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    print(f"{name:10s} Otsu={otsu}")
```

Expected Output: Bimodal histogram shows two clear humps; unimodal is flat. Otsu still returns a value for the unimodal case (~127) but the result is meaningless.

Example 3: Pre-blur for stability

```
import cv2
from skimage.data import coins

img = coins()
blur = cv2.GaussianBlur(img, (5, 5), 0)
otsu1, mask1 = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
otsu2, mask2 = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
print(f"no blur: {otsu1}    with blur: {otsu2}")
```

Expected Output: Both threshold values are close (~108); blur typically nudges things a few units.

Example 4: Multi-level Otsu via skimage

```
import numpy as np
from skimage.data import camera
from skimage.filters import threshold_multiotsu

img = camera()
thresholds = threshold_multiotsu(img, classes=3)
print("multi-Otsu thresholds:", thresholds)
labels = np.digitize(img, bins=thresholds)
print("classes per pixel range:", labels.min(), labels.max())
```

Expected Output: Two threshold values printed (e.g. [58, 142]); labels is 0/1/2 indicating which intensity class each pixel belongs to.

4. Parameter Sensitivity

Param	Effect
pre-blur σ	bigger = more stable threshold, less detail
number of classes (multi-Otsu)	2 (single Otsu) for binary; 3+ for richer segmentation

5. Common Mistakes

Mistake	What Happens	Fix
Pass non-zero threshold	OpenCV ignores it but it's confusing	Use 0 as the argument
Use Otsu on uneven lighting	Big chunks fail	Use adaptive threshold
Use Otsu on unimodal	Meaningless threshold	Check histogram first

6. Practice Tasks

- Otsu on coins(); print chosen value.
- Plot histograms of coins() vs uniform-random; observe bimodality difference.
- Pre-blur with $\sigma=2$; compare Otsu threshold values.

7. Key Takeaways

- Otsu picks the threshold by maximising between-class variance.
- Pass 0 to cv2.threshold with THRESH_OTSU flag.
- Works on bimodal histograms; fails on unimodal or uneven lighting.
- Pre-blur slightly for stability.

8. What's Next

Adaptive thresholding — for images with uneven lighting where one global value can't work.

Lesson 3: Adaptive Thresholding

Module: 11 — Segmentation & Thresholding

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Apply `cv2.adaptiveThreshold` with `ADAPTIVE_THRESH_MEAN_C` and `_GAUSSIAN_C`.
- Tune `blockSize` and `C`.
- Recognise when adaptive beats Otsu (uneven lighting).

Prerequisites

- Module 11 Lesson 2

1. Why This Matters

Global thresholds — even Otsu — fail when one side of an image is brighter than the other. Adaptive thresholding picks a local threshold per region, perfect for documents photographed under varying light.

2. Concept

2.1 How it works

For each pixel, look at its neighbourhood (e.g. 11×11), compute a local threshold value, and compare. Neighbours brighter than the pixel $\rightarrow$ pixel becomes foreground; darker $\rightarrow$ background.

The local threshold is:

- `MEAN_C`: $\text{mean}(\text{neighbours}) - C$
- `GAUSSIAN_C`: weighted Gaussian mean $- C$

C is a small constant subtracted to tune sensitivity.

2.2 OpenCV API

```
out = cv2.adaptiveThreshold(  
    src, maxval, adaptiveMethod, thresholdType, blockSize, C)
```

- `adaptiveMethod`: `cv2.ADAPTIVE_THRESH_MEAN_C` or `_GAUSSIAN_C`.
- `thresholdType`: `THRESH_BINARY` or `THRESH_BINARY_INV`.
- `blockSize`: odd, ≥ 3 . Size of the neighbourhood.
- C : integer subtracted from mean. Larger C = stricter (less foreground).

2.3 Picking `blockSize`

- Too small (3-7): noisy mask, every speck triggers.

- Just right (11-25): captures local features without noise.
- Too large (50+): approaches global thresholding.

The rule: blockSize should be larger than the features you want to detect.

2.4 Picking C

Typical values 2-10. Start at 5 and tune.

2.5 MEAN vs GAUSSIAN

GAUSSIAN tends to look smoother (less staircasing on edges). Use it as the default.

2.6 The classic use case

Old paper photographed with uneven flash:

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
out = cv2.adaptiveThreshold(gray, 255,
                           cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                           cv2.THRESH_BINARY, 25, 8)
```

Text comes out black on white, regardless of which corner of the paper is in shadow.

3. Code Examples

Example 1: Compare on uneven-lit text

```
import cv2, numpy as np
from skimage.data import text

img = text()
# Simulate uneven lighting
h, w = img.shape
ramp = np.tile(np.linspace(0.4, 1.4, w), (h, 1))
uneven = np.clip(img * ramp, 0, 255).astype(np.uint8)
cv2.imwrite("uneven.png", uneven)

_, otsu = cv2.threshold(uneven, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
adapt = cv2.adaptiveThreshold(uneven, 255,
                             cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                             cv2.THRESH_BINARY, 25, 10)

cv2.imwrite("uneven_otsu.png", otsu)
cv2.imwrite("uneven_adapt.png", adapt)
```

Expected Output: otsu fails — bright side mostly white, dark side mostly black. adapt produces clean text across the whole image.

Example 2: blockSize sensitivity

```
import cv2
from skimage.data import text

img = text()
```



```

for bs in [3, 11, 25, 51]:
    out = cv2.adaptiveThreshold(img, 255,
                               cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                               cv2.THRESH_BINARY, bs, 8)
    cv2.imwrite(f"adapt_bs{bs}.png", out)

```

Expected Output: bs=3 is noisy; bs=25 captures text cleanly; bs=51 starts to behave like global.

Example 3: C sensitivity

```

import cv2
from skimage.data import text

img = text()
for c in [0, 5, 10, 20]:
    out = cv2.adaptiveThreshold(img, 255,
                               cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                               cv2.THRESH_BINARY, 25, c)
    cv2.imwrite(f"adapt_c{c}.png", out)

```

Expected Output: Low C: more foreground (text + noise). High C: less foreground (text might thin out).

Example 4: MEAN vs GAUSSIAN

```

import cv2
from skimage.data import text

img = text()
m = cv2.adaptiveThreshold(img, 255, cv2.ADAPTIVE_THRESH_MEAN_C,
                          cv2.THRESH_BINARY, 25, 10)
g = cv2.adaptiveThreshold(img, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                          cv2.THRESH_BINARY, 25, 10)
cv2.imwrite("adapt_mean.png", m)
cv2.imwrite("adapt_gauss.png", g)

```

Expected Output: Both capture text; GAUSSIAN looks slightly smoother / less jaggy.

4. Parameter Sensitivity

Param	Effect
blockSize	small = noisy / fine; large = global-like
C	small = more foreground; large = stricter
MEAN vs GAUSSIAN	GAUSSIAN smoother by a small margin

5. Common Mistakes

Mistake	What Happens	Fix
Even blockSize	Error	Use odd
blockSize too small	Noise everywhere	11+ for typical images
C = 0	Foreground includes noise	Try 5-10
Use on uniform-lit image	OK but no advantage over Otsu	Use simpler method

6. Practice Tasks

- Apply adaptive Gaussian on `text()` with `blockSize=25`, `C=10`.
- Compare to Otsu on the same image with simulated uneven lighting.
- Tune `blockSize` $\in \{3, 11, 25, 51\}$ and pick a good value.

7. Key Takeaways

- `cv2.adaptiveThreshold` picks per-region threshold = mean – C.
- Use `GAUSSIAN_C` as default; `blockSize` odd, larger than features.
- Best for uneven lighting (documents, scans).
- C tunes how strict the foreground assignment is.

8. What's Next

Watershed — handle touching objects that thresholding alone can't separate.

Lesson 4: Watershed

Module: 11 — Segmentation & Thresholding

Duration: 35 minutes

Difficulty: Intermediate-Advanced

Learning Objectives

- Run the OpenCV watershed pipeline.
- Build sure foreground / sure background / unknown markers.
- Use distance transform to seed markers.

Prerequisites

- Module 11 Lesson 3, morphology (M10).

1. Why This Matters

Touching objects (overlapping coins, cells, packed nuts) merge into one blob after thresholding. Watershed treats the image as a topographic surface and "floods" basins from marker seeds — separating touching objects into distinct labels.

2. Concept

2.1 The intuition

Imagine the inverse of the image as a 3D landscape: bright spots are valleys, dark spots are hills. Drop water into each marker; water flows downhill and meets at "watershed lines" — the boundaries between objects.

2.2 The OpenCV pipeline

- Threshold (often Otsu) → rough binary mask.
- Clean with morphology (open / close).
- Sure background = dilation of mask.
- Distance transform of mask; threshold high values → sure foreground (object cores).
- Unknown = sure_bg – sure_fg.
- Connected components on sure_fg → marker labels.
- Add 1 to all markers (so background isn't 0); set unknown region to 0.
- `cv2.watershed(image_color, markers)` — modifies markers in place: -1 along watershed lines, integer label elsewhere.

2.3 Distance transform

`cv2.distanceTransform(mask, distanceType, maskSize)` — value at each white pixel = distance to nearest black pixel. Object cores (far from any boundary) get the largest values; edge pixels get small values.

```
dist = cv2.distanceTransform(mask, cv2.DIST_L2, 5)
```

Threshold the top X% of dist to get sure cores.

2.4 Why "markers"?

Watershed needs to know where every object's "seed" is. Markers = integer-labeled image: 0 for unknown, 1 for background, 2, 3, 4, ... for each object.

2.5 OpenCV API

```
cv2.watershed(img_bgr, markers)
```

- `img_bgr` must be 3-channel.
- `markers` is `int32`, modified in place.
- After: pixels at watershed boundaries have value -1; other pixels have their marker label.

3. Code Examples

Example 1: Full watershed pipeline on touching circles

```
import cv2, numpy as np

# Build 3 touching circles
img = np.full((300, 600, 3), 240, dtype=np.uint8)
cv2.circle(img, (200, 150), 80, (50, 50, 50), -1)
cv2.circle(img, (300, 150), 80, (50, 50, 50), -1)
cv2.circle(img, (400, 150), 80, (50, 50, 50), -1)
cv2.imwrite("ws_input.png", img)

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, mask = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

# Cleanup
se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, se, iterations=2)

# sure bg
sure_bg = cv2.dilate(mask, se, iterations=3)

# sure fg via distance transform
dist = cv2.distanceTransform(mask, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(dist, 0.5 * dist.max(), 255, 0)
sure_fg = sure_fg.astype(np.uint8)

unknown = cv2.subtract(sure_bg, sure_fg)

n, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[unknown == 255] = 0

cv2.watershed(img, markers)

out = img.copy()
out[markers == -1] = (0, 0, 255)          # watershed lines in red
```

```
cv2.imwrite("ws_output.png", out)
print("number of objects found:", n - 1)
```

Expected Output: ws_output.png shows the 3 circles with red boundary lines between them — successful separation. Console prints number of objects found: 3.

Example 2: Watershed on touching coins

```
import cv2, numpy as np
from skimage.data import coins

gray = coins()
img = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)

_, mask = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, se, iterations=2)

sure_bg = cv2.dilate(mask, se, iterations=3)
dist = cv2.distanceTransform(mask, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(dist, 0.5 * dist.max(), 255, 0)
sure_fg = sure_fg.astype(np.uint8)
unknown = cv2.subtract(sure_bg, sure_fg)

n, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[unknown == 255] = 0

cv2.watershed(img, markers)
out = img.copy()
out[markers == -1] = (0, 0, 255)
cv2.imwrite("coins_ws.png", out)
print("coins detected:", n - 1)
```

Expected Output: Coins outlined in red where they touch — typically separates most of them correctly.

Example 3: Distance transform visualisation

```
import cv2, numpy as np
from skimage.data import coins

_, mask = cv2.threshold(coins(), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
dist = cv2.distanceTransform(mask, cv2.DIST_L2, 5)
dist_vis = cv2.normalize(dist, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
cv2.imwrite("coins_dist.png", dist_vis)
print("max distance:", dist.max())
```

Expected Output: coins_dist.png is a grayscale image where coin centres are bright (far from edge) and coin boundaries are dark. Max distance $\approx$ coin radius in pixels.

4. Parameter Sensitivity

Param	Effect
-------	--------

Distance threshold (% of max)	Too low: cores merge again; too high: small objects missed
Cleanup iterations	Too few: noise leaks in; too many: cores too small

5. Common Mistakes

Mistake	What Happens	Fix
Pass grayscale to cv2.watershed	Error	Convert to BGR first
Marker dtype not int32	Error	markers.astype(np.int32)
Marker label 0 used for an object	Conflict with "unknown"	Add 1 to all markers
Skip distance transform; just use mask	Touching objects stay merged	Use distance + threshold for seeds

6. Practice Tasks

- Build 3 touching circles; run watershed pipeline; verify 3 objects.
- Apply to binarised coins(); count separated objects.
- Visualise the distance transform of any mask.

7. Key Takeaways

- Watershed needs markers: background, sure foreground, unknown.
- Distance transform gives object "cores" for sure-foreground seeds.
- cv2.watershed(bgr, markers_int32) modifies markers in place.
- Watershed lines = pixels where markers == -1.

8. What's Next

GrabCut — interactive foreground extraction.

Lesson 5: GrabCut (Interactive Foreground)

Module: 11 — Segmentation & Thresholding

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Use cv2.grabCut for one-click foreground extraction.
- Provide a bounding-box init.
- Refine with mask input.

Prerequisites

- Module 11 Lesson 4

1. Why This Matters

GrabCut produces clean foreground masks from a rough bounding box (or scribbles). It's the engine behind tools like "magic wand" in image editors and is far more sophisticated than simple thresholding.

2. Concept

2.1 The algorithm (very brief)

GrabCut models foreground / background as Gaussian Mixture Models on color, then uses graph cuts to find the optimal segmentation under those models. Iterates 5-10 times for convergence.

2.2 OpenCV API — rectangle init

```
mask = np.zeros(img.shape[:2], dtype=np.uint8)
bgd_model = np.zeros((1, 65), dtype=np.float64)
fgd_model = np.zeros((1, 65), dtype=np.float64)
rect = (x, y, w, h) # bounding box around target
cv2.grabCut(img, mask, rect, bgd_model, fgd_model, 5, cv2.GC_INIT_WITH_RECT)
```

After running:

```
mask_final = np.where((mask == cv2.GC_FGD) | (mask == cv2.GC_PR_FGD), 255,
0).astype(np.uint8)
```

2.3 Mask values

Constant	Meaning
cv2.GC_BGD (0)	definite background
cv2.GC_FGD (1)	definite foreground
cv2.GC_PR_BGD (2)	probable background
cv2.GC_PR_FGD (3)	probable foreground

2.4 Refinement

After the initial run, you can paint into the mask manually (definite FG / BG) and re-run with `cv2.GC_INIT_WITH_MASK`:

```
mask[manual_fg] = cv2.GC_FGD
mask[manual_bg] = cv2.GC_BGD
cv2.grabCut(img, mask, None, bgd_model, fgd_model, 5, cv2.GC_INIT_WITH_MASK)
```

2.5 Use cases

- Cutout for product photos.
- Replacing backgrounds (poor man's green-screen).
- Initial mask for further refinement / training data.

2.6 Limitations

- Slow (several seconds for HD).
- Needs good initial input — randomly placed rect won't help.
- Struggles with similar foreground / background colors.

3. Code Examples

Example 1: GrabCut on astronaut with bounding box

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
mask = np.zeros(img.shape[:2], dtype=np.uint8)
bgd, fgd = np.zeros((1, 65), np.float64), np.zeros((1, 65), np.float64)
rect = (60, 30, 380, 460) # x, y, w, h covering the astronaut
cv2.grabCut(img, mask, rect, bgd, fgd, 5, cv2.GC_INIT_WITH_RECT)

m_final = np.where((mask == cv2.GC_FGD) | (mask == cv2.GC_PR_FGD), 255,
0).astype(np.uint8)
out = cv2.bitwise_and(img, img, mask=m_final)
cv2.imwrite("astro_grabcut.png", out)
cv2.imwrite("astro_mask.png", m_final)
```

Expected Output: `astro_grabcut.png` shows the astronaut cleanly extracted with the background blackened.

Example 2: Replace background after GrabCut

```
import cv2, numpy as np
from skimage.data import astronaut, coffee

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
mask = np.zeros(img.shape[:2], dtype=np.uint8)
bgd, fgd = np.zeros((1, 65), np.float64), np.zeros((1, 65), np.float64)
cv2.grabCut(img, mask, (60, 30, 380, 460), bgd, fgd, 5, cv2.GC_INIT_WITH_RECT)
m = np.where((mask == cv2.GC_FGD) | (mask == cv2.GC_PR_FGD), 255,
0).astype(np.uint8)
```



```

new_bg = cv2.resize(cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR), (img.shape[1],
img.shape[0]))
inv = cv2.bitwise_not(m)
fg = cv2.bitwise_and(img, img, mask=m)
bg = cv2.bitwise_and(new_bg, new_bg, mask=inv)
comp = cv2.add(fg, bg)
cv2.imwrite("astro_on_coffee.png", comp)

```

Expected Output: Astronaut placed over the coffee photo.

Example 3: Iterations matter

```

import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
rect = (60, 30, 380, 460)
for it in [1, 3, 5, 10]:
    mask = np.zeros(img.shape[:2], dtype=np.uint8)
    bgd, fgd = np.zeros((1, 65), np.float64), np.zeros((1, 65), np.float64)
    cv2.grabCut(img, mask, rect, bgd, fgd, it, cv2.GC_INIT_WITH_RECT)
    m = np.where((mask == cv2.GC_FGD) | (mask == cv2.GC_PR_FGD), 255,
0).astype(np.uint8)
    cv2.imwrite(f"grabcut_it{it}.png", m)

```

Expected Output: 1 iteration: rough mask. 10 iterations: cleaner mask. Returns plateau around 5 iterations.

4. Parameter Sensitivity

Param	Effect
Rectangle accuracy	Tight rect = better result
Iterations (5 default)	More = better, slow
Mask-mode refinement	Adds user knowledge → big quality bumps

5. Common Mistakes

Mistake	What Happens	Fix
Rect outside or too tight	Bad mask	Encompass target with a small margin
Apply on grayscale	Error	Must be 3-channel BGR
Reusing bgd_model / fgd_model from prior call	Tainted state	Reinitialise to zeros for each new image
Expecting perfect cut on similar fg/bg colors	Mask leaks	Switch to mask init + manual scribbles

6. Practice Tasks

- Run GrabCut on astronaut() with the rect from Example 1. Save mask + extracted FG.
- Composite the astronaut onto coffee() as a new background.
- Try iterations 1, 3, 5, 10; compare visually.

7. Key Takeaways

- GrabCut = graph-cut + GMM-based interactive foreground.
- Initialise with a bounding box, then run 5+ iterations.
- Final mask = (FGD | PR_FGD).
- Slow but high-quality; pair with mask-mode refinement for tough cases.

8. What's Next

End of Module 11. Next: Module 12 — Contours & Shape Analysis — analyse the regions you just segmented.

Module 11 — Exercises

15 exercises.

11.1 Thresholding

- (E) Threshold coins() at 80; save mask.
- (M) Apply all 5 threshold types on camera().
- (H) Use mask to keep only the coins; blacken background.

11.2 Otsu

- (E) Otsu on coins(); print chosen value.
- (M) Compare bimodal vs unimodal histograms.
- (H) Use skimage threshold_multiotsu with 3 classes.

11.3 Adaptive

- (E) Adaptive Gaussian on text() (blockSize=25, C=10).
- (M) Simulate uneven lighting; compare Otsu vs adaptive.
- (H) Tune blockSize $\in$ {3,11,25,51}; pick a good one.

11.4 Watershed

- (E) Build 3 touching circles; run full watershed; expect 3 objects.
- (M) Apply watershed to binarised coins().
- (H) Visualise distance transform of any mask.

11.5 GrabCut

- (E) GrabCut on astronaut() with bounding box; save mask + extracted FG.
- (M) Composite onto coffee() background.
- (H) Try iterations 1, 3, 5, 10; compare.

Module 11 — Quiz

10 MCQs.

Q1

`cv2.threshold(src, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)` returns: A. only the mask B. (otsu_value, mask) C. only Otsu value D. error

Answer: B (L2)

Q2

Otsu works well when the histogram is: A. flat B. unimodal C. bimodal D. random

Answer: C (L2)

Q3

For uneven-lit text scans, the best threshold is: A. fixed 128 B. Otsu C. adaptive D. random

Answer: C (L3)

Q4

`adaptiveThreshold` blockSize must be: A. even B. odd, ≥ 3 C. ≥ 51 D. any

Answer: B (L3)

Q5

In adaptive threshold, larger C means: A. more foreground B. less foreground C. blurrier D. inverted

Answer: B (L3)

Q6

Watershed separates touching objects by treating the image as: A. random B. text C. topographic surface D. graph

Answer: C (L4)

Q7

For watershed seeds, the distance transform value at a pixel is: A. distance to centre B. distance to nearest black pixel C. brightness D. gradient

Answer: B (L4)

Q8

GrabCut needs the image as: A. grayscale B. binary C. BGR (3 channel) D. RGBA

Answer: C (L5)

Q9

After GrabCut, the foreground mask is: A. $\text{mask} == 1$ B. $(\text{mask} == \text{FGD}) \mid (\text{mask} == \text{PR_FGD})$ C. $\text{mask} == 0$ D. $\text{mask} > 128$

Answer: B (L5)

Q10

For "old paper photographed with uneven flash", the right segmentation method is: A. simple threshold B. Otsu C. adaptive D. GrabCut

Answer: C (L3)

Module 11 — Assignment: Coin Counter

Estimated time: 2 hours

Combines: Module 11 + Modules 9, 10

Brief

Build a CLI tool that counts the coins in an image using full segmentation pipeline (threshold → morphology → watershed).

Requirements

- `count.py INPUT [--output OUT]`
- Read as grayscale.
- Otsu threshold.
- Open + close cleanup.
- Distance transform + watershed.
- Count distinct labels (excluding background and watershed lines).
- Draw red watershed boundaries on the original color version.
- Print count, save the annotated result.

Constraints

- Modules 1-11 only.
- Use `cv2.watershed`.
- Use `argparse`.

Expected Output

```
$ python count.py datasets/starter/coins.png
detected: 24 objects
saved -> coins_counted.png
```

Grading Rubric

Criterion	Weight
Threshold + cleanup	20%
Distance transform	15%
Watershed correct	30%
Count + annotation	20%
Style	15%

Submission

Save as `assignment_module11.py`.

Module 11 — Mini Project: One-Click GrabCut Cutout

Overview

User clicks two opposite corners of a bounding box around an object; GrabCut runs and produces a cleanly cut-out PNG with transparent background.

Concepts Used

- GrabCut (M11 L5)
- Mouse callbacks (M2 L3)
- Alpha PNG output (M2 L6)

Features

- cutout.py INPUT [--output OUT.png]
- User clicks 2 corners (top-left, bottom-right).
- GrabCut runs 5 iterations on that rect.
- Result saved as PNG with alpha (transparent background).

Starter Code

cutout.py:

```
import sys, cv2, numpy as np

pts = []

def on_mouse(event, x, y, flags, param):
    global pts
    if event == cv2.EVENT_LBUTTONDOWN:
        pts.append((x, y))
        cv2.circle(param, (x, y), 5, (0, 255, 0), -1)

def run(path, out):
    img = cv2.imread(path)
    if img is None: raise FileNotFoundError(path)
    if max(img.shape) > 900:
        img = cv2.resize(img, None, fx=0.6, fy=0.6)
    disp = img.copy()
    cv2.namedWindow("click 2 corners")
    cv2.setMouseCallback("click 2 corners", on_mouse, disp)
    while True:
        cv2.imshow("click 2 corners", disp)
        if len(pts) >= 2: break
        if cv2.waitKey(20) & 0xFF == ord('q'): return
    cv2.destroyAllWindows()

    x1, y1 = pts[0]; x2, y2 = pts[1]
    rect = (min(x1, x2), min(y1, y2), abs(x2 - x1), abs(y2 - y1))

    mask = np.zeros(img.shape[:2], dtype=np.uint8)
```

```

bgd, fgd = np.zeros((1, 65), np.float64), np.zeros((1, 65), np.float64)
cv2.grabCut(img, mask, rect, bgd, fgd, 5, cv2.GC_INIT_WITH_RECT)
m = np.where((mask == cv2.GC_FGD) | (mask == cv2.GC_PR_FGD), 255,
0).astype(np.uint8)

# Build BGRA result with alpha mask
bgra = cv2.cvtColor(img, cv2.COLOR_BGR2BGRA)
bgra[..., 3] = m
cv2.imwrite(out, bgra)
print("saved", out)

if __name__ == "__main__":
    path = sys.argv[1] if len(sys.argv) > 1 else "datasets/starter/astronaut.png"
    out = sys.argv[2] if len(sys.argv) > 2 else "cutout.png"
    run(path, out)

```

Expected Interaction

```

$ python cutout.py datasets/starter/astronaut.png
(window opens; click top-left of astronaut, then bottom-right)
saved cutout.png

```

cutout.png is a transparent PNG with the astronaut on transparent background.

Extension Challenges

- Allow rectangle drag (instead of two clicks).
- After initial mask, allow scribble refinement (LMB=FG, RMB=BG) and re-run.
- Output a separate alpha-only PNG for further use.

Grading Rubric

Criterion	Weight
Two-click rect input	25%
GrabCut correct	30%
BGRA save with alpha	20%
Code organisation	15%
Style	10%

Python E-Course

Module 12 — Contours & Shape Analysis

Detailed Notes

Module Overview

Level: Segmentation

Duration: ~1 week

Prerequisites: Modules 1-11

What You'll Learn

- Find and draw contours.
- Use hierarchy modes (RETR_EXTERNAL, RETR_TREE).
- Compute shape descriptors: area, perimeter, moments, aspect ratio, extent, solidity.
- Compute bounding boxes and convex hulls.

Lessons

#	Lesson	Duration
1	Finding Contours	25 min
2	Drawing & Hierarchy	25 min
3	Moments & Centroid	25 min
4	Bounding Boxes & Hull	25 min
5	Shape Descriptors	30 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 13: Features

Lesson 1: Finding Contours

Module: 12 — Contours & Shape Analysis

Duration: 25 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Run `cv2.findContours` on a binary mask.
- Understand what a contour is structurally.
- Filter contours by area.

Prerequisites

- Modules 1-11

1. Why This Matters

Contours are the polygonal outlines of connected regions. Once you have them, you can count objects, measure shapes, recognise letters, filter by size — they're the bridge between pixel-level segmentation and object-level reasoning.

2. Concept

2.1 What a contour is

A NumPy array of (x, y) points along an object's boundary, shape (N, 1, 2). OpenCV returns a Python list of these arrays — one per detected region.

2.2 The function

```
contours, hierarchy = cv2.findContours(mask, mode, method)
```

- `mask` — binary uint8 (0 or non-zero).
- `mode` — retrieval mode (next lesson). `cv2.RETR_EXTERNAL` for just outer contours; `cv2.RETR_TREE` for full hierarchy.
- `method` — approximation. `cv2.CHAIN_APPROX_SIMPLE` (compress straight runs to endpoints) or `cv2.CHAIN_APPROX_NONE` (every point).

2.3 Output

`contours`: list of arrays of points. `hierarchy`: parent/child/sibling info per contour (Lesson 2).

2.4 Source must be binary

Threshold, edge-detect, or mask first. `cv2.findContours` on a grayscale image is undefined.

2.5 Filtering by area

```
contours = [c for c in contours if cv2.contourArea(c) > 100]
```

Drops tiny noise blobs. Almost always the first thing you do after finding.

2.6 In-place modification (versions ≤ 4.0)

Older OpenCV versions modified the input mask. Modern versions (4.x) don't. Still good practice to pass `mask.copy()` to avoid surprises.

3. Code Examples

Example 1: Find contours of binarised coins

```
import cv2
from skimage.data import coins

_, mask = cv2.threshold(coins(), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
print("contours found:", len(contours))
print("first contour shape:", contours[0].shape)
print("sample points (first 3):", contours[0][:3, 0])
```

Expected Output:

```
contours found: ~25
first contour shape: (~50, 1, 2)
sample points (first 3): [[210  19] [209  20] [...]]
```

Example 2: Filter by area

```
import cv2
from skimage.data import coins

_, mask = cv2.threshold(coins(), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
big = [c for c in contours if cv2.contourArea(c) > 500]
print("all:", len(contours), " big (area > 500):", len(big))
```

Expected Output: All ~25 but only the genuine coins (e.g. ~20) survive after filtering noise specks.

Example 3: Draw on the image

```
import cv2
from skimage.data import coins

_, mask = cv2.threshold(coins(), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

img = cv2.cvtColor(coins(), cv2.COLOR_GRAY2BGR)
cv2.drawContours(img, contours, -1, (0, 255, 0), 2)
cv2.imwrite("coins_contours.png", img)
```

Expected Output: Coin image with green outlines around each coin.

Example 4: CHAIN_APPROX comparison

```
import cv2
from skimage.data import coins

_, mask = cv2.threshold(coins(), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

```

c_none, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
c_simple, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
print("approx NONE points (largest): ", max(len(c) for c in c_none))
print("approx SIMPLE points (largest):", max(len(c) for c in c_simple))

```

Expected Output: NONE returns ~hundreds of points per coin (every boundary pixel); SIMPLE returns far fewer (just the endpoints of straight runs).

4. Parameter Sensitivity

Param	Effect
mode	EXTERNAL → only outer; TREE → outer + holes + nesting
method	NONE → full points; SIMPLE → compressed
area filter	bigger threshold = fewer contours

5. Common Mistakes

Mistake	What Happens	Fix
Pass grayscale to findContours	Undefined	Threshold first
Forget to filter tiny contours	Hundreds of bogus blobs	Filter by area early
Treat contours as one array	TypeError	It's a list of arrays
Pass hierarchy is None confusingly	Just ignore if using EXTERNAL	Use _

6. Practice Tasks

- Otsu-threshold coins(); find contours; print count.
- Filter to area > 500; count remaining.
- Draw all contours on the color version; save.

7. Key Takeaways

- cv2.findContours(mask, mode, method) returns (contours, hierarchy).
- Each contour is an (N, 1, 2) array of points.
- Filter by cv2.contourArea to drop noise.
- Use RETR_EXTERNAL + CHAIN_APPROX_SIMPLE as default.

8. What's Next

Drawing and hierarchy — how to render contours and understand parent/child relationships.

Lesson 2: Drawing & Hierarchy

Module: 12 — Contours & Shape Analysis

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Use `cv2.drawContours` for outlining and filling.
- Read the 4-tuple hierarchy structure.
- Compare `RETR_EXTERNAL`, `RETR_LIST`, `RETR_CCOMP`, `RETR_TREE`.

Prerequisites

- Module 12 Lesson 1

1. Why This Matters

Without hierarchy you can't tell "outer object" from "hole inside object". Most real images have nested shapes — letters with holes, donuts, gears.

2. Concept

2.1 `drawContours`

```
cv2.drawContours(img, contours, contourIdx, color, thickness)
```

- `contourIdx = -1` → draw all.
- `thickness = -1` (or `cv2.FILLED`) → fill the contour.

2.2 Retrieval modes

Mode	Returns
<code>RETR_EXTERNAL</code>	only the outermost contours
<code>RETR_LIST</code>	all, no hierarchy info
<code>RETR_CCOMP</code>	2-level hierarchy (outer + holes)
<code>RETR_TREE</code>	full nested hierarchy

For "find each object once, regardless of holes inside": `EXTERNAL`. For "find objects and their holes": `CCOMP` or `TREE`.

2.3 Hierarchy format

```
hierarchy.shape    # (1, N, 4)
# Per contour: [Next, Previous, First_Child, Parent]
```

- `Next`: next contour at the same hierarchy level.
- `Previous`: previous at the same level.
- `First_Child`: first hole inside this contour.
- `Parent`: enclosing contour.

-1 means "no such relationship".

2.4 Practical hierarchy use

Find outer contours that have at least one child (i.e., objects with holes):

```
outer_with_holes = []
for i, h in enumerate(hierarchy[0]):
    if h[3] == -1 and h[2] != -1:    # no parent + has child
        outer_with_holes.append(contours[i])
```

2.5 When you don't need hierarchy

Most everyday work uses RETR_EXTERNAL and ignores hierarchy. Reach for TREE only when nested structure matters.

3. Code Examples

Example 1: Draw all contours

```
import cv2
from skimage.data import coins

_, mask = cv2.threshold(coins(), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
img = cv2.cvtColor(coins(), cv2.COLOR_GRAY2BGR)
cv2.drawContours(img, contours, -1, (0, 255, 0), 2)
cv2.imwrite("coins_drawn.png", img)
```

Expected Output: Coins outlined in green.

Example 2: Draw a single contour, filled

```
import cv2
from skimage.data import coins

_, mask = cv2.threshold(coins(), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
largest = max(contours, key=cv2.contourArea)

img = cv2.cvtColor(coins(), cv2.COLOR_GRAY2BGR)
cv2.drawContours(img, [largest], 0, (0, 0, 255), thickness=cv2.FILLED)
cv2.imwrite("largest_filled.png", img)
print("largest area:", cv2.contourArea(largest))
```

Expected Output: Only the biggest coin filled red; others untouched.

Example 3: Hierarchy with TREE

```
import cv2, numpy as np

# Build a shape with holes
img = np.zeros((400, 400), dtype=np.uint8)
cv2.rectangle(img, (50, 50), (350, 350), 255, -1)
cv2.circle(img, (150, 200), 30, 0, -1)
cv2.circle(img, (250, 200), 30, 0, -1)
```

```

contours, hierarchy = cv2.findContours(img, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
print("contours:", len(contours))
print("hierarchy:\n", hierarchy[0])

```

Expected Output: 3 contours (outer square + 2 holes). Hierarchy shows outer has children, holes have parent = 0.

Example 4: Distinguish outer vs holes

```

import cv2, numpy as np

img = np.zeros((400, 400), dtype=np.uint8)
cv2.rectangle(img, (50, 50), (350, 350), 255, -1)
cv2.circle(img, (200, 200), 50, 0, -1)

contours, hierarchy = cv2.findContours(img, cv2.RETR_CCOMP, cv2.CHAIN_APPROX_SIMPLE)
disp = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
for i, c in enumerate(contours):
    parent = hierarchy[0][i][3]
    color = (0, 255, 0) if parent == -1 else (0, 0, 255)
    cv2.drawContours(disp, [c], 0, color, 2)
cv2.imwrite("outer_vs_hole.png", disp)

```

Expected Output: Outer rectangle in green, inner hole in red.

4. Parameter Sensitivity

(See L1 for mode/method effects.)

5. Common Mistakes

Mistake	What Happens	Fix
RETR_EXTERNAL but expecting holes	Holes missed	Use CCOMP or TREE
Hierarchy indexing: hierarchy[i]	wrong: it's hierarchy[0][i]	Always hierarchy[0] first
Drawing with non-list contour	TypeError	Wrap single contour: [contour]

6. Practice Tasks

- Draw all contours of binarised coins() in red.
- Fill the largest contour with green.
- Build a square-with-holes and identify outer vs hole using RETR_CCOMP hierarchy.

7. Key Takeaways

- cv2.drawContours(img, list, idx, color, thickness).
- RETR_EXTERNAL default; RETR_TREE for nested.
- Hierarchy is (1, N, 4): Next, Prev, Child, Parent.
- Use hierarchy[0][i][3] == -1 to identify outer contours.

8. What's Next

Moments and centroid — the first shape descriptor.

Lesson 3: Moments & Centroid

Module: 12 — Contours & Shape Analysis

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Compute image / contour moments with `cv2.moments`.
- Use moments to find centroid, area, and orientation.
- Recognise the 7 Hu invariants for shape matching.

Prerequisites

- Module 12 Lesson 2

1. Why This Matters

Moments are weighted sums of pixel positions. They tell you where a shape's centre is, how big it is, and (via Hu invariants) what it looks like — independent of position, rotation, and scale. Used everywhere from blob detection to character recognition.

2. Concept

2.1 Spatial moments

A moment is a sum over pixels:

$$m_{ij} = \sum (x^i \times y^j \times \text{pixel})$$

For a contour, the "pixel" is essentially 1 inside the shape, 0 outside.

Moment	Meaning
m00	area
m10, m01	first moments → use to find centroid
m11, m20, m02	second moments → used for orientation

2.2 OpenCV API

```
M = cv2.moments(contour)      # or cv2.moments(image)
print(M.keys())
```

Returns a dict with all the moments, plus normalised central moments (`nu`) and Hu invariants (`hu`).

2.3 Centroid

```
cx = m10 / m00
cy = m01 / m00
```

```
M = cv2.moments(contour)
cx = int(M["m10"] / M["m00"])
cy = int(M["m01"] / M["m00"])
```

2.4 Hu moments

Seven scalars invariant to translation, rotation, and scale. Useful for matching shapes:

```
hu = cv2.HuMoments(M).ravel() # shape (7,)
```

Often work in log space because the values span many orders of magnitude:

```
import numpy as np
log_hu = -np.sign(hu) * np.log10(np.abs(hu) + 1e-30)
```

2.5 matchShapes for similarity

```
score = cv2.matchShapes(contour_a, contour_b, cv2.CONTOURS_MATCH_I1, 0.0)
```

Lower = more similar. Internally uses Hu moments.

3. Code Examples

Example 1: Centroid of each coin

```
import cv2
from skimage.data import coins

_, mask = cv2.threshold(coins(), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
img = cv2.cvtColor(coins(), cv2.COLOR_GRAY2BGR)

for c in contours:
    if cv2.contourArea(c) < 100:
        continue
    M = cv2.moments(c)
    cx = int(M["m10"] / M["m00"])
    cy = int(M["m01"] / M["m00"])
    cv2.circle(img, (cx, cy), 4, (0, 0, 255), -1)
cv2.imwrite("coins_centroids.png", img)
```

Expected Output: Coin image with red dots at the centre of each coin.

Example 2: Area from moments vs contourArea

```
import cv2, numpy as np

contour = np.array([[10,10],[100,10],[100,100],[10,100]], dtype=np.int32)
M = cv2.moments(contour)
print("m00 (area):", M["m00"], " contourArea:", cv2.contourArea(contour))
```

Expected Output: Both 8100 (90×90 = 8100). The two functions agree.

Example 3: Hu moments

```
import cv2, numpy as np
```

```

square = np.array([[10,10],[100,10],[100,100],[10,100]], dtype=np.int32)
triangle = np.array([[200,50],[300,150],[100,150]], dtype=np.int32)

for name, c in [("square", square), ("triangle", triangle)]:
    hu = cv2.HuMoments(cv2.moments(c)).ravel()
    print(f"{name}: {hu}")

```

Expected Output: Two arrays of 7 numbers; clearly different — that's why Hu moments are good for shape matching.

Example 4: matchShapes

```

import cv2, numpy as np

square_a = np.array([[10,10],[100,10],[100,100],[10,100]], dtype=np.int32)
square_b = np.array([[200,200],[300,200],[300,300],[200,300]],
dtype=np.int32) # same shape, moved
triangle = np.array([[400,50],[500,150],[300,150]], dtype=np.int32)

print("square vs square:", cv2.matchShapes(square_a, square_b,
cv2.CONTOURS_MATCH_I1, 0.0))
print("square vs tri    :", cv2.matchShapes(square_a, triangle,
cv2.CONTOURS_MATCH_I1, 0.0))

```

Expected Output: First call near 0 (very similar); second much larger (dissimilar).

4. Parameter Sensitivity

Match method	Behaviour
CONTOURS_MATCH_I1	most-used default
CONTOURS_MATCH_I2	different weighting
CONTOURS_MATCH_I3	even different weighting

5. Common Mistakes

Mistake	What Happens	Fix
m00 == 0 (empty contour)	ZeroDivisionError	Filter by area first
Use Hu moments raw (huge range)	Comparisons skewed	Use log magnitudes
Expect Hu to be illumination-invariant	They're scale/rotation/translation only	Pair with proper preprocessing

6. Practice Tasks

- Find each coin's centroid; draw a red dot.
- Compute area via `cv2.moments(c)["m00"]` and compare to `cv2.contourArea`.
- Compare two shapes with `cv2.matchShapes`.

7. Key Takeaways

- `cv2.moments(contour)` returns spatial + central + normalised + Hu moments.
- Centroid: `(m10/m00, m01/m00)`.

- Hu moments are invariant to translation, rotation, scale.
- `cv2.matchShapes` for shape similarity; lower = more similar.

8. What's Next

Bounding boxes and convex hulls — the most common contour summaries.

Lesson 4: Bounding Boxes & Convex Hull

Module: 12 — Contours & Shape Analysis

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Compute axis-aligned and rotated bounding rectangles.
- Compute the convex hull of a contour.
- Approximate a contour with `cv2.approxPolyDP`.

Prerequisites

- Module 12 Lesson 3

1. Why This Matters

Bounding boxes are the universal "where is this object" summary used by detectors, trackers, and downstream code. Hulls and polygon approximations simplify complex contours into something tractable.

2. Concept

2.1 Axis-aligned bounding rect

```
x, y, w, h = cv2.boundingRect(contour)
```

Smallest upright rectangle enclosing the contour. Fast.

2.2 Rotated min-area rect

```
rotrect = cv2.minAreaRect(contour)          # ((cx, cy), (w, h), angle)
box = cv2.boxPoints(rotrect).astype(np.int32)
```

Smallest rectangle of any orientation. Better fit for elongated rotated objects.

2.3 Minimum enclosing circle / ellipse

```
(cx, cy), radius = cv2.minEnclosingCircle(contour)
ellipse = cv2.fitEllipse(contour)           # ((cx, cy), (w, h), angle)
```

2.4 Convex hull

The smallest convex polygon enclosing the contour. "Wrap a rubber band around the points."

```
hull = cv2.convexHull(contour)
# returns_points=True (default) → array of points; False → indices
```

Useful for:

- Simplifying contours.
- Detecting concavities (compare contour area to hull area).

- Hand / shape recognition.

2.5 Polygon approximation

```
epsilon = 0.02 * cv2.arcLength(contour, closed=True)
approx = cv2.approxPolyDP(contour, epsilon, closed=True)
```

Ramer-Douglas-Peucker algorithm — approximates the contour with fewer points. epsilon is the max deviation; typical 1-5% of perimeter.

For a near-rectangular shape, approx will have 4 corners.

2.6 Putting it together

For each contour:

- Get bounding rect → draw bounding box for visualisation.
- Get hull → simplify.
- Get approxPolyDP → detect shape type from vertex count.

3. Code Examples

Example 1: Axis-aligned vs rotated bounding boxes

```
import cv2, numpy as np

# Synthesize a rotated rectangle
img = np.zeros((400, 600), dtype=np.uint8)
rect = ((300, 200), (200, 80), -30) # cx, cy, (w, h), angle
box = cv2.boxPoints(rect).astype(np.int32)
cv2.fillPoly(img, [box], 255)

contours, _ = cv2.findContours(img, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
c = contours[0]

x, y, w, h = cv2.boundingRect(c)
rot = cv2.minAreaRect(c)
rot_box = cv2.boxPoints(rot).astype(np.int32)

disp = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
cv2.rectangle(disp, (x, y), (x + w, y + h), (0, 255, 0), 2)
cv2.drawContours(disp, [rot_box], 0, (0, 0, 255), 2)
cv2.imwrite("bbox_compare.png", disp)
print("axis-aligned area:", w * h, " rotated area:", rot[1][0] * rot[1][1])
```

Expected Output: Green axis-aligned box is big and loose; red rotated box is tight around the rotated rectangle. Console shows the rotated area is much smaller — better fit.

Example 2: Convex hull

```
import cv2, numpy as np

# A star-shaped polygon
pts = []
```

```

for i in range(10):
    angle = i * np.pi / 5
    r = 100 if i % 2 == 0 else 40
    pts.append((int(200 + r * np.cos(angle)), int(200 + r * np.sin(angle))))
pts = np.array(pts, dtype=np.int32)

img = np.zeros((400, 400), dtype=np.uint8)
cv2.fillPoly(img, [pts], 255)

contours, _ = cv2.findContours(img, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
c = contours[0]
hull = cv2.convexHull(c)

disp = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
cv2.drawContours(disp, [hull], 0, (0, 0, 255), 2)
cv2.imwrite("star_hull.png", disp)
print("contour area:", cv2.contourArea(c), " hull area:", cv2.contourArea(hull))

```

Expected Output: Star with red convex hull (outer pentagon). Hull area is larger than contour area — the difference is the "concavity".

Example 3: Polygon approximation — identify shape

```

import cv2, numpy as np

# 4 shapes on a canvas
img = np.zeros((400, 800), dtype=np.uint8)
cv2.rectangle(img, (40, 80), (180, 320), 255, -1) # rect
cv2.fillPoly(img, [np.array([[300, 80], [400, 320], [200, 320]])], 255) # tri
cv2.circle(img, (550, 200), 100, 255, -1) # circle
cv2.fillPoly(img, [np.array([[700, 100], [780, 180], [720, 320], [680, 320]])], 255)
# quad

contours, _ = cv2.findContours(img, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
for c in contours:
    eps = 0.02 * cv2.arcLength(c, True)
    approx = cv2.approxPolyDP(c, eps, True)
    print("vertices:", len(approx))

```

Expected Output: 4 (rect), 3 (triangle), ~6-8 (circle — many vertices), 4 (quad).

Example 4: Min enclosing circle on each coin

```

import cv2
from skimage.data import coins

_, mask = cv2.threshold(coins(), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

disp = cv2.cvtColor(coins(), cv2.COLOR_GRAY2BGR)
for c in contours:
    if cv2.contourArea(c) < 200: continue
    (cx, cy), r = cv2.minEnclosingCircle(c)

```



```
cv2.circle(dis, (int(cx), int(cy)), int(r), (0, 255, 0), 2)
cv2.imwrite("coins_mincirc.png", dis)
```

Expected Output: Each coin enclosed by a tight green circle.

4. Parameter Sensitivity

Param	Effect
epsilon in approxPolyDP	smaller = closer to original (more vertices)
Hull "returnPoints=True"	gives points; False gives indices

5. Common Mistakes

Mistake	What Happens	Fix
Pass empty contour	Error	Filter by area first
Forget cv2.boxPoints after minAreaRect	TypeError when drawing	Convert to points
approxPolyDP epsilon too small	Hundreds of vertices	Use $0.01-0.05 \times \text{arcLength}$

6. Practice Tasks

- Draw axis-aligned vs rotated bounding box on a rotated rectangle.
- Compute convex hull of a star shape.
- Identify shapes (triangle / square / circle) by approxPolyDP vertex count.

7. Key Takeaways

- cv2.boundingRect (upright) vs cv2.minAreaRect (rotated).
- cv2.minEnclosingCircle for circles.
- cv2.convexHull for outer wrapping.
- cv2.approxPolyDP simplifies contours; useful for shape classification.

8. What's Next

Shape descriptors — aspect ratio, extent, solidity.

Lesson 5: Shape Descriptors

Module: 12 — Contours & Shape Analysis

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Compute aspect ratio, extent, solidity, equivalent diameter.
- Use these to filter contours by shape.
- Combine multiple descriptors for shape classification.

Prerequisites

- Module 12 Lesson 4

1. Why This Matters

A coin and a pencil have similar areas but very different shapes. Aspect ratio, solidity, extent — these small numbers tell you the shape's character. Build them into your contour filter and you can reliably keep only the objects you care about.

2. Concept

2.1 The descriptors

Descriptor	Formula	Tells you
Aspect ratio	w / h of bounding rect	elongation
Extent	contour_area / (w × h)	how rectangular
Solidity	contour_area / hull_area	how convex (1 = convex, < 1 = concave)
Equivalent diameter	$\sqrt{4 \times \text{area} / \pi}$	"diameter if it were a circle"
Orientation	from fitEllipse	angle of major axis

2.2 Typical values

Shape	aspect	extent	solidity
Filled square	1.0	1.0	1.0
Circle	1.0	$\pi/4 \approx 0.785$	1.0
Long pencil	5+	high	1.0
Star	~1	low	low
L-shape	~1	low	low-medium

2.3 Code

```
x, y, w, h = cv2.boundingRect(c)
area = cv2.contourArea(c)
hull_area = cv2.contourArea(cv2.convexHull(c))

aspect    = w / h
extent    = area / (w * h)
```

```
solidity = area / hull_area if hull_area else 0
eq_diam = np.sqrt(4 * area / np.pi)
```

2.4 Filter recipes

- Keep round coins: $0.85 < \text{extent} < 0.9$ and $0.95 < \text{solidity}$.
- Keep elongated lines: $\text{aspect} > 3$ or $\text{aspect} < 1/3$.
- Drop concave noise blobs: $\text{solidity} > 0.9$.

2.5 Orientation

```
if len(c) >= 5:
    (cx, cy), (MA, ma), angle = cv2.fitEllipse(c)
```

Angle is in degrees, 0-180. Major axis MA, minor ma.

3. Code Examples

Example 1: Print descriptors for sample shapes

```
import cv2, numpy as np

shapes = []
# Square
sq = np.array([[100, 100], [200, 100], [200, 200], [100, 200]], dtype=np.int32)
shapes.append(("square", sq))
# Circle
img = np.zeros((300, 300), dtype=np.uint8)
cv2.circle(img, (150, 150), 60, 255, -1)
ct, _ = cv2.findContours(img, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
shapes.append(("circle", ct[0]))
# L-shape
img = np.zeros((300, 300), dtype=np.uint8)
cv2.fillPoly(img,
[ np.array([[50,50],[200,50],[200,100],[100,100],[100,250],[50,250]]) ], 255)
ct, _ = cv2.findContours(img, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
shapes.append(("L", ct[0]))

print(f"{'shape':6s} {'aspect':>8s} {'extent':>8s} {'solidity':>9s}")
for name, c in shapes:
    x, y, w, h = cv2.boundingRect(c)
    area = cv2.contourArea(c)
    hull = cv2.contourArea(cv2.convexHull(c))
    aspect = w / h
    extent = area / (w * h)
    solidity = area / hull if hull else 0
    print(f"{'name':6s} {'aspect':8.3f} {'extent':8.3f} {'solidity':9.3f}")
```

Expected Output:

shape	aspect	extent	solidity
square	1.000	1.000	1.000
circle	1.000	0.785	1.000
L	1.000	~0.39	~0.55

Example 2: Filter by shape

```
import cv2
from skimage.data import coins

_, mask = cv2.threshold(coins(), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

round_objects = []
for c in contours:
    area = cv2.contourArea(c)
    if area < 200: continue
    x, y, w, h = cv2.boundingRect(c)
    extent = area / (w * h)
    if 0.7 < extent < 0.85:          # roughly circular
        round_objects.append(c)

print("round objects:", len(round_objects))
```

Expected Output: round objects: ~20 (coins typically have extent around $\pi/4 \approx 0.785$).

Example 3: Orientation via fitEllipse

```
import cv2, numpy as np

# Rotated rectangle
img = np.zeros((300, 600), dtype=np.uint8)
rect = ((300, 150), (200, 50), 30)
box = cv2.boxPoints(rect).astype(np.int32)
cv2.fillPoly(img, [box], 255)

contours, _ = cv2.findContours(img, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
(cx, cy), (MA, ma), angle = cv2.fitEllipse(contours[0])
print(f"orientation: {angle:.1f}° major={MA:.0f} minor={ma:.0f}")
```

Expected Output: angle close to 30° (the angle we used for the rectangle), MA much larger than ma.

Example 4: Combined classifier

```
import cv2

def classify(c):
    area = cv2.contourArea(c)
    if area < 50: return "noise"
    x, y, w, h = cv2.boundingRect(c)
    aspect = w / h
    extent = area / (w * h)
    if aspect > 3 or aspect < 1/3: return "long-thin"
    if 0.95 < extent <= 1.0:      return "rectangle"
    if 0.7 < extent < 0.85:      return "ellipse / circle"
    return "other"

# pretend we have a contour `c`:
# print(classify(c))
```

4. Parameter Sensitivity

Descriptor	Discriminator
Aspect	elongation
Extent	rectangularity
Solidity	convexity

5. Common Mistakes

Mistake	What Happens	Fix
Divide by zero on small contours	Exception	Filter by area first
Treat aspect > 1 as "wide" only	aspect = w/h depends on orientation	Use min/max of aspect and 1/aspect
Use one descriptor for classification	False positives	Combine 2-3 descriptors

6. Practice Tasks

- Print aspect / extent / solidity for square, circle, L-shape.
- Filter binarised coins() keeping only "circular" extent (~0.785).
- For a rotated rectangle contour, get orientation via fitEllipse.

7. Key Takeaways

- Aspect = w/h; extent = area / bbox; solidity = area / hull.
- Each tells you something different about shape.
- Combine for robust classification.
- Filter contours with descriptor thresholds.

8. What's Next

End of Module 12. Next: Module 13 — Features — template matching, ORB, Hough.

Module 12 — Exercises

15 exercises.

12.1 Finding

- (E) Otsu mask coins(); find contours; print count.
- (M) Filter to area > 500; count.
- (H) Draw all contours on the color image; save.

12.2 Hierarchy

- (E) Draw all contours of coins() in red.
- (M) Fill the largest contour green.
- (H) Build square-with-hole; identify outer vs hole via RETR_CCOMP.

12.3 Moments

- (E) Compute and draw centroids of each coin.
- (M) Verify `m00 == cv2.contourArea`.
- (H) Compare 2 shapes with `cv2.matchShapes`.

12.4 BBox / Hull

- (E) Draw axis-aligned + rotated bbox on rotated rectangle.
- (M) Convex hull of star shape; print areas.
- (H) Classify shapes by `approxPolyDP` vertex count.

12.5 Descriptors

- (E) Print aspect / extent / solidity for square / circle / L-shape.
- (M) Filter coins keeping only extent ~ 0.785 .
- (H) `fitEllipse` orientation on rotated rectangle.

Module 12 — Quiz

10 MCQs.

Q1

cv2.findContours requires: A. color image B. binary mask C. grayscale D. float

Answer: B (L1)

Q2

cv2.RETR_EXTERNAL returns: A. all contours B. only inner contours C. only outermost D. hierarchy info

Answer: C (L2)

Q3

hierarchy[0][i] = [Next, Previous, ?, Parent]. The ? is: A. First_Child B. Last_Child C. Sibling D. Area

Answer: A (L2)

Q4

Centroid is computed as: A. (m10/m00, m01/m00) B. mean of all points C. bounding-box center D. all of these are valid

Answer: A (L3); they happen to coincide for symmetric shapes.

Q5

Hu moments are invariant to: A. translation only B. rotation only C. translation, rotation, scale D. lighting

Answer: C (L3)

Q6

cv2.minAreaRect returns: A. axis-aligned bounding rect B. rotated bounding rect C. circle D. ellipse

Answer: B (L4)

Q7

Convex hull of a star: A. is the star itself B. is the outer pentagon enclosing it C. is empty D. equals the bounding rect

Answer: B (L4)

Q8

cv2.approxPolyDP for a square with epsilon = 2% of perimeter: A. ~50 vertices B. 4 vertices C. 8 vertices D. 1 vertex

Answer: B (L4)

Q9

"Solidity" is: A. area / bbox area B. area / hull area C. $\text{perimeter}^2 / \text{area}$ D. width / height

Answer: B (L5)

Q10

A long thin pencil contour has: A. extent ~ 1, aspect ~ 5 B. extent ~ 0.5, aspect ~ 1 C. solidity 0 D. all-zero descriptors

Answer: A (L5)

Module 12 — Assignment: Coin Sorter

Estimated time: 2 hours

Combines: Module 12 + Modules 9-11

Brief

Take a coin image, segment each coin, and classify by size into 3 categories: small / medium / large. Annotate each with category name and area.

Requirements

- `sort.py INPUT [--output OUT]`
- Threshold → morphological cleanup → contours.
- Filter by area > 200 (drop noise).
- Compute area for each; sort areas; classify by terciles (smallest third / middle / largest).
- Annotate each coin on the original image:
- Draw bounding rectangle.
- Put text: small/med/large + area value.
- Print counts per category.

Constraints

- Modules 1-12 only.
- Use `cv2.findContours`, `cv2.contourArea`.
- Use `argparse`.

Expected Output

```
$ python sort.py datasets/starter/coins.png
small: 6  med: 8  large: 6
saved -> coins_sorted.png
```

`coins_sorted.png` shows boxes + labels per coin.

Grading Rubric

Criterion	Weight
Segmentation + contours	25%
Area-based sort	20%
Annotation correct	25%
Counts printed	10%
Style	20%

Submission

Save as `assignment_module12.py`.

Module 12 — Mini Project: Shape Recogniser

Overview

Take an image with various shapes (triangles, squares, circles, polygons), find each contour, classify by vertex count + descriptors, and annotate.

Concepts Used

- Contours (M12 L1)
- approxPolyDP (M12 L4)
- Shape descriptors (M12 L5)

Features

- shapes.py INPUT
- Otsu threshold; find contours.
- For each contour:
- Compute polygon approximation (epsilon = 2-4% of perimeter).
- Classify by vertex count: 3 = triangle, 4 = square/rect, 5+ = circle / polygon.
- Use extent and solidity to refine (square vs rect, circle vs polygon).
- Annotate label and color outline.

Starter Code

shapes.py:

```
import sys, cv2, numpy as np

def classify(c):
    area = cv2.contourArea(c)
    if area < 100: return None
    peri = cv2.arcLength(c, True)
    approx = cv2.approxPolyDP(c, 0.03 * peri, True)
    v = len(approx)
    x, y, w, h = cv2.boundingRect(c)
    aspect = w / h
    extent = area / (w * h)
    if v == 3: return "triangle"
    if v == 4:
        return "square" if 0.85 < aspect < 1.15 else "rectangle"
    if v == 5: return "pentagon"
    if extent > 0.75: return "circle"
    return f"polygon-{v}"

def run(path):
    img = cv2.imread(path)
    if img is None: raise FileNotFoundError(path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    _, mask = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

```

out = img.copy()
counts = {}
for c in contours:
    label = classify(c)
    if label is None: continue
    counts[label] = counts.get(label, 0) + 1
    cv2.drawContours(out, [c], 0, (0, 255, 0), 2)
    M = cv2.moments(c)
    if M["m00"]:
        cx = int(M["m10"] / M["m00"])
        cy = int(M["m01"] / M["m00"])
        cv2.putText(out, label, (cx - 30, cy),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 0, 255), 2)

print(counts)
cv2.imwrite("shapes_classified.png", out)

if __name__ == "__main__":
    run(sys.argv[1] if len(sys.argv) > 1 else "datasets/starter/checkerboard.png")

```

Expected Output

For an image containing 1 triangle, 2 squares, 1 circle:

```

{'triangle': 1, 'square': 2, 'circle': 1}
saved -> shapes_classified.png

```

Extension Challenges

- Test on a real "shape sorter toy" photo.
- Add color classification per shape (M4 HSV) — "blue triangle", "red square".
- Use cv2.matchShapes against templates instead of vertex count.

Grading Rubric

Criterion	Weight
Classification rule	30%
Per-shape label drawn	20%
Threshold + segmentation	20%
Counts printed	10%
Style	20%

Python E-Course

Module 13 — Feature Detection & Matching

Detailed Notes

Module Overview

Level: Features

Duration: ~1.5 weeks

Prerequisites: Modules 1-12

What You'll Learn

- Detect templates with `cv2.matchTemplate`.
- Detect corners with Harris and `goodFeaturesToTrack`.
- Use ORB to compute keypoints + descriptors.
- Match descriptors between two images with `BFMatcher`.
- Detect lines and circles with the Hough transform.

Lessons

#	Lesson	Duration
1	Template Matching	25 min
2	Harris & Corner Detection	30 min
3	ORB Keypoints & Descriptors	35 min
4	Feature Matching	35 min
5	Hough Transform — Lines & Circles	35 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 14: Frequency Domain (DFT)

Lesson 1: Template Matching

Module: 13 — Feature Detection & Matching

Duration: 25 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Use `cv2.matchTemplate` to find a small template inside a larger image.
- Pick the right matching method.
- Recognise template matching's limitations (rotation, scale).

Prerequisites

- Modules 1-12

1. Why This Matters

When the thing you're looking for has a fixed appearance — a logo, a button, an icon — template matching is the fastest, simplest detector. It's the first thing to try before reaching for ORB or deep learning.

2. Concept

2.1 The idea

Slide a small template image over a larger image, compute a similarity score at every position. The brightest (or darkest, depending on method) location is the match.

2.2 The function

```
result = cv2.matchTemplate(image, template, method)
```

result has shape $(H - h + 1, W - w + 1)$ — a scalar score per valid position.

2.3 Match methods

Method	Best =	Notes
<code>cv2.TM_SQDIFF</code>	minimum	sum of squared differences
<code>cv2.TM_SQDIFF_NORMED</code>	minimum	normalised SQDIFF
<code>cv2.TM_CCORR</code>	maximum	cross-correlation
<code>cv2.TM_CCORR_NORMED</code>	maximum	normalised CCORR
<code>cv2.TM_CCOEFF</code>	maximum	mean-centred correlation
<code>cv2.TM_CCOEFF_NORMED</code>	maximum	best default — invariant to brightness/contrast

Use `TM_CCOEFF_NORMED` unless you have a reason not to.

2.4 Finding the match

```
min_val, max_val, min_loc, max_loc = cv2.minMaxLoc(result)
# For CCOEFF_NORMED: top-left of best match is max_loc
```

```
top_left = max_loc
bottom_right = (top_left[0] + tw, top_left[1] + th)
```

2.5 Multi-match

For more than one match, threshold the result map:

```
threshold = 0.85
ys, xs = np.where(result >= threshold)
for x, y in zip(xs, ys):
    cv2.rectangle(image, (x, y), (x + tw, y + th), (0, 255, 0), 2)
```

2.6 Limitations

Template matching fails when the target is:

- Rotated (works only at the rotation in the template).
- Scaled (size must match).
- Occluded (partial occlusion drops the score).
- Photometrically different (use `_NORMED` variants to help).

For invariance, use ORB / SIFT (next lessons).

3. Code Examples

Example 1: Find an eye in the astronaut image

```
import cv2
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Template: crop a 50x50 region around the right eye
template = gray[220:270, 260:310].copy()
print("template shape:", template.shape)

result = cv2.matchTemplate(gray, template, cv2.TM_CCOEFF_NORMED)
_, max_val, _, max_loc = cv2.minMaxLoc(result)
print("best match score:", max_val, "at", max_loc)

h, w = template.shape
cv2.rectangle(img, max_loc, (max_loc[0] + w, max_loc[1] + h), (0, 255, 0), 2)
cv2.imwrite("astro_template_match.png", img)
```

Expected Output: Green box drawn around the original eye location (the template was cut from there, so best match is exactly the original spot with score ~1.0).

Example 2: Multi-match — find all instances

```
import cv2, numpy as np

# Build a synthetic scene with 3 copies of a small "X"
scene = np.full((300, 600), 200, dtype=np.uint8)
```

```

template = np.full((30, 30), 50, dtype=np.uint8)
cv2.line(template, (3, 3), (27, 27), 255, 3)
cv2.line(template, (27, 3), (3, 27), 255, 3)

for x, y in [(50, 50), (300, 100), (450, 200)]:
    scene[y:y+30, x:x+30] = template

result = cv2.matchTemplate(scene, template, cv2.TM_CCOEFF_NORMED)
threshold = 0.95
ys, xs = np.where(result >= threshold)
print("matches found:", len(xs))
out = cv2.cvtColor(scene, cv2.COLOR_GRAY2BGR)
for x, y in zip(xs, ys):
    cv2.rectangle(out, (x, y), (x + 30, y + 30), (0, 255, 0), 2)
cv2.imwrite("multi_match.png", out)

```

Expected Output: 3 boxes drawn around the 3 placed Xs. (May be slightly more than 3 if neighbouring pixels also exceed threshold; can be deduplicated with NMS.)

Example 3: Compare match methods

```

import cv2
from skimage.data import astronaut

gray = cv2.cvtColor(cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR),
                    cv2.COLOR_BGR2GRAY)
tpl = gray[220:270, 260:310]

for name, method, find in [
    ("SQDIFF", cv2.TM_SQDIFF, "min"),
    ("CCORR_NORMED", cv2.TM_CCORR_NORMED, "max"),
    ("CCOEFF_NORMED", cv2.TM_CCOEFF_NORMED, "max"),
]:
    res = cv2.matchTemplate(gray, tpl, method)
    mn, mx, mn_loc, mx_loc = cv2.minMaxLoc(res)
    print(f"{name:14s} best={'min' if find=='min' else 'max'} loc {mn_loc if
        find=='min' else mx_loc}")

```

Expected Output: All three methods locate the eye in approximately the same place (around (260, 220)).

4. Parameter Sensitivity

Param	Effect
Match method	NORMED variants handle brightness shifts
Threshold (for multi-match)	lower = more matches (incl false positives)
Template size	bigger = stricter match but slower

5. Common Mistakes

Mistake	What Happens	Fix
Search rotated template	Misses	Try ORB / SIFT
Template size doesn't match	Misses	Multi-scale: resize template

scene scale		at various scales
Use raw CCORR on varying brightness	False positives	Use _NORMED variants
Find min instead of max for CCOEFF	Wrong location	Check direction per method

6. Practice Tasks

- Crop a region from any image and use it as a template; find its location.
- Multi-match for repeated icons in a synthetic scene.
- Compare 3 match methods on the same input.

7. Key Takeaways

- Template matching = slide template, score, find best.
- cv2.TM_CCOEFF_NORMED is the safe default.
- Threshold the score map for multi-match.
- No rotation/scale invariance — use ORB for that.

8. What's Next

Harris corner detection — find "interesting" points without a template.

Lesson 2: Harris & Corner Detection

Module: 13 — Feature Detection & Matching

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Apply `cv2.cornerHarris` and `cv2.goodFeaturesToTrack`.
- Understand why corners are good features (gradient in 2 directions).
- Tune parameters for typical scenes.

Prerequisites

- Module 13 Lesson 1

1. Why This Matters

A corner is a pixel where intensity changes sharply in two directions — distinctive enough to track across frames, recognise across images, or use as a keypoint for higher-level features (ORB, SIFT). Harris was the first major corner detector; Shi-Tomasi (`cv2.goodFeaturesToTrack`) is its everyday successor.

2. Concept

2.1 What makes a corner

- Flat region: gradient ≈ 0 in any direction. Not a corner.
- Edge: gradient large in one direction (perpendicular to edge). Not a corner.
- Corner: gradient large in two perpendicular directions.

2.2 Harris response

For each pixel, Harris computes a response based on the gradient covariance matrix:

$$R = \det(M) - k \cdot \text{trace}(M)^2$$

Where M is built from local Sobel gradients. Pixels with R above a threshold are corners.

2.3 OpenCV API — Harris

```
gray = np.float32(gray)
dst = cv2.cornerHarris(gray, blockSize=2, ksize=3, k=0.04)
dst = cv2.dilate(dst, None)
mask = dst > 0.01 * dst.max()
img[mask] = (0, 0, 255)
```

Param	Meaning
blockSize	Neighbourhood size for M (2 default)
ksize	Sobel aperture (3)
k	Harris parameter (0.04-0.06)

2.4 Shi-Tomasi (`goodFeaturesToTrack`)

Easier-to-use, more reliable variant. Returns the top-N strongest corners directly:

```
corners = cv2.goodFeaturesToTrack(gray, maxCorners=100, qualityLevel=0.01,
minDistance=10)
```

Param	Meaning
maxCorners	maximum number to return
qualityLevel	fraction of best corner's quality required (0.01 = corners $\geq$ 1% of best)
minDistance	min pixels between corners (anti-clustering)

2.5 Sub-pixel refinement

For precise corner positions:

```
corners = cv2.goodFeaturesToTrack(gray, 100, 0.01, 10)
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)
refined = cv2.cornerSubPix(gray, corners, (5, 5), (-1, -1), criteria)
```

2.6 Pre-blur

Like every gradient-based op, light Gaussian blur ($\sigma=1$) before corner detection makes results more robust.

3. Code Examples

Example 1: Harris on a checkerboard

```
import cv2, numpy as np
from skimage.data import checkerboard

img = checkerboard().astype(np.uint8) * 255 if checkerboard().max() <= 1 else
checkerboard().astype(np.uint8)
gray = np.float32(img)
dst = cv2.cornerHarris(gray, blockSize=2, ksize=3, k=0.04)
dst = cv2.dilate(dst, None)

vis = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
vis[dst > 0.01 * dst.max()] = (0, 0, 255)
cv2.imwrite("checker_harris.png", vis)
print("max response:", dst.max())
```

Expected Output: Checkerboard with red dots at the corners of the squares.

Example 2: `goodFeaturesToTrack`

```
import cv2, numpy as np
from skimage.data import astronaut

gray = cv2.cvtColor(cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR),
cv2.COLOR_BGR2GRAY)
corners = cv2.goodFeaturesToTrack(gray, maxCorners=100, qualityLevel=0.01,
minDistance=10)
```

```
vis = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)
for c in corners.astype(int):
    x, y = c.ravel()
    cv2.circle(vis, (x, y), 3, (0, 255, 0), -1)
cv2.imwrite("astro_corners.png", vis)
print("corners found:", len(corners))
```

Expected Output: Astronaut image with 100 green dots at strong corners — eyes, helmet edges, mission patch text.

Example 3: Tune qualityLevel

```
import cv2, numpy as np
from skimage.data import camera

gray = camera()
for q in [0.005, 0.01, 0.05, 0.2]:
    c = cv2.goodFeaturesToTrack(gray, 200, q, 10)
    print(f"q={q}: {len(c) if c is not None else 0} corners")
```

Expected Output: Higher q → fewer corners (only the strongest). q=0.2 typically returns only a handful.

Example 4: Sub-pixel refinement

```
import cv2, numpy as np
from skimage.data import checkerboard

img = (checkerboard().astype(np.uint8) * 255) if checkerboard().max() <= 1 else checkerboard()
gray = np.float32(img)

corners = cv2.goodFeaturesToTrack(gray, 81, 0.01, 10)
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)
refined = cv2.cornerSubPix(gray, corners, (5, 5), (-1, -1), criteria)
print("raw corner 0:", corners[0].ravel())
print("refined    0:", refined[0].ravel())
```

Expected Output: Refined coordinates are slightly different — typically sub-pixel adjustments like (20.000, 20.000) → (19.998, 19.998).

4. Parameter Sensitivity

Param	Effect
qualityLevel	higher = stricter / fewer corners
minDistance	spacing between detections
k (Harris)	rarely changed; 0.04-0.06

5. Common Mistakes

Mistake	What Happens	Fix
Pass uint8 to cornerHarris	Should be float	np.float32(gray)

qualityLevel too low	Lots of false corners	Try 0.01-0.05
Forget to dilate Harris dst	Tiny single-pixel hotspots	cv2.dilate(dst, None) for visibility
Use as descriptor	Just coordinates, no descriptor	Use ORB/SIFT for descriptors

6. Practice Tasks

- Harris on a checkerboard; show red dots at corners.
- goodFeaturesToTrack (top 100) on astronaut().
- Compare qualityLevel $\in$ {0.005, 0.05, 0.2}.

7. Key Takeaways

- Corner = strong gradient in 2 directions.
- Harris: cornerHarris returns a response map.
- Shi-Tomasi: goodFeaturesToTrack returns top-N corners directly.
- Use cornerSubPix for precise coordinates (calibration).

8. What's Next

ORB — keypoints + descriptors, the practical replacement for SIFT.

Lesson 3: ORB Keypoints & Descriptors

Module: 13 — Feature Detection & Matching

Duration: 35 minutes

Difficulty: Intermediate

Learning Objectives

- Use `cv2.ORB_create` to detect keypoints and compute descriptors.
- Distinguish keypoint vs descriptor.
- Visualise keypoints with `cv2.drawKeypoints`.

Prerequisites

- Module 13 Lesson 2

1. Why This Matters

ORB (Oriented FAST + Rotated BRIEF) is free, fast, and built into OpenCV. It produces rotation- and scale-invariant features that can be matched across images — the foundation for image alignment, panorama stitching, and basic visual recognition.

2. Concept

2.1 Keypoint vs descriptor

- Keypoint = a 2D location (often with a scale + orientation).
- Descriptor = a small vector that "describes" the local image patch around the keypoint, so two patches with similar content have similar descriptors.

You need both for matching: location says where, descriptor says what.

2.2 ORB pipeline (very brief)

- Detect FAST corners at multiple scales.
- Assign orientation per corner from intensity moments.
- Compute a 256-bit BRIEF descriptor at each keypoint, rotated to the keypoint's orientation.

2.3 OpenCV API

```
orb = cv2.ORB_create(nfeatures=500)
keypoints, descriptors = orb.detectAndCompute(gray, mask=None)
```

- keypoints: list of `cv2.KeyPoint` objects (with `.pt`, `.angle`, `.size`).
- descriptors: (N, 32) `uint8` — each row is a 256-bit binary string.

2.4 ORB parameters

Param	Default	Meaning
nfeatures	500	max keypoints to return
scaleFactor	1.2	pyramid scale

nlevels	8	pyramid levels
edgeThreshold	31	edge pixels to ignore
WTA_K	2	descriptor type

2.5 Visualising

```
vis = cv2.drawKeypoints(img, keypoints, None,
                        flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
```

RICH_KEYPOINTS draws circles with size and orientation.

2.6 SIFT vs ORB (briefly)

- SIFT — patented for years (free since 2020); higher quality matches; slower; descriptor is 128 floats.
- ORB — always free; fast; binary descriptor; slightly less accurate.

For most practical work, ORB is fine. SIFT is preferred when you need more reliable matches and can afford the time.

3. Code Examples

Example 1: Detect ORB keypoints

```
import cv2
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

orb = cv2.ORB_create(nfeatures=500)
kp, desc = orb.detectAndCompute(gray, None)
print("keypoints:", len(kp))
print("descriptor shape:", desc.shape, desc.dtype)

vis = cv2.drawKeypoints(img, kp, None,
                        flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
cv2.imwrite("astro_orb_kp.png", vis)
```

Expected Output:

```
keypoints: 500
descriptor shape: (500, 32) uint8
```

Image shows ~500 circles of various sizes covering distinctive areas (eyes, helmet text, suit edges).

Example 2: Inspect a keypoint

```
import cv2
from skimage.data import astronaut

gray = cv2.cvtColor(cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR),
                    cv2.COLOR_BGR2GRAY)
```

```

kp, _ = cv2.ORB_create(50).detectAndCompute(gray, None)
k = kp[0]
print(f"pt = ({k.pt[0]:.1f}, {k.pt[1]:.1f}) size = {k.size:.1f} angle = {k.angle:.1f} response = {k.response:.3f}")

```

Expected Output (varies):

```

pt = (45.4, 312.7) size = 31.0 angle = 75.2 response = 0.000178

```

Example 3: ORB on two views — same image

```

import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Slightly rotated version
h, w = gray.shape
M = cv2.getRotationMatrix2D((w/2, h/2), 15, 1.0)
rotated = cv2.warpAffine(gray, M, (w, h))

orb = cv2.ORB_create(500)
kp1, d1 = orb.detectAndCompute(gray, None)
kp2, d2 = orb.detectAndCompute(rotated, None)
print(f"original: {len(kp1)} kp, rotated: {len(kp2)} kp")

```

Expected Output: Both around 500 keypoints; ORB recovers most after rotation.

Example 4: Tune nfeatures

```

import cv2
from skimage.data import coffee

gray = cv2.cvtColor(cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR), cv2.COLOR_BGR2GRAY)
for n in [50, 200, 1000]:
    kp, _ = cv2.ORB_create(n).detectAndCompute(gray, None)
    print(f"nfeatures={n}: {len(kp)} keypoints returned")

```

Expected Output: Larger nfeatures returns more keypoints (up to what ORB can find).

4. Parameter Sensitivity

Param	Effect
nfeatures	how many keypoints to return
scaleFactor	scale-space granularity
edgeThreshold	excludes near-edge keypoints

5. Common Mistakes

Mistake	What Happens	Fix
Pass color image	Works but gray is preferred	Convert to grayscale
Forget desc may be None if no keypoints	NoneType errors downstream	Check if desc is None or len(kp) == 0:

Use <code>cv2.SIFT_create()</code> thinking it's free in old OpenCV	May fail on older builds	OpenCV $\geq$ 4.4 has it free
---	--------------------------	-------------------------------

6. Practice Tasks

- Detect 500 ORB keypoints on `astronaut()`; visualise with `RICH_KEYPOINTS`.
- Inspect the first keypoint's `pt`, `size`, `angle`, `response`.
- Rotate the image 15° ; detect again; compare keypoint count.

7. Key Takeaways

- ORB detects keypoints AND computes a 256-bit descriptor per point.
- Always free, fast, rotation-invariant.
- `orb.detectAndCompute(gray, None)` is the standard call.
- Pair with a binary matcher (next lesson).

8. What's Next

Matching descriptors between two images.

Lesson 4: Feature Matching

Module: 13 — Feature Detection & Matching

Duration: 35 minutes

Difficulty: Intermediate

Learning Objectives

- Match ORB descriptors with `cv2.BFMatcher`.
- Filter matches with Lowe's ratio test.
- Visualise matches with `cv2.drawMatches`.
- Use `cv2.findHomography` to recover the geometric relationship.

Prerequisites

- Module 13 Lesson 3

1. Why This Matters

Matching is the bridge from keypoints to actual alignment / recognition / stitching. ORB + BF + ratio test + homography is the recipe for panorama stitching, image registration, and basic visual SLAM front-ends.

2. Concept

2.1 Brute Force matcher

For each descriptor in image A, find the nearest descriptor in image B by distance.

For ORB (binary descriptors), use Hamming distance:

```
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=False)
matches = bf.knnMatch(desc1, desc2, k=2)
```

`k=2` requests the 2 nearest neighbours — needed for the ratio test.

2.2 Lowe's ratio test

A match is "good" if the best neighbour is much better than the second-best. Otherwise the match is ambiguous (texture, repeats).

```
good = []
for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good.append(m)
```

Typical threshold 0.7-0.8.

2.3 `crossCheck=True` alternative

Simpler than ratio: each match must be mutual (A's best in B and B's best in A). Use with `bf.match`, not `bf.knnMatch`:

```
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = bf.match(desc1, desc2)
matches = sorted(matches, key=lambda m: m.distance)
```

2.4 Drawing matches

```
img_matches = cv2.drawMatches(img1, kp1, img2, kp2, good, None,
                              flags=cv2.DRAW_MATCHES_FLAGS_NOT_DRAW_SINGLE_POINTS)
```

2.5 Homography from matches

If both images show the same planar surface, the geometric map between them is a 3×3 homography (M5 L3):

```
src_pts = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
dst_pts = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
```

RANSAC filters out outlier matches. mask marks which matches survived.

2.6 Use cases

- Panorama stitching (M16 project): find H between overlapping photos, warp one onto the other.
- Image registration: align satellite photos taken months apart.
- Object detection (limited): given a template object, find it in a scene by matching ORB features and checking H consistency.

3. Code Examples

Example 1: Match ORB between original and rotated

```
import cv2, numpy as np
from skimage.data import astronaut

img1 = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)

# Slightly rotated version
h, w = gray1.shape
M = cv2.getRotationMatrix2D((w/2, h/2), 20, 1.0)
gray2 = cv2.warpAffine(gray1, M, (w, h))

orb = cv2.ORB_create(500)
kp1, d1 = orb.detectAndCompute(gray1, None)
kp2, d2 = orb.detectAndCompute(gray2, None)

bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=False)
matches = bf.knnMatch(d1, d2, k=2)
good = [m for m, n in matches if m.distance < 0.75 * n.distance]
print(f"keypoints: {len(kp1)} / {len(kp2)}   raw matches: {len(matches)}   good: {len(good)}")

img2 = cv2.cvtColor(gray2, cv2.COLOR_GRAY2BGR)
```

```
vis = cv2.drawMatches(img1, kp1, img2, kp2, good[:30], None,
                      flags=cv2.DRAW_MATCHES_FLAGS_NOT_DRAW_SINGLE_POINTS)
cv2.imwrite("matches.png", vis)
```

Expected Output: Console shows ~100+ good matches. Image is two views side-by-side with green lines connecting matching keypoints.

Example 2: crossCheck path

```
import cv2
from skimage.data import astronaut

gray = cv2.cvtColor(cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR),
                    cv2.COLOR_BGR2GRAY)
orb = cv2.ORB_create(500)
kp1, d1 = orb.detectAndCompute(gray, None)
kp2, d2 = orb.detectAndCompute(gray, None)

bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = sorted(bf.match(d1, d2), key=lambda m: m.distance)
print(f"matches: {len(matches)}    best distance: {matches[0].distance}")
```

Expected Output: Same image matched to itself → all keypoints match perfectly with distance 0.

Example 3: Homography between matched views

```
import cv2, numpy as np
from skimage.data import astronaut

img1 = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
h, w = gray1.shape
M = cv2.getRotationMatrix2D((w/2, h/2), 20, 1.0)
gray2 = cv2.warpAffine(gray1, M, (w, h))
img2 = cv2.cvtColor(gray2, cv2.COLOR_GRAY2BGR)

orb = cv2.ORB_create(1000)
kp1, d1 = orb.detectAndCompute(gray1, None)
kp2, d2 = orb.detectAndCompute(gray2, None)

bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=False)
matches = bf.knnMatch(d1, d2, k=2)
good = [m for m, n in matches if m.distance < 0.75 * n.distance]
print(f"good: {len(good)}")

if len(good) >= 4:
    src = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
    dst = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
    H, mask = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)
    print("inliers:", int(mask.sum()), "/", len(good))
    print("H =\n", H)
```

Expected Output: Most matches are inliers (RANSAC keeps the consistent ones). Homography matrix is roughly a rotation by 20° around the centre.

4. Parameter Sensitivity

Param	Effect
Ratio threshold	0.7 strict, 0.8 lenient
nfeatures (ORB)	more keypoints → more potential matches
RANSAC threshold	larger = more inliers but less precise H

5. Common Mistakes

Mistake	What Happens	Fix
cv2.NORM_L2 with ORB	Wrong distance	Use NORM_HAMMING for binary descriptors
Skip ratio test	Tons of bogus matches	Always apply Lowe's
< 4 good matches for findHomography	Not enough constraints	Check $\text{len}(\text{good}) \geq 4$
Treat all matches as correct after RANSAC	Outliers may still slip in	Use mask to filter

6. Practice Tasks

- Rotate astronaut() 20°. Match ORB descriptors; count good.
- Match an image to itself; show all matches have distance 0.
- Compute a homography from matched ORBs; print inlier count.

7. Key Takeaways

- cv2.BFMatcher(cv2.NORM_HAMMING) for ORB.
- Use knnMatch(k=2) + Lowe's ratio test (0.75).
- cv2.findHomography(..., cv2.RANSAC) survives outliers.
- ORB + BF + RANSAC = the classic stitching / registration recipe.

8. What's Next

Hough transform — detect lines and circles without matching.

Lesson 5: Hough Transform — Lines & Circles

Module: 13 — Feature Detection & Matching

Duration: 35 minutes

Difficulty: Intermediate

Learning Objectives

- Detect straight lines with `cv2.HoughLines` and `cv2.HoughLinesP`.
- Detect circles with `cv2.HoughCircles`.
- Tune accumulator threshold and minimum line length.

Prerequisites

- Module 9 (Canny), Module 13 Lessons 1-4.

1. Why This Matters

When you need parametric shapes (straight lines, circles), Hough is the classic tool — robust to gaps, occlusions, and noise. Used everywhere from lane detection to coin counting.

2. Concept

2.1 The accumulator idea

Each edge pixel votes for all parametric shapes (lines / circles) it could lie on. After all votes, peaks in the parameter space correspond to actual shapes in the image.

For a line, the parameter space is (ρ, θ) — distance from origin and angle.

2.2 `cv2.HoughLines` — infinite lines

```
edges = cv2.Canny(gray, 50, 150)
lines = cv2.HoughLines(edges, rho=1, theta=np.pi/180, threshold=150)
# returns array of (rho, theta) pairs, shape (N, 1, 2)
```

Convert each (ρ, θ) to two points for drawing:

```
for rho, theta in lines[:, 0]:
    a, b = np.cos(theta), np.sin(theta)
    x0, y0 = a * rho, b * rho
    p1 = (int(x0 + 1000 * (-b)), int(y0 + 1000 * a))
    p2 = (int(x0 - 1000 * (-b)), int(y0 - 1000 * a))
    cv2.line(img, p1, p2, (0, 255, 0), 2)
```

2.3 `cv2.HoughLinesP` — line segments (preferred)

Probabilistic Hough returns finite segments with endpoints — much more useful:

```
segs = cv2.HoughLinesP(edges, rho=1, theta=np.pi/180, threshold=80,
                        minLineLength=40, maxLineGap=10)
# returns (N, 1, 4) of (x1, y1, x2, y2)
```

Param	Meaning
threshold	min votes for a line
minLineLength	discard segments shorter than this
maxLineGap	bridge gaps up to this length

2.4 cv2.HoughCircles

```

circles = cv2.HoughCircles(gray, cv2.HOUGH_GRADIENT, dp=1, minDist=20,
                           param1=100, param2=30, minRadius=10, maxRadius=80)
# returns (1, N, 3) of (x, y, r)

```

Param	Meaning
dp	inverse ratio of accumulator resolution to image (1 = same, 2 = half)
minDist	min distance between detected circle centres
param1	Canny high threshold (low = param1 / 2)
param2	accumulator threshold; smaller = more circles
minRadius, maxRadius	radius range

Pre-blur is essential — circles are noise-sensitive.

2.5 When to use Hough

Use	Notes
Lane detection	HoughLinesP with line angle filter
Coin / cell detection (round)	HoughCircles
Detecting structured shapes (paper edges)	HoughLinesP to find dominant lines
Generalised parametric matching	Generalised Hough (advanced)

2.6 Failure modes

- Noisy edges → too many spurious lines/circles. Increase threshold; pre-blur.
- Circle params off → no detections. Tune param2, radius range.
- Too many duplicate lines → use NMS (cluster nearby (ρ , θ)).

3. Code Examples

Example 1: HoughLinesP on a chessboard

```

import cv2, numpy as np
from skimage.data import checkerboard

img = (checkerboard().astype(np.uint8) * 255) if checkerboard().max() <= 1 else
checkerboard()
edges = cv2.Canny(img, 50, 150)
segs = cv2.HoughLinesP(edges, 1, np.pi/180, threshold=80,
                      minLineLength=40, maxLineGap=10)
vis = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
for x1, y1, x2, y2 in segs[:, 0]:
    cv2.line(vis, (x1, y1), (x2, y2), (0, 0, 255), 2)
print("segments:", len(segs))
cv2.imwrite("checker_hlines.png", vis)

```

Expected Output: Checkerboard with red lines drawn along its grid edges.

Example 2: HoughCircles on coins

```
import cv2, numpy as np
from skimage.data import coins

gray = cv2.medianBlur(coins(), 5)
circles = cv2.HoughCircles(gray, cv2.HOUGH_GRADIENT, dp=1, minDist=30,
                           param1=100, param2=30, minRadius=20, maxRadius=40)
print("circles:", 0 if circles is None else circles.shape[1])
vis = cv2.cvtColor(coins(), cv2.COLOR_GRAY2BGR)
if circles is not None:
    for x, y, r in circles[0].astype(int):
        cv2.circle(vis, (x, y), r, (0, 255, 0), 2)
        cv2.circle(vis, (x, y), 2, (0, 0, 255), 3)
cv2.imwrite("coins_hcircles.png", vis)
```

Expected Output: Coin image with green circles drawn around each detected coin (radius approximately matching coin radius).

Example 3: Tune HoughLinesP threshold

```
import cv2, numpy as np
from skimage.data import camera

edges = cv2.Canny(camera(), 60, 150)
for t in [50, 100, 200]:
    segs = cv2.HoughLinesP(edges, 1, np.pi/180, threshold=t, minLineLength=20,
                           maxLineGap=5)
    n = 0 if segs is None else len(segs)
    print(f"threshold={t}: {n} segments")
```

Expected Output: Lower threshold → more segments. Higher threshold → only the strongest lines survive.

Example 4: Detect lane-like lines (horizontal filter)

```
import cv2, numpy as np

# Synthesize a "road" image
img = np.full((300, 600, 3), 100, dtype=np.uint8)
cv2.line(img, (50, 250), (300, 100), (255, 255, 255), 4)
cv2.line(img, (550, 250), (300, 100), (255, 255, 255), 4)
cv2.line(img, (100, 280), (500, 280), (200, 200, 200), 2)

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 50, 150)
segs = cv2.HoughLinesP(edges, 1, np.pi/180, 50, minLineLength=50, maxLineGap=20)

for x1, y1, x2, y2 in segs[:, 0]:
    angle = np.degrees(np.arctan2(y2 - y1, x2 - x1))
    if abs(angle) > 20: # keep slanted lines (lanes)
        cv2.line(img, (x1, y1), (x2, y2), (0, 0, 255), 2)
cv2.imwrite("lanes.png", img)
```


Expected Output: Only the slanted "lane" lines outlined in red; the near-horizontal stripe is ignored.

4. Parameter Sensitivity

Param	Direction
threshold (lines)	higher = fewer lines
minLineLength	larger = filter short noise
maxLineGap	larger = bridge bigger gaps
param2 (circles)	smaller = more circles
minRadius/maxRadius	narrow range = faster + less noise

5. Common Mistakes

Mistake	What Happens	Fix
Skip Canny before HoughLines	No edges to vote	Run Canny first
Wrong radius range for circles	No detections	Inspect typical radius first
Skip pre-blur for circles	Noisy false positives	medianBlur(5) is a safe default
HoughLines vs HoughLinesP confusion	Different output formats	Prefer LinesP for segments

6. Practice Tasks

- HoughLinesP on a checkerboard; draw red lines.
- HoughCircles on coins(); draw a green circle on each.
- Filter to keep only slanted "lane" lines in a synthetic road scene.

7. Key Takeaways

- Hough = voting in parameter space.
- HoughLinesP for line segments; HoughCircles for circles.
- Always pre-process: Canny for lines, blur for circles.
- Tune thresholds for sensitivity.

8. What's Next

End of Module 13. Next: Module 14 — Frequency Domain (DFT).

Module 13 — Exercises

15 exercises.

13.1 Template Matching

- (E) Crop 50×50 from astronaut(); match back; verify max_loc.
- (M) Multi-match on synthetic scene with 3 copies of an "X".
- (H) Compare TM_SQDIFF / TM_CCORR_NORMED / TM_CCOEFF_NORMED on the same input.

13.2 Harris / Shi-Tomasi

- (E) Harris on checkerboard(); show red corner dots.
- (M) goodFeaturesToTrack (100) on astronaut().
- (H) Compare qualityLevel $\in$ {0.005, 0.05, 0.2}.

13.3 ORB

- (E) ORB(500) on astronaut(); draw RICH_KEYPOINTS.
- (M) Inspect kp[0].pt, .size, .angle, .response.
- (H) Rotate image 15°; detect again; compare kp counts.

13.4 Matching

- (E) Rotate astronaut() 20°. Match ORB; count good.
- (M) Match an image to itself; verify distance 0.
- (H) Estimate homography with RANSAC; print inlier count.

13.5 Hough

- (E) HoughLinesP on checkerboard(); draw red lines.
- (M) HoughCircles on coins(); draw green circles.
- (H) Filter lanes by angle from a synthetic road scene.

Module 13 — Quiz

10 MCQs.

Q1

For brightness-invariant template matching use: A. TM_SQDIFF B. TM_CCORR C. TM_CCOEFF_NORMED D. TM_CCORR_NORMED

Answer: C (L1)

Q2

Template matching is invariant to: A. brightness (with NORMED) but not rotation/scale B. all transforms C. rotation D. nothing

Answer: A (L1)

Q3

Harris corner = strong gradient in: A. one direction B. two directions C. four directions D. all directions

Answer: B (L2)

Q4

cv2.goodFeaturesToTrack returns: A. all corners B. response map C. top-N strongest corners D. only the best

Answer: C (L2)

Q5

ORB descriptor is: A. 128 floats B. 256-bit binary (32 bytes uint8) C. JPEG-encoded D. RGB color

Answer: B (L3)

Q6

For ORB matching, the right distance metric is: A. NORM_L2 B. NORM_HAMMING C. NORM_INF D. NORM_L1

Answer: B (L4)

Q7

Lowe's ratio test discards matches where: A. $\text{best} > 0.75 \times \text{second-best}$ B. $\text{distance} > 0.75$ C. crosscheck fails D. RANSAC rejects

Answer: A (L4)

Q8

`cv2.findHomography(..., cv2.RANSAC, 5.0)`: A. requires exact data B. tolerates outliers C. fits a line D. returns a 2x2

Answer: B (L4)

Q9

For finite line segments, use: A. `HoughLines` B. `HoughLinesP` C. `HoughCircles` D. `matchTemplate`

Answer: B (L5)

Q10

For circle detection, the typical pre-processing is: A. Canny B. `medianBlur` C. erosion D. `equalizeHist`

Answer: B (L5)

Module 13 — Assignment: Logo Finder

Estimated time: 2 hours

Combines: Module 13 + Modules 7, 9

Brief

Given a small "logo" template and a larger "scene", find the logo in the scene using:

- Template matching (rigid).
- ORB + matching + homography (rotation/scale invariant).

Compare results.

Requirements

- `find_logo.py SCENE LOGO [--mode template|orb] [--output OUT]`
- template mode: `cv2.matchTemplate` + threshold; draw bounding box of best match.
- orb mode: ORB on both → `BFMatcher` + Lowe ratio → `findHomography` → warp logo bbox into scene; draw resulting quadrilateral.
- Print number of matches (orb) or best score (template).
- Save annotated scene.

Constraints

- Modules 1-13 only.
- Use `argparse`.
- Handle the case "no good match" gracefully.

Expected Output

```
$ python find_logo.py scene.png logo.png --mode template
best score: 0.92 at (x=325, y=180)
saved -> scene_template.png
```

```
$ python find_logo.py scene.png logo.png --mode orb
good matches: 84 inliers: 71
saved -> scene_orb.png
```

Grading Rubric

Criterion	Weight
Template path	30%
ORB + matching path	35%
<code>findHomography</code> use	15%
CLI + reporting	10%
Style	10%

Submission

Save as `assignment_module13.py`.

Module 13 — Mini Project: Coin Counter via Hough

Overview

Count circular coins in a photo using `cv2.HoughCircles`. Output an annotated image with each coin numbered and the total.

Concepts Used

- `cv2.medianBlur` (M7 L4)
- `cv2.HoughCircles` (M13 L5)
- Drawing (M3 L2)

Features

- `count_coins.py INPUT [--minR 20] [--maxR 60] [--p2 30] [--output OUT]`
- Pre-blur with `medianBlur(5)`.
- Detect circles via `HOUGH_GRADIENT`.
- Number each detected coin; print total.

Starter Code

`count_coins.py`:

```
import sys, cv2, numpy as np, argparse

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("input")
    ap.add_argument("--minR", type=int, default=20)
    ap.add_argument("--maxR", type=int, default=60)
    ap.add_argument("--p2", type=int, default=30)
    ap.add_argument("--output", default="coins_counted.png")
    args = ap.parse_args()

    img = cv2.imread(args.input)
    if img is None:
        raise SystemExit("can't load: " + args.input)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    gray = cv2.medianBlur(gray, 5)

    circles = cv2.HoughCircles(gray, cv2.HOUGH_GRADIENT, dp=1, minDist=args.minR,
                               param1=100, param2=args.p2,
                               minRadius=args.minR, maxRadius=args.maxR)

    if circles is None:
        print("no circles found"); return

    for i, (x, y, r) in enumerate(circles[0].astype(int), 1):
        cv2.circle(img, (x, y), r, (0, 255, 0), 2)
        cv2.putText(img, str(i), (x - 10, y + 5),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)
    print("coins:", circles.shape[1])
```

```
cv2.imwrite(args.output, img)

if __name__ == "__main__":
    main()
```

Expected Output

```
$ python count_coins.py datasets/starter/coins.png
coins: 24
```

coins_counted.png shows green circles around each coin, each numbered 1..N.

Extension Challenges

- Add trackbars for live param tuning before commit.
- Cluster coins by radius into 2 "denominations".
- Robust mode: try several param2 values and pick one that gives circles within expected count range.

Grading Rubric

Criterion	Weight
Circle detection works	35%
Pre-blur applied	10%
Annotation + numbering	25%
CLI / argparse	15%
Style	15%

Python E-Course

Module 14 — Frequency Domain (DFT)

Detailed Notes

Module Overview

Level: Advanced

Duration: ~1 week

Prerequisites: Modules 1-13

What You'll Learn

- Why frequencies matter (1D audio analogy → 2D images).
- Compute and visualise the 2D DFT with `cv2.dft` / `np.fft`.
- Design low-pass, high-pass, and band-pass filters in the frequency domain.
- Build notch filters to remove periodic noise.
- Reason about spatial filtering as frequency multiplication.

Lessons

#	Lesson	Duration
1	1D Fourier — Audio Analogy	30 min
2	2D DFT — Magnitude Spectrum	35 min
3	Low-Pass & High-Pass Filters	30 min
4	Notch Filters — Periodic Noise	30 min
5	Spatial ↔ Frequency Equivalence	25 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 15: Image Restoration & Noise

Lesson 1: 1D Fourier — Audio Analogy

Module: 14 — Frequency Domain (DFT)

Duration: 30 minutes

Difficulty: Intermediate-Advanced

Learning Objectives

- Explain frequency intuitively via the audio analogy.
- Compute a 1D DFT with NumPy.
- Read a magnitude spectrum.
- Filter a 1D signal by zeroing frequencies.

Prerequisites

- Modules 1-13, basic high-school math (sine waves).

1. Why This Matters

Frequencies are easier to grasp in 1D (audio) than in 2D (images). Once you've seen "a low rumble + a high whistle = a recorded sound", the 2D version ("slow gradient + fine texture = a photo") falls into place.

2. Concept

2.1 A signal is a sum of sines

Any signal can be decomposed into a sum of sine waves of different frequencies. That's what Fourier showed.

In sound:

- Low frequencies = bass (slow oscillations).
- High frequencies = treble (fast oscillations).
- A musical note = many frequencies summed together.

In a 1D image row:

- Low frequencies = slow brightness changes (the overall gradient, smooth shading).
- High frequencies = fast brightness changes (edges, fine texture, noise).

2.2 The Discrete Fourier Transform (DFT)

For a length-N signal $x[n]$:

$$X[k] = \sum_n x[n] \cdot \exp(-2\pi i \cdot kn/N) \quad \text{for } k = 0..N-1$$

$X[k]$ is complex: magnitude (how strong frequency k is) and phase (where its peaks are).

You'll almost never compute this by hand. NumPy's `np.fft.fft` does it.

2.3 The magnitude spectrum

```
X = np.fft.fft(x)
mag = np.abs(X)
```

Peaks in mag tell you which frequencies dominate. A pure sine wave produces one tall peak; a noisy signal has many small peaks.

2.4 Frequency bins

$X[0]$ is the DC component (average / zero frequency). $X[1]$ is the lowest non-zero frequency. $X[N/2]$ is the highest representable frequency (Nyquist). $X[k]$ for $k > N/2$ is the negative-frequency mirror.

For an image row of 1024 pixels, that's 512 distinct frequencies.

2.5 Filtering in frequency space

To remove fast oscillations from a signal:

- Compute the DFT.
- Zero out the high-frequency coefficients.
- Inverse-DFT back to time/space.

The result is a low-pass-filtered signal — equivalent to a smoothing filter applied in time/space.

2.6 fftshift

By default fft puts the DC at index 0. For plotting and centered filtering, shift it to the middle:

```
X_shifted = np.fft.fftshift(X)      # DC now at N/2
```

3. Code Examples

Example 1: Pure sine + DFT

```
import numpy as np
import matplotlib.pyplot as plt

N = 256
t = np.arange(N)
x = 1.5 * np.sin(2 * np.pi * 8 * t / N) + 0.8 * np.sin(2 * np.pi * 20 * t / N)

X = np.fft.fft(x)
mag = np.abs(X)

fig, ax = plt.subplots(2, 1, figsize=(8, 5))
ax[0].plot(x); ax[0].set_title("signal")
ax[1].plot(mag[:N/2]); ax[1].set_title("magnitude (positive freq half)")
ax[1].set_xlabel("frequency bin")
plt.tight_layout(); plt.show()
print("peaks at:", np.argsort(mag[:N/2])[-2:][::-1])
```

Expected Output: Top: a wavy signal. Bottom: two tall spikes at bins 8 and 20 (the frequencies we put in). Console prints peaks at: [8, 20].

Example 2: Noisy signal → low-pass via DFT

```
import numpy as np
import matplotlib.pyplot as plt

N = 256
t = np.arange(N)
clean = np.sin(2 * np.pi * 5 * t / N)
np.random.seed(0)
noisy = clean + 0.4 * np.random.randn(N)

X = np.fft.fft(noisy)
# Keep only frequencies 0..15 (and their mirrors at N-15..N-1)
mask = np.zeros(N)
mask[:16] = 1; mask[-15:] = 1
X_lp = X * mask
clean_est = np.real(np.fft.ifft(X_lp))

fig, ax = plt.subplots(2, 1, figsize=(8, 5))
ax[0].plot(noisy, label="noisy"); ax[0].plot(clean, label="true clean", alpha=0.5);
ax[0].legend()
ax[1].plot(clean_est); ax[1].set_title("recovered via low-pass DFT")
plt.tight_layout(); plt.show()
```

Expected Output: Top: jittery noisy signal overlaid with the smooth target. Bottom: recovered signal nearly matches the clean target.

Example 3: Inspect an image row's spectrum

```
import numpy as np
import matplotlib.pyplot as plt
from skimage.data import camera

img = camera()
row = img[256].astype(np.float32) # row through middle

X = np.fft.fft(row)
mag = np.abs(np.fft.fftshift(X)) # shifted: 0 freq in the middle

fig, ax = plt.subplots(2, 1, figsize=(8, 5))
ax[0].plot(row); ax[0].set_title("middle row intensity")
ax[1].plot(mag); ax[1].set_title("|DFT|, shifted")
plt.tight_layout(); plt.show()
print("DC magnitude:", mag[len(mag)//2])
```

Expected Output: Top: gray-level profile along the row. Bottom: a symmetric spectrum, peaked at the centre (DC = average intensity × N), with smaller side peaks for textures.

Example 4: Verify FFT round-trip

```
import numpy as np
```

```

x = np.random.randn(64)
back = np.fft.ifft(np.fft.fft(x))
print("max imag part:", np.abs(back.imag).max())
print("max real diff:", np.abs(x - back.real).max())

```

Expected Output: Both numbers tiny ($\sim 1e-15$) — `ifft(fft(x))` recovers `x` exactly (up to floating-point error).

4. Parameter Sensitivity

(Conceptual lesson — see L3 for filter design parameters.)

5. Common Mistakes

Mistake	What Happens	Fix
Plot real part of <code>np.fft.fft</code> directly	Negative values, confusing	Use <code>np.abs</code> for magnitude
Forget Hermitian symmetry	Confused by "negative frequencies"	They're the mirror; for real signals, equivalent info
Filter without symmetry	Result has imaginary part	Symmetric masks keep result real

6. Practice Tasks

- Build a signal with sines at frequencies 5 and 25; verify FFT shows peaks there.
- Add Gaussian noise to a sine wave; recover it by zeroing high frequencies.
- Inspect the DFT of one row of `camera()`; print the DC magnitude.

7. Key Takeaways

- Any signal = sum of sines at different frequencies.
- DFT: signal $\leftrightarrow$ complex spectrum (`np.fft.fft` / `np.fft.ifft`).
- `np.abs` for magnitude; `np.fft.fftshift` to centre DC.
- Filtering in frequency = zero out unwanted frequencies, IFFT back.

8. What's Next

Same idea in 2D — the DFT of an image.

Lesson 2: 2D DFT — Magnitude Spectrum

Module: 14 — Frequency Domain (DFT)

Duration: 35 minutes

Difficulty: Intermediate-Advanced

Learning Objectives

- Compute the 2D DFT of an image with `np.fft.fft2` and `cv2.dft`.
- Visualise the magnitude spectrum (log-scaled, shifted).
- Read the centred spectrum: where are low/high frequencies?
- Interpret common visual patterns in the spectrum.

Prerequisites

- Module 14 Lesson 1

1. Why This Matters

Once you can read a 2D spectrum, you can diagnose images at a glance: blurry → bright centre only, fine texture → energy at the edges, periodic noise → bright spots off-centre. Frequency-domain filtering is then "draw a mask in the spectrum and invert."

2. Concept

2.1 2D DFT — two passes of 1D DFT

The 2D DFT decomposes a (H, W) image into a 2D grid of complex coefficients of the same shape. Each coefficient (u, v) says "how much sine-wave content at horizontal frequency u and vertical frequency v is in the image".

Mathematically:

$$F(u, v) = \sum_x \sum_y I(x, y) \cdot \exp(-2\pi i \cdot (ux/W + vy/H))$$

But you'll never compute this by hand. Use:

```
F = np.fft.fft2(img)          # complex (H, W)
```

2.2 Centring with `fftshift`

By default, frequency (0, 0) (the DC) lives at array position [0, 0]. For viewing and centred masks, shift it to the middle:

```
Fs = np.fft.fftshift(F)      # DC at (H//2, W//2)
```

Now low frequencies sit near the centre and high frequencies are at the corners (of the shifted array, that means farther from centre).

2.3 Magnitude spectrum

```
mag = np.abs(Fs)
```

Values span many orders of magnitude — the DC alone is often $>10^6$ while textures are 10^2 . Use log for display:

```
mag_vis = np.log1p(mag)
mag_vis = cv2.normalize(mag_vis, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
```

2.4 Reading the spectrum

Pattern in spectrum	What it means about the image
Bright DC, dim elsewhere	Mostly uniform / very blurry
Bright cross + DC	Strong horizontal AND vertical edges (a building scene)
Bright diagonal smear	Diagonal edges / a tilted dominant texture
Bright dots off-centre	Periodic patterns (screens, fences, stripes)
Energy spread to corners	Lots of fine texture / noise

The spectrum is symmetric about the centre for real-valued images (Hermitian symmetry).

2.5 Inverse DFT

```
F = np.fft.fft2(img)
img_back = np.real(np.fft.ifft2(F))
```

Within rounding, identical to the input. Filtering happens between `fft2` and `ifft2`.

2.6 OpenCV vs NumPy

API	Notes
<code>np.fft.fft2 / ifft2</code>	Simplest; returns complex array
<code>cv2.dft(..., flags=cv2.DFT_COMPLEX_OUTPUT)</code>	Faster on large images, returns (H, W, 2) real+imag
<code>cv2.idft</code>	Inverse

For learning, stick with NumPy. For production with big images, OpenCV.

3. Code Examples

Example 1: Spectrum of camera()

```
import cv2, numpy as np
import matplotlib.pyplot as plt
from skimage.data import camera

img = camera().astype(np.float32)
F = np.fft.fft2(img)
Fs = np.fft.fftshift(F)
mag_vis = np.log1p(np.abs(Fs))
mag_vis = cv2.normalize(mag_vis, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

fig, ax = plt.subplots(1, 2, figsize=(10, 5))
ax[0].imshow(img, cmap="gray"); ax[0].set_title("image"); ax[0].axis("off")
ax[1].imshow(mag_vis, cmap="gray"); ax[1].set_title("log |DFT|, shifted");
ax[1].axis("off")
plt.tight_layout(); plt.show()
print("F shape:", F.shape, "dtype:", F.dtype)
```

Expected Output (visual): Left — cameraman. Right — bright spot in the middle (DC), with horizontal and vertical bright lines crossing it (from building edges) and a dim "haze" away from centre (other textures). Console prints (512, 512) complex128.

Example 2: Round-trip

```
import numpy as np
from skimage.data import camera

img = camera().astype(np.float32)
F = np.fft.fft2(img)
back = np.real(np.fft.ifft2(F))
print("max abs diff:", np.abs(back - img).max())
```

Expected Output: max abs diff: <1e-9 — perfect round-trip up to floating point.

Example 3: Compare spectra of different content

```
import cv2, numpy as np
import matplotlib.pyplot as plt
from skimage.data import camera

def spectrum(img):
    F = np.fft.fft2(img.astype(np.float32))
    Fs = np.fft.fftshift(F)
    v = np.log1p(np.abs(Fs))
    return cv2.normalize(v, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

img = camera()
blur = cv2.GaussianBlur(img, (0, 0), 4)
noisy = np.clip(img + np.random.normal(0, 25, img.shape), 0, 255).astype(np.uint8)

fig, ax = plt.subplots(2, 3, figsize=(12, 7))
for col, (name, x) in enumerate([("sharp", img), ("blurred", blur), ("noisy",
noisy)]):
    ax[0, col].imshow(x, cmap="gray"); ax[0, col].set_title(name); ax[0,
col].axis("off")
    ax[1, col].imshow(spectrum(x), cmap="gray"); ax[1, col].axis("off")
plt.tight_layout(); plt.show()
```

Expected Output: Blurred image has spectrum concentrated near the centre (highs gone). Noisy image has noticeably brighter outer regions (highs amplified). Sharp image is in between.

Example 4: OpenCV DFT

```
import cv2, numpy as np
from skimage.data import camera

img = camera().astype(np.float32)
dft = cv2.dft(img, flags=cv2.DFT_COMPLEX_OUTPUT) # shape (H, W, 2)
dft_shift = np.fft.fftshift(dft, axes=(0, 1))
mag = cv2.magnitude(dft_shift[..., 0], dft_shift[..., 1])
mag_vis = cv2.normalize(np.log1p(mag), None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)
```



```
cv2.imwrite("cam_spectrum_cv.png", mag_vis)
print("dft shape:", dft.shape)
```

Expected Output: dft shape: (512, 512, 2) and a saved spectrum visually similar to Example 1's right panel.

4. Parameter Sensitivity

Choice	Effect
Pre-resize to power of 2	DFT slightly faster (negligible for fft2)
np.log vs np.log1p	log1p handles zeros gracefully
fftshift vs not	Just visualisation convenience

5. Common Mistakes

Mistake	What Happens	Fix
Display np.abs(F) directly	Huge DC overshadows everything	Take log1p then normalize
Forget fftshift	DC stuck in the corner; harder to interpret	Always shift for display
Treat np.fft.fft2(img) as float	It's complex	Use np.abs for magnitude
ifft2 returns complex	.real to drop tiny imaginary residue	np.real(ifft2(...))

6. Practice Tasks

- Compute and visualise the log-magnitude spectrum of camera().
- Do the same for a Gaussian-blurred copy ($\sigma=4$); compare brightness distribution.
- Compute spectrum via OpenCV cv2.dft; verify magnitude matches NumPy.

7. Key Takeaways

- 2D DFT: np.fft.fft2(img) $\rightarrow$ complex (H, W).
- Display: log1p(|fftshift(F)|) then cv2.normalize.
- Centre = low frequencies; corners = high frequencies.
- Spectrum patterns diagnose what's in the image.

8. What's Next

Filtering — drawing low-pass and high-pass masks in the spectrum and inverting back to the image.

Lesson 3: Low-Pass & High-Pass Filters

Module: 14 — Frequency Domain (DFT)

Duration: 30 minutes

Difficulty: Advanced

Learning Objectives

- Design ideal low-pass and high-pass masks in frequency space.
- Apply Gaussian and Butterworth alternatives that avoid ringing.
- Inverse-DFT back to the image.
- Recognise ringing and how to control it.

Prerequisites

- Module 14 Lesson 2

1. Why This Matters

Frequency-domain filtering gives you fine control over what spatial scales survive. It's slower than a convolution for everyday blurs but irreplaceable for unusual filters (band-stops, custom shapes) and for understanding why spatial filters do what they do.

2. Concept

2.1 The pipeline

- $F = \text{fft2}(\text{image})$, then $F_s = \text{fftshift}(F)$.
- Multiply F_s by a mask $H(u, v)$ — same shape as F_s .
- Unshift: $F_2 = \text{ifftshift}(F_s * H)$.
- Inverse DFT: $\text{out} = \text{real}(\text{ifft2}(F_2))$.
- Clip to display range.

Ideal LP keeps a centred disk of radius D_0 ; ideal HP keeps the complement (centre zeroed out).

2.2 Distance-from-centre map

A reusable helper:

```
def distance_map(shape):
    h, w = shape
    cy, cx = h // 2, w // 2
    y, x = np.ogrid[:h, :w]
    return np.sqrt((y - cy)**2 + (x - cx)**2)
```

2.3 Ideal filters (sharp boundary)

```
D = distance_map(img.shape)
mask_lp = (D <= D0).astype(np.float32)      # ideal LP
mask_hp = 1.0 - mask_lp                     # ideal HP
```

Problem: ideal filters cause ringing — visible ripples around edges. The hard boundary in frequency space produces a sinc pattern in image space.

2.4 Gaussian filters (smooth boundary, no ringing)

```
mask_lp = np.exp(-(D**2) / (2 * sigma**2))  
mask_hp = 1.0 - mask_lp
```

Smooth roll-off → no ringing. Use these by default.

2.5 Butterworth filters (controlled steepness)

```
mask_lp = 1.0 / (1.0 + (D / D0)**(2 * n))    # order n
```

Higher n → steeper transition (closer to ideal, more ringing). n=2 is a good default.

2.6 Reading the result

LP: removes fine detail → smooth, blurry image. Roughly equivalent to spatial Gaussian blur with related σ .

HP: removes the slow-changing content → only edges and texture visible, centred around zero (offset for display).

2.7 D0 — the cutoff

D0 is the radius (in spectrum pixels) at which the filter transitions. For a 512×512 image:

- D0 = 30 → strong LP (removes most detail)
- D0 = 80 → moderate LP
- D0 = 200 → mild LP

For HP, same numbers — but they mean "remove frequencies below this radius".

3. Code Examples

Example 1: Ideal vs Gaussian LP — see ringing

```
import cv2, numpy as np  
import matplotlib.pyplot as plt  
from skimage.data import camera  
  
img = camera().astype(np.float32)  
F = np.fft.fft2(img)  
Fs = np.fft.fftshift(F)  
  
h, w = img.shape  
cy, cx = h // 2, w // 2  
y, x = np.ogrid[:h, :w]  
D = np.sqrt((y - cy)**2 + (x - cx)**2)  
  
D0 = 40  
mask_ideal = (D <= D0).astype(np.float32)  
mask_gaussian = np.exp(-(D**2) / (2 * D0**2))
```

```

def filt(Fs, mask):
    F2 = np.fft.ifftshift(Fs * mask)
    return np.clip(np.real(np.fft.ifft2(F2)), 0, 255).astype(np.uint8)

lp_ideal = filt(Fs, mask_ideal)
lp_gauss = filt(Fs, mask_gaussian)

fig, ax = plt.subplots(1, 3, figsize=(12, 4))
ax[0].imshow(img.astype(np.uint8), cmap="gray"); ax[0].set_title("original")
ax[1].imshow(lp_ideal, cmap="gray"); ax[1].set_title("ideal LP
(ringing)")
ax[2].imshow(lp_gauss, cmap="gray"); ax[2].set_title("Gaussian LP")
for a in ax: a.axis("off")
plt.tight_layout(); plt.show()

```

Expected Output (visual): Ideal LP has visible halos / ripples around the cameraman's silhouette. Gaussian LP is just smoothly blurred — no ringing.

Example 2: HP filter — only edges remain

```

import cv2, numpy as np
from skimage.data import camera

img = camera().astype(np.float32)
F = np.fft.fft2(img)
Fs = np.fft.fftshift(F)

h, w = img.shape
cy, cx = h // 2, w // 2
y, x = np.ogrid[:h, :w]
D = np.sqrt((y - cy)**2 + (x - cx)**2)
D0 = 30
mask_hp = 1.0 - np.exp(-(D**2) / (2 * D0**2))

F2 = np.fft.ifftshift(Fs * mask_hp)
hp = np.real(np.fft.ifft2(F2))
hp_vis = cv2.normalize(hp, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
cv2.imwrite("cam_hp.png", hp_vis)
print("HP range:", hp.min(), hp.max())

```

Expected Output (visual): Edges of the cameraman + buildings pop out as bright/dark traces on a near-uniform mid-gray. Console shows symmetric range around 0 before normalisation.

Example 3: Butterworth LP — tune order

```

import cv2, numpy as np
import matplotlib.pyplot as plt
from skimage.data import camera

img = camera().astype(np.float32)
F = np.fft.fft2(img); Fs = np.fft.fftshift(F)
h, w = img.shape
cy, cx = h // 2, w // 2

```

```

y, x = np.ogrid[:h, :w]
D = np.sqrt((y - cy)**2 + (x - cx)**2)
D0 = 40

fig, ax = plt.subplots(1, 3, figsize=(12, 4))
for i, n in enumerate([1, 2, 8]):
    mask = 1.0 / (1.0 + (D / D0)**(2 * n))
    out = np.clip(np.real(np.fft.ifft2(np.fft.ifftshift(Fs * mask))), 0,
255).astype(np.uint8)
    ax[i].imshow(out, cmap="gray"); ax[i].set_title(f"Butterworth n={n}");
ax[i].axis("off")
plt.tight_layout(); plt.show()

```

Expected Output: n=1 is a very gentle roll-off (close to Gaussian); n=8 is sharper (close to ideal, mild ringing creeping in).

Example 4: D0 sensitivity (Gaussian LP)

```

import cv2, numpy as np
from skimage.data import camera

img = camera().astype(np.float32)
F = np.fft.fft2(img); Fs = np.fft.fftshift(F)
h, w = img.shape
cy, cx = h // 2, w // 2
y, x = np.ogrid[:h, :w]
D = np.sqrt((y - cy)**2 + (x - cx)**2)

for D0 in [10, 30, 80, 200]:
    mask = np.exp(-(D**2) / (2 * D0**2))
    out = np.clip(np.real(np.fft.ifft2(np.fft.ifftshift(Fs * mask))), 0,
255).astype(np.uint8)
    cv2.imwrite(f"lp_D{D0}.png", out)

```

Expected Output: D0=10 is heavily smoothed (almost flat). D0=200 looks nearly identical to the original (most frequencies kept).

4. Parameter Sensitivity

Param	Effect
D0 (cutoff radius)	small = aggressive filter; large = mild
Mask shape (ideal/Gaussian/Butterworth)	ideal rings; Gaussian smooth; Butterworth tunable
Butterworth order n	1 (gentle) → 8 (near-ideal)

5. Common Mistakes

Mistake	What Happens	Fix
Forget ifftshift before ifft2	Image appears shifted by half	Always undo fftshift
Apply ideal mask without expecting ringing	Halos around edges	Use Gaussian / Butterworth
Mismatched shapes	Broadcast error	Build mask with img.shape

Discard imaginary part incorrectly	Lose phase info	<code>np.real(fft2(...))</code> only after multiplication
------------------------------------	-----------------	---

6. Practice Tasks

- Apply Gaussian LP with $D_0=40$ to `camera()`. Compare to `cv2.GaussianBlur`.
- Apply Gaussian HP with $D_0=30$. Compare to a Laplacian.
- Try Butterworth $n \in \{1, 2, 4, 8\}$ and observe ringing onset.

7. Key Takeaways

- Filter in frequency = multiply spectrum by a mask.
- Ideal mask $\rightarrow$ ringing; Gaussian $\rightarrow$ smooth; Butterworth $\rightarrow$ tunable steepness.
- D_0 is the cutoff radius in spectrum pixels.
- Always `fftshift` before `fft2`.

8. What's Next

Notch filters — surgically remove single periodic noise patterns.

Lesson 4: Notch Filters — Periodic Noise

Module: 14 — Frequency Domain (DFT)

Duration: 30 minutes

Difficulty: Advanced

Learning Objectives

- Identify periodic noise from its spectrum.
- Build notch-reject filters that zero out specific frequency dots.
- Remove screen/scanner moiré with a notch filter.

Prerequisites

- Module 14 Lesson 3

1. Why This Matters

Some noise has structure — scanned magazines have screen patterns, old TVs have horizontal scan lines, fluorescent lights cause banding. These show up as bright dots off-centre in the spectrum, exactly the kind of thing a notch filter is designed to surgically kill.

2. Concept

2.1 Periodic noise → spectrum dots

A sine pattern of frequency (u_0, v_0) in the image appears as two bright dots at (u_0, v_0) and $(-u_0, -v_0)$ (mirrored about DC) in the spectrum. Multiple periodic components → multiple dot pairs.

2.2 Notch reject filter

Zero out small circles around each noise dot, leave the rest of the spectrum alone:

```
def notch_reject(shape, centers, radius=10):
    h, w = shape
    cy, cx = h // 2, w // 2
    y, x = np.ogrid[:h, :w]
    mask = np.ones(shape, dtype=np.float32)
    for u, v in centers:
        # mask out a disk around (cy+v, cx+u) AND its mirror
        d1 = np.sqrt((y - (cy + v))**2 + (x - (cx + u))**2)
        d2 = np.sqrt((y - (cy - v))**2 + (x - (cx - u))**2)
        mask[d1 < radius] = 0
        mask[d2 < radius] = 0
    return mask
```

centers is a list of (u, v) offsets from DC where you spotted noise dots.

2.3 Smooth notches (Gaussian)

A hard disk causes mild ringing too. Smooth version:

```
mask *= 1 - np.exp(-(d1**2) / (2 * radius**2))
```

Replace the assignment with multiplication and a Gaussian — the notch becomes a smooth dip.

2.4 Finding the centres

- Display the log-magnitude spectrum.
- Visually locate bright dots that aren't on the centre axis or expected edges.
- Read their (u, v) offsets from the centre.

For programmatic detection: threshold the spectrum, find connected components, exclude the central blob.

2.5 Common periodic patterns

Source	Pattern
Scanned print	Diagonal pair of dots (the screening pattern)
Halftone newsprint	A rosette / multiple dot pairs
Old TV scan lines	Vertical strip of dots (one along the v-axis)
Power-line interference	Single horizontal dot
CCD readout patterns	Faint vertical band

2.6 When to use notch vs LP

- Periodic noise: notch filter only — LP would also kill the image's high-frequency detail.
- Random noise (grain): use spatial filters (M7) or non-local means (M15).
- Mixed: notch first, then a mild denoiser.

3. Code Examples

Example 1: Add synthetic periodic noise, then remove it

```
import cv2, numpy as np
import matplotlib.pyplot as plt
from skimage.data import camera

img = camera().astype(np.float32)
h, w = img.shape

# Add a sinusoidal pattern (diagonal stripes at frequency ~10)
yy, xx = np.mgrid[:h, :w]
noise = 30 * np.sin(2 * np.pi * (xx + yy) / 16)
noisy = np.clip(img + noise, 0, 255)

cv2.imwrite("striped.png", noisy.astype(np.uint8))

# Spectrum
F = np.fft.fft2(noisy)
Fs = np.fft.fftshift(F)
mag = np.log1p(np.abs(Fs))
mag_vis = cv2.normalize(mag, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
cv2.imwrite("striped_spectrum.png", mag_vis)
print("inspect striped_spectrum.png – note the symmetric bright dots off DC")
```


Expected Output (visual): striped.png is the cameraman with diagonal stripes overlaid.
striped_spectrum.png shows the usual centre cross PLUS a pair of bright dots symmetric about the centre — the noise frequencies.

Example 2: Apply a notch filter to remove the stripes

```
import cv2, numpy as np
from skimage.data import camera

img = camera().astype(np.float32)
h, w = img.shape
yy, xx = np.mgrid[:h, :w]
noisy = np.clip(img + 30 * np.sin(2 * np.pi * (xx + yy) / 16), 0, 255)

F = np.fft.fft2(noisy)
Fs = np.fft.fftshift(F)
cy, cx = h // 2, w // 2

# 16-pixel period diagonal => spectrum frequency at (h/16, w/16) and mirror
# (approximate; exact location depends on resolution)
y, x = np.ogrid[:h, :w]
def notch_disk(u, v, r=8):
    d1 = np.sqrt((y - (cy + v))**2 + (x - (cx + u))**2)
    d2 = np.sqrt((y - (cy - v))**2 + (x - (cx - u))**2)
    return (d1 < r) | (d2 < r)

mask = np.ones((h, w), dtype=np.float32)
mask[notch_disk(32, 32, r=8)] = 0 # tune offsets visually if needed

cleaned = np.real(np.fft.ifft2(np.fft.ifftshift(Fs * mask)))
cleaned = np.clip(cleaned, 0, 255).astype(np.uint8)
cv2.imwrite("destriped.png", cleaned)
print("if stripes remain, adjust the (u, v) offsets")
```

Expected Output (visual): destriped.png is the cameraman without (or with greatly reduced) diagonal stripes.

Example 3: Gaussian smoothing on notches — kills ringing

```
import cv2, numpy as np
from skimage.data import camera

img = camera().astype(np.float32)
h, w = img.shape
yy, xx = np.mgrid[:h, :w]
noisy = np.clip(img + 30 * np.sin(2 * np.pi * (xx + yy) / 16), 0, 255)

F = np.fft.fft2(noisy); Fs = np.fft.fftshift(F)
cy, cx = h // 2, w // 2
y, x = np.ogrid[:h, :w]

mask = np.ones((h, w), dtype=np.float32)
for u, v in [(32, 32), (-32, -32)]:
    D = np.sqrt((y - (cy + v))**2 + (x - (cx + u))**2)
```

```

mask *= 1 - np.exp(-D**2 / (2 * 6**2))

cleaned = np.real(np.fft.ifft2(np.fft.ifftshift(Fs * mask)))
cleaned = np.clip(cleaned, 0, 255).astype(np.uint8)
cv2.imwrite("destriped_gauss.png", cleaned)

```

Expected Output (visual): Same destriping as Example 2 but no ringing artefacts around edges — softer / cleaner.

Example 4: Interactive workflow

```

# 1. Display log-magnitude spectrum.
# 2. Identify dot pairs that don't belong (not on the cross / not DC).
# 3. Read their (u, v) coordinates (offsets from centre).
# 4. Build a notch mask with `notch_disk(u, v)` for each pair.
# 5. Apply and inspect.
# 6. Iterate.
print("(no code – this lesson's recipe in narrative form)")

```

4. Parameter Sensitivity

Param	Effect
Notch radius	small = surgical (precise); large = also kills nearby useful frequencies
Number of notches	more = handles complex patterns but risk killing real content
Soft (Gaussian) vs hard (disk)	soft = no ringing; hard = simpler to reason about

5. Common Mistakes

Mistake	What Happens	Fix
Forget the mirror pair	Noise only half-removed	Always notch (u, v) AND (-u, -v)
Notch too large	Lose real image content too	Shrink radius
Locate dot wrong	No improvement	Re-inspect spectrum; tune (u, v)
Hard notch on photo	Ringing along edges	Use Gaussian-shaped notches

6. Practice Tasks

- Add diagonal sinusoid noise to camera(); inspect spectrum; locate dot positions.
- Build a notch filter at those positions; remove the stripes.
- Switch to Gaussian notches with a wider σ ; compare to hard disks.

7. Key Takeaways

- Periodic noise $\rightarrow$ bright dots off-centre in the spectrum.
- Notch reject: zero (or soften) small regions around the dots.
- Always mirror about DC for real images.
- Soft (Gaussian) notches prevent ringing.

8. What's Next

The big idea: spatial filters and frequency filters are mathematically the same operation in different domains.

Lesson 5: Spatial ↔ Frequency Equivalence

Module: 14 — Frequency Domain (DFT)

Duration: 25 minutes

Difficulty: Advanced

Learning Objectives

- State the convolution theorem (spatial convolution = frequency multiplication).
- Verify it numerically with NumPy.
- Decide when to filter in spatial vs frequency domain.

Prerequisites

- Module 14 Lessons 1-4, Module 7 (convolution).

1. Why This Matters

Knowing that Gaussian blur in space is the same as multiplying by a Gaussian in frequency is the unifying insight of this module. It explains why spatial filters look the way they do and tells you exactly when to switch domains.

2. Concept

2.1 The convolution theorem

spatial:	$\text{out} = \text{kernel} \circledast \text{image}$	(convolution)
frequency:	$\text{OUT} = \text{KERNEL} \cdot \text{IMAGE}$	(element-wise product)

If you take a kernel's DFT and the image's DFT, multiply, then IFFT — you get the same result as convolving the image with the kernel in the spatial domain.

In Python:

```
F_img = np.fft.fft2(img)
F_kernel = np.fft.fft2(kernel, s=img.shape) # pad kernel to image size
out = np.real(np.fft.ifft2(F_img * F_kernel))
```

(Note: `s=img.shape` zero-pads the kernel so the DFTs have matching shapes.)

2.2 What this tells you intuitively

Spatial filter	Frequency interpretation
Gaussian blur	Low-pass: multiply by a centred Gaussian in spectrum
Box blur	Low-pass: multiply by a sinc (which causes ringing!)
Sharpen kernel	High-pass boost: amplify away from DC
Laplacian	High-pass: zero at DC, growing with frequency

This is why box blur looks slightly boxy — its frequency response is a sinc with side-lobes that let some high frequencies leak through asymmetrically.

2.3 When to pick which domain

Use frequency domain when	Use spatial domain when
Custom shape (notches, band-pass, rings)	Standard small kernels (3×3, 5×5)
Very large kernel ($\geq 30 \times 30$)	Small kernel — spatial faster
You want to design by drawing in spectrum	You want speed and simplicity
Educational / debugging	Production code (usually)

Rule of thumb: for kernels under $\sim 21 \times 21$, spatial is faster (less overhead). For huge kernels or non-standard shapes, frequency wins.

2.4 Periodic boundary

fft2 assumes the image is periodic — i.e. wraps around. If your image's edges don't match, you'll see slight ringing near the borders. Two fixes:

- Pad before FFT (zero or reflect), crop after.
- Use cv2.dft with cv2.DFT_REAL_OUTPUT and OpenCV's optimal size: cv2.getOptimalDFTSize(N).

2.5 Closing the loop

You can now read any spatial filter as a frequency operation, and vice versa. Practical takeaway: when a spatial filter behaves unexpectedly (ringing, aliasing, halo), look at its DFT — the answer is usually visible there.

3. Code Examples

Example 1: Numerically verify the convolution theorem

```
import cv2, numpy as np
from skimage.data import camera

img = camera().astype(np.float32)
kernel = np.array([[0, -1, 0],
                   [-1, 5, -1],
                   [0, -1, 0]], dtype=np.float32)

# Spatial
spatial = cv2.filter2D(img, -1, kernel, borderType=cv2.BORDER_CONSTANT)

# Frequency
F_img = np.fft.fft2(img)
F_k = np.fft.fft2(kernel, s=img.shape)
freq = np.real(np.fft.ifft2(F_img * F_k))

# Align: filter2D centres the kernel; the freq method places it at (0,0)
# We can compare after recentering or just check inner regions.
spatial = np.clip(spatial, 0, 255).astype(np.uint8)
freq = np.clip(freq, 0, 255).astype(np.uint8)
```

```
print("spatial range:", spatial.min(), spatial.max())
print("freq    range:", freq.min(), freq.max())
```

Expected Output: Both produce a sharpened image. Exact pixel-by-pixel match requires careful centring of the kernel; the qualitative result is identical.

Example 2: Gaussian blur — equivalent in both domains

```
import cv2, numpy as np
from skimage.data import camera

img = camera().astype(np.float32)
sigma = 3.0

# Spatial
spatial = cv2.GaussianBlur(img, (0, 0), sigma)

# Frequency: multiply by Gaussian in spectrum
F = np.fft.fft2(img); Fs = np.fft.fftshift(F)
h, w = img.shape; cy, cx = h // 2, w // 2
y, x = np.ogrid[:h, :w]
D = np.sqrt((y - cy)**2 + (x - cx)**2)
#  $\sigma$  in image space  $\rightarrow \sigma' \approx N/(2\pi \sigma)$  in frequency space (for a 1D case)
freq_sigma = max(1.0, min(h, w) / (2 * np.pi * sigma))
mask = np.exp(-D**2 / (2 * freq_sigma**2))
freq = np.real(np.fft.ifft2(np.fft.ifftshift(Fs * mask)))

print("max abs diff (loose):",
      np.abs(spatial.astype(np.float32) - freq.astype(np.float32)).max())
```

Expected Output: Diff is non-zero but small. Both are visibly the same Gaussian blur. (Exact agreement requires careful boundary handling; this demonstrates the principle.)

Example 3: Show that box blur has sinc response

```
import numpy as np
import matplotlib.pyplot as plt

k = np.ones(21) / 21 # 21-wide box kernel (1D)
K = np.abs(np.fft.fft(k, n=512))
K = np.fft.fftshift(K)

plt.plot(K)
plt.title("DFT of box kernel — sinc-like ripples")
plt.xlabel("frequency bin"); plt.ylabel("|K|"); plt.show()
print("note the oscillating side-lobes — they cause aliasing")
```

Expected Output: A sinc-shaped curve with several oscillating side-lobes. That's why box blur looks "boxy" — the side-lobes admit some high frequencies and reject others non-monotonically.

Example 4: Timing — when does frequency win?

```
import cv2, numpy as np, time
from skimage.data import astronaut
```

```

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2GRAY).astype(np.float32)
img = cv2.resize(img, (1024, 1024))

for ks in [5, 21, 101]:
    k = np.ones((ks, ks), np.float32) / (ks * ks)

    t = time.perf_counter()
    cv2.filter2D(img, -1, k)
    t_sp = (time.perf_counter() - t) * 1000

    t = time.perf_counter()
    F_img = np.fft.fft2(img)
    F_k = np.fft.fft2(k, s=img.shape)
    _ = np.real(np.fft.ifft2(F_img * F_k))
    t_fq = (time.perf_counter() - t) * 1000

    print(f"ksize={ks:3d}: spatial={t_sp:6.1f} ms frequency={t_fq:6.1f} ms")

```

Expected Output (timings vary):

```

ksize=  5: spatial=  2.0 ms frequency=  90.0 ms
ksize= 21: spatial= 25.0 ms frequency=  88.0 ms
ksize=101: spatial= 400.0 ms frequency=  90.0 ms

```

Spatial wins for small kernels; frequency wins as kernel grows.

4. Parameter Sensitivity

Param	Effect
Kernel size	small → spatial faster; large → frequency faster
Image size	bigger images make the frequency overhead more amortised

5. Common Mistakes

Mistake	What Happens	Fix
Forget to pad kernel to image size in fft2	Shape mismatch	<code>np.fft.fft2(k, s=img.shape)</code>
Compare DFT result without recentering	Pixel-wise mismatch but qualitative match	Centre the kernel or compare cropped interiors
Assume frequency is always faster	Small kernels are slower in frequency	Profile first

6. Practice Tasks

- Apply a 3×3 sharpening kernel via both `cv2.filter2D` and frequency multiplication; compare qualitatively.
- Plot the DFT magnitude of a 21-wide box kernel; note the side-lobes.
- Time spatial vs frequency for kernel sizes 5/21/101 on a 1024² image.

7. Key Takeaways

- Convolution theorem: spatial convolution = frequency multiplication.
- Pad kernel to image size for the FFT product.
- Frequency wins on large or custom-shape kernels; spatial wins on small standard kernels.
- The "boxy" look of box blur is its sinc-shaped frequency response.

8. What's Next

End of Module 14. Next: Module 15 — Image Restoration & Noise.

Module 14 — Exercises

15 exercises.

14.1 1D Fourier

- (E) Build a 256-sample signal with sines at frequencies 8 and 20; FFT and identify peaks.
- (M) Add Gaussian noise to a 1D sine; recover it by zeroing high-frequency bins.
- (H) Inspect the DFT of one row of `camera()`; print the DC magnitude.

14.2 2D DFT

- (E) Compute and log-display the spectrum of `camera()`.
- (M) Compare spectra of original vs Gaussian-blurred ($\sigma=4$) vs noisy versions.
- (H) Use `cv2.dft` instead of `np.fft.fft2`; verify magnitudes match.

14.3 LP / HP filters

- (E) Apply a Gaussian LP with $D_0=40$ to `camera()`; compare to `cv2.GaussianBlur`.
- (M) Apply a Gaussian HP with $D_0=30$; compare visually to a Laplacian edge map.
- (H) Try Butterworth orders $n \in \{1, 2, 4, 8\}$; describe ringing onset.

14.4 Notch

- (E) Add a diagonal sinusoidal noise pattern to `camera()`; inspect spectrum.
- (M) Locate the noise dots; build a hard-disk notch filter; remove the stripes.
- (H) Switch to Gaussian-shaped notches; compare ringing.

14.5 Equivalence

- (E) Apply a 3×3 sharpen kernel via both `cv2.filter2D` and FFT-multiply.
- (M) Plot the DFT magnitude of a 21-wide 1D box kernel; identify side-lobes.
- (H) Time spatial vs frequency for kernel sizes 5/21/101 on a 1024^2 image.

Module 14 — Quiz

10 MCQs.

Q1

The DC (zero frequency) component of a DFT represents: A. the brightest pixel B. the image's average value $\times N$ C. random noise D. the edges

Answer: B (L1, L2)

Q2

`np.fft.fftshift` is used to: A. compress B. centre the DC in the spectrum for visualisation C. invert D. blur

Answer: B (L1, L2)

Q3

For real-valued images, the spectrum is: A. random B. Hermitian-symmetric about the centre C. always real D. always positive

Answer: B (L2)

Q4

The visual cross pattern in a spectrum corresponds to: A. periodic noise B. strong horizontal and vertical edges in the image C. DC term D. blur

Answer: B (L2)

Q5

An ideal LP filter causes: A. blur with no artefacts B. ringing (sinc artefacts) C. sharpening D. noise

Answer: B (L3)

Q6

For a smooth, ringing-free LP, the natural mask shape is: A. ideal disk B. Gaussian C. random D. uniform

Answer: B (L3)

Q7

Periodic noise (e.g. screen pattern) appears in the spectrum as: A. uniform haze B. bright dots off-centre, symmetric pairs C. central spike D. radial lines

Answer: B (L4)

Q8

A notch filter: A. zeros (or attenuates) small regions around noise peaks B. blurs everywhere C. only acts on DC D. is the inverse of LP

Answer: A (L4)

Q9

The convolution theorem says: A. spatial convolution = frequency multiplication B. they're unrelated C. only Gaussians qualify D. only with DCT

Answer: A (L5)

Q10

For a 5×5 kernel on a 512^2 image, the faster domain is usually: A. spatial B. frequency C. either — same D. parallel only

Answer: A (L5)

Module 14 — Assignment: Frequency-Domain Denoiser

Estimated time: 2 hours

Combines: Module 14 + Module 7

Brief

Build a CLI that denoises an image using frequency-domain filtering and compares the result against an equivalent spatial-domain filter.

Requirements

- `freq_denoise.py INPUT --mode lp|hp|notch [--D0 40] [--notch-uv u,v[;u,v...]] [--output OUT]`
- lp mode: Gaussian low-pass with cutoff D0. Output = LP result.
- hp mode: Gaussian high-pass with cutoff D0. Output = HP-only image (normalised for display).
- notch mode: notch filter at each (u, v) plus its mirror (radius 8, Gaussian shape).
- Always also produce a spatial baseline:
- For lp → `cv2.GaussianBlur` with σ matched to D0.
- For hp → spatial high-pass = original – `GaussianBlur`.
- For notch → spatial baseline = original (no easy equivalent — note in output).
- Save a 3-up side-by-side: [original | frequency-result | spatial-baseline].
- Print PSNR (frequency vs spatial) and any other stats you choose.

Constraints

- Modules 1-14 only.
- Use `np.fft.fft2` / `ifft2` and `np.fft.fftshift` / `ifftshift`.
- Use `argparse`.
- For `--notch-uv`, parse "32,32;40,10" into a list of (u, v) pairs.

Expected Output

```
$ python freq_denoise.py noisy.png --mode lp --D0 30
mode: lp  D0: 30
PSNR(freq vs spatial): 42.1 dB
saved -> noisy_freq.png
```

Grading Rubric

Criterion	Weight
FFT + shift pipeline	25%
Three modes correct	30%
Spatial baseline	15%
3-up output	15%
Argparse + style	15%

Submission

Save as assignment_module14.py.

Module 14 — Mini Project: Interactive Frequency Filter Tuner

Overview

An interactive cv2-window tool: load an image, see its log-magnitude spectrum side-by-side with the filtered result, and tune cutoff D_0 , filter type (LP / HP / Band-pass), and notch positions live.

Concepts Used

- 2D DFT (M14 L2)
- LP / HP / Butterworth masks (M14 L3)
- Notch filters (M14 L4)
- Trackbars (M2 L3)

Features

- Trackbars: D_0 , mode (LP/HP/BP/notch), Butterworth order n , notch radius.
- 3-panel display: original | filtered | spectrum (log).
- `s` saves the filtered image; `q` quits.

Starter Code

freq_tuner.py:

```
import sys, cv2, numpy as np

MODES = ["LP", "HP", "BP", "notch"]

def filter_freq(img, mode,  $D_0$ , n, notch_radius):
    f = img.astype(np.float32)
    F = np.fft.fft2(f)
    Fs = np.fft.fftshift(F)
    h, w = f.shape
    cy, cx = h // 2, w // 2
    y, x = np.ogrid[:h, :w]
    D = np.sqrt((y - cy)**2 + (x - cx)**2)

    if mode == "LP":
        mask = 1 / (1 + (D / max(1,  $D_0$ ))**(2 * max(1, n)))
    elif mode == "HP":
        mask = 1 - 1 / (1 + (D / max(1,  $D_0$ ))**(2 * max(1, n)))
    elif mode == "BP":
        lp1 = 1 / (1 + (D / max(1,  $D_0$ ))**(2 * max(1, n)))
        lp2 = 1 / (1 + (D / max(1, int( $D_0$  * 1.6)))**(2 * max(1, n)))
        mask = lp2 - lp1 # band-pass
    else: # notch - fixed diagonal pair as a demo
        u, v = int( $D_0$ ), int( $D_0$ )
        mask = np.ones((h, w), dtype=np.float32)
        for du, dv in [(u, v), (-u, -v)]:
            d = np.sqrt((y - (cy + dv))**2 + (x - (cx + du))**2)
            mask *= 1 - np.exp(-d**2 / (2 * max(1, notch_radius)**2))
```

```

out = np.real(np.fft.ifft2(np.fft.ifftshift(Fs * mask)))
out = cv2.normalize(out, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
spec = cv2.normalize(np.log1p(np.abs(Fs * mask)), None, 0, 255,
                    cv2.NORM_MINMAX).astype(np.uint8)

return out, spec

def run(path):
    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if img is None: raise FileNotFoundError(path)
    if max(img.shape) > 600: img = cv2.resize(img, (512, 512))

    cv2.namedWindow("tuner")
    cv2.createTrackbar("D0", "tuner", 40, 250, lambda v: None)
    cv2.createTrackbar("mode 0LP 1HP 2BP 3notch", "tuner", 0, 3, lambda v: None)
    cv2.createTrackbar("order n", "tuner", 2, 8, lambda v: None)
    cv2.createTrackbar("notch radius", "tuner", 8, 30, lambda v: None)

    while True:
        D0 = cv2.getTrackbarPos("D0", "tuner")
        m = MODES[cv2.getTrackbarPos("mode 0LP 1HP 2BP 3notch", "tuner")]
        n = cv2.getTrackbarPos("order n", "tuner")
        nr = cv2.getTrackbarPos("notch radius", "tuner")

        out, spec = filter_freq(img, m, D0, n, nr)
        triple = np.hstack([img, out, spec])
        cv2.putText(triple, f"{m} D0={D0} n={n} notch_r={nr}",
                    (10, 25), cv2.FONT_HERSHEY_SIMPLEX, 0.6, 255, 2)
        cv2.imshow("tuner", triple)
        k = cv2.waitKey(30) & 0xFF
        if k == ord('q'): break
        if k == ord('s'): cv2.imwrite("freq_out.png", out); print("saved
freq_out.png")
        cv2.destroyAllWindows()

if __name__ == "__main__":
    run(sys.argv[1] if len(sys.argv) > 1 else "datasets/starter/camera.png")

```

Expected Interaction

Move D0 to change cutoff; switch mode to see LP/HP/BP/notch effects; spectrum panel updates with the masked spectrum.

Extension Challenges

- Add a mouse-click handler to place notch centres interactively.
- Add a "spatial-equivalent" baseline panel for comparison.
- Save the mask alongside the result.

Grading Rubric

Criterion	Weight
Four modes work	35%
Spectrum panel updates	20%

Trackbars responsive	15%
Save / quit	10%
Style	20%

Python E-Course

Module 15 — Image Restoration & Noise

Detailed Notes

Module Overview

Level: Advanced

Duration: ~1 week

Prerequisites: Modules 1-14

What You'll Learn

- Identify and synthesise common noise types.
- Pick the right denoiser per noise type.
- Apply non-local means and bilateral for high-quality denoising.
- Use Wiener filtering for deconvolution / restoration.
- Measure quality objectively with PSNR and SSIM.

Lessons

#	Lesson	Duration
1	Noise Types & Synthesis	30 min
2	Median Revisit + Non-Local Means	30 min
3	Bilateral & Edge-Preserving Denoising	25 min
4	Wiener Filter (Deconvolution Intro)	30 min
5	Quality Metrics — PSNR & SSIM	25 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 16: Capstone Projects

Lesson 1: Noise Types & Synthesis

Module: 15 — Image Restoration & Noise

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Distinguish Gaussian, salt-and-pepper, speckle, and periodic noise visually and statistically.
- Synthesise each noise type to test denoisers.
- Read a noise histogram.

Prerequisites

- Modules 1-14

1. Why This Matters

You can't pick a denoiser without naming the enemy. Each noise type has a characteristic signature, a typical cause, and a "best" denoiser. Five minutes here saves trial-and-error.

2. Concept

2.1 The four noise types you'll meet

Type	Looks like	Caused by	Best denoiser
Gaussian	grain everywhere, bell-shaped distribution	sensor read noise, low light	NLM, bilateral, Gaussian
Salt-and-pepper	random pure-white & pure-black specks	bit errors, dropped pixels	median
Speckle	multiplicative grain (more in bright areas)	radar, ultrasound, laser	NLM, Lee filter
Periodic	repeating patterns (stripes, screens)	electronic interference, scans	notch filter (M14)

2.2 Synthesizing each (for testing your denoiser)

Gaussian:

```
noisy = np.clip(img.astype(np.int16) +
                np.random.normal(0, sigma, img.shape).astype(np.int16),
                0, 255).astype(np.uint8)
```

Salt-and-pepper:

```
out = img.copy()
m = np.random.rand(*img.shape[:2])
out[m < p] = 0
out[m > 1 - p] = 255
```

Speckle (multiplicative):

```
n = np.random.normal(1.0, 0.1, img.shape).astype(np.float32)
noisy = np.clip(img.astype(np.float32) * n, 0, 255).astype(np.uint8)
```

Periodic (sinusoidal stripe):

```
yy, xx = np.mgrid[:img.shape[0], :img.shape[1]]
stripe = 30 * np.sin(2 * np.pi * (xx + yy) / 16)
noisy = np.clip(img.astype(np.float32) + stripe, 0, 255).astype(np.uint8)
```

2.3 Diagnosing real noise

Steps:

- Look at a flat area (sky, paper, wall) — what do you see?
- Plot histogram of that flat area:
- Bell curve → Gaussian.
- Two spikes near 0 and 255 → salt-and-pepper.
- Asymmetric, brightness-dependent spread → speckle.
- Check the DFT spectrum — bright dots off-centre → periodic.

2.4 Noise level estimation

For Gaussian, a quick estimate from a flat region:

```
flat = img[10:60, 10:60] # known-flat patch
sigma_est = flat.std()
```

Or scikit-image's wavelet-based estimator:

```
from skimage.restoration import estimate_sigma
print(estimate_sigma(img, average_sigmas=True))
```

2.5 The denoising trade-off

Every denoiser averages neighbouring pixels in some way. Result: noise drops AND fine detail blurs. Quality denoisers maximise noise removal per unit detail loss. Quantify with PSNR/SSIM (Lesson 5).

3. Code Examples

Example 1: Synthesise all four noise types

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
np.random.seed(0)

# Gaussian
sigma = 25
g = np.clip(img.astype(np.int16) +
```

```

        np.random.normal(0, sigma, img.shape).astype(np.int16), 0,
255).astype(np.uint8)

# Salt-and-pepper
sp = img.copy()
m = np.random.rand(*img.shape)
sp[m < 0.03] = 0; sp[m > 0.97] = 255

# Speckle
n = np.random.normal(1.0, 0.1, img.shape).astype(np.float32)
spk = np.clip(img.astype(np.float32) * n, 0, 255).astype(np.uint8)

# Periodic
yy, xx = np.mgrid[:img.shape[0], :img.shape[1]]
per = np.clip(img.astype(np.float32) + 30 * np.sin(2 * np.pi * (xx + yy) / 16),
              0, 255).astype(np.uint8)

for name, x in [("gauss", g), ("sp", sp), ("speckle", spk), ("periodic", per)]:
    cv2.imwrite(f"noise_{name}.png", x)
    print(f"{name:8s} std={x.std():.1f} range={x.min()}-{x.max()}")

```

Expected Output: Four PNG files demonstrating each noise type, with statistics that match the type (e.g. salt-and-pepper has wide range 0-255 but median near the original).

Example 2: Histogram diagnosis

```

import cv2, numpy as np
import matplotlib.pyplot as plt
from skimage.data import camera

img = camera()
np.random.seed(0)
gauss = np.clip(img + np.random.normal(0, 25, img.shape), 0, 255).astype(np.uint8)
sp = img.copy()
m = np.random.rand(*img.shape); sp[m < 0.03] = 0; sp[m > 0.97] = 255

fig, ax = plt.subplots(1, 2, figsize=(10, 4))
ax[0].hist(gauss.ravel(), 256, [0, 256]); ax[0].set_title("Gaussian-noisy histogram")
ax[1].hist(sp.ravel(), 256, [0, 256]); ax[1].set_title("Salt-and-pepper histogram")
plt.tight_layout(); plt.show()

```

Expected Output: Gaussian histogram is a smoothed bell-shaped curve. Salt-and-pepper has two prominent spikes at 0 and 255 (the salt and pepper) plus the original distribution.

Example 3: Estimate Gaussian σ from a flat patch

```

import numpy as np
import cv2
from skimage.data import camera

img = camera()
np.random.seed(0)

```

```
noisy = np.clip(img + np.random.normal(0, 20, img.shape), 0, 255).astype(np.uint8)

flat = noisy[20:80, 20:80]
print("estimated sigma:", flat.std())
```

Expected Output: estimated sigma: ~20 (close to the true value we used).

Example 4: scikit-image estimator

```
import numpy as np
from skimage.restoration import estimate_sigma
from skimage.data import camera

img = camera()
np.random.seed(0)
noisy = np.clip(img + np.random.normal(0, 20, img.shape), 0, 255).astype(np.uint8)
print("skimage sigma estimate:", estimate_sigma(noisy, average_sigmas=True))
```

Expected Output: value near 20 — sometimes a slight under/overshoot depending on image content.

4. Parameter Sensitivity

Noise param	Effect
Gaussian σ	larger = more grain
S&P prob p	larger = more dropouts
Speckle std	larger = more brightness-modulated grain
Periodic amplitude	larger = stronger stripes

5. Common Mistakes

Mistake	What Happens	Fix
Add Gaussian noise on uint8 directly	overflow at clipping → distribution skewed	Cast to int16 first
Sample uniform speckle	wrong distribution shape	Use multiplicative Gaussian (mean=1)
Treat salt-and-pepper as Gaussian	filter fails	Diagnose first by checking for 0/255 spikes

6. Practice Tasks

- Synthesise Gaussian noise ($\sigma=25$) on camera(); save.
- Plot histograms of a Gaussian-noisy vs salt-and-pepper-noisy image; identify each from its histogram alone.
- Estimate σ from a flat patch; compare to ground truth.

7. Key Takeaways

- Four noise types: Gaussian, salt-and-pepper, speckle, periodic.
- Each has a characteristic signature in image and histogram.
- Synthesise to test denoisers reproducibly.
- Estimate σ from flat regions or estimate_sigma.

8. What's Next

Median (M7 revisited) and the king of Gaussian-noise denoisers: non-local means.

Lesson 2: Median Revisit + Non-Local Means

Module: 15 — Image Restoration & Noise

Duration: 30 minutes

Difficulty: Intermediate-Advanced

Learning Objectives

- Apply `cv2.medianBlur` for impulse noise (recap of M7).
- Apply `cv2.fastNlMeansDenoising` / `Colored` for Gaussian noise.
- Tune NLM parameters: `h`, `templateWindowSize`, `searchWindowSize`.

Prerequisites

- Module 7 Lesson 4, Module 15 Lesson 1

1. Why This Matters

Non-Local Means (NLM) is the best general-purpose denoiser available without deep learning. It produces results that look more natural than Gaussian / bilateral on real photos. Median is still the right answer for impulse noise.

2. Concept

2.1 Median for impulse noise (recap)

Salt-and-pepper noise → `cv2.medianBlur(img, 3)` is unbeatable. Median ignores outliers because they sit at the extremes of the sorted neighbourhood.

For higher densities (15-30% noise) try `ksize=5` or `7`. Above 30%, you'll lose real detail; combine with morphology or use a deep model.

2.2 The NLM idea (intuition)

A traditional Gaussian filter averages spatial neighbours. NLM averages similar pixels — even if they're far away. For each pixel `p`, search a window for patches similar to `p`'s patch; weight each by similarity; average.

Result: noise (which is random) cancels out, while genuine detail (which repeats across the image in similar patches) is preserved.

2.3 OpenCV API

```
cv2.fastNlMeansDenoising(src, h=10, templateWindowSize=7, searchWindowSize=21)
cv2.fastNlMeansDenoisingColored(src, h=10, hColor=10, ...)
```

Param	Meaning
<code>h</code>	Filter strength. Larger = more denoising AND more blurring. Typical 5-15.
<code>templateWindowSize</code>	Patch size for similarity (default 7). Should be

	odd.
searchWindowSize	Neighbourhood to search for similar patches (default 21). Should be odd.
hColor (Colored version)	Same as h but for color channels

2.4 Picking h

A common heuristic: $h \approx \text{estimated_sigma}$. If you don't know σ , start with 10 and tune visually.

Too small → noise remains. Too large → detail wiped out.

2.5 Performance

NLM is slow — typically 1-10 seconds for an HD image. Downsample first if you can:

```
small = cv2.resize(img, None, fx=0.5, fy=0.5)
out_small = cv2.fastNlMeansDenoisingColored(small, None, 10, 10, 7, 21)
out = cv2.resize(out_small, img.shape[1::-1]) # may lose some quality
```

For video, OpenCV provides `cv2.fastNlMeansDenoisingMulti` which uses several frames.

2.6 NLM vs Bilateral

NLM:

- Slower but higher quality on photographic noise.
- Better at preserving textures (uses patch similarity).

Bilateral:

- Faster.
- Better when you want strong edge preservation (e.g. skin smoothing).

Use NLM as the quality benchmark; bilateral when speed matters.

3. Code Examples

Example 1: NLM denoise grayscale

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
np.random.seed(0)
noisy = np.clip(img + np.random.normal(0, 20, img.shape), 0, 255).astype(np.uint8)
clean = cv2.fastNlMeansDenoising(noisy, None, h=15, templateWindowSize=7,
searchWindowSize=21)

cv2.imwrite("noisy.png", noisy)
cv2.imwrite("clean_nlm.png", clean)
print("noisy std on flat patch:", noisy[10:60, 10:60].std())
print("clean std on flat patch:", clean[10:60, 10:60].std())
```

Expected Output: Noisy has flat-patch std ~20; clean ~3-5. Visually, the cameraman is recognisable and most detail is preserved.

Example 2: NLM color (faster than per-channel)

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
np.random.seed(0)
noisy = np.clip(img + np.random.normal(0, 20, img.shape), 0, 255).astype(np.uint8)
clean = cv2.fastNlMeansDenoisingColored(noisy, None, h=10, hColor=10,
                                       templateWindowSize=7, searchWindowSize=21)

cv2.imwrite("astro_noisy.png", noisy)
cv2.imwrite("astro_nlm.png", clean)
```

Expected Output: astro_nlm.png shows the astronaut with grain dramatically reduced; fine helmet text and hair detail preserved.

Example 3: Tune h

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
np.random.seed(0)
noisy = np.clip(img + np.random.normal(0, 20, img.shape), 0, 255).astype(np.uint8)

for h in [5, 10, 20, 40]:
    out = cv2.fastNlMeansDenoising(noisy, None, h=h, templateWindowSize=7,
                                  searchWindowSize=21)
    cv2.imwrite(f"nlm_h{h}.png", out)
print("compare files: h=5 underdenoised, h=40 oversmoothed")
```

Expected Output: h=5 still has visible grain. h=40 looks plasticky / loses detail. h=10–15 is the sweet spot for $\sigma=20$ noise.

Example 4: Median for salt-and-pepper (recap)

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
np.random.seed(0)
m = np.random.rand(*img.shape)
sp = img.copy(); sp[m < 0.05] = 0; sp[m > 0.95] = 255

med = cv2.medianBlur(sp, 3)
nlm = cv2.fastNlMeansDenoising(sp, None, h=15)

cv2.imwrite("sp_median.png", med)
cv2.imwrite("sp_nlm.png", nlm)
```

Expected Output: Median: clean image, sharp. NLM on salt-and-pepper: visible grey smudges where the impulse noise was — NLM doesn't handle outliers well. Confirms: pick the filter for the noise type.

4. Parameter Sensitivity

Param	Lower	Higher
h (NLM)	residual noise	smoother but more detail loss
templateWindowSize	local similarity	broader patch similarity
searchWindowSize	small region	longer runtime, more patches considered

5. Common Mistakes

Mistake	What Happens	Fix
Use NLM on salt-and-pepper	grey smudges	Use median
Run NLM at full HD without thinking	slow	Downsample, denoise, upsample
Per-channel NLM on color	color shifts possible	Use fastNlMeansDenoisingColored
h too large	plasticky output	Match h to estimated σ

6. Practice Tasks

- Add Gaussian noise ($\sigma=20$) to camera(); denoise with NLM, $h=15$.
- Add salt-and-pepper noise; compare median vs NLM.
- Tune NLM h in {5, 10, 20, 40}; pick a visual sweet spot.

7. Key Takeaways

- Median for impulse / salt-and-pepper.
- NLM for Gaussian / photographic grain.
- $h \approx \sigma$ is a good starting point for NLM.
- NLM is slow — downsample if needed.

8. What's Next

Bilateral filter in this restoration context — when and how to combine with NLM.

Lesson 3: Bilateral & Edge-Preserving Denoising

Module: 15 — Image Restoration & Noise

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Apply `cv2.bilateralFilter` for edge-preserving denoising.
- Compare bilateral with NLM, Gaussian, and median.
- Combine filters in a denoising pipeline.

Prerequisites

- Module 7 Lesson 5, Module 15 Lesson 2

1. Why This Matters

Bilateral was introduced in M7 for general smoothing. Here we focus on its restoration role: when edges matter more than texture (e.g. portraits, scanned documents), bilateral wins over NLM in speed and over Gaussian in edge preservation.

2. Concept

2.1 Bilateral in one line

`cv2.bilateralFilter` smooths spatially close pixels with similar intensity. Edges (large intensity difference) survive because dissimilar pixels are excluded from the average.

```
out = cv2.bilateralFilter(img, d=9, sigmaColor=75, sigmaSpace=75)
```

Param	Meaning
d	spatial neighbourhood diameter (9 typical)
sigmaColor	intensity-difference tolerance (50-100 typical)
sigmaSpace	spatial extent (50-100 typical)

2.2 Restoration recipes

Mild Gaussian noise + sharp edges (portraits):

```
out = cv2.bilateralFilter(noisy, 9, 50, 50)
```

Heavy Gaussian noise:

```
# Stack bilateral several times
out = noisy
for _ in range(3):
    out = cv2.bilateralFilter(out, 9, 75, 75)
```

Stacking trades small detail for stronger smoothing — much cheaper than NLM at heavy h.

Mixed noise (impulse + Gaussian):

```
out = cv2.medianBlur(noisy, 3)          # remove salt-and-pepper
out = cv2.bilateralFilter(out, 9, 50, 50) # smooth the rest
```

2.3 Edge-preserving variants in OpenCV

- `cv2.edgePreservingFilter(img, flags=cv2.RECURS_FILTER, sigma_s=60, sigma_r=0.4)` — fast edge-preserving smoothing.
- `cv2.detailEnhance` — bilateral + sharpening combo.
- `cv2.stylization` — bilateral + edge map (cartoon effect).

These wrap variants of bilateral for one-call effects.

2.4 Bilateral vs NLM — when to pick which

You want	Use
Smooth skin / strong edge preservation	bilateral
Best photographic-noise removal	NLM
Fast (real-time / video)	bilateral
Quality first, time available	NLM
Mix of texture + noise	NLM, then mild bilateral

2.5 Anisotropic diffusion (advanced sibling)

Perona-Malik anisotropic diffusion is the academic ancestor of bilateral. scikit-image has `skimage.restoration.denoise_tv_chambolle` (total variation) — preserves piecewise-flat regions even better. Useful for cartoon-style images.

3. Code Examples

Example 1: Bilateral on Gaussian noise

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
np.random.seed(0)
noisy = np.clip(img + np.random.normal(0, 15, img.shape), 0, 255).astype(np.uint8)
out = cv2.bilateralFilter(noisy, 9, 75, 75)
cv2.imwrite("astro_bilat.png", out)
```

Expected Output: Astronaut with much less grain but edges (helmet rim, mission patch text) preserved. Compare to NLM — bilateral is a touch less smooth in textured regions but ~10× faster.

Example 2: Stacked bilateral for heavy noise

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
np.random.seed(0)
```

```

noisy = np.clip(img + np.random.normal(0, 30, img.shape), 0, 255).astype(np.uint8)

out = noisy
for _ in range(3):
    out = cv2.bilateralFilter(out, 9, 75, 75)
cv2.imwrite("cam_bilat_x3.png", out)

```

Expected Output: Heavy noise is dramatically reduced; fine grain replaced by smooth shading; edges of buildings still crisp.

Example 3: Pipeline — median + bilateral for mixed noise

```

import cv2, numpy as np
from skimage.data import camera

img = camera()
np.random.seed(0)
mixed = np.clip(img + np.random.normal(0, 15, img.shape), 0, 255).astype(np.uint8)
m = np.random.rand(*img.shape)
mixed[m < 0.02] = 0; mixed[m > 0.98] = 255

step1 = cv2.medianBlur(mixed, 3)
step2 = cv2.bilateralFilter(step1, 9, 50, 50)
cv2.imwrite("cam_pipeline.png", step2)

import numpy as np
print("flat patch std before:", mixed[10:60, 10:60].std())
print("flat patch std after :", step2[10:60, 10:60].std())

```

Expected Output: Std drops from ~30 to ~5. Visually: speckles removed by median, residual grain smoothed by bilateral.

Example 4: edgePreservingFilter — one-call recipe

```

import cv2, numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
np.random.seed(0)
noisy = np.clip(img + np.random.normal(0, 20, img.shape), 0, 255).astype(np.uint8)

out = cv2.edgePreservingFilter(noisy, flags=cv2.RECURS_FILTER, sigma_s=60,
sigma_r=0.4)
cv2.imwrite("coffee_eps.png", out)

```

Expected Output: Slightly painterly-looking coffee image — noise gone, edges crisp, slight cartoonish flat-fill effect.

4. Parameter Sensitivity

Param	Lower	Higher
d	faster, less smoothing	slower, more smoothing
sigmaColor	keep more edges	smooth across stronger edges

Stacks (re-applications)	mild	strong / loses detail
--------------------------	------	-----------------------

5. Common Mistakes

Mistake	What Happens	Fix
Use bilateral on salt-and-pepper	Specks survive, look smeared	Median first
sigmaColor too large	Acts like Gaussian (no edge preservation)	Try 25-75
Stack too many times	Plasticky	2-3 stacks max

6. Practice Tasks

- Add Gaussian noise $\sigma=15$ to astronaut(); denoise with bilateral 9, 75, 75.
- Add heavy noise $\sigma=30$; stack bilateral 3 times.
- Add mixed noise (Gaussian + S&P); apply median $\rightarrow$ bilateral pipeline.

7. Key Takeaways

- Bilateral = fast, edge-preserving denoising.
- Stack 2-3 times for heavier noise.
- Combine with median for mixed noise.
- edgePreservingFilter wraps bilateral-like recipes for one-call use.

8. What's Next

Wiener filtering — when the noise is mixed with blur (deconvolution).

Lesson 4: Wiener Filter (Deconvolution Intro)

Module: 15 — Image Restoration & Noise

Duration: 30 minutes

Difficulty: Advanced

Learning Objectives

- Distinguish denoising from deblurring (deconvolution).
- Apply `scipy.signal.wiener` for adaptive denoising.
- Implement a frequency-domain Wiener deconvolution for motion blur.

Prerequisites

- Module 14 (frequency domain), Module 15 Lesson 3

1. Why This Matters

When you know HOW the image got corrupted (blur kernel + noise), you can mathematically reverse it. Wiener filtering is the classic optimal solution in the least-squares sense. Used in microscopy, radar, and any imaging where the point-spread function is known.

2. Concept

2.1 Image formation model

$$g = f \otimes h + n$$

- g — what you observed (blurry + noisy).
- f — the true image you want.
- h — the blur kernel (point-spread function / PSF).
- n — additive noise.
- $\otimes$ — convolution.

Goal: recover f given g and (ideally) h .

2.2 Naive inverse — and why it fails

In frequency space:

$$G = F \cdot H + N \quad \rightarrow \quad F_{\text{est}} = G / H - N/H$$

If H has small values (and it usually does at high frequencies), dividing by H blows up noise. So naive inversion is a disaster.

2.3 Wiener — regularised inverse

$$F_{\text{est}} = (\bar{H} / (|H|^2 + K)) \cdot G$$

Where K is the noise-to-signal power ratio. Small $K \rightarrow$ close to inverse (sharp but noisy); large $K \rightarrow$ safe (over-smooth).

In practice, treat K as a hyperparameter and tune.

2.4 `scipy.signal.wiener` — adaptive variance method

A simpler form: estimates local variance and shrinks pixels toward the local mean based on noise variance. Works well as a general denoiser when you don't know the PSF.

```
from scipy.signal import wiener
out = wiener(img.astype(np.float32), mysize=5, noise=None) # noise=None auto-estimates
```

2.5 Frequency-domain Wiener for motion blur

When you know the PSF (or can estimate it):

```
def wiener_deconvolve(g, h, K=0.01):
    g = g.astype(np.float32) / 255
    H = np.fft.fft2(h, s=g.shape)
    G = np.fft.fft2(g)
    H_conj = np.conj(H)
    W = H_conj / (np.abs(H)**2 + K)
    F_est = W * G
    out = np.real(np.fft.ifft2(F_est))
    return np.clip(out * 255, 0, 255).astype(np.uint8)
```

2.6 Limitations

- Need to know (or estimate) the PSF.
- Noise model assumed Gaussian.
- Ringing at image borders.
- For real photos, deep-learning deblurring (e.g. Restormer, DeepDeblur) is usually better.

2.7 When to use what

You have	Use
Noisy image, no blur, unknown statistics	NLM / bilateral
Noisy + known blur (PSF)	Wiener deconvolution
Heavy motion blur, no PSF	Blind deconvolution (advanced)
Modern photo, want highest quality	Deep model (Restormer, NAFNet, ...)

3. Code Examples

Example 1: `scipy.signal.wiener` as a denoiser

```
import numpy as np, cv2
from scipy.signal import wiener
from skimage.data import camera

img = camera()
np.random.seed(0)
noisy = np.clip(img + np.random.normal(0, 20, img.shape), 0, 255).astype(np.uint8)

out = wiener(noisy.astype(np.float32), mysize=5, noise=None)
out = np.clip(out, 0, 255).astype(np.uint8)
```

```

cv2.imwrite("cam_wiener.png", out)

print("noisy flat std:", noisy[10:60, 10:60].std())
print("wiener flat std:", out[10:60, 10:60].std())

```

Expected Output: Flat-patch std drops dramatically (e.g. 20 → 6). Image is visibly cleaner with slight loss of fine texture.

Example 2: Synthesize motion blur + recover via Wiener

```

import numpy as np, cv2
from skimage.data import camera

img = camera().astype(np.float32) / 255.0
h, w = img.shape

# Horizontal motion-blur PSF (15-pixel line)
psf = np.zeros((h, w), dtype=np.float32)
psf[h//2, w//2 - 7:w//2 + 8] = 1
psf /= psf.sum()

# Apply blur via FFT
H = np.fft.fft2(np.fft.ifftshift(psf))
blurred = np.real(np.fft.ifft2(np.fft.fft2(img) * H))
np.random.seed(0)
blurred_noisy = np.clip(blurred + np.random.normal(0, 0.01, img.shape), 0, 1)

# Wiener deconvolve
def wiener_deconv(g, H, K):
    G = np.fft.fft2(g)
    W = np.conj(H) / (np.abs(H)**2 + K)
    return np.real(np.fft.ifft2(W * G))

for K in [0.0001, 0.001, 0.01, 0.1]:
    restored = wiener_deconv(blurred_noisy, H, K)
    cv2.imwrite(f"deblur_K{K}.png", np.clip(restored * 255, 0,
    255).astype(np.uint8))

cv2.imwrite("blurred_input.png", np.clip(blurred_noisy * 255, 0,
255).astype(np.uint8))

```

Expected Output: blurred_input.png is the noisy horizontally-blurred cameraman. deblur_K0.0001.png is sharp but noisy; deblur_K0.1.png is over-smooth. deblur_K0.001 tends to be the sweet spot — visibly sharper than input with moderate noise.

Example 3: Wiener vs bilateral on Gaussian-noisy image

```

import numpy as np, cv2
from scipy.signal import wiener
from skimage.data import camera

img = camera()
np.random.seed(0)
noisy = np.clip(img + np.random.normal(0, 20, img.shape), 0, 255).astype(np.uint8)

```

```
w_out = np.clip(wiener(noisy.astype(np.float32), mysize=5), 0, 255).astype(np.uint8)
b_out = cv2.bilateralFilter(noisy, 9, 75, 75)

cv2.imwrite("cam_wiener.png", w_out)
cv2.imwrite("cam_bilat.png", b_out)
```

Expected Output: Both look denoised. Bilateral keeps edges sharper; Wiener is slightly more uniform-looking. For pure denoising without blur, bilateral / NLM are usually preferred.

4. Parameter Sensitivity

Param	Effect
K (Wiener regulariser)	small → sharper but noisy; large → safer but blurrier
mysize (scipy wiener)	local window; 3-7 typical
PSF accuracy	wrong PSF → ringing / smearing

5. Common Mistakes

Mistake	What Happens	Fix
Try Wiener on photo without PSF	Just acts like denoiser	Use scipy version or another method
K = 0	Naive inverse, noise explodes	K > 0 always
Wrong PSF shape	Artefacts	Estimate or measure PSF
Use uint8 in math	Overflow	Cast to float, work in [0, 1]

6. Practice Tasks

- Apply `scipy.signal.wiener(mysize=5)` to `Gaussian-noisy camera()`.
- Synthesize horizontal motion blur on `camera()`; recover with Wiener; tune K.
- Compare Wiener vs bilateral on the same Gaussian-noisy input.

7. Key Takeaways

- Wiener = noise-aware deconvolution.
- Need (or estimate) the PSF for deblurring.
- K trades sharpness for noise.
- For pure denoising, simpler filters (NLM/bilateral) often suffice.

8. What's Next

Measuring "how good is my denoiser" objectively — PSNR and SSIM.

Lesson 5: Quality Metrics — PSNR & SSIM

Module: 15 — Image Restoration & Noise

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Compute PSNR and SSIM with scikit-image.
- Interpret typical values.
- Build a small denoiser benchmark.

Prerequisites

- Module 15 Lessons 1-4

1. Why This Matters

"Looks better" isn't reproducible. PSNR and SSIM give you numbers — you can rank denoisers, set quality thresholds, and report results without arguing about subjective opinion.

2. Concept

2.1 MSE — the foundation

Mean Squared Error:

$$\text{MSE} = (1/N) \cdot \sum (\text{true} - \text{estimate})^2$$

Lower = better. But MSE on its own is unitless and doesn't relate to image properties.

2.2 PSNR

Peak Signal-to-Noise Ratio:

$$\text{PSNR} = 10 \cdot \log_{10}(\text{MAX}^2 / \text{MSE})$$

For 8-bit images, MAX = 255. Units: decibels (dB). Higher = better.

PSNR (dB)	Quality
> 40	Near-imperceptible difference
30-40	Good (typical for compressed photos)
20-30	Visible degradation
< 20	Significant degradation

2.3 SSIM — Structural Similarity

PSNR is per-pixel and ignores perceptual structure. SSIM considers luminance, contrast, and structure within local windows. Range -1 to 1; 1 = identical.

SSIM	Quality
------	---------

> 0.98	Visually identical
0.9-0.98	Very good
0.7-0.9	Recognisable but degraded
< 0.7	Significantly different

SSIM correlates better with human perception than PSNR for many distortions (blur, blocking, contrast shifts).

2.4 scikit-image API

```
from skimage.metrics import peak_signal_noise_ratio, structural_similarity
psnr = peak_signal_noise_ratio(reference, test, data_range=255)
ssim = structural_similarity(reference, test, data_range=255)
```

For color, pass `channel_axis=2`:

```
ssim = structural_similarity(ref_color, test_color, data_range=255, channel_axis=2)
```

2.5 The benchmark pattern

For comparing denoisers:

```
methods = {
    "gaussian": lambda x: cv2.GaussianBlur(x, (5, 5), 0),
    "median":   lambda x: cv2.medianBlur(x, 3),
    "bilateral": lambda x: cv2.bilateralFilter(x, 9, 75, 75),
    "nlm":      lambda x: cv2.fastNlMeansDenoising(x, None, h=15),
}

for name, f in methods.items():
    out = f(noisy)
    p = peak_signal_noise_ratio(clean, out, data_range=255)
    s = structural_similarity(clean, out, data_range=255)
    print(f"{name:10s} PSNR={p:5.2f} SSIM={s:.3f}")
```

This gives you a quantitative leaderboard.

2.6 Caveat — metrics ≠ perception

- PSNR rewards pixel-exact match. A barely-noticeable shift can tank PSNR.
- SSIM is better but not perfect.
- For final QA, use BOTH plus visual inspection.
- Modern: LPIPS / DISTs use deep features for perceptual quality.

3. Code Examples

Example 1: Compute PSNR / SSIM

```
import numpy as np, cv2
from skimage.data import camera
from skimage.metrics import peak_signal_noise_ratio, structural_similarity

clean = camera()
np.random.seed(0)
noisy = np.clip(clean + np.random.normal(0, 20, clean.shape), 0,
```

```

255).astype(np.uint8)

print("PSNR (noisy vs clean):", peak_signal_noise_ratio(clean, noisy,
data_range=255))
print("SSIM (noisy vs clean):", structural_similarity(clean, noisy, data_range=255))

```

Expected Output:

```

PSNR (noisy vs clean): ~22.0
SSIM (noisy vs clean): ~0.40

```

Example 2: Benchmark denoisers

```

import cv2, numpy as np
from skimage.data import camera
from skimage.metrics import peak_signal_noise_ratio as psnr, structural_similarity
as ssim

clean = camera()
np.random.seed(0)
noisy = np.clip(clean + np.random.normal(0, 20, clean.shape), 0,
255).astype(np.uint8)

methods = {
    "gaussian": lambda x: cv2.GaussianBlur(x, (5, 5), 1.0),
    "median": lambda x: cv2.medianBlur(x, 3),
    "bilateral": lambda x: cv2.bilateralFilter(x, 9, 75, 75),
    "nlm": lambda x: cv2.fastNlMeansDenoising(x, None, h=15),
}

print(f"{'method':10s}{ 'PSNR':>8s}{ 'SSIM':>8s}")
for name, f in methods.items():
    out = f(noisy)
    print(f"{'name':10s}{psnr(clean, out, data_range=255):8.2f}{ssim(clean, out,
data_range=255):8.3f}")

```

Expected Output:

method	PSNR	SSIM
gaussian	27.1	0.72
median	25.4	0.65
bilateral	28.3	0.78
nlm	30.1	0.84

NLM wins (expected); median is worst on Gaussian noise (also expected).

Example 3: SSIM map (per-pixel)

```

import cv2, numpy as np
from skimage.data import camera
from skimage.metrics import structural_similarity

clean = camera()
np.random.seed(0)
noisy = np.clip(clean + np.random.normal(0, 20, clean.shape), 0,

```

```

255).astype(np.uint8)

score, ssim_map = structural_similarity(clean, noisy, data_range=255, full=True)
print("global SSIM:", score)
ssim_vis = cv2.normalize(ssim_map, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
cv2.imwrite("ssim_map.png", ssim_vis)

```

Expected Output: Score ~0.40. ssim_map.png shows where the noisy and clean images differ most — uniform regions look identical (bright in the map), textured regions less so.

Example 4: Track quality during stacked filtering

```

import cv2, numpy as np
from skimage.data import camera
from skimage.metrics import peak_signal_noise_ratio as psnr

clean = camera()
np.random.seed(0)
out = np.clip(clean + np.random.normal(0, 25, clean.shape), 0, 255).astype(np.uint8)

print("step 0:", psnr(clean, out, data_range=255))
for i in range(1, 6):
    out = cv2.bilateralFilter(out, 9, 60, 60)
    print(f"step {i}:", psnr(clean, out, data_range=255))

```

Expected Output: PSNR rises in the first 2-3 passes, then plateaus or starts decreasing as detail is eroded. Tells you when to stop stacking.

4. Parameter Sensitivity

Metric	Best when...
PSNR	Identifying pixel-exact match
SSIM	Comparing perceived quality
LPIPS (deep)	Comparing different artefact types

5. Common Mistakes

Mistake	What Happens	Fix
data_range wrong	Inflated / deflated numbers	Pass data_range=255 for uint8
Compute on different shapes	Error	Resize to match
Compare uint8 vs float without normalising	Bad numbers	Use the same dtype/range
Use PSNR alone	Misleading for blurs	Pair with SSIM

6. Practice Tasks

- Compute PSNR and SSIM between camera() and a $\sigma=20$ noisy version.
- Benchmark Gaussian/median/bilateral/NLM on the same noisy image.
- Stack bilateral 5 times; plot PSNR per step; find the optimum.

7. Key Takeaways

- PSNR (dB) and SSIM (0-1) quantify image quality.
- skimage.metrics has both.
- Use BOTH plus visual inspection for QA.
- data_range=255 for uint8 — don't forget it.

8. What's Next

End of Module 15. Next: Module 16 — Capstone Projects and the 8 standalone projects in projects/.

Module 15 — Exercises

15 exercises.

15.1 Noise types

- (E) Synthesise $\sigma=25$ Gaussian noise on `camera()`; save.
- (M) Plot histograms of Gaussian-noisy vs S&P-noisy versions; identify each from histogram alone.
- (H) Estimate σ from a flat patch and via `skimage.restoration.estimate_sigma`; compare to true σ .

15.2 Median + NLM

- (E) NLM denoise $\sigma=20$ Gaussian noise on `camera()` with $h=15$.
- (M) Apply `median(3)` vs `NLM(h=15)` on salt-and-pepper noise; compare.
- (H) Tune NLM $h \in \{5, 10, 20, 40\}$; pick the sweet spot for $\sigma=20$ noise.

15.3 Bilateral

- (E) `bilateralFilter(9, 75, 75)` on $\sigma=15$ noise on `astronaut()`.
- (M) Stack bilateral 3 times on $\sigma=30$ noisy `camera()`.
- (H) Mixed noise pipeline: median $\rightarrow$ bilateral; report flat-patch std before/after.

15.4 Wiener

- (E) Apply `scipy.signal.wiener(mysize=5)` to Gaussian-noisy `camera()`.
- (M) Synthesize horizontal motion blur via FFT; recover with Wiener ($K=0.001$).
- (H) Sweep Wiener $K \in \{1e-4, 1e-3, 1e-2, 1e-1\}$; pick the best output.

15.5 Metrics

- (E) Compute PSNR/SSIM between `camera()` and $\sigma=20$ noisy version.
- (M) Benchmark Gaussian/median/bilateral/NLM on the same noisy input.
- (H) Stack bilateral 5 $\times$; plot PSNR per step; find the optimum.

Module 15 — Quiz

10 MCQs.

Q1

Salt-and-pepper noise is best removed by: A. Gaussian blur B. median C. bilateral D. NLM

Answer: B (L1, L2)

Q2

For photographic Gaussian noise, the best classical denoiser is: A. median B. Gaussian C. NLM D. erosion

Answer: C (L2)

Q3

In cv2.fastNlMeansDenoising, h controls: A. patch size B. filter strength C. window stride D. iterations

Answer: B (L2)

Q4

Bilateral preserves edges because: A. it computes Laplacian B. range kernel down-weights dissimilar intensities C. it's iterative D. it works in frequency

Answer: B (L3)

Q5

A frequency-domain notch filter targets: A. Gaussian noise B. salt-and-pepper C. periodic noise D. speckle

Answer: C (L1)

Q6

Wiener deconvolution requires knowledge of: A. the noise standard deviation only B. the blur (PSF) C. ground-truth image D. SSIM target

Answer: B (L4)

Q7

PSNR is measured in: A. pixels B. dB C. percent D. milliseconds

Answer: B (L5)

Q8

SSIM range is: A. 0..1 B. 0..255 C. -1..1 D. 0..100

Answer: C (L5)

Q9

For a uint8 image, the correct data_range for PSNR is: A. 1 B. 100 C. 255 D. depends

Answer: C (L5)

Q10

Stacking bilateral too many times: A. always improves PSNR B. removes color C. eventually loses detail D. inverts the image

Answer: C (L3, L5)

Module 15 — Assignment: Denoiser Benchmark Suite

Estimated time: 2 hours

Combines: Module 15 + Modules 7, 14

Brief

Build a CLI that takes a clean image, synthesises noise of a chosen type, applies multiple denoisers, and produces a benchmark table with PSNR/SSIM.

Requirements

- `bench.py INPUT --noise gauss|sp|speckle [--sigma 20] [--p 0.03]`
- Load the clean reference; synthesise noise per the chosen type.
- Apply at least 4 denoisers: `cv2.GaussianBlur`, `cv2.medianBlur`, `cv2.bilateralFilter`, `cv2.fastNlMeansDenoising` (+ optionally Wiener).
- Compute PSNR and SSIM for each output vs reference.
- Print a sortable table (best PSNR first).
- Save a side-by-side image: `[clean | noisy | best-method-output]` with method label.

Constraints

- Modules 1-15 only.
- `argparse`.
- Use `skimage.metrics.peak_signal_noise_ratio` and `structural_similarity`.

Expected Output

```
$ python bench.py datasets/starter/camera.png --noise gauss --sigma 20
```

method	PSNR	SSIM
nlm	30.12	0.842
bilateral	28.34	0.781
gaussian	27.05	0.722
median	25.41	0.651

```
saved -> camera_bench.png (clean | noisy | nlm)
```

Grading Rubric

Criterion	Weight
Noise synthesis	15%
4 denoisers correct	30%
PSNR/SSIM computed	20%
Sorted table output	15%
Side-by-side image	10%
Argparse + style	10%

Submission

Save as `assignment_module15.py`.

Module 15 — Mini Project: Interactive Denoiser Comparator

Overview

A cv2-window comparator. Pick a noise type and intensity via trackbars; see four denoisers running side-by-side with live PSNR/SSIM numbers.

Concepts Used

- All of Module 15 (noise synthesis, NLM, bilateral, median, metrics).
- Trackbars (M2 L3).

Features

- Trackbars: noise type (0..2), noise intensity, NLM h, bilateral sigmaColor.
- 5-panel display: clean | noisy | bilateral | median | NLM (with labels + PSNR).
- s saves the whole comparison; q quits.

Starter Code

denoise_compare.py:

```
import sys, cv2, numpy as np
from skimage.metrics import peak_signal_noise_ratio as psnr

NOISES = ["gauss", "sp", "speckle"]

def add_noise(img, kind, intensity):
    rng = np.random.default_rng(0)
    if kind == "gauss":
        return np.clip(img + rng.normal(0, intensity, img.shape), 0,
255).astype(np.uint8)
    if kind == "sp":
        out = img.copy()
        m = rng.random(img.shape)
        p = intensity / 100
        out[m < p] = 0; out[m > 1 - p] = 255
        return out
    if kind == "speckle":
        n = rng.normal(1.0, intensity / 100, img.shape).astype(np.float32)
        return np.clip(img.astype(np.float32) * n, 0, 255).astype(np.uint8)

def annotate(img, label, score):
    out = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR) if img.ndim == 2 else img.copy()
    cv2.rectangle(out, (0, 0), (out.shape[1], 30), (0, 0, 0), -1)
    cv2.putText(out, f"{label} PSNR={score:.1f}",
        (5, 22), cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 0), 1)
    return out

def run(path):
    clean = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if clean is None: raise FileNotFoundError(path)
    if max(clean.shape) > 400: clean = cv2.resize(clean, (320, 320))
```

```

cv2.namedWindow("denoise")
cv2.createTrackbar("noise 0gauss 1sp 2speckle", "denoise", 0, 2, lambda v: None)
cv2.createTrackbar("intensity", "denoise", 20, 80, lambda v: None)
cv2.createTrackbar("nlm h", "denoise", 15, 40, lambda v: None)
cv2.createTrackbar("bilat sc", "denoise", 75, 200, lambda v: None)

while True:
    kind = NOISES[cv2.getTrackbarPos("noise 0gauss 1sp 2speckle", "denoise")]
    intensity = cv2.getTrackbarPos("intensity", "denoise") or 1
    h_nlm = cv2.getTrackbarPos("nlm h", "denoise") or 1
    sc = cv2.getTrackbarPos("bilat sc", "denoise") or 1

    noisy = add_noise(clean, kind, intensity)
    bi = cv2.bilateralFilter(noisy, 9, sc, sc)
    me = cv2.medianBlur(noisy, 3)
    nlm = cv2.fastNlMeansDenoising(noisy, None, h=h_nlm)

    panels = [
        annotate(clean, f"clean ({kind})", 99.0),
        annotate(noisy, "noisy", psnr(clean, noisy, data_range=255)),
        annotate(bi, f"bilat sc={sc}", psnr(clean, bi, data_range=255)),
        annotate(me, "median k=3", psnr(clean, me, data_range=255)),
        annotate(nlm, f"NLM h={h_nlm}", psnr(clean, nlm, data_range=255)),
    ]
    combo = np.hstack(panels)
    cv2.imshow("denoise", combo)
    k = cv2.waitKey(30) & 0xFF
    if k == ord('q'): break
    if k == ord('s'): cv2.imwrite("denoise_compare.png", combo); print("saved")
cv2.destroyAllWindows()

if __name__ == "__main__":
    run(sys.argv[1] if len(sys.argv) > 1 else "datasets/starter/camera.png")

```

Expected Interaction

Window shows 5 strips side-by-side with PSNR labels. Slide "intensity" to make noise stronger; see PSNRs drop. Switch noise type; observe median dominating on S&P.

Extension Challenges

- Add Wiener as a 6th panel.
- Add an "SSIM" toggle alongside PSNR.
- Click any panel to save just that one image.

Grading Rubric

Criterion	Weight
Three noise types	20%
Four denoisers	30%
PSNR labels live	20%

Slider responsiveness	15%
Style	15%

Python E-Course

Module 16 — Capstone Projects

Detailed Notes

Module Overview

Level: Application

Duration: ~2 weeks

Prerequisites: Modules 1-15

What This Module Is

Build, polish, and ship at least one substantial project that exercises everything you've learned. Four short scaffolding lessons cover the how of project building (planning, structure, testing CV pipelines, shipping). The real work lives in the `projects/` folder, which holds 8 self-contained capstone projects you can pick from.

Lessons

#	Lesson	Duration
1	Planning a Computer-Vision Project	25 min
2	CV Pipeline Architecture	25 min
3	Testing Computer-Vision Code	25 min
4	Shipping & Portfolio	25 min

Standalone Projects (in `projects/`)

#	Project	Modules Combined
1	Photo Editor CLI	2-8
2	Document Scanner	5, 9, 11, 12
3	License Plate Detector	9-13
4	Color-Based Object Tracker	4, 10, 12
5	Panorama Stitcher	5, 13
6	Barcode / QR Reader	6, 8, 11
7	Surface Defect Detection	6, 10-12
8	Image Forensics Tool	2, 7, 14

Each project README includes problem statement, ASCII architecture, step-by-step guide, starter code with TODOs, a complete reference solution, test plan, and 3 extension challenges.

Assessment

- Exercises — light warm-ups for the scaffolding lessons.
- Quiz — 10 MCQ on project-building methodology.
- Assignment — the Tier-A capstone specification.
- Project — the Tier-B mini-capstone specification (pick one of the smaller standalone projects).

What's Next

After this module: see [career/career-guide.md](#) and [career/portfolio-ideas.md](#). Run the certification test.

Lesson 1: Planning a Computer-Vision Project

Module: 16 — Capstone

Duration: 25 minutes

Difficulty: Applied

Learning Objectives

- Write a one-page spec for a CV project before any code.
- Decompose a vision pipeline into named stages.
- Pick the smallest v1 that's still meaningful.

Prerequisites

- Modules 1-15.

1. Why This Matters

The biggest failure mode for hobby CV projects is "I'll figure it out as I go." Five minutes of planning beats five hours of refactoring. CV in particular benefits from explicit stage thinking because pipelines are linear and each stage's output can be inspected.

2. Concept

2.1 The spec template

Write before opening an editor:

```
What:      <one sentence>
For:       <who or what use case>
Inputs:    <image source, size, format>
Outputs:   <annotated image / JSON / count / classification>
v1 stages: <bulleted pipeline, 3-7 items>
v1 NOT:    <explicit non-goals>
Success:   <one concrete sentence: "I can detect X with Y precision">
```

Example for a document scanner:

```
What:      Take a phone photo of a tilted document and produce a flat scan.
For:       Personal use — replace dragging out a scanner.
Inputs:    Phone photo, 8-12 MP, JPG.
Outputs:   Top-down rectangular PNG, b/w-thresholded option.
v1 stages: load → resize → Canny → contours → largest 4-vertex polygon →
perspective warp → adaptive threshold.
v1 NOT:    OCR, multi-page, perspective from corners < 50px gap, fancy lighting models.
Success:   I can scan 8 of 10 typical phone photos to a readable result, total
processing < 2 seconds.
```

2.2 Decompose into pipeline stages

Every CV project is a sequence of operations. Sketch them as stages with clear inputs/outputs.

Each stage should be:

- Inspectable — you can visualise its input and output.
- Replaceable — swap one implementation for another without touching the rest.
- Testable — assert something about its output given a known input.

2.3 Smallest meaningful v1

Don't try to be robust before you've been complete. Get the walking skeleton:

- Load one specific test image.
- Run all stages.
- Save the result.

Done? Now make it work on 5 more images, then handle edge cases.

2.4 Failure-mode list

Before coding, write down what could go wrong:

- Camera is upside down → handle EXIF rotation.
- Document fills frame → no margin for contours.
- Glare on glossy paper → adaptive threshold fails.
- Background has many edges → wrong contour selected.

Each entry hints at a stage that needs hardening.

2.5 Time-box the v1

Pick a hard limit (e.g. "I'll have v1 working by Friday"). If something's hard to do in v1, defer it explicitly to v2 in the spec.

3. Code Examples

(This lesson is process, not code. Templates instead.)

Template — spec.md

```
# <project name>

## What
<sentence>

## For
<sentence>

## Inputs
- size:
- format:
- source:

## Outputs
- format:
- where saved:
```

```

## v1 stages
1.
2.
3.

## v1 non-goals
-
-

## Success
"I can ..."

```

Template — pipeline sketch (ASCII)

[input] → load → resize → preprocess → detect → annotate → [output]

Template — failure modes

```

| Failure | Stage | Mitigation |
|-----|-----|-----|
| ... | ... | ... |

```

4. Parameter Sensitivity

n/a — process lesson.

5. Common Mistakes

Mistake	What Happens	Fix
Skip the spec	Endless yak-shaving	Write 1 page first
Try to be robust before complete	Stuck on edge cases	Walking skeleton first
No success metric	Never declare done	Write "I can ..." sentence
Ignore failure modes	Brittle code	List them, mitigate explicitly

6. Practice Tasks

- Pick a project from the projects/ folder. Write its spec using the template above.
- Sketch the ASCII pipeline.
- List 5 failure modes and mitigations.

7. Key Takeaways

- Spec first, code second.
- Pipeline = sequence of named stages.
- Walking skeleton before robustness.
- Explicit non-goals are as valuable as goals.

8. What's Next

How to organise the code itself — folder layout, modules, configuration.

Lesson 2: CV Pipeline Architecture

Module: 16 — Capstone

Duration: 25 minutes

Difficulty: Applied

Learning Objectives

- Lay out a small CV project (folders, modules, CLI entry).
- Separate I/O, pipeline stages, and visualisation.
- Add per-stage debug image dumps.

Prerequisites

- Module 16 Lesson 1

1. Why This Matters

A clean structure makes CV pipelines easy to debug. The trick: every stage saves an intermediate image on demand. That makes "why is the output wrong?" a 30-second question instead of a 30-minute one.

2. Concept

2.1 Default layout

```
myproject/
├── README.md
├── pyproject.toml      # or requirements.txt
├── .gitignore
├── myproject/         # the package
│   ├── __init__.py
│   ├── __main__.py    # so `python -m myproject` works
│   ├── cli.py         # argparse + dispatch
│   ├── pipeline.py    # the stages
│   ├── io.py          # load / save / dump
│   └── viz.py         # annotation / overlay helpers
├── samples/           # test images
└── tests/             # smoke tests
```

For very small projects, collapse pipeline.py, viz.py, and io.py into one file. Add structure when it earns itself.

2.2 Pipeline as named functions

Each stage is a function `f(img, **params) -> img`:

```
def preprocess(img):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    return cv2.GaussianBlur(gray, (5, 5), 0)

def detect_edges(gray):
```

```

    return cv2.Canny(gray, 50, 150)

def find_quad(edges):
    contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE)
    # ... returns 4 points

```

The pipeline driver chains them and optionally saves intermediates:

```

def run(img, debug_dir=None):
    pre = preprocess(img)
    if debug_dir: cv2.imwrite(f"{debug_dir}/01_pre.png", pre)
    edges = detect_edges(pre)
    if debug_dir: cv2.imwrite(f"{debug_dir}/02_edges.png", edges)
    quad = find_quad(edges)
    ...

```

2.3 Visualisation module

Keep annotation code (rectangles, text overlays, color masks) in viz.py. Don't mix it with the pipeline — your unit tests will thank you.

2.4 Config + defaults

For tunable parameters, use a Config dataclass or simple dict:

```

from dataclasses import dataclass

@dataclass
class Config:
    canny_low: int = 50
    canny_high: int = 150
    min_area: int = 500

```

Pass Config() to run(img, cfg). Easier to test, easier to tune.

2.5 CLI entry

```

# myproject/cli.py
import argparse, cv2
from .pipeline import run, Config

def main():
    p = argparse.ArgumentParser()
    p.add_argument("input")
    p.add_argument("--output", default="result.png")
    p.add_argument("--debug-dir")
    p.add_argument("--canny-low", type=int)
    p.add_argument("--canny-high", type=int)
    args = p.parse_args()

    cfg = Config()
    if args.canny_low is not None: cfg.canny_low = args.canny_low
    if args.canny_high is not None: cfg.canny_high = args.canny_high

```

```

img = cv2.imread(args.input)
out = run(img, cfg, debug_dir=args.debug_dir)
cv2.imwrite(args.output, out)

```

Then `python -m myproject INPUT --debug-dir dump/` writes every stage to `dump/`.

2.6 The "explain this output" workflow

When something looks wrong:

- Re-run with `--debug-dir`.
- Open the numbered intermediate images in order.
- Find the first one that looks wrong.
- Tune that stage's parameters.

This turns vague "doesn't work" into specific "Canny threshold too high."

3. Code Examples

Example 1: Minimal pipeline structure

```

# project/pipeline.py
from dataclasses import dataclass
import cv2

@dataclass
class Config:
    canny_low: int = 50
    canny_high: int = 150

def preprocess(bgr):
    gray = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY)
    return cv2.GaussianBlur(gray, (5, 5), 0)

def detect_edges(gray, cfg):
    return cv2.Canny(gray, cfg.canny_low, cfg.canny_high)

def run(bgr, cfg=Config(), debug_dir=None):
    pre = preprocess(bgr)
    edges = detect_edges(pre, cfg)
    if debug_dir:
        from pathlib import Path
        Path(debug_dir).mkdir(parents=True, exist_ok=True)
        cv2.imwrite(f"{debug_dir}/01_pre.png", pre)
        cv2.imwrite(f"{debug_dir}/02_edges.png", edges)
    return edges

```

Example 2: Tiny CLI wrapper

```

# project/cli.py
import argparse, cv2
from .pipeline import run, Config

def main():

```



```

p = argparse.ArgumentParser()
p.add_argument("input")
p.add_argument("--output", default="out.png")
p.add_argument("--debug-dir")
args = p.parse_args()
img = cv2.imread(args.input)
out = run(img, Config(), debug_dir=args.debug_dir)
cv2.imwrite(args.output, out)

if __name__ == "__main__":
    main()

```

Example 3: Test a single stage in isolation

```

# tests/test_preprocess.py
import numpy as np
from project.pipeline import preprocess

def test_preprocess_output_shape():
    img = np.zeros((100, 200, 3), dtype=np.uint8)
    out = preprocess(img)
    assert out.shape == (100, 200)
    assert out.dtype == np.uint8

```

4. Parameter Sensitivity

n/a — architecture lesson.

5. Common Mistakes

Mistake	What Happens	Fix
One giant function	Impossible to debug	Split into stages
Mix viz with logic	Untestable	Separate viz.py
Hardcoded params	Re-edit code to tune	Config dataclass
No debug-dir	Have to add cv2.imwrite ad-hoc	Build it in from v1

6. Practice Tasks

- Lay out the recommended package structure for one project from projects/.
- Refactor the project's starter code into 3 named stages.
- Add a --debug-dir flag that dumps each stage.

7. Key Takeaways

- Pipeline = sequence of small named stages.
- Config dataclass over loose globals.
- Add --debug-dir from the start.
- Separate I/O, logic, and visualisation.

8. What's Next

Testing — how to write meaningful tests for code where the "correct" output is fuzzy.

Lesson 3: Testing Computer-Vision Code

Module: 16 — Capstone

Duration: 25 minutes

Difficulty: Applied

Learning Objectives

- Write smoke tests for CV pipelines.
- Use synthesised fixtures (small drawn images) for stage-level tests.
- Use PSNR/SSIM thresholds for regression checks.
- Set up a sample-image evaluation harness.

Prerequisites

- Module 16 Lesson 2

1. Why This Matters

Testing CV code is awkward — there's rarely a single "correct" output. But you can still pin down properties: shape, dtype, score thresholds, regression vs a known-good output. Five tests cover the cases that actually break.

2. Concept

2.1 Three test layers

- Smoke tests — "does it run without crashing on a typical input?"
- Stage tests — "given a known input, does this stage produce the expected output?" (assert shape, type, count of detections).
- End-to-end regression — "is the output close enough (PSNR/SSIM) to a saved reference?"

2.2 Synthetic fixtures

Build tiny images programmatically — much faster than loading samples:

```
import numpy as np, cv2
def make_square_doc():
    img = np.full((400, 400), 250, dtype=np.uint8)
    cv2.rectangle(img, (50, 50), (350, 350), 100, -1)
    return img
```

Use these for stage-level tests where you control everything.

2.3 Pytest pattern

```
import numpy as np
from myproject.pipeline import detect_quad

def test_detect_quad_finds_4_corners():
    img = make_square_doc()
    quad = detect_quad(img)
```

```
assert quad is not None
assert quad.shape == (4, 2)
```

2.4 PSNR-based regression

For pipelines where the result is an image (deblur, restore, recolor):

```
from skimage.metrics import peak_signal_noise_ratio
def test_regression():
    img = cv2.imread("samples/in.png")
    expected = cv2.imread("samples/expected.png")
    actual = run(img)
    assert peak_signal_noise_ratio(expected, actual, data_range=255) > 30
```

Set the threshold based on what you observed during development.

2.5 Property-based tests

Useful properties to check on outputs:

- Mask is uint8 in {0, 255}.
- Detected count > 0.
- Bounding box inside image bounds.
- Centroid coordinates positive.
- Output image same size as input.

2.6 Visual review folder

Even with tests, keep a small folder of samples/before/ + samples/after/ that you spot-check after each commit. CV bugs that pass numerical tests often catch your eye instantly.

2.7 CI considerations

If putting this in CI:

- Pin OpenCV version (small algorithm changes between versions can flip outputs).
- Set a random seed in tests that involve noise.
- Use small synthetic images, not 4K samples.

3. Code Examples

Example 1: Smoke test

```
# tests/test_smoke.py
import numpy as np
from myproject.pipeline import run

def test_smoke_runs():
    img = np.full((300, 400, 3), 128, dtype=np.uint8)
    out = run(img)
    assert out is not None
```

Example 2: Stage test with synthetic fixture

```
import numpy as np, cv2
from myproject.pipeline import find_blobs

def make_two_blobs():
    img = np.zeros((200, 400), dtype=np.uint8)
    cv2.circle(img, (100, 100), 30, 255, -1)
    cv2.circle(img, (300, 100), 40, 255, -1)
    return img

def test_finds_two_blobs():
    blobs = find_blobs(make_two_blobs())
    assert len(blobs) == 2
```

Example 3: PSNR-based regression

```
import cv2
from skimage.metrics import peak_signal_noise_ratio
from myproject.pipeline import run

def test_regression_quality():
    inp = cv2.imread("samples/01_in.jpg")
    expected = cv2.imread("samples/01_expected.png")
    actual = run(inp)
    assert peak_signal_noise_ratio(expected, actual, data_range=255) > 28
```

Example 4: Eval harness on a folder of samples

```
import cv2
from pathlib import Path
from myproject.pipeline import run

def evaluate(folder):
    folder = Path(folder)
    results = []
    for path in sorted(folder.glob("*.jpg")):
        img = cv2.imread(str(path))
        out = run(img)
        cv2.imwrite(str(folder / "out" / path.name), out)
        results.append(path.name)
    print(f"processed {len(results)} images")

if __name__ == "__main__":
    evaluate("samples")
```

4. Parameter Sensitivity

n/a — process lesson.

5. Common Mistakes

Mistake	What Happens	Fix
No tests at all	Refactor with fear	Add at least smoke test
Test with random images	Flaky on order changes	Use synthetic + seed
Test against pixel-exact	Tiny OpenCV changes break	Use PSNR/SSIM thresholds

output	things	
Test only happy path	Edge cases ship broken	Add "empty input" / "no detection" tests

6. Practice Tasks

- Write a smoke test for one project from projects/.
- Write a stage test using a synthetic fixture.
- Add a PSNR-based regression test for a denoising pipeline.

7. Key Takeaways

- Smoke + stage + regression = 3 layers.
- Synthetic fixtures > samples for stage tests.
- PSNR/SSIM thresholds for fuzzy outputs.
- Keep a visual review folder.

8. What's Next

The final lesson: shipping the project (README, demo gif, releases).

Lesson 4: Shipping & Portfolio

Module: 16 — Capstone

Duration: 25 minutes

Difficulty: Applied

Learning Objectives

- Write a README that demos a CV project clearly.
- Capture before/after images for the README.
- Push to GitHub and tag a release.
- Talk about a CV project in an interview.

Prerequisites

- Module 16 Lesson 3

1. Why This Matters

A great CV project no one understands is invisible. CV projects in particular live or die on visuals — a before/after image in the README does more than any paragraph. Spend the last 10% on packaging; it pays back hugely.

2. Concept

2.1 README structure for a CV project

```
# Project Name

One-line tagline.

## Demo

[BEFORE → AFTER image]
[GIF if the project handles video]

## Features
- bullet 1
- bullet 2

## Install
$ python -m venv .venv
$ source .venv/bin/activate
$ pip install -r requirements.txt

## Use
$ python -m project INPUT --output OUT.png
[more examples]

## How it works
- stage 1: ...
```

```
- stage 2: ...
```

```
## Limitations
```

```
- ...  
- ...
```

```
## License
```

```
MIT
```

The "How it works" section is what differentiates a CV project on a portfolio — it shows you understand your pipeline, not just stitched library calls.

2.2 The before/after image

Compose a single PNG with a small dividing line:

```
import cv2, numpy as np  
before = cv2.imread("samples/in.jpg")  
after = cv2.imread("samples/out.png")  
divider = np.full((before.shape[0], 4, 3), 255, dtype=np.uint8)  
combo = np.hstack([before, divider, after])  
cv2.imwrite("demo.png", combo)
```

Show that on top of the README. Reviewers will see the result before reading a word.

2.3 Animated GIFs

For video / live projects: record a screen capture (15-30 s), reduce to GIF with ffmpeg:

```
ffmpeg -i demo.mp4 -vf "fps=10,scale=640:-1" demo.gif
```

GitHub renders inline.

2.4 Limitations section

A short, honest list of what your project doesn't handle ("works only on landscape orientation", "fails under glare", "max 1 document per image") is more impressive than over-claiming. Senior engineers read these to gauge maturity.

2.5 Releases and tags

When v1 works:

```
git tag -a v0.1.0 -m "First release"  
git push origin v0.1.0
```

On GitHub: Releases → Draft new release → pick the tag → list features. Even small projects benefit from a tagged release — it's a milestone you can reference later.

2.6 Talking about a CV project in an interview

Pattern:

- Problem in one sentence: "Phone photos of documents look tilted; I built a tool that flattens them."

- Stack and stages: "OpenCV, scikit-image. Canny → contour → largest 4-vertex polygon → perspective warp → adaptive threshold."
- One thing you learned: "Adaptive threshold beats Otsu when lighting is uneven — I learned that the hard way after Otsu kept failing on documents shot near windows."
- Limitations + next steps: "Doesn't handle multi-page; corners < 50 px from frame edges fail. Next: train a small UNet for corner detection to handle those."

This 60-second answer tells a reviewer you can plan, build, evaluate, and improve.

2.7 Ship checklist

- [] README with demo image at the top.
- [] Install + Use sections work as-is on a clean machine.
- [] Tests pass (even just smoke).
- [] .gitignore excludes samples that aren't yours, .venv, __pycache__.
- [] LICENSE (MIT is fine for personal projects).
- [] One tagged release.

3. Code Examples

Example 1: Build a side-by-side demo image

```
import cv2, numpy as np

before = cv2.imread("samples/01_in.jpg")
after = cv2.imread("samples/01_out.png")
# Match heights (if needed) before stacking
h = min(before.shape[0], after.shape[0])
before = cv2.resize(before, (int(before.shape[1] * h / before.shape[0]), h))
after = cv2.resize(after, (int(after.shape[1] * h / after.shape[0]), h))
divider = np.full((h, 4, 3), 255, dtype=np.uint8)
combo = np.hstack([before, divider, after])
cv2.imwrite("docs/demo.png", combo)
```

Example 2: Generate a 4-up grid for the README

```
import cv2, numpy as np

imgs = [cv2.imread(p) for p in
        ["samples/in1.jpg", "samples/in2.jpg",
         "samples/out1.png", "samples/out2.png"]]
top = np.hstack(imgs[:2])
bottom = np.hstack(imgs[2:])
grid = np.vstack([top, bottom])
cv2.imwrite("docs/grid.png", grid)
```

Example 3: requirements.txt example

```
opencv-python>=4.5
scikit-image>=0.20
numpy>=1.23
Pillow>=9.0
```


Example 4: Tag and release (shell)

```
git add .  
git commit -m "v0.1.0: working document scanner"  
git tag -a v0.1.0 -m "First release"  
git push origin main --tags
```

4. Parameter Sensitivity

n/a — process lesson.

5. Common Mistakes

Mistake	What Happens	Fix
No demo image	Reviewer scrolls past	Demo at the very top
README walls of text	Skipped	Use sections; lead with visuals
Hide limitations	Looks oblivious	Honest "Limitations" section
No tagged release	Project feels in-progress	Tag v0.1.0 the moment v1 works

6. Practice Tasks

- Write a README for one of the projects/ projects using the template.
- Build a side-by-side demo PNG.
- Tag v0.1.0 once your v1 works.

7. Key Takeaways

- README leads with a before/after image.
- "How it works" beats library lists for CV.
- Honest "Limitations" wins respect.
- Tag releases; record GIFs for video projects.

8. What's Next

End of Module 16. Pick a project from projects/ and ship it.

Module 16 — Exercises

These are light scaffolding tasks. The real work is in the assignment and the projects/ directory.

16.1 Planning

- (E) Write a one-page spec for any project from projects/ using the template.
- (M) Sketch its pipeline as an ASCII diagram with named stages.
- (H) List 5 failure modes for that project with mitigations.

16.2 Architecture

- (E) Lay out the recommended package directory for that project.
- (M) Refactor the starter code into 3 named stages.
- (H) Add a --debug-dir flag that dumps each stage as a PNG.

16.3 Testing

- (E) Write a smoke test that runs the pipeline on a synthetic image.
- (M) Write a stage-level test using a synthetic fixture (e.g. a drawn square).
- (H) Add a PSNR-based regression test against a saved expected output.

16.4 Shipping

- (E) Write a README for your chosen project using the lesson-4 template.
- (M) Build a side-by-side demo PNG and embed it at the top of the README.
- (H) Tag v0.1.0 on the repo and write release notes.

Module 16 — Quiz

10 MCQs on capstone methodology.

Q1

The first artefact in a CV project should be: A. the README B. one-page spec C. starter code D. unit tests

Answer: B (L1)

Q2

"Walking skeleton" means: A. fastest implementation B. smallest version that runs end-to-end C. final UI shell D. README

Answer: B (L1)

Q3

Each pipeline stage should be: A. inline in one function B. inspectable, replaceable, testable C. written in C D. parallelised

Answer: B (L2)

Q4

A --debug-dir flag should: A. enable verbose logging B. dump each stage's image to disk C. open a GUI D. profile timings

Answer: B (L2)

Q5

For tunable parameters, prefer: A. globals B. Config dataclass / dict C. environment variables D. hardcoded constants

Answer: B (L2)

Q6

For testing image outputs whose pixel-exact value is unstable: A. assertEquals B. PSNR/SSIM thresholds C. md5 hash D. visual review only

Answer: B (L3)

Q7

A synthetic fixture for tests should: A. be a 10 MP photo B. be drawn programmatically; small and reproducible C. live on disk D. be random

Answer: B (L3)

Q8

What goes at the very top of a CV project README? A. install instructions B. a before/after demo image C. licence D. authors

Answer: B (L4)

Q9

A "Limitations" section is: A. discouraged — looks bad B. helpful — shows engineering maturity C. only for academic projects D. optional

Answer: B (L4)

Q10

When v1 of a project works, you should: A. start v2 immediately B. tag a release (e.g., v0.1.0) and stop to polish docs C. discard and rewrite D. wait for feedback

Answer: B (L4)

Module 16 — Tier-A Capstone Assignment

Estimated time: 12-20 hours

Combines: All of Modules 1-15

Brief

Pick one of the 8 standalone projects in projects/, build it end-to-end, and ship it. The goal isn't "use every module" — it's "build something good enough to put on a CV."

Required deliverables

For the project you choose:

- Package layout per Module 16 Lesson 2 (myproject/__main__.py, pipeline.py, cli.py, viz.py, tests/).
- pyproject.toml or requirements.txt with pinned versions.
- README following the lesson-4 template:
- Title + one-line tagline.
- Before/after demo image at the very top.
- Features bullet list.
- Install + Use sections.
- "How it works" (pipeline stages explained).
- Limitations section.
- LICENSE.
- CLI: `python -m myproject INPUT [--output] [--debug-dir] [project-specific flags]`.
- Tests: at minimum 1 smoke + 2 stage tests; ideally a regression test.
- Sample images in samples/ (use the starter images or your own).
- Tagged release (v0.1.0 once it works).

Recommended workflow

- Read the project's projects/XX-name/README.md to understand the requirements.
- Write a one-page spec.md in your repo (Lesson 1 template).
- Build the walking skeleton (load → core stage → save) on one test image.
- Add the remaining stages one at a time.
- Run on 5 more sample images; fix what breaks.
- Add tests, README, demo image.
- Tag.

Constraints

- Use OpenCV, NumPy, scikit-image, matplotlib, Pillow as needed.
- A third-party CV library beyond these (e.g. pyzbar for QR) is fine; cite it in README.
- No deep learning — this is a classical CV course. (You can add a DL stage in v2.)

Grading Rubric

Criterion	Weight
Walking skeleton works	20%
All required features	30%
Code organisation	15%
Tests pass	10%
README + demo image	15%
Tagged release	5%
Style / clean code	5%

Submission

A GitHub repo URL (or zip), with:

- The package
- Tests
- README + sample images
- Tagged release
- A short (3-5 min) screencast or written walkthrough

Module 16 — Tier-B Mini-Capstone Project

Estimated time: 4-8 hours

Combines: Selected modules per the chosen project.

Brief

Build a shorter version of any project from projects/ — implement the core pipeline only, skip extension challenges. Use this for a quick portfolio piece if you don't have time for the full Tier-A capstone.

Required Deliverables

- Single-file Python script in your repo (no package structure required at this level).
- One README.md with:
 - 2-paragraph description.
 - Install one-liner.
 - Use one-liner.
 - Before/after demo image.
- Sample images demonstrating it working on at least 3 inputs.

Suggested Mini-Capstones (lighter scope of each project)

Pick	Scope reduction
Photo Editor CLI	Just 3 ops (brightness, contrast, blur) + save
Document Scanner	Manual 4-corner click (not auto-detect) + warp + save
License Plate Detector	Locate plate region only; don't read characters
Color Object Tracker	Single image (not video); just bounding box around colored object
Panorama Stitcher	Two pre-aligned images (skip Lowe + RANSAC reliability)
Barcode/QR Reader	Direct pyzbar.decode on input; no preprocessing
Defect Detection	One known good vs one inspect image; threshold absdiff
Image Forensics	Just ELA (Error Level Analysis), nothing else

Constraints

- Use OpenCV / NumPy / scikit-image / matplotlib / Pillow.
- A single .py file is fine; no need for a package.
- README must lead with a demo image.

Grading Rubric

Criterion	Weight
-----------	--------

Script runs as-described	40%
Demo image in README	20%
Quality of result	20%
Code clarity	10%
Sample inputs included	10%

Submission

A GitHub repo or zip with the script, README, and samples/ folder.